



人工智能的数学思维

组织：华东师范大学软件工程学院

时间：2024

目录

前言	4
0.1 从“困难重重”到“显而易见”：智能发展的历史辩证法	4
0.2 数学史：智能追求的发展主线	5
0.3 智能的层次：从模仿到创造	5
0.4 本书的目标与挑战	6
0.5 致读者	6
第一部分 追问智能	8
0.6 什么是智能与人工智能	9
0.7 鹦鹉与乌鸦：何种智能?	11
第1章 数学发展史：从计数到抽象的智慧之路	16
1.1 萌芽时期：从计数到位值制	16
1.1.1 史前数学：日常需求与实用计算	16
1.1.2 手指：长身上的计数器	16
1.1.3 荒岛上的自然教育，人类数字文明的缩影	17
1.1.4 从刻痕到符号：表示系统的演进	17
1.2 初等数学时期：从计算到推理	17
1.2.1 几何学的诞生：从测量到证明	17
1.2.2 代数的萌芽：从具体到抽象	18
1.3 变量数学时期：从静态到动态	18
1.3.1 解析几何：代数与几何的统一	18
1.3.2 微积分：变化的数学	18
1.3.3 概率论：不确定性的数学	19
1.4 现代数学时期：从具体到抽象	19
1.4.1 集合论：数学的统一基础	19
1.4.2 抽象代数：结构的数学	19
1.4.3 拓扑学：连续性的抽象	20
1.5 三次数学危机：从无理数到不完备	20
1.5.1 第一次危机：无理数的发现与算术信仰的破灭	20
1.5.2 第二次数学危机：微积分基础的争议与解析	20
1.5.3 第三次危机：集合论与逻辑的悖论	21
1.6 小结	22

第 2 章 智能，成于计算机	
简明 AI 史	23
2.1 AI 的萌芽期：理论的起源与计算机的诞生	23
2.2 1950-1970 年代：AI 的奠基与早期探索	25
2.2.1 达特茅斯会后早期黄金期：符号主义 AI	26
2.3 1980 年-1987 年：专家系统与知识工程的兴起	28
2.3.1 代表性专家系统	29
2.3.2 知识表示与规则推理	29
2.3.3 知识工程的广泛应用	29
2.3.4 新兴技术：模糊逻辑与贝叶斯网络	30
2.3.5 第二次“AI 寒冬”的反思与转型	30
2.4 人工智能的分化	30
2.5 1990-2010 年代：联结主义和机器学习的兴起	31
2.5.1 2020 年代：大语言模型（LLM）与生成式人工智能（AIGC）的崛起	32
2.5.2 符号主义学派 (Symbolicism)	33
2.5.3 联结学派/连接学派 (Connectionism)：从神经网络到深度学习	35
2.5.4 行为主义学派 (Actionism)	36
2.5.5 三大学派的比较	37
2.6 小结：危机与机遇中的 AI 发展之旅	38
参考文献	41

第二部分 学习的逻辑 43

第 3 章 人工智能的三种思维：从理论到应用的跨越	44
3.1 数学思维：定义和可能性的框架	44
3.2 算法思维：从理论到可计算的路径	46
3.2.1 算法复杂度	46
3.3 工程思维：从可计算到可用的实际落地	49
3.4 两个从数学思维到工程思维的例子	51
3.4.1 数字图像处理：从理论到实际应用的三重思维	51
3.4.1.1 数学思维：图像处理的理论基础	51
3.4.1.2 算法思维：从理论模型到具体算法	52
3.4.1.3 工程思维：图像处理的实际落地	53
3.4.2 总结	54
3.4.2.1 数学思维：推荐系统的理论建模	54
3.4.2.2 工程思维：推荐系统的实际部署	56

3.5 小结：三种思维的交融与驱动	57
参考文献	58
第 4 章 人工智能的数学工具	60
4.1 预备知识	61
4.2 浅谈图像处理：从矩阵到深度学习	61
4.3 第 1 章引言	62
4.3.1 1.1 图像识别如何作用于我们的生活?	62
4.3.2 1.2 追根溯源—其中的数学方法	62
4.3.3 1.3 报告主体结构与主要内容	62
4.3.4 2.1 矩阵	63
4.3.5 矩阵的应用现状	64
4.4 体会启发与结语	72
4.5 次线性算法	73
4.6 迈进后严谨阶段!	75
参考文献	77
第 5 章 从传统机器学习到深度学习	79
5.1 从机器学习到深度学习	79
5.1.1 从人工筛选到自动归纳	79
5.1.2 从“人工扶持”到“自主学习”	80
5.1.3 像侦探一样找线索	80
5.1.4 学骑自行车的启示	80
5.1.5 深度学习的“洋葱式思考”	81
5.1.6 深度学习为何如此强大?	81
5.1.6.1 超强泛化能力	81
5.1.6.2 数据越多，模型越强	81
5.2 生物神经元的启发：模拟大脑	82
5.2.1 深度学习的起源：向人脑学习	82
5.2.2 智能如何产生：生物神经元的启发	82
5.2.3 生物神经元计算模型的启示	85
5.3 M-P 模型：人工神经网络的起点	86
5.3.1 M-P 模型的基本结构	88
5.3.2 M-P 模型如何实现逻辑运算	91
小故事：数学天才遇上神经科学家	93
5.4 感知机：第一个可训练的神经网络，神经网络的“Hello World”	94

5.4.1	感知机的工作原理	95
5.4.2	感知机示例：“西瓜 vs 香蕉”分类	97
5.4.3	感知机的几何意义	98
5.4.4	感知机的局限性：XOR 问题与神经网络的停滞	100
5.4.5	感知机的实际应用	100
5.4.6	感知机的历史影响	101
	小故事：感知机的传奇人物：弗兰克·罗森布拉特	102
5.5	多层感知机（1980s 复兴）：突破单层感知机的局限	102
5.5.1	增加隐藏层：解决线性不可分问题	102
5.5.2	两层感知机如何解决“异或”问题	103
5.5.3	真正的突破：反向传播（Backpropagation）	104
5.5.4	反向传播的影响：神经网络的真正复兴	106
5.6	前馈神经网络（Forward Neural Networks）：从简单到复杂	106
5.6.1	前馈神经网络的基本结构	107
5.7	深度学习（2006 及之后）：突破梯度消失，实现多层神经网络的高效训练	107
5.7.0.1	激活函数（Activation Function）	109
5.7.1	神经网络的数学灵魂：通用逼近定理（Universal Approximation Theorem）	117
5.8	人工神经网络的主要类型	118
5.9	通用逼近定理的局限性：为何我们需要深度学习？	120
5.10	从浅层网络到深度学习	121
5.11	小结	121
	参考文献	123
	第 6 章 深度学习，真的行	125
6.1	引言—深度学习：从理论走向现实	125
6.2	深度学习的方法论	126
6.2.1	端到端 (end-to-end) 学习：自动特征学习的突破	126
6.2.2	优化方法的改进：让深度网络真正可训练	127
6.2.2.1	梯度稳定性问题及解决方案	128
6.2.2.2	梯度消失问题（Vanishing Gradient）	128
6.2.2.3	梯度爆炸问题（Exploding Gradient）	128
6.2.2.4	自适应优化算法（Adaptive Optimization Algorithms）	129
6.2.3	正则化：防止过拟合，提高泛化能力	129
6.2.4	优化与泛化的平衡	132
6.3	全连接神经网络：深度学习的“原型”与局限性	132
6.4	神经网络的不同架构：从认知方式到计算方式	134
6.4.1	卷积神经网络（CNN）：如何高效处理图像？	135

6.4.1.1	从人类视觉到人工智能：CNN 如何“看”世界	136
6.4.1.2	人类视觉的分层感知机制	136
6.4.1.3	CNN 的关键结构及其数学原理	139
6.4.1.4	CNN 让计算机“看”世界更高效	144
6.4.2	循环神经网络 (RNN)：如何建模序列数据?	147
6.4.2.1	Why：为什么需要 RNN?	148
6.4.2.2	How—RNN 的数学直觉：如何让网络“记住”信息?	148
6.4.2.3	RNN 的局限性：记忆并不总是可靠	149
6.4.2.4	LSTM 与 GRU：赋予 RNN 更强的记忆能力	149
6.4.2.5	RNN 在现实中的应用	150
6.4.2.6	RNN 与 CNN：互补还是竞争?	150
6.4.2.7	总结	150
6.4.3	Transformer：为什么能取代 RNN?	151
6.4.3.1	小结：神经网络的不同架构与数学思维	154
参考文献		161
参考文献		163

自序

在动笔之前，我常被一个朴素而执拗的问题追着走：我们究竟为什么要谈人工智能？是为了追逐新技术的光亮，还是为了在这片光亮背后照见更久远的智性传统——数学、逻辑与经验之间的来回穿梭。写这本书的动机，便是在繁复的术语与迅疾的迭代之上，给读者留一段可以驻足的路径：从问题的提出、到模型的表达、再到工程的落地，层层看清“智能”如何成为可讨论、可计算、可实现的对象。

我坚持从“问题意识”出发。任何技术看似强大，若没有问题意识，也只会迷失在工具的丰盛里。人类的智能之所以动人，是因为它总能把世界重新描述为一组可以被比较、被检验的命题。数学给出形式化的骨架，算法让推理可走，工程则把抽象按到现实的尺度上。三者并非线性递进，而是彼此往复：一个好问题往往逼迫我们改写模型，一个可靠的模型又会反过来修正问题的边界，工程细节最终决定了这些想法能否在真实世界里站住脚。

本书的结构因此尽量保持朴素。每一章都会交代“为何提出这个主题”“我们用怎样的数学去刻画”“具体的算法与实现有哪些关键环节”。读者不必在意术语的完整性，而应关心一个观点能否自洽、能否经受例子与反例的考验。与其把人工智能全盘描述成一架精密仪器，不如把它视为一种持续改进的思维习惯：敢于提出假设，愿意暴露不确定，允许在数据面前修正自信。

在写作上，我尽量避免炫技式的密度。知识固然需要准确，但准确不必与艰涩同义。许多概念其实可以借助朴素比喻来理解：梯度不过是“朝着更好的方向迈一步”的量化；正则化不过是“别让模型把噪声当规律”的提醒；泛化能力说到底，是“在陌生数据上仍不失准度”的朴素期望。我们在复杂与可读之间保持一条窄路，让读者感到“可跟上”，又不会牺牲严谨。

我也希望把“限制”写在书页上。人工智能并不是一切问题的万灵药。从计算理论到工程实践，处处存在代价：算法需要时间与空间，数据需要清洗与标注，硬件有能耗与延迟，更重要的是，真实世界的目标函数往往多维而矛盾。理解这些限制并不让人气馁，反而会使我们在选择上更克制，在评价上更诚实，在改进上更有方向。

读者可能来自不同背景。有工程经验的读者可以从案例出发，逆推背后的数学与假设；偏理论的读者则可以在公式之间找到“可实现性”的锚点；初学者不妨从术语表与插图开始，把大图景先搭起来，再逐步填充细节。我建议以“问题—模型—实现”的节奏来阅读，每到一个关键术语，问自己三个问题：它解决了什么；它假设了什么；它牺牲了什么。

在写作过程中，我反复提醒自己：不要让结论跑在论证前面。数据的可变、环境的复杂、人的目标的多样性，都意味着我们要在多条路径上并行行走。一次实验的成功可能来自巧合；一次算法的失效也许只是样本不够。把不确定写出来，正是科学精神的底色。愿这本书在带来启发的同时，也保留谦卑：承认未知，拥抱修正。

如果要用一句话概括本书希望留下的印象，那便是：智能是一种改写世界描述的能力。我们通过数学给出可检验的形式，通过算法让这种形式可运行，再通过工程把它纳入真实约束之中。每一次从“描述”到“实现”的往返，都是对世界的再认识。无论你身处研究、产业还是教

育，我都希望你能在这些往返里，找到属于自己的步伐。

最后，感谢一路同行的人：同事与学生提供的讨论，行业伙伴分享的实践，读者反馈出的困惑。它们共同塑造了本书的节奏与取舍。也许我们无法穷尽“智能”的边界，但我们可以在边界附近不断点亮路标。把复杂讲清楚，把清楚做出来，这便是写作与研究最朴素的初心。

愿你在阅读的过程中，不仅收获概念与方法，更收获提出好问题的勇气、承受不确定的耐心以及在现实约束下推进改进的恒心。我们在下一章继续前行，但并不急于抵达；重要的是在每一步停下时，都能回答：为什么在此处停下，下一步又将通向何方。

他序

前言

内容提要

□ 现在看起来的“显而易见”，在当年却是“困难重重”。这句话深刻地概括了人工智能发展的历史轨迹，也揭示了

人类智慧演进的根本规律。今天我们习以为常的技术成就，在其诞生之初都曾面临着巨大的挑战和质疑。

0.1 从“困难重重”到“显而易见”：智能发展的历史辩证法

现在看起来的“显而易见”，在当年却是“困难重重”。这句话，也许是理解人工智能发展历程的最好钥匙。

今天我们轻松使用语音助手、个性化推荐、智能搜索摘要、实时翻译，甚至让大模型生成文本、代码与图像，驱动办公 Copilot 与多模态对话——很难想象这些能力背后经历过怎样的艰难探索。每一个今天看来理所当然的功能，都是无数研究者在黑暗中摸索、在失败中坚持、在质疑中前行的结果。

任何一种计算技术要想发挥其全部影响力，都必须得到充分的理解、充分的文档记录，并得到成熟的、维护良好的工具的支持。关键思想应该被清楚地提炼出来，尽可能减少需要让新的从业者跟上时代的入门时间。成熟的库应该自动化常见的任务，示例代码应该使从业者可以轻松地修改、应用和扩展常见的应用程序，以满足他们的需求。以动态网页应用为例。尽管许多公司，如亚马逊，在 20 世纪 90 年代开发了成功的数据库驱动网页应用程序。但在过去的 10 年里，这项技术在帮助创造性企业家方面的潜力已经得到了更大程度的发挥，部分原因是开发了功能强大、文档完整的框架。

测试深度学习的潜力带来了独特的挑战，战，因为任何一个应用都会将不同的学科结合在一起。应用深度学习需要同时了解（1）以特定方式提出问题的动机；（2）给定建模方法的数学；（3）将模型拟合数据的优化算法；（4）能够有效训练模型、克服数值计算缺陷并最大限度地利用现有硬件的工程方法。同时教授表述问题所需的批判性思维技能、解决问题所需的数学知识，以及实现这些解决方案所需的软件工具，这是一个巨大的挑战。

人类历史上，每一次重大的技术突破都遵循着相似的轨迹：从被认为不可能，到被视为困难，再到变得显而易见，最终成为日常生活的一部分。这种认知转变的背后，是科学与工程共同推动的深层逻辑。

在古代，人们认为飞行是神的特权，莱特兄弟的尝试曾被视为“杂技”；几十年后，航空成为日常。人工智能亦如此：从图灵的“机器能思考吗？”，到深蓝与 AlphaGo，再到通用大模型的涌现——不可能逐步变为显然。

在神话传统中，人造“智能”的想象从未缺席——塔罗斯与哥连都在提醒我们：人类对智能的创造有着恒久的渴望。要理解这种“从困难到显而易见”的逻辑，最清晰的路径，是回到数

学与计算的演化主线。

0.2 数学史：智能追求的发展主线

本质上，人类发展史就是一部智能简史。从最早的数概念，到算术、代数、几何、抽象代数、拓扑、群论等日益复杂的分支，数学史的主线背后，是对更强智能的追求。从最初的计数开始，人类就在不断追求计算更高效、更自动化、更智能。算盘让计算摆脱手指；对数表使复杂运算变得简单；机械计算器标志自动化的开端。每一步进展，都是在拓展智能的边界。从人脑和手算发展到一定程度，对更高效计算效率的追求必然导致计算机的发明，进而催生人工智能——这不是偶然，而是历史发展的必然逻辑。“语言是思想的边界，技术是思想的实现。”新概念的诞生拓展思维边界，抽象理论化为算法与程序，思想由此获得改变世界的力量。深度学习的历程即是典型：从人工神经元到反向传播，再到卷积网络的突破，理论与实践相互促进，推动领域飞跃。数学在其中扮演了底座角色：线性代数提供计算框架，概率论奠定学习理论，微积分使梯度优化成为可能，信息论指导数据压缩与特征提取。这些工具也经历了从“困难重重”到“显而易见”的转变——微积分自争议而至基础课程，概率论自“数学游戏”而成科学支柱。欧几里得几何的公理化、牛顿微积分的发明、非欧几何的发现、集合论的建立、哥德尔不完备定理的证明——这些“宿命时刻”如星光点亮历史，照亮了智能进化的道路。这条主线直接推动了智能层次的跃迁——从模仿到创造。这条主线也具体映射到人工智能能力的形成：优化与凸分析支撑支持向量机与深度网络的训练，图论与组合推动知识图谱与图神经网络，数值方法与稳定性理论保障大规模训练的可行性，几何与群论解释视觉中的不变性与等变性。由抽象到算法，由算法到系统，数学不断把“不可为”改写为“可为”。

0.3 智能的层次：从模仿到创造

智能不仅是计算能力，更是对复杂情境的拿捏与适度响应。“过犹不及”正是智能的外在表现之一。最初的人工智能依赖规则与逻辑推理（符号主义），在复杂现实面前力有未逮；随后机器学习兴起，依靠数据统计与模式识别，能学能泛化却仍以模仿为主；深度学习通过多层非线性映射，展现出某种“理解”，能处理语言、图像与跨模态生成，但本质上仍以统计关联为核心。从规则到学习、从统计到生成，智能的跃迁不是凭空发生；它背后站着一整套可操作的数学框架。要让这种跃迁站稳脚跟，还需要它的底座——数学。真正的挑战在于让机器具备创造性思维与抽象推理能力。这一挑战在今日呈现为“工具使用与代理化”的能力：大模型结合检索增强、函数调用、程序生成与规划执行，开始从模仿走向协作与解决问题；但要让这种能力可控、可证、可复现，仍需符号推理与神经表征的更紧密融合。我们将在后文讨论“统计—符号”混合范式如何为此提供可能路径。

0.4 本书的目标与挑战

本书试图通过数学的视角，揭示人工智能发展的内在逻辑。我们将探讨那些看似“显而易见”的技术背后的数学原理，回顾那些曾经“困难重重”的问题如何被一步步解决。人工智能高度跨学科，涉及数学、计算机科学、认知科学、神经科学、哲学等。要在有限篇幅内既保持严谨，又保证可读性，需要在深度与广度之间取得平衡。写作方式上，我们采用“公式—直觉—应用—限制—展望”的五段式展开，并以“案例—数学—实现”三层结构贯穿，确保读者既能把握数学内核，又能落到工程实践。我们的目标不仅是传授知识，更是启发思考：今天的“显而易见”，来源于昨日的“困难重重”；明天的突破，或许就蕴藏在今天看似不可能的想法之中。每一次低谷都被视为“AI 冬天”，每一次复兴都被视为“奇迹”。在这个快速演进的时代，我们既要敬畏技术，也要清醒面对挑战。数学作为人工智能的基石，将继续指引我们探索智能的奥秘，推动文明迈向更高层次。让我们带着这样的认识，开始这段探索人工智能数学基础的旅程。在这个旅程中，我们将见证“困难重重”如何转化为“显而易见”，体验数学之美如何照亮智能之路。

0.5 致读者

这本书不是传统教科书，也不是纯粹的学术专著。它更像是一场思想漫游——一次关于智能本质的哲学思辨，一段追溯数学与人工智能关系的历史探索。我们将从基础数学概念出发，逐步深入到人工智能的核心算法。每一个公式背后都有思想；每一个算法背后都有故事。我们不仅要理解“是什么”“怎么做”，更要思考“为什么”“意味着什么”。正如史蒂夫·乔布斯所言，连接点只能在回望时发生——当你一路走来，再回看这些“点”，你会发现它们自有逻辑地连成了线。愿这本书为你提供有意义的“点”，让你在未来的某刻完成自己的连接。技术会过时，思想永恒；工具会更新，智慧长存。愿这本书成为你探索智能奥秘的指南针，在“困难重重”的学习路上点亮一盏明灯，最终让那些看似高深的理论变得“显而易见”。

这本书代表了我们的尝试——让深度学习可平易近人，教会人们概念、背景和思维。

我们以读者熟悉的案例说明方法的边界与取舍，避免抽象堆砌。

目标受众

本书面向学生（本科生或研究生）、工程师和研究人员，他们希望扎实掌握深度学习的实用技术。因为我们从头开始解释每个概念，所以不需要过往的深度学习或机器学习背景。全面解释深度学习的方法需要一些数学和编程，但我们只假设读者了解一些基础知识，包括线性代数、微积分、概率和非常基础的 Python 编程。此外，在附录中，我们提供了本书所涵盖的大多数数学知识的复习。大多数时候，我们会优先考虑直觉和想法，而不是数学的严谨性。有许多很棒的书可以引导感兴趣的读者走得更远。Bela Bollobas 的《线性分析》([Bollobás, 1999] (https://zh-v2.d2l.ai/chapter_references/zreferences.html#id13)) 对线性代数和函数分析进行了深入的研究。([Wasserman, 2013] (https://zh-v2.d2l.ai/chapter_references/zreferences.html#id178)) 是一本很好的统计学指南。如果读者以前没有使用过 Python 语言，那么可以仔细阅读这个 [Python

教程]

第一部分

追问智能

引言 什么是智能？

内容提要

- 智能的多维度定义与理解
- 人工智能与生物智能的本质区别
- 从哲学、科学到工程的智能观
- 数学思维作为解码智能的钥匙
- 机器智能的进化与发展轨迹

这一章将带领你从哲学、科学和工程的角度出发，追问“什么是智能”。接下来的章节中，我们将深入探讨数学在人工智能中的具体应用，展示数学如何帮助我们创造出越来越强大的智能系统。

0.6 什么是智能与人工智能

智能 (intelligence)，如今已经成为现代生活中很常见的一个词，比如智能手机、智能家居、智能驾驶等。在不同使用场合中，智能的含义也不太一样。比如“智能手机”中的“智能”一般是指由计算机控制并具有某种智能行为的意思。这里的“计算机控制”+“智能行为”隐含了对人工智能的简单定义。简单地讲，人工智能 (Artificial Intelligence, AI) 就是让机器具有人类的智能，这也是人们长期追求的目标。然而，关于什么是“智能”并没有一个很明确的定义，但一般认为智能是知识和智力的总和，都和大脑的思维活动有关。人类大脑经过了上亿年的进化才形成了如此复杂的结构，但至今仍然没有完全了解其运作原理，以及如何产生意识、情感、记忆等功能。因此，通过“复制”一个人脑来实现人工智能在目前阶段是不切实际的。从生物学角度看，智能并非人类独有。许多动物，如海豚、鸟类、猩猩，甚至章鱼，都表现出高度的学习能力、问题解决能力以及某种形式的社交智能。然而，人类的智能以其复杂性和多样性独树一帜，在抽象和创造性上尤为突出。人类具有抽象思维，创造了诸多符号系统，并基于此进行高度复杂的推理和想象。从数学公式到哲学思辨，人类通过语言、文字和符号，建立起了庞大的知识体系和文化遗产。我们可以预见未来、反思过去，并通过科学、技术、工具改变现实世界。正是这种智能上的深度和广度，推动了人类文明的发展。智能是生命进化过程中形成的一种适应性特征。随着人工智能的发展，我们正在见证一种新型智能的崛起。20 世纪中期计算机的发明，标志着一种全新“智能”形式的诞生。这种基于二进制系统和算法逻辑的机器，能够完成许多过去被认为只有人类才能做到的任务，如图像识别、语言翻译、甚至写作创作。机器可以通过算法模拟复杂的推理过程，甚至在某些领域超越人类的能力——它们能够比人类更快地进行大规模计算、更高效地分析海量数据。这促使我们重新思考：究竟什么是“智能”？它是基于生物特性的一种特有思维能力，还是某种可以通过数学和逻辑建模来实现的计算逻辑？对于智能的定义，在不同学科中有着不同的阐释：在心理学中，智能被视为一种个体解决问题、学习和适应环境的能力；在生物学中，智能更多与生物体的行为、神经系统的复杂性相关；而在计算机科学和人工智能领域，智能则被看作是一种

能够在复杂环境中执行任务并达成特定目标的能力。从人工智能的角度看，智能可以被定义为“机器执行通常需要人类智能才能完成的任务的能力”。这些任务包括视觉识别、自然语言处理、学习、推理、决策等。然而，计算机和人类智能之间仍然存在着巨大的差异，特别是在理解、情感和创造力等方面。

机器“智能”是否真的与人类的智能一样？人类的智能伴随着进化而来，经过了数百万年的演变和适应，它不仅仅是一种计算能力，更是情感、直觉、创造力和推理能力的综合体。相对来说，人工智能的发展历史尚短，但其进化速度却让人惊叹。机器智能的本质在于其基于算法和数据的计算逻辑，而人类智能则包含了情感、意识、创造力等更为复杂和难以定义的元素。智能不仅仅是任务的完成能力，更涉及到意识、理解和体验。也许机器可以在规则内完成任务，优化目标，但它们仍然无法像人类那样拥有感知、情感或独立的创造思维。尽管如此，在这场由人类主导的智能革命中，我们依然是规则的制定者和方向的掌控者。这也是为什么，理解数学思维如何推动人工智能，能够帮助我们更好地把握未来智能发展的关键脉络。

数学在智能中的角色

为便于阅读，本节（数学在智能中的角色）承接上节（什么是智能与人工智能），先交代差异与共同点，再聚焦本节关键问题。

自古以来，数学不仅是解决问题的工具，更是智能表达与实现的基础语言。数学的力量在于它能够抽象出复杂问题的本质，提供精确的描述和推理方法。人工智能中的诸多任务，如模式识别、数据分类、预测等，背后都依赖数学模型进行量化和求解。例如，图像识别问题看似复杂，但可以被归纳为一个线性代数问题：图像被分解为像素点，每个像素点通过向量和矩阵的运算来进行处理。同样，神经网络的训练也依赖于微积分中的优化技术，以最小化损失函数，找到最优参数组合。

机器智能的进化：从计算到学习

为便于阅读，本节（机器智能的进化：从计算到学习）承接上节（数学在智能中的角色），先交代差异与共同点，再聚焦本节关键问题。

数学在推动人工智能从简单计算迈向学习与适应中起到了关键作用。最初，计算机的智能主要体现在通过预先编写的规则来执行任务，模拟人类的某些智能行为。随着大数据和深度学习技术的出现，人工智能开始从数据中“学习”，即所谓“机器学习”。这是一场范式的转变：机器智能从依赖人工预定义的规则，转变为通过模型的不断训练和优化进行自我学习。在这场范式的转变中，数学再次发挥了至关重要的作用。统计学、线性代数、概率论等数学工具，使机器能够从复杂的海量数据中提取有价值的模式和信息，并进行合理决策。这种数学思维下的机器学习能力，使得智能不仅仅是计算结果的简单输出，而是一种能够学习、适应和自我优化的动态过程。这场智能的进化，无疑奠定了人工智能发展的新高度。

数学思维：解码智能的钥匙

为便于阅读，本节（数学思维：解码智能的钥匙）承接上节（机器智能的进化：从计算到学习），先交代差异与共同点，再聚焦本节关键问题。

人工智能是模拟、延伸和扩展人类智能的理论、方法、技术及其应用系统的综合学科。AI 作为一门跨学科领域，从多个学科中汲取灵感与启发，融合了不同领域的知识与技术。数学思维不仅仅是对现有工具的应用，更代表着一种解码世界的方式。数学能够帮助我们从表象中剥离出系统的本质，理清复杂事物之间的逻辑关系。在人工智能领域，数学思维是帮助我们更好理解并发展智能的核心钥匙。人工智能的本质是一种通过数学和计算实现的智能，它利用算法、数据处理和计算资源模拟生物智能的行为。为了理解智能如何通过数学实现，我们可以将其分解为几个关键组成部分：

- **感知与反应**：任何智能体首先需要能够感知环境，并对环境做出适当的反应。这是智能的基础，也是进化中所有生物生存的关键。数学在感知数据的处理上扮演重要角色，例如通过卷积神经网络处理视觉输入。
- **学习与记忆**：智能体不仅是对环境做出反应，还包括从经验（数据）中学习，储存经验用于未来决策。
- **推理与决策**：智能体需要能够进行推理，从已有信息中得出新的结论，从而做出合理的决策。这离不开统计推断、优化等数学方法。
- **创造与适应**：更高级的智能不仅仅是反应式的，而是能够创造性地应对未知的挑战，并在复杂的环境中灵活适应。

人工智能（AI）在这些维度上逐步模仿人类的智能，但它的本质是通过数学和计算实现的。AI 的智能是通过算法、数据处理以及计算资源，以我们在生物智能中所见的行为为目标进行的复杂优化。

0.7 鹦鹉与乌鸦：何种智能？

为了更好地理解智能的不同层次，我们可以借用一个生动的比喻——鹦鹉与乌鸦。

鹦鹉式智能：模仿而不理解

为便于阅读，本节（鹦鹉式智能：模仿而不理解）承接上节（鹦鹉与乌鸦：何种智能？），先交代差异与共同点，再聚焦本节关键问题。

鹦鹉是模仿高手，能够复制人类的语言，甚至在特定情境下做出看似有意识的反应。然而，鹦鹉并不真正理解它模仿的内容，它只是重复我们的话语，而无法将这些词汇与现实世界中的人、物或情景建立联系。这种行为类似于一些现有的人工智能系统，特别是那些专注于模仿和执行特定任务的 AI 模型。这类 AI 可以通过大量数据训练，在特定任务中表现优异，但它们并不真正理解背后的语义或意图。这种“鹦鹉式智能”可以类比为**弱 AI** 或**窄 AI**。它们擅长于特定任务，但缺乏广泛的推理和创造能力。当前的许多 AI 模型，如语言生成或图像识

别，都是在这一框架下运作的——它们通过模式识别和数据驱动的方式执行任务，却没有真正的自主意识或深度理解。

乌鸦式智能：推理与创造

为便于阅读，本节（乌鸦式智能：推理与创造）承接上节（鹦鹉式智能：模仿而不理解），先交代差异与共同点，再聚焦本节关键问题。

乌鸦向来被认为是最聪明的鸟类之一。与鹦鹉不同，乌鸦不仅擅长模仿，还展现出复杂推理能力、问题解决能力以及创造性的智能。乌鸦是少数能够制造工具的鸟类，能够通过创新思维应对物理世界中的挑战。我们从小都听过各种“乌鸦喝水”的故事，而国外科学家也曾做过一系列“乌鸦喝水”的变化实验。在这些实验中，研究人员使用了各种不同形状的瓶子和不同密度的物体，观察乌鸦如何应对。令人惊讶的是，乌鸦总能在几次尝试后，快速识别并选择那些能够最快让水位上升的物体。这些实验不仅证实了乌鸦对物理规律的深刻理解，还展示了它们高度的认知能力和推理能力，以及面对复杂问题时的创造性和灵活性。另一个著名的例子是，乌鸦会将坚果放在马路上，利用来往车辆碾压坚果壳，并观察红绿灯、斑马线等信号来推理交通规则，最终推理出红绿灯与车辆停靠及行人停走之间的复杂因果关系，从而安全地过马路，吃到被碾碎的坚果肉。这是一种远远超出简单反应的行为，展示了乌鸦对物理世界和人类行为之间复杂因果链的深刻理解。乌鸦的这一行为没有经过大量数据训练或监督学习，它通过少量的尝试与观察，直接推理出解决问题的方法。乌鸦智能正是我们期望的真正的智能——完全自主的智能：观察、感知、认知、学习、推理、执行。这种智能可以比喻为**强 AI 或通用 AI**，具备自主学习、推理和适应能力，能够在不同的环境中灵活应对各种任务。更令人惊叹的是，乌鸦的智能不仅功能强大，功耗也极低。人类大脑的功耗大约在 10-25 瓦之间，而乌鸦的大脑尺寸约为人类的 1%，功耗约为 0.1-0.2 瓦。这意味着乌鸦无需复杂的能源供应系统或冷却设备，就能完成复杂的推理和问题解决任务。相比之下，现代 AI 系统需要高性能计算设备和大量能源，显示了生物智能的高效性。

智能的层次：从鹦鹉到乌鸦

为便于阅读，本节（智能的层次：从鹦鹉到乌鸦）承接上节（乌鸦式智能：推理与创造），先交代差异与共同点，再聚焦本节关键问题。

“鹦鹉”与“乌鸦”的比喻在人工智能领域有着深刻的象征意义，这个比喻不仅生动地描绘了智能的不同表现形式，更揭示了**智能发展的本质差异和层次结构**。通过对比这两种截然不同的智能模式，我们能够更清晰地理解当前人工智能的发展阶段，以及未来可能的突破方向。

- **鹦鹉式智能**：擅长特定任务，通过数据驱动的模式识别和模仿来实现，针对某些垂直应用有效，但缺乏深度理解和推理能力。
- **乌鸦式智能**：具备复杂的推理和创造性，能够自主学习、适应环境，展现出高水平的智能灵活性。

这个比喻的深层含义在于，它揭示了智能发展的两条不同路径：一条是基于大规模数据训练和模式匹配的“统计智能”，另一条是基于理解、推理和创新的“认知智能”。

鹦鹉式智能：精确模仿的艺术与局限

为便于阅读，本节（鹦鹉式智能：精确模仿的艺术与局限）承接上节（智能的层次：从鹦鹉到乌鸦），先交代差异与共同点，再聚焦本节关键问题。

鹦鹉作为自然界中最出色的模仿者之一，能够以惊人的准确度复制人类的语言、音调甚至情感表达。这种能力看似简单，实际上需要复杂的神经机制来支撑。鹦鹉通过听觉系统捕捉声音模式，经过大脑处理后，通过精确的肌肉控制重现这些声音。然而，这种模仿虽然在表面上极其逼真，但缺乏对语言内容的真正理解。当前的人工智能系统，特别是深度学习模型，在很大程度上体现了“鹦鹉式智能”的特征。以计算机视觉为例，现代的卷积神经网络 (CNN) 能够在图像识别任务中达到甚至超越人类的准确率。这些系统通过分析数百万张标注图像，学习到了从像素级特征到高级语义概念的复杂映射关系。然而，这种“理解”本质上是统计相关性的体现，而非真正的概念理解。在自然语言处理领域，大型语言模型如 GPT 系列、BERT 等，展现了令人印象深刻的语言生成和理解能力。这些模型通过训练海量文本数据，学会了语言的统计规律和上下文关系。它们能够生成语法正确、语义连贯的文本，甚至能够进行复杂的对话和推理任务。然而，深入分析会发现，这些模型的“理解”更多是基于模式匹配和统计推断，而非真正的语义理解。鹦鹉式智能的优势在于其**可扩展性和一致性**。一旦训练完成，这类系统能够以极高的效率处理大量任务，且性能稳定可预测。它们不会因为情绪、疲劳或其他主观因素而影响表现，这使得它们在许多实际应用中具有重要价值。例如，在医疗影像诊断、金融风险评估、工业质量控制等领域，鹦鹉式 AI 系统已经展现出了巨大的应用潜力。然而，鹦鹉式智能也存在明显的**局限性**。首先是**泛化能力有限**，当面对训练数据中未曾出现的情况时，这类系统往往表现不佳。其次是**缺乏创造性**，它们只能在已有模式的基础上进行组合和变换，难以产生真正原创的想法。最重要的是，这类系统**缺乏对“意义”的理解**，它们处理的是符号和模式，而非概念和意义本身。

乌鸦式智能：推理与创新的典范

为便于阅读，本节（乌鸦式智能：推理与创新的典范）承接上节（鹦鹉式智能：精确模仿的艺术与局限），先交代差异与共同点，再聚焦本节关键问题。

与鹦鹉形成鲜明对比的是乌鸦，这种被誉为“羽毛界爱因斯坦”的鸟类展现出了令人惊叹的认知能力。乌鸦的智能不仅体现在问题解决上，更体现在其**理解因果关系、进行抽象思维和创造性解决问题的能力**上。科学研究表明，乌鸦具备多项高级认知能力。在著名的“弯钩实验”中，新喀里多尼亚乌鸦贝蒂 (Betty) 能够将直铁丝弯曲成钩状工具，用来从管子中取出食物。这个行为展现了乌鸦对**工具功能的理解**和**目标导向的问题解决能力**。更令人惊讶的是，乌鸦还能够进行**多步骤推理**。在一项实验中，乌鸦需要使用短棍获得长棍，再用长棍获得食物，展现了复杂的**序列规划能力**。乌鸦的社交智能同样令人印象深刻。它们能够识别个体面

孔，记住友好或敌对的人类，并将这种记忆传递给后代。这种**社会学习**和**文化传承**能力表明，乌鸦不仅具备个体智能，还具备**集体智能**的特征。在城市环境中，乌鸦展现出了惊人的**适应性创新**。它们学会了利用汽车碾压坚果，观察交通信号灯的变化规律，甚至学会了使用人类的工具。这些行为都体现了乌鸦对**复杂环境的理解**和**创造性适应**能力。从神经科学角度看，乌鸦大脑的结构为其高级认知能力提供了基础。虽然乌鸦没有哺乳动物的新皮层，但其前脑区域高度发达，神经元密度极高。这种独特的大脑结构支撑了乌鸦的**工作记忆**、**执行控制**和**抽象思维能力**。乌鸦式智能的核心特征包括：**因果推理能力**——能够理解事件之间的因果关系；**抽象思维能力**——能够从具体情况中抽取一般规律；**创造性问题解决**——能够发明新的解决方案；**元认知能力**——对自己的认知过程有一定的认识和控制。

两种智能模式的深层对比

为便于阅读，本节（两种智能模式的深层对比）承接上节（乌鸦式智能：推理与创新的典范），先交代差异与共同点，再聚焦本节关键问题。

鹦鹉式智能和乌鸦式智能代表了两种根本不同的信息处理方式。鹦鹉式智能基于**模式匹配和统计学习**，通过大量数据训练获得特定任务的高性能表现。这种智能模式的优势在于**可扩展性、一致性和效率**，但缺乏**灵活性、创造性和真正的理解**。乌鸦式智能则基于**因果推理和概念理解**，通过相对较少的经验就能够理解世界的运作规律，并创造性地解决新问题。这种智能模式的优势在于**灵活性、创造性和深度理解**，但在**一致性和可预测性**方面可能不如鹦鹉式智能。从计算复杂度角度看，鹦鹉式智能通常需要**大量的计算资源和数据**，但一旦训练完成，推理过程相对简单。乌鸦式智能则可能在训练阶段需要较少的数据，但推理过程更加复杂，需要动态的因果推理和创造性思维。从应用角度看，鹦鹉式智能更适合**结构化、重复性的任务**，如图像识别、语音识别、机器翻译等。乌鸦式智能则更适合**开放性、创造性的任务**，如科学发现、艺术创作、复杂问题解决等。

AI 的未来：向“乌鸦式智能”的艰难迈进

为便于阅读，本节（AI 的未来：向“乌鸦式智能”的艰难迈进）承接上节（两种智能模式的深层对比），先交代差异与共同点，再聚焦本节关键问题。

当前的人工智能发展主要集中在鹦鹉式智能的完善上。深度学习、大数据、云计算等技术的发展，使得我们能够构建越来越强大的模式识别和统计学习系统。然而，要实现真正的人工智能——即具备人类水平的通用智能，我们必须向乌鸦式智能迈进。现有的 AI 系统，即使是最先进的语言模型和图像识别系统，都还处于“鹦鹉式智能”阶段。这些系统依赖于预定义规则、大量数据、大量计算资源来完成特定任务，但缺乏真正的推理和创造能力。虽然这类 AI 能够生成看似“智能”的对话，但它们的智能基于模式识别和数据驱动，与鹦鹉能模仿人类发音但不理解其语义一样。以 ChatGPT 为代表的大型语言模型，虽然在模仿人类语言并生成连贯对话方面表现出色，但它们并不具备关于“意义”的主观理解。这些模型无法感知、体验或有意识地推理，只是基于统计模式和相关性，根据输入生成最有可能符合上下文的回应。

因此，大语言模型更多是“聪明的猜测”，而非真正的理解。这与人类的理解方式存在本质差异。向乌鸦式智能迈进面临着多重**技术挑战**：**因果推理的实现**：当前的 AI 系统主要基于相关性学习，而非因果关系理解。要实现乌鸦式智能，需要开发能够理解和利用因果关系的算法和模型。**常识推理的建模**：人类和乌鸦都具备丰富的常识知识，能够在新情况下灵活运用。如何将常识知识有效地编码到 AI 系统中，仍然是一个重大挑战。**创造性思维的算法化**：创造性是乌鸦式智能的核心特征，但如何用算法实现真正的创造性思维，目前还没有明确的技术路径。**元认知能力的构建**：乌鸦式智能需要具备对自身认知过程的认识和控制能力，这要求 AI 系统具备自我反思和自我改进的能力。尽管挑战巨大，但已经有一些**有希望的研究方向**：**神经符号 AI**：结合神经网络的学习能力和符号系统的推理能力，试图实现既能学习又能推理的 AI 系统。

因果 AI：专门研究因果推理的 AI 技术，试图让机器理解事件之间的因果关系。**元学习**：研究如何让 AI 系统“学会学习”，具备快速适应新任务的能力。**认知架构**：借鉴认知科学的研究成果，构建模拟人类认知过程的 AI 系统。AI 的未来发展目标是朝着“乌鸦式的智能”迈进，不仅能够模仿，还能推理、学习、适应，在跨任务和多变的环境中展现出灵活的创造力。实现与生物智能相媲美的灵活性和创造力，正是“强 AI”或“通用 AI”所要追求的方向。这个转变不仅是技术上的挑战，也是**哲学和伦理上的挑战**。当 AI 系统真正具备了乌鸦式的智能——能够理解、推理、创造时，我们需要重新思考机器与人类的关系，以及智能的本质和意义。综上所述，人工智能是模拟、延伸和扩展人类智能的理论、方法、技术及应用系统的综合学科。从鹦鹉式智能到乌鸦式智能的发展，不仅代表了技术的进步，更代表了我们对智能本质理解的深化。而数学，作为描述和分析复杂系统的精确语言，正是我们理解和实现这一宏伟目标的核心工具。无论是当前的统计学习方法，还是未来的因果推理和创造性算法，都离不开数学的支撑和指导。

第一章 智能，生于数学

简明数学史

内容提要

- 萌芽：结绳计数与位值制
- 初等：几何、公理化与证明
- 近代：变量/函数、微积分与解析几何
- 现代：集合论、逻辑与可计算性
- 三次数学危机（无理数、微积分基础、集合论悖论）

“如果人们不相信数学很简单，那是因为他们没有意识到生活有多复杂。”

— 约翰·冯·诺伊曼

智能的历史可以追溯到人类最早期的思维活动，而数学是其中最重要的基础工具之一。本章将按“萌芽—初等—近代—现代”的时间脉络，说明数学观念如何层层抽象，并在每一步为后来的计算与智能埋下线索。对于古代人来说，数字的发明源于对数量的需求。无论是借手指计数，还是用其他物体辅助，人类通过千年的发展，逐渐完善了从简单符号到抽象数字的演变。今天看似“自然”的自然数概念，其实是无数社会实践与思想试验的沉淀。正是这些最初的表征与规则，使“可计算的表示”成为可能。人类对“智能”的探讨源远流长。远在古代希腊时期，哲学家们就已经开始追问：什么是智慧？什么是理性？我们如何通过思维去理解这个世界？从那时起，人们就梦想着创造能自主思考的机器：皮格马利翁（Pygmalion）、代达罗斯（Daedalus）、赫淮斯托斯（Hephaestus）等可被视为传说中的发明家，而加拉蒂亚（Galatea）、塔洛斯（Talos）与潘多拉（Pandora）则可被视为人造生命 [OvidMartin2004, Sparkes1996, Tandy1997]。这些想象后来在科学与工程中转化为“问题—表示—算法—数据”的框架，神话中的愿景开始落地为可检验的系统。

这些问题不仅关乎哲学，还对现代科学和技术的发展产生了深远影响。随着时间推移，这种对智能的追问从哲学的纯理论思辨，逐渐转向了科学与工程的实践领域。早期的数学家如欧几里得、阿基米德以及笛卡尔等，通过逻辑和结构化的思维方式，为计算与智能提供了基础：欧氏公理化对应模型的可验证性；笛卡尔坐标促成连续空间的特征化；阿基米德近似展示了可计算思想的雏形。数学逐渐从实用算术走向系统推理，成为连接抽象理念与工程实现的桥梁。数学的发展史可以粗分为四期：萌芽、初等、近代与现代。这些分期并非泾渭分明，而是层层叠加与概念外延的扩展。为把握演化主线，接下来我们依次回顾四个阶段的关键观念与方法，并在节末用 *examplebox* 收束为“AI 建模要点”。

1.1 萌芽时期：从计数到位值制

数学的萌芽时期从远古时代延续至公元前 5 世纪，是人类最早利用数学工具应对日常生活需求的阶段。通过手指计数、刻痕和结绳，人类逐渐形成自然数的概念与四则运算雏形，为

后续的几何与代数提供了素材和直觉。总体上，这一时期以“就事论理”的经验法则为主，但已显露出从数与形中提炼稳定规则的趋势。

1.1.1 史前数学：日常需求与实用计算

在公元前 5 世纪之前，数学并非独立学科，而是伴随人类生活的需求逐步发展的：牲畜计数、土地丈量、历法制定与贸易核算等场景催生了各种方法。古代美索不达米亚与埃及的泥板和石刻清楚记录了这些活动，展现了数学与生活实践的紧密联系。“数学”活动最古老的痕迹之一是在美索不达米亚发现的一块泥板（约公元前 2500 年）。泥板记录了这样一个计算：如果谷仓里有 1 152 000 份粮食，每个人分得 7 份，一共可以分给多少人？结果是 164 571 人，即用 1 152 000 除以 7 的结果。这说明在算术“成书”之前，美索不达米亚的会计师们已经能熟练地进行除法运算。甚至，书写完全有可能就是为了记账才发明的一若真如此，数字就比字母发明得还要早了。古埃及人发展了基础的几何公式，在建造金字塔时已经能够运用高度和角度的几何关系；美索不达米亚的会计师不仅掌握了加法和乘法，还能解简单的二次方程，用于管理粮食和资源。巴比伦人对天体运动的观察与历法推算也推动了数学的发展。为了精确预测日食、月相以及制定历法，天文学家需要处理大量的时间和角度计算。这些天文观测不仅进一步提升了测量和计算的要求，也推动了几何与代数的发展。总体而言，这一阶段的数学活动是解决问题的经验性工具，尚未形成系统理论，但奠定了“数”与“形”的概念基础。这些先于理论的操作共同催化出对“可复用规则”的渴望，从而引向更稳健的记数手段。问题也由一次性的分配与测度，转向对“如何表示与传递数量”的一般方法。

1.1.2 手指：长身上的计数器

相信你在许多影视剧中都见过这样的一幕：一位算命先生，掐指一算，口中念念有词。这种“掐指一算”看似神秘，实则源自于古老而简便的计数方法。在远古时代，用手指计数是再自



图 1.1: “掐指一算”

然不过的选择，手指是人类最早的“随身计算器”。它帮助人们轻松应对日常计数需求，如记录牲畜数量、物品交换量，推算季节与节气，安排农耕节日等活动。由此，一个从身体到符号的过渡被打开：手指结构为进制与位权提供了直觉。

手指的构造与多种进制系统有着天然的联系。稍加思考，我们就能找到十进制、二十进制、十二进制等计数法与手指构造的对应关系。十进制 (Decimal System) 是最常见、最自然的计数方式，与双手十指自然对应。十进制系统在全球的广泛使用，源于人类天生拥有十根手指的解剖学特征。正如爱德华·卢卡斯所言，这是“自然界的一个偶然事件”。十二进制 (Duodecimal System) 则适用于更复杂的计数需求。除大拇指外，四根手指各有三段指节。一只手的拇指轮流指向其余四根手指的各段指节，可以计数 1 到 12。这种方式在古代美索不达米亚和古埃及广泛应用，至今仍用于时间和角度的计数系统中，如 12 小时制、一年 12 个月等。六十进制 (Sexagesimal System) 则更加复杂。它结合双手来计数。一只手用于计 12（与十二进制相同），另一只手的五根手指则记录 12 的倍数，五轮 12 等于 60。六十进制在古巴比伦时期被发明，沿用至今，广泛用于时间和角度的计量，例如 1 小时 = 60 分钟，1 分钟 = 60 秒，以及 360 度的圆周划分。二十进制 (Vigesimal System) 也是一种古老的计数法。为了更大范围的计算，一些古代民族不仅用手指，还加上脚趾来计数。例如，中美洲的玛雅文明和阿兹特克文明，以及现今的西非和欧洲部分地区，采用了以 20 为基数的计数法。二十进制的痕迹在许多语言中依然存在。现代英语中的“score”表示 20 的用法便保留了这一印记。例如，莎士比亚在《圣经》英译本中使用“three score and ten”（20 的 3 倍加 10，表示 70 岁）来指代古稀之年；林肯在《葛底斯堡演讲》中使用“four score and seven years ago”（20 的 4 倍加 7，表示 87）来表达 87 年前的历史。一百年后，马丁·路德·金在其著名的演讲《I have a dream》中说“Five score years ago”（20 的 5 倍，即 100 年前）。英国曾有一个 12 与 20 进制组合的货币系统：1 英镑 = 20 先令，1 先令 = 12 便士。1971 年，英国引入了十进制货币系统，1 英镑 = 100 便士，先令在 1990 年后彻底退出流通。在法语、丹麦语和巴斯克语中也能找到二十进制的痕迹。例如，法语中的数词从 quatre-vingts(80)、quatre-vingts-dix(90)，到 quatre-vingts-dix-neuf(99)。同时，这些语言中还混杂了六十进制的痕迹，如数词 soixante(60) 到 soixante-dix-neuf(79)。手指不仅是身体的一部分，还是人类最早的天然计算工具。它帮助我们理解和组织世界，并为现代数学中的位值制和进制系统提供了最初的灵感。直到今天，手指仍是许多人在进行简单计算时的本能工具。

1.1.3 荒岛上的自然教育，人类数字文明的缩影

设想你在一次航海探险中遭遇风暴，不幸漂流到了一个无人岛。没有手表和手机，也没有任何现代工具。你很快面临一个问题——如何计算时间？如何记录每天收集的食物？在这种完全回归自然的环境中，你必须依靠最原始的方式来解决这些问题。这个经历，其实正是早期人类如何发明和发展数字的缩影。

手指计数：最简单的工具

为便于阅读，本节（手指计数：最简单的工具）承接上节（荒岛上的自然教育，人类数字文明的缩影），先交代差异与共同点，再聚焦本节关键问题。

你的第一反应可能是用手指来计数。十根手指天生就是人类最便捷的“计算器”，帮助你快速记录简单的数量——比如你捕到的鱼或采摘的果实。手指计数是直观而方便，几乎无需思考就能完成。但很快，你意识到这种方法有明显的局限性：当数字变大，或需要记录更复杂的信息时，手指显然不够用了。更重要的是，手指计数无法保存这些数据，一旦忘记，你的记录便随之消失。（抽象：易得的“工作记忆”vs. 可复查的“外部记忆”。）

刻痕计数：更长久的记录方式

为便于阅读，本节（刻痕计数：更长久的记录方式）承接上节（手指计数：最简单的工具），先交代差异与共同点，再聚焦本节关键问题。

为了更长久地保存记录，你开始在岛上的树木或石头上刻下痕迹。每道刻痕代表一天的过去，或你每天捕获的鱼或采摘的果实数量……。刻痕计数法让你能够更长久地保存这些信息。每天的刻痕不断累积，形成一个记录时间和活动的系统。这种方法虽然简单，却比手指计数更加持久、可靠，也能应对稍微复杂的需求，比如记录日期或特定的事件。然而，随着时间的推移，刻痕的数量增加，管理这些信息变得越来越困难。你开始需要一种更便捷、更灵活的方式来记录和传递这些信息。（AI 启发：从短上下文到外部存储/检索—RAG 的原型。）

结绳记数：更高级的记数法

为便于阅读，本节（结绳记数：更高级的记数法）承接上节（刻痕计数：更长久的记录方式），先交代差异与共同点，再聚焦本节关键问题。

幸运的是，你发现了一根绳子，这给了你一个新的选择——结绳记数法。通过在绳子上打不同的结，你可以标记不同的天数、事件或物品的数量。不同形状和位置的结可以表示不同的信息。相比于刻痕，结绳计数法更便捷灵活，更易于携带和传递，成为你在荒岛生活中的高级计数工具。（抽象：符号化与压缩编码；启发：离散化/量化。）你在荒岛上的探索，正是人类数字文明进化的缩影。无论是刻痕还是结绳，这些方法都体现了人类从最初的物理标记到复杂符号系统的演化路径。通过这些自然的计数方法，你逐步掌握了记录、比较和计算的基本概念。这不仅是你个人的“自然教育”，也是数字发明的最初动力——让生活更加便捷和高效。（一句话抽象：需求—表示—外部化—标准化的演化链。）从需求出发，人的记数方式沿着“即时—留痕—编码”的路径外化与标准化。下文据此上升到一般而稳定的表示——位值制。

1.1.4 数字的位值系统

在人类早期文明中，简单的手指计数和结绳、刻痕等方法已广泛用于记录数字。然而，随着社会的发展，这些物理标记在处理更复杂和更大范围的数字时，显得低效而繁琐。于是，更

加抽象的符号系统应运而生，催生了**位值制**这一革命性的数字表示方式，使得复杂的数值能够通过简单的符号位置来表示和计算。早期的计数系统大多采用**加法规则**。例如，在古代巴比伦的黏土板上，数字是通过符号的重复来表示的：数字“1”用一个符号表示，数字“2”用两个相同符号表示，数字“3”则用三个相同符号。这种计数方式在中文数字、罗马数字、古印度的婆罗门数字和笈多王朝使用的数字，以及大洋彼岸的玛雅数字中都很常见。大多数古代文明在表示前三个数字时，都采取了同样的方式：重复表示“1”的符号一次、两次、三次。古代的中文“一、二、三”、钟表盘上的罗马数字“I、II、III”、阿拉伯数字也体现了同样的重复方式。可是，从数字4开始，所有的文明就都放弃了这种重复。为什么呢？研究发现，人类的数感对少于3的数字有天然的反应，但超出这个范围后，单纯的重复符号便变得困难。对一些出生几个月的婴孩进行试验，结果证实人类与生俱来的数感的容量不超过三。试想一下，数字13若用重复排列“1”的符号13次来表示，无论是对书写还是阅读，都会很低效——怎么能一眼就看出它是13，而不是12或者14呢？类似的问题在罗马数字系统中也很常见。罗马数字系统中，V表示5，X表示10，L表示50，C表示100。此外，向右添加符号为加，如，VI表示数字6，XI表示数字11；相反地，向左添加一个更小数值的符号为减，如IV表示4，IX表示9。尽管使用了加减法规则的计数系统十分直观，但在表示大数时显得十分低效。例如1888年写成罗马数字就是MDCCCLXXXVIII，甚至无法有效表示类似100万那样的数字——除非把1000个M排成一排！

早期加法型记数（如罗马数字）在表达大数与运算效率上受限。位值制用“位置决定权重”的方式，以极少符号表达极大数量，长度近似 $\log x$ 。这不仅简化书写与计算，也带来压缩与编码的思想（最短码长度 $\approx -\log p$ ）。位值制因此成为现代信息处理与计算架构的共通底座。随着人类文明的发展，许多地区开始引入更为先进的计数法，其中最具革命性的是**位值制计数法**。古巴比伦、中国，尤其是印度和阿拉伯，率先采用了这种创新的位值制计数法。这种计数法的关键在于符号的位置决定了其数值。例如，数字“12”和“21”虽然使用相同的符号，但因为符号位置不同，代表的数值也不同。位值计数法具有出众的简洁性。通过有限的符号，可以表达任意大的数值。例如， 10^{100} 这个超出了物理限制的天文数字，在位值制下仅需101个符号即可表示——一个“1”后面跟着100个“0”。位值计数法用极短的符号链表示极其巨大的数值。位值制不仅提高了数字的表示效率，还反映了数字的**对数增长**特性，即随着数字的增大，所需的符号长度呈对数增长，符号数量远远少于数字本身的大小。例如，表达 10^{100} 所需的符号长度约为 $\log_{10}(10^{100}) = 100$ 。一般地，表达任意整数 x 所需的符号数量接近 $\log_{10} x$ 。

因此，从某种意义上，位值计数系统是最简洁、甚至最优的。（信息论视角：最短码长度 $\approx -\log p$ 。）位值制的引入是人类计数方法的一次重大飞跃。从简单的重复符号到高度抽象的符号系统，位值制极大简化了计算过程，推动了计算和数学的发展，奠定了现代数字系统的基础。（一句话抽象：位权 $\Rightarrow$ 表达与计算的指数增益；AI启发：词表/位宽/量化。）

1.1.5 位值制与对数增长在现代计算机中的应用

在计算机科学中，在已经排好序的数组中查找某个元素的时间也是数组长度的对数。即使数组规模如同宇宙般庞大，经过几百次迭代后，便能够找到所需的元素——唯一的限制就是光速有限性所带来的信息传播和处理速度的物理极限。互联网的 URL 是位值制与对数增长在数据处理和网络寻址中的一个典型应用。一个简短的 URL(Uniform Resource Locator, URL, 统一资源定位符) 迅速定位到海量信息。想象一下，你希望在网上搜索某项信息。尽管互联网上的数据量可能以 EB(10^{18} 字节) 甚至 ZB(10^{21} 字节) 计量，只要拥有对应信息的 URL，你几乎瞬间就能找到目标信息。这正是对数增长的强大之处：即使面对巨大规模的数据集，只需少量步骤就能完成检索。此外，URL 的简短程度令人惊叹，其长度是整个网络大小的对数！这意味着我们可以轻松将 URL 存储在内存中，而 URL 指向的信息量却远远超出了其自身长度。这种简洁有效的特性，展示了对数增长在计算与信息处理中的关键作用。IPv4 和 IPv6 的设计正是对数增长在实际应用中的另一个实例。IPv4 地址使用 **32 位** 二进制数来表示，每个 IP 地址可以被写成 4 个以点分隔的十进制数（如 192.168.1.1），每部分对应一个 8 位的二进制数。

32 位的二进制能够生成 2^{32} ，即约 **43 亿** 个唯一 IP 地址。这在 IPv4 设计之初被认为是足够庞大的数目。然而，随着互联网的迅速扩展，特别是物联网(IoT)设备和移动设备的广泛普及后，IPv4 的 43 亿个地址空间将很快耗尽。为了解决这一问题，IPv6 应运而生。IPv6 使用 **128 位** 二进制数表示，由八组以冒号分隔的 16 进制数构成（如 2001:0db8:85a3:0000:0000:8a2e:0370:7334）。这种 128 位的二进制表示能生成 2^{128} ，即约 3.4×10^{38} 个唯一 IP 地址，相当于为地球上的每粒沙子分配多个 IP 地址。这一数量级足以应对未来几十年甚至几百年内所有联网设备的需求。IPv4 和 IPv6 的位数与它们能够支持的地址数量之间的巨大落差，展示了对数增长的强大力量。IPv4 仅用 32 位符号就能表示 43 亿个地址，而 IPv6 仅用 128 位符号就能支持 3.4×10^{38} 个地址。随着位数的增加，IP 地址系统实现了指数级的扩展，同时保持了地址表示的简洁性。（AI 去向：索引结构/向量检索/稀疏路由—见 8.2、8.5。）下一节我们从“表示的效率”转向“表示的抽象度”。

1.1.6 数字：从具象到抽象

几千年来，人类都在探索如何建立数与物的对应关系，现代数字系统的定义正是基于这种探索的延续。一旦人类掌握了计数，数字系统的应用就变得“自然而然”了。或许正因如此，我们才称自然数为“自然”数 (natural number)。然而，实际上，数字的概念一点也不“自然”。无论是在早期人类在骨头及木棍上有序地刻下划痕，还是结绳或是在容器中放置物品无论是结绳，这些早期方法已代表了相当抽象的符号系统，其中抽象的数学对象与自然中的实际物体之间的分离，同一个数字既可以表示动物的数量，也可以记录借贷量或一个月的天数。自此之后，数学研究的对象不再是必须与实际物体相关的几何图形和数字。研究对象性质的变化，最终引发了解决数学问题的方法的革命。（AI 启发：可迁移表征与多任务共享—抽象层提高，泛化/可解释间的取舍加剧。）抽象提高了可迁移性，也带来语义漂移与解释性的张力。

1.2 初等数学时期：从计算到推理

公元前 5 世纪开始，数学进入了初等数学时期。这个时期的数学探索更加系统和深入，尤其是在欧几里得几何和代数方程的研究中，数与形的关系得到了更加严谨的探讨，算术、几何、代数和三角等数学分支逐渐成形。公元前 5 世纪起，欧几里得几何与代数方程的研究使数与形的关系走向严密，算术、几何、代数与三角逐渐定型。核心转变在于：从求值走向论证，从技巧走向体系。数学作为一种学科，经历了漫长的演化过程。起初，数学的诞生是为了解决生活中的实际问题——如何计数、测量、交易、管理时间等。在公元前 5 世纪之前，埃及和美索不达米亚等古老文明的数学发展更多地依赖于经验和实际操作，主要集中于如何进行计算，服务于土地测量、建筑工程、会计管理以及天文学等日常需求。这种实用性的数学（包括算术和几何）在数学史上称为“史前”数学，因为它并不涉及抽象推理，而是围绕日常需求展开。古代文明的数学多服务于实践；希腊学者把关注点转向命题的可证性。等腰直角三角形三边皆为整数的设想，化为方程 $2x^2 = y^2$ 。穷举无效，转而假设互素并分类奇偶，导出矛盾，因而无解。由此，数学的目标从“得到一个数”变为“给出必然理由”。这一转折打开了更广阔的对象与方法空间。因此，教科书中的数学史往往是从公元前 5 世纪的希腊开始讲起的，因为希腊数学家让数学从“如何计算”进化到了“为什么如此”。在公元前 5 世纪的希腊，毕达哥拉斯学派提出了这样一个问题：找出一个等腰直角三角形，使其三边的长度都是自然数。

假设等腰直角三角形的直角边长度为 x ，斜边长度为 y 。根据毕达哥拉斯定理（勾股定理）， y^2 就等于 $x^2 + x^2$ 。因此，这个问题最终归结为：找出两个自然数 x 和 y ，使得

$$2x^2 = y^2. \quad (1.1)$$

表 1.1: 4 以内 x 与 y 的所有可能性

x	y	$2x^2$	y^2
1	1	2	1
1	2	2	4
1	3	2	9
1	4	2	16
2	1	8	1
2	2	8	4
2	3	8	9
2	4	8	16
3	1	18	1
3	2	18	4
3	3	18	9
3	4	18	16
4	1	32	1
4	2	32	4
4	3	32	9
4	4	32	16

让我们来试试 4 以内所有 x 和 y 的可能性吧（见表 ??）。在上述所有情况里， $2x^2$ 都不等

于 y^2 。我们当然还可以在更大的数字范围里继续寻找。事实上，毕达哥拉斯学派很可能寻找了很久，却没能找到解。后来，他们终于相信这个解不存在。他们是怎么说服自己这个解不存在的呢？显然不是穷尽所有的数对，因为这样的数对有无穷多个（且为可数集）。你就算试到 1000 甚至 100 万，证实没有数对能满足条件，可你还是没法保证在更大的数字里面不可能有解……既然不能用穷举法，那就试试逻辑推理。不妨假设 x, y 两个数互素。为什么呢？因为若 x, y 至少有一个是奇数时，两者自然互素；反之，若 x, y 都是偶数，则总是可以通过消去公因子来保证 x, y 至少有一个是奇数。据此，我们把数对 x, y 分成以下 3 种情况：

1. x, y 都是奇数；
2. x 是偶数， y 是奇数；
3. x 是奇数， y 是偶数；

我们先从第一种情况开始： x 和 y 都是奇数。此时 (??) 的解不存在。因为 y 是奇数，则 y^2 也是奇数。它不可能等于 $2x^2$ ，因为后者必然是偶数。这一论证也同样适用于第二种情况，即 x 是偶数而 y 是奇数。第三种情况即 x 是奇数而 y 是偶数。但在这种情况下，若 $2x^2 = y^2$ 成立，则这个数的一半 x^2 是奇数，同时 y^2 的一半却是偶数——这两个数不可能相等。综上所述，找不到两个自然数 x, y 满足 (??)。为了证明这个问题没有解，毕达哥拉斯学派采用了逻辑推理而不是计算来实现。他们通过分类讨论和推理，证明了无论 x 和 y 是奇数还是偶数，方程 $2x^2 = y^2$ 都不可能有整数解。这一问题的解决在数学史的发展上具有划时代的意义。它的革命性不仅在于其对未来数学的巨大影响，还在于其抽象性及解决的方法。相比于美索不达米亚泥板上铭刻的诸如“将 1152000 份粮食除以 7 份”的具体问题，毕达哥拉斯学派的问题更为抽象：美索不达米亚的会计师关注粮食的数量，而毕达哥拉斯学派的问题仅涉及数字本身。同样，他们处理的是抽象的几何形式，而非具体的三角形田地。从从粮食的份数到数字，从实际的土地测量转向几何学的抽象思考，迈向抽象这一步的意义不可小觑。田地的面积无非几平方千米，而抽象的几何对象则可以轻松超越这一尺度，进入无限的范围。古希腊数学家不再满足于计算结果，而是开始重视推导过程。他们通过严谨的逻辑推理，逐步构建了数学证明和公理系统。这一从计算到推理的转变，被视为数学的真正诞生，使得数学从日常的计算工具演变为以推理和抽象为核心、具有高度抽象和严谨逻辑的独立学科。一个平方数不可能是另外一个平方数的两倍——这个由毕达哥拉斯学派在 2500 多年前得到的结果，迄今在数学上仍占有重要地位。它证明了，一个短边为 1 米的等腰直角三角形中，斜边的长度为 $\sqrt{2}$ ，约 1.414，但这个数无法通过两个自然数 y 和 x 的比值得到。这揭示了，某些几何关系无法通过自然数的四则运算（加、减、乘、除）运算得到，暗示了更广泛的数系——实数的存在。这一发现让数学进入了一个全新的领域——从实际问题中抽象出纯粹的数学对象并通过推理得出结论。虽然这一发现具有革命性，但毕达哥拉斯学派并没有进一步发展出实数的概念。对他们而言，这一结论动摇了他们对自然数“万物皆数”的信仰，甚至引发了哲学上的困惑与争议。然而，几百年后，这一发现启发了数学家们发展出实数理论，为数论和分析学的诞生奠定了基础。（AI 启发：超出“有理”可表达性的现象推动了更大状态空间的建模——如连续表征与实值网络。）

1.2.1 毕达哥拉斯：数与宇宙的关系

数学的哲学化起点源自古希腊的毕达哥拉斯学派。毕达哥拉斯 (Pythagoras) 及其追随者提出了“万物皆数”的思想，认为宇宙的本质可以通过数来理解。这一理念激励了希腊数学家们将数学视作解读自然现象的工具。毕达哥拉斯定理（即勾股定理）是这一学派最著名的成就之一，标志着数学开始从经验性计算向理性推理的迈进，开启了数与形结合几何学研究。（AI 去向：几何先验 → 物理世界建模；见 9.2 几何深度学习。）

1.2.2 欧几里得：公理化体系的奠基人

如果说毕达哥拉斯奠定了数学的哲学基础，那么欧几里得 (Euclid) 则为数学建立了逻辑结构。他的《几何原本》(《Elements》) 是古代数学史上最具影响力的著作之一，也是现代数学公理化体系的奠基之作。欧几里得通过公理和定理的形式，系统地将几何学结构化，强调了证明的重要性。欧几里得展示了如何从少数的基础假设（公理）出发，通过严密的逻辑推导，推导出一系列复杂的几何命题。这种公理化方法对后世的数学家产生了深远影响，甚至成为了现代数学研究的重要框架。欧几里得的几何不仅仅是关于形状和空间的学问，更标志着数学作为一种基于推理的演绎学科的正式诞生。（AI 启发：明确假设集 $\Rightarrow$ 可复现实验与安全边界；见 10.1 可验证与可信 AI。）

1.2.3 阿基米德：数学与物理的桥梁

阿基米德 (Archimedes) 是古希腊另一位杰出的数学家，他不仅在数学领域取得了重大的突破，还在物理学、机械学等领域做出了重要贡献。阿基米德将几何与物理现象紧密结合，开创了利用数学模型解决物理问题的先例。他在几何领域的突破，如圆周率的近似计算、浮力定律及流体静力学基本定理的证明，展示了数学在物理学应用中的强大力量。阿基米德的工作不仅扩展了数学的应用范围，还展现了数学在解释自然现象中的力量。（AI 去向：连续量的近似—数值稳定/误差控制；见 8.2 优化与数值性。）

1.2.4 亚里士多德：逻辑与推理

亚里士多德 (Aristotle) 是古希腊著名的哲学家。亚里士多德认为，数学不应只停留在经验性的层面，而应该通过演绎推理去发现隐藏在事物背后的逻辑关系。他提出的三段论和归纳法，帮助数学家们更好地理解和构建逻辑推理过程。这种逻辑推理体系推动了古希腊数学的进一步发展，数学家们开始通过推理和证明而非直观的经验得出结论。（AI 启发：符号推理与统计学习的互补；见 11.3 神经—符号融合。）算术与几何这两大分支的创立，标志着数学开始作为一门理论化学科的诞生，逻辑推理和证明逐渐成为数学的核心方法。与此同时，代数学在这一时期取得了重要进展，自然数、有理数和无理数的运算逐渐成为代数的核心主题。除此之外，这一阶段的代数学还开始引入虚数，推动了数的概念向更广的领域扩展。

1.2.5 古希腊数学的遗产

通过抽象思维和逻辑推理，建立了数学作为一种独立学科的基础。古希腊数学家们的贡献不仅仅是发现了一些几何定理和代数法则，更重要的是他们发展了数学思维的方式——通过演绎、证明、公理化等方法，将数学从经验性工具转变为一种严谨的科学。这一传统不仅奠定了现代数学的基础，还推动了之后数千年数学和科学的进步。（一句话抽象：证据链 $\Rightarrow$ 可靠知识；AI 去向：可解释与可证明的系统。）中世纪的阿拉伯数学家翻译了希腊经典，并引入了印度的十进制数和“零”的概念。阿尔·花刺子密（al-Khwarizmi）等数学家对代数学的发展做出了巨大贡献，他的著作奠定了代数理论奠定了坚实的基础。（“algorithm”一词即源于其姓名的拉丁化形式。）中国的《九章算术》和印度的基础代数成就，也为初等数学的发展增添了重要内容。中国古代数学强调实用性，广泛应用于农业、建筑和税务管理；而印度数学则对数论和代数的早期发展做出了杰出贡献。这些不同文化的数学成果，最终共同塑造了初等数学的广阔基础。这一时期涵盖了公元前 5 世纪到 17 世纪前的数学发展。数学不再仅限于解决实际问题，而是向抽象领域迈进。古希腊数学通过几何学和数论的研究，构建了数学推理和证明的数学体系。这一阶段的数学成就不仅构成了今天中学数学的主要内容，还对现代数学的基本理论结构产生了深远的影响。

1.3 近代数学时期：变量、函数与微积分

从 17 世纪开始，随着资本主义的崛起和工业革命的推动，数学进入了一个全新的阶段——变量数学时期。这一时期的数学从研究静态图形和数量问题逐步转向关注变化与运动问题，变量和函数的引入是更好描述运动和变化的关键。牛顿和莱布尼茨分别独立发明了微积分，标志着数学史上一个重要的转折点。微积分提供了一套精确描述连续变化与瞬时变化的数学工具，成为解决诸如速度、加速度、曲线下面积等问题的强大工具。牛顿通过微积分为物理学中的万有引力定律奠定了理论基础，而莱布尼茨则将微积分符号化并推广到欧洲，极大地推动了数学的发展。微积分不仅让数学得以分析瞬时变化，还为后续的数学分析开创了新的方向。解析几何的诞生是这一时期的另一个重大突破。笛卡尔通过引入坐标系，将几何与代数结合起来，使得复杂的几何图形、物理的运动及其空间关系可以通过代数方程来精确描述。借助于笛卡尔坐标系，数学家们能够用代数表达曲线、直线的关系，从而简化了几何中的很多问题。解析几何不仅为物理学中的动力学提供了有力工具，还为现代数学的发展铺平了道路。近代数学时期的另一个重要分支是概率论。概率论起初由费马和帕斯卡为了研究赌博中的随机事件而诞生，但很快超越了这一领域，成为研究不确定性和随机现象的重要工具。费马还对数论做出了重要贡献，提出了许多关于素数与数之间关系的问题，激发了后世数学家对数论的广泛研究。欧拉、拉格朗日等人进一步推动了数论的发展，探讨了代数结构和数论的基本问题。与此同时，分析学作为微积分的延续，成为这一时期的核心数学分支。通过函数和极限概念的引入，数学家们能够处理无穷小量和变化中的量，开创了分析学的时代。数学分析不仅成为这一时期的核心理论，不仅解决了古希腊时期悬而未决的几何问题，还广泛渗透

进了物理学、天文学和工程学，极大地推动了科学的进步。（AI 启发：梯度/最优化—现代深度学习的计算内核。）

总的来说，近代数学时期不仅为数学的进一步抽象化奠定了基础，还通过微积分、解析几何和概率论的兴起，彻底改变了人类对世界的理解方式。数学在这一时期的变革性进展，成为了现代数学发展的根基，也为科学技术的进一步进步提供了强有力的工具。（一句话抽象：变化、空间与不确定性三件套；AI 去向：优化、几何表示、概率图模型。）

1.4 现代数学时期：抽象、结构与计算

从 19 世纪中叶起，数学进入了一个高度抽象化和系统化的阶段，被称为现代数学时期。这个时期的数学阶段不仅延续了之前的分析学发展，还引入了全新的几何和代数理论。代数、几何和分析学等传统学科在此期间发生了质的飞跃，数学的研究对象和方法也逐渐向高度抽象化发展，推动了数学成为一门更加系统和广泛应用的科学。现代数学的一个显著特征是对数学基础的重新审视与抽象概念的扩展。罗巴切夫斯基和黎曼的非欧几何，颠覆了传统欧几里得平面几何观，提出了球面和鞍面等不同几何结构，揭示了更为广泛的空间和结构关系。拓扑学作为新的分支，也在 19 世纪逐渐兴起，打破了传统几何的局限，成为数学研究连续性、边界与形态的重要工具。同时，现代数学的发展离不开对无穷概念和数学基础的深入研究。康托尔的集合论，为处理无穷大的数学研究提供了严格的工具，将无穷大和无穷小的概念精确化，成为现代数学和数理逻辑的基础理论之一。集合论的发展不仅使数学在抽象层面上得以统一，还激发了对数学自身基础的研究。集合论和数理逻辑的兴起为数学理论提供了全新的理论框架，希尔伯特、哥德尔等数学家在此领域的工作，使得数学的公理化体系进一步得到发展与完善。20 世纪，统计学、拓扑学、泛函分析等新分支陆续创立，使数学更多元化、更富实用性。数学在众多领域的应用范围得到了前所未有的扩展，数学在物理学、工程学、经济学以及其他自然科学和社会科学等各个学科中的作用变得愈加重要。（AI 去向：统计学习理论、泛函空间中的核方法与 RKHS。）现代数学时期的另一个重要推动力是计算机的发明和信息技术的迅猛发展。20 世纪中期，图灵、冯·诺依曼等科学家为计算机科学奠定了理论基础，计算理论逐渐成为数学的重要分支。计算机的强大计算能力使得复杂的数学问题得以快速求解，同时也极大地推动了数理逻辑和算法理论的发展，数学的潜能和边界更进一步发掘。（一句话抽象：可计算/复杂度的边界塑造了“可做之事”。）随着数学的抽象化与理论化趋势日益加深，数学在深入自然科学领域的同时，还广泛渗透到信息科学、工程学、经济学、社会科学等多个领域，推动了现代社会各个层面的进步。现代数学逐渐成为理解自然世界、构建理论模型和推动技术创新的核心力量。

总之，现代数学时期是数学高度抽象化、系统化的阶段。数学基础理论的深入研究与应用范围的广泛扩展，使得数学不仅在理论层面得到了重大发展，也成为了推动科学和技术进步的关键推动力。通过集合论、非欧几何、拓扑学和计算理论的引入与发展，数学在 20 世纪的进步为现代社会的各个领域带来了不可忽视的影响。（AI 去向：可计算性/复杂度 → 算法选择与系统设计的第一性约束。）

1.5 三次数学危机：从无理数到不完备

数学发展的历程中，曾三次面临深刻的理论挑战，这些危机不仅推动了数学的变革，也影响了其作为一门科学的根本性质。这三次数学危机分别发生在古希腊时期、17 世纪微积分的创立期以及 19 世纪末的集合论时代。每一次危机都促使数学家重新审视基本概念，重整方法论，进而把数学从工具性实践推高到更为抽象而严密的层次。（一句话抽象：边界被突破—再公理化—再出发。）

1.5.1 第一次危机：无理数的发现与算术信仰的破灭

第一次数学危机发生在古希腊，起因是**无理数**的发现。毕达哥拉斯学派主张“万物皆数”，但这里的“数”是指整数或整数之比，即有理数。他们认为世界的规律可以通过数与比例来完美解释，这一思想赋予了数学一种神秘的普适性。然而，对边长为 1 的等腰直角三角形的考察显示斜边为 $\sqrt{2}$ ，不能表示为任意两个整数之比。这类“不可公度性”的发现，彻底动摇了毕达哥拉斯学派“万物皆数”（即万物皆有理数之比）的信念，引发了数学史上的第一次重大危机。无理数的存在打破了古希腊数学家对数字和比例的简化认识，表明数的体系并不完备，迫使数学家们不得不重新审视数的定义。（AI 去向：连续表征与几何先验一见 9 章。）希腊人的直接应对不是立刻“发明”新数，而是**退回几何**：以线段、面积等几何量为操作对象，通过相似与比例关系间接处理“不可公度”量，暂不强求把一切写成有理数。欧几里得在《几何原本》系统化了这一做法，特别是第十卷区分可公度与不可公度线段，并借助欧多克索斯的**比与比例理论**回避了对“比值算术化”的执念—即便两段线没有公共度量单位，也能在几何中讨论“比”的相等与传递性；黄金分割便是典型例子。与此并行，连分数与欧几里得算法揭示了不可公度的结构性：当比例化归过程无法在有限步终止时，意味着超出有理数的尺度。经过两个多千年的几何化安置，“无理量”主要以几何客体存在。17 世纪以后，随着代数和解析几何的发展，人们逐步承认仅靠有理数不足以支撑连续量的计算，无理量从几何比率过渡为算术中的合法公民，在方程、函数与近似计算中获得常规地位。19 世纪，柯西以极限刻画收敛与导数，魏尔斯特拉斯建立 ε - δ 语言，德德金用“截断”定义实数，柯西—康托尔以柯西列完备化给出等价刻画： $\mathbb{R}$ 被严格定义为完备有序域，有理与“无理”由此统一。此番危机在对象层面扩张了“数”的外延，在方法层面巩固了“由证明而非经验近似给出理由”的标准。（AI 启发：当离散词表不足以承载现象，引入连续表征与可微优化；见第 9 与第 8 章。）无理数危机的解决是数学史上的一大转折点，经过了数代数数学家的努力。虽然无理数的发现带来了危机，但最终促使了数学向更高的层次发展。几何学的抽象化、代数学的进步以及实数体系的确立，标志着数学摆脱了毕达哥拉斯学派的局限，迈入了新的发展阶段。这场危机不仅将数的概念从有理数扩展到无理数，还为后来的数学分析和数论奠定了坚实的基础。

1.5.2 第二次数学危机：微积分基础的争议与解析

第二次数学危机出现在 17 世纪，伴随着**微积分**的诞生。微积分由牛顿和莱布尼茨独立提出，是解决变化率和曲线面积等问题的强大工具。虽然在解决实际问题上取得了巨大的成功，但微积分的早期发展在理论上存在许多模糊之处，尤其是关于**无穷小量**（infinitesimals）的使用，引发了深刻的数学危机。微积分的核心思想之一是使用无穷小量来定义变化率和累积变化。比如，导数被定义为“函数的瞬时变化率”，而积分则表示曲线下面积的无限累加。然而，当时的数学家并没有严格定义什么是无穷小量，它既不能被归为零，也不是某个确定的数，以致于在某些推导过程中无穷小量一会等于 0，一会又不等于 0。这种模糊性使得微积分缺乏严密的逻辑基础，许多数学家无法接受微积分作为严格数学工具的合理性。**贝克莱**等学者猛烈批评了微积分理论，指出其在逻辑上存在的缺陷，“微积分似乎依赖于‘幽灵般的消失量’”。解决这一危机的关键在于**极限**（limit）概念的引入。17 世纪末至 18 世纪初，数学家们逐渐认识到，无穷小量可以通过极限概念加以精确描述。极限理论通过定义函数在某一点附近的行为，能够描述无穷小量趋于零的过程，避免对模糊无穷小量概念的依赖。19 世纪，柯西和魏尔斯特拉斯等数学家通过引入**极限理论**，对极限、连续性、导数和积分的定义进行了重新审视，对微积分进行了严密的逻辑重构。柯西首次明确使用极限概念来定义导数和积分，奠定了现代分析学的基础。而魏尔斯特拉斯则通过对极限的精确定义，彻底摆脱了无穷小量的概念，构建了严谨的微积分体系。极限概念取代了无穷小量，导数被定义为函数在某一点的瞬时变化率，积分则作为求和的极限。这一重新定义消除了无穷小量的模糊性，将微积分从模糊的直觉推导转变为明确的、严格的数学定义，确保了微积分中的推导和计算具有严格的逻辑基础。

通过这种方式，微积分危机得到了全面解决，数学家们能够自信地使用微积分来解决变化率和复杂的动态问题。（AI 去向：梯度传播与稳定性分析一见 8.2。）第二次数学危机的解决，标志着数学从经验性工具走向了更加严谨的理论体系。通过引入极限概念和分析学的系统化构建，数学家们不仅解决了微积分中的基础问题，还推动了现代数学分析的诞生。这一危机的解决具有深远的意义，不仅使微积分成为物理学、天文学、工程学等领域的基础工具，还推动了整个数学的发展。微积分危机的解决使得数学家们对数、函数和极限有了更深刻的理解，并奠定了未来分析学和数理逻辑的基石。

1.5.3 第三次危机：集合论与逻辑的悖论

第三次危机发生在 19 世纪末至 20 世纪初。康托尔创立**集合论**，用对角论证区分了可数与不可数（ $\mathbb{N}$ 与 $[0, 1]$ 基数不同），证明幂集 $\mathcal{P}(X)$ 严格大于 X 本身；无穷不再是单一概念，而有层级与算术。辉煌背后，朴素的“任意性质确定一个集合”（不受限制的理解公理）却诱发自指式矛盾：**罗素悖论**断言“所有不包含自身的集合所成的集合”无论是否包含自身都矛盾；**Burali-Forti** 悖论显示“所有序数的集合”不可成立。问题骤然从“算得对”转为“体系是否自洽”。化解路径分叉而互补：其一，**形式化与公理化**。希尔伯特计划期望用有限方法证明数学的无矛盾性；证明论与可计算性随之萌芽。其二，**类型论**（罗素）以分层语法禁止自指，从源头切

断悖论结构；后经演化影响了编程语言与验证（简单类型、依赖类型、同伦类型论等）。其三，**策梅洛—弗兰克尔 (ZF) 集合论**以外延、公理分离、替换、无穷、正则等公理约束“能被当作集合”的对象，连同**选择公理**形成 **ZFC**，成为主流基础。更具颠覆性的，是**哥德尔不完备性定理**：只要体系足够强（能编码算术），就存在真而不可证的命题（第一定理），且体系无法用自身证明自身无矛盾（第二定理）。随后的独立性成果（如连续统假设 **CH** 由哥德尔与科恩分别证明在 **ZFC** 下不可判定）进一步揭示：一致性可相对证明，完备性不可苛求。为了形象地说明罗素悖论，我们可以借用著名的**理发师悖论**：“他只为那些不自己剃须的人剃须”的理发师，谁给他剃须？无论理发师是否自己剃须，都会违反规则，这是一种**自指性悖论**，在逻辑上无法自洽。罗素悖论暴露出在集合论的基本定义中存在自相矛盾的逻辑缺陷，动摇了整个数学体系的逻辑一致性。为了解决这一危机，数学家们开始重新审视数学的逻辑基础，试图通过更加严密的逻辑系统来修补集合论的悖论，并确保数学体系的完整性和一致性。解决这一危机的主要路径有以下几方面：

1. **形式系统的引入** 以希尔伯特为代表的数学家提出了**形式系统**的思想，即通过定义一组公理和推理规则，将所有数学命题都归结为形式符号的推理。在这种系统中，所有推理必须严格按照形式化规则进行，以确保逻辑的自洽性。希尔伯特的“公理化计划”试图通过一组完备的公理和逻辑规则构建数学的基础，确保数学推理的一致性和完备性，并证明数学体系的无矛盾性。然而，这一计划在哥德尔的不完备性定理面前遭遇了挑战，但它奠定了数学逻辑的基础，为后来的形式逻辑系统和数学结构提供了工具。（AI 去向：形式规约与可验证合约—见 10 章。）
2. **类型论的提出** 罗素本人提出了**类型论 (Theory of Types)**，该理论通过对集合进行分层，将集合划分为不同的类型，禁止集合包含自身或同类类型的集合，从而避免自指性带来的矛盾。这种层次化的集合结构避免了自指的产生，有效消除了罗素悖论带来的自相矛盾。类型论在逻辑学和计算机科学中的应用仍然具有重要意义，尤其在编程语言理论中得到了广泛应用。（AI 去向：类型系统 → 静态检查与安全沙箱。）
3. **公理化集合论** 集合论的进一步发展得益于**策梅洛-弗兰克尔公理系统 (Zermelo-Fraenkel set theory, ZF)**。这一系统通过一套精心设计的公理，对集合的构成和操作进行了规范化处理，避免了自指和无限回归等问题，消除了罗素悖论的根源。**ZF 公理化系统**通过严格的公理定义，为集合论提供了一个无悖论的逻辑基础，确保了数学推理的可靠性。举个例子：以“正则性公理”禁止集合自含，从根本上排除了“自我成员”的构型。
4. **哥德尔不完备性定理** 尽管公理化的集合论解决了逻辑悖论问题，但库尔特·哥德尔(Kurt Gödel)的**不完备性定理 (Gödel's Incompleteness Theorem)**揭示了一个新的局限性：在任何足够复杂的公理系统中，都存在一些命题，既无法在系统内部证明其真伪，也无法推翻。这一结果否定了希尔伯特的完备性计划，证明了即便数学可以通过形式系统避免自相矛盾，也无法保证其逻辑的完备性。数学在本质上是无法完备的。（AI 去向：可证明性边界对“全能对齐”的否证—务实转向评测与约束优化。）尽管如此，哥德尔不完备性定理并未动摇数学的基础，反而让数学家更加意识到逻辑系统的局限性。数学家因此更加重视逻辑学的精密性，推动了数理逻辑和元数学的进一步发展。

第三次数学危机通过公理化、形式系统、类型论和逻辑分析的多重努力得到了有效解决。这一危机不仅推动了数学基础理论的深入研究，也促使数学家对逻辑一致性和严密性的要求更为严格。通过策梅洛-弗兰克尔公理化体系，数学家为现代集合论提供了无悖论的基础。尽管哥德尔的不完备性定理提出了数学体系不完备的本质限制，指出数学系统永远不可能完美无缺，但它同时也为数学逻辑的发展提供了新的方向。（一句话抽象：形式系统保障一致性，不完备性—工程上以规约与监控兜底。）这一危机的解决，使得数学从逻辑上更加稳固，并为后续的数学研究奠定了更高标准的严谨性。数学基础的重建不仅加强了数学逻辑的力量，还推动了现代数学向更高层次的抽象发展，为数理逻辑、模型论和计算理论等新兴学科的诞生铺平了道路。这些三次数学危机深刻影响了数学的演进，促使数学家们不断重新审视基本概念，并推动了数学向更高的抽象层次发展。每一次危机的解决不仅重塑了数学的结构，还推动了新领域的诞生，使数学始终在挑战中突破，走向新的高度。总体而言，三次危机推动数学沿三条主轴升级：其一，扩张对象—从 $\mathbb{Q}$ 到 $\mathbb{R}$ （容纳无理）；其二，澄清语义—以极限、测度等精化“可算性”的语义；其三，自知其限—以公理与证明论重塑基础并承认不完备。危机并未削弱数学，反而让它在更高层级上达成稳定与自治。

1.6 小结

纵观以上四个时期，数学的发展脉络清晰呈现。数学从最初解决日常问题的工具，逐步演化为一门以逻辑推理和抽象思维为核心的科学。数学史既是一部理论创新史，也是一部与社会发展和科学进步紧密相连的演进史。从萌芽阶段的经验性计算到古希腊数学家奠定的逻辑推理基础，再到近代变量数学的飞跃和现代数学的高度抽象化，数学经历了从实用工具到抽象学科的跨越。随着社会与科学的持续发展，数学也成为了理解世界、解决问题和推动技术进步的核心力量。（复盘：本章每节“examplebox”即为 AI 建模的落地锚点。）纵观历史，数学的应用几乎渗透到了每一个领域中。它不仅是科学的语言，更是推动人类文明进步的强大引擎。人类历史上的每一个重大事件的背后都有数学的身影，它以其独特的抽象性和严谨性，塑造了我们对世界的理解和改造：

- 达芬奇的绘画（15-16 世纪）：文艺复兴巨匠达芬奇在其艺术创作中广泛运用了数学原理，如透视法、黄金比例和人体解剖学的精确测量，使得其作品在视觉上达到和谐与真实，展现了艺术与科学的完美融合。
- 哥白尼的日心说（16 世纪）：哥白尼通过复杂的数学计算和天文观测数据，构建了日心说模型，挑战了统治西方思想千年的地心说。他的工作依赖于几何学和三角学，为天文学和物理学奠定了基础。
- 牛顿的万有引力定律（17 世纪）：牛顿运用其发明的微积分工具，精确描述了物体间的引力作用，揭示了行星运动的普遍规律。万有引力定律的提出，是数学与物理学结合的典范，极大地推动了科学革命。
- 马尔萨斯的人口论（18 世纪）：马尔萨斯运用数学模型，特别是指数增长的概念，分析了人口增长与资源供给之间的关系，提出了人口增长快于食物供给的理论，对经济学、社

会学和生态学产生了深远影响。

- 巴赫的 12 平均律（18 世纪）：巴赫的《平均律键盘曲集》是音乐与数学完美结合的典范。12 平均律的音高关系基于精确的数学比例，使得音乐在任何调性下都能保持和谐，展现了数学在美学和艺术中的深层结构。
- 孟德尔的遗传学（19 世纪）：孟德尔通过对豌豆杂交实验数据的统计分析，发现了遗传的基本规律，如分离定律和自由组合定律。他的工作是生物学中运用概率和统计学方法的早期范例，开创了现代遗传学。
- 晶体结构的确定（19 世纪）：晶体学的发展离不开群论和几何学的应用。数学家和物理学家利用对称性原理和 X 射线衍射数据，精确推导了晶体的内部原子排列结构，这对于材料科学和化学的发展至关重要。
- 人人平等（所有的直角都相等）、三权分立的政治结构（19 世纪）：虽然“人人平等”和“三权分立”是政治哲学概念，但其背后蕴含着数学的公理化思想和逻辑结构。例如，“所有的直角都相等”体现了普遍性和一致性原则，这与法律和政治制度追求的公平、普遍适用性有异曲同工之妙。三权分立的制衡机制，也体现了系统设计中的平衡与优化思想。
- 无线电波的发现（19 世纪）：麦克斯韦基于数学方程组（麦克斯韦方程组）预言了电磁波的存在，并揭示了光是一种电磁波。他的理论统一了电、磁、光现象，为无线电通信技术的发展奠定了坚实的数学物理基础。
- 巴贝奇的“机械”计算机（19 世纪）：查尔斯·巴贝奇设计的差分机和分析机，是现代计算机的先驱。这些机械装置的运作原理完全基于数学逻辑和算法，旨在实现自动化计算，标志着计算科学的萌芽。
- 达尔文的进化论（19 世纪）：达尔文的自然选择理论虽然主要基于生物观察，但其核心思想，如变异、遗传和适应性，在现代生物学中已广泛通过统计学和群体遗传学模型进行量化和验证，数学为理解进化过程提供了严谨的框架。
- 爱因斯坦的相对论（20 世纪初）：爱因斯坦的狭义相对论和广义相对论，彻底改变了人类对时间、空间、质量和引力的理解。这些理论的提出和验证，都离不开复杂的张量分析、微分几何等高级数学工具。
- DNA 双螺旋疑结的打开（20 世纪）：沃森和克里克在构建 DNA 双螺旋结构模型时，运用了晶体学数据（由罗莎琳德·富兰克林和莫里斯·威尔金斯提供）和几何对称性原理。数学在解析分子结构、理解遗传信息编码方面发挥了关键作用。
- 冯·诺依曼的二进制编码和现代计算机（20 世纪）：冯·诺依曼体系结构奠定了现代计算机的基础，其核心是二进制编码和存储程序概念。这使得复杂的逻辑运算和数据处理能够通过简单的 0 和 1 进行，是数学逻辑和计算理论的伟大实践。
- 图灵机（20 世纪）：阿兰·图灵提出的图灵机模型，是现代计算机科学的理论基石。它以抽象的数学方式定义了“可计算性”，为算法、程序设计和人工智能的理论发展提供了根本性的框架。
-

（总收束：本章以“现象—抽象—启发—取舍—去向”形成可迁移的学习闭环，后续章节将据此

展开各模块的可操作细节与练习。)

第二章 智能，成于计算机

简明 AI 史

内容提要

- Definition of Theorem
- Property of Cauchy Series
- Ask for help
- Angle of Corner
- Optimization Problem

— 傅鹰

— 约翰·冯·诺伊曼

人工智能（AI）的发展史和数学史一样，是一部不断突破技术限制、追求理解智能本质的进化历程。从最初的理论奠基，到今天无处不在的智能应用，AI 的发展经历了多个阶段，每一个阶段都为未来奠定了新的基础。

2.1 AI 的萌芽期：理论的起源与计算机的诞生

AI 的概念可以追溯到 20 世纪中叶，然而其理论根基则来自更早的数学与逻辑学研究。事实上，AI 的出现并非偶然，它是人类在探索计算的更高效率和智能模拟道路上的自然延续。回顾数学史，不难发现，人类的进步始终伴随着对**更高计算效率**和**更强智能模拟**的追求。从最早的数数工具，到代数、几何的兴起，再到微积分和分析学的创立，每一次数学理论的突破，都是为了解决日益复杂的问题，并提升计算的效率和智能化程度。**举个例子**。随着数学理论的发展，特别是近现代数学时期数理逻辑和计算理论的提出，人工智能的理论基础逐步形成。然而，在 AI 真正出现之前，人类已经通过多个世纪的努力，逐步发明了各种计算机器，以减轻人类在计算方面的负担。人类很早就认识到，计算不必局限于脑力或手工操作，一些重复性和繁复的计算可以通过机械的方法来执行，从而提高效率，减少人为误差。其中最著名的早期计算工具之一便是中国古老的**算盘**。**算盘**的发明可以追溯到公元前数百年，它是一种基于十进制系统的手动计算工具，广泛应用于商业、税务和日常生活中，用于快速进行加减乘除运算。尽管原理简单，但算盘展示了人类利用工具来加速复杂计算的思想，代表了早期机械化计算工具的雏形。随着时间的推移，机械计算设备逐渐变得更为复杂和精密。1642 年，法国数学家**布莱兹·帕斯卡**（Blaise Pascal）发明了第一台机械计算器——“帕斯卡计算器”，它可以执行十进制加法运算，标志着机械与数学计算结合的开端。1678 年，德国数学家**莱布尼兹**（Gottfried Wilhelm Leibniz）进一步改进了帕斯卡的设计，发明了能够执行十进制数乘除运算的机械计算机，揭示了机械计算在复杂运算中的巨大潜力。

不仅是机械计算工具，自动化的思想也融入了各类生产工具的设计中。19 世纪，**雅卡尔织机**通过打孔卡片来控制织物图案的变化，展示了早期的编程思想。这种通过机械手段实现将繁复的任务自动化的理念，预示着计算机自动化编程的雏形，间接推动了后来智能技术的发展。19 世纪，随着工业革命的推进，人类对计算的需求愈加迫切，特别是在天文学、工程学和金融领域。大量数字和复杂方程式问题已远超出人类的手工能力，催生了更多计算机器的发明。1821 年，英国数学家**查尔斯·巴贝奇**（Charles Babbage）设计了“差分机”，这是一种专门用于提高乘法速度和改进对数表精度的机械计算装置。尽管由于设计成本过高，差分机的后续发展以失败告终，但巴贝奇在 1834 年设计的“分析机”被认为是通用计算机的雏形，通过打孔纸带输入程序和数据，预示了现代计算机的基本理念。尽管“分析机”最终因技术局限也未能实现，但巴贝奇的思想奠定了现代计算机的概念基础：通过程序控制的通用计算机。20 世纪初，数学家如**大卫·希尔伯特**、**库尔特·哥德尔**和**阿兰·图灵**提出了关于计算与逻辑的基本理论，为人工智能的萌芽提供了肥沃的土壤。**阿兰·图灵**在 1936 年发表的《论可计算数》一文中，提出了**图灵机**的概念，奠定了现代计算理论的基础和框架。图灵机是一种模拟计算过程的抽象数学模型，这一思想不仅为后来电子计算机的发明打下了坚实的理论基础，也为人工智能提出了一个关键问题——**机器能否像人类一样思考？**1950 年，图灵提出了著名的**图灵测试**，作为判断机器是否具备智能的标准，开启了对机器智能的早期探索。图灵的思想对随后的人工智能发展产生了深远的影响，确定了未来人工智能的发展方向。20 世纪，电子技术的突破使计算机摆脱了传统机械部件的限制，转向实用电子开关、电路板和真空管，使计算速度提高了无数倍，实现了前所未有的效率飞跃。真正具有突破性的发明出现在 1942 年，美国的**约翰·阿塔纳索夫**（John Vincent Atanasoff）和**克利福德·贝瑞**（Clifford Berry）制造了第一台电子计算机——**Atanasoff-Berry 计算机（ABC）**，首次使用二进制来表示数据，并通过电子开关代替了机械部件，标志着从机械计算向电子计算的重大转变，尽管它尚无法进行编程。随后，**ENIAC**，由**约翰·冯·诺依曼**（John von Neumann）等人设计，成为世界上第一台通用电子计算机。**ENIAC**不仅能进行复杂的计算，还采用了冯·诺依曼的存储程序架构，即将程序和数据存储在计算机内存中。这一架构成为现代计算机的基础，大幅提高了计算机的灵活性和计算效率。这些电子计算机的发明，不仅标志着计算能力的巨大飞跃，也为人工智能的早期发展提供了关键的技术平台。

电子计算机使得机器能够在更大规模上执行复杂的计算和推理任务，推动了智能模拟从理论走向实践，开启了自动化计算的新纪元。**AI**作为一种思想和技术，诞生在数学、逻辑和计算科学的交汇点上，标志着智能模拟从理论走向实践的新时代。因此，**AI**的发展历程，既是人类追求越来越高效计算的延续，也是计算思维逐步外化并加速进化的体现。从算盘、机械计算器到电子计算机，每个阶段的技术进步都使计算变得更加高效和智能化，推动了从人工计算到机器计算的转变。这一趋势也为 **AI** 的诞生铺平了道路，使得机器不仅能执行运算，还具备了模拟人类思维、学习和决策的潜力。**AI**正是这一智能化趋势的巅峰成果，是人类不断努力提升计算能力和模拟智能行为的必然延伸。

2.2 1950-1970 年代：AI 的奠基与早期探索

1950 到 1970 年代，人工智能开始从理论走向实践。1950 年代被视为人工智能的正式起点。在此期间，数学、逻辑学、计算机科学和神经科学的交汇，为人工智能的早期发展铺设了道路。**阿兰·图灵**的贡献无疑是这个时期最为重要的起点。1950 年，他在论文《计算机与智能》中提出了衡量机器是否具备智能的标准——“图灵测试”。图灵通过设想一台机器与人类对话的情境，开启了对机器是否能够模拟人类智能的讨论，奠定了早期人工智能研究的理论基础。**约翰·冯·诺依曼**的工作对 AI 的发展也产生了重要影响。1940 年代，冯·诺依曼提出了自复制自动机的理论，并在计算机设计中推广了“存储程序”的概念，这使得计算机能够按照程序指令自动执行复杂任务。冯·诺依曼的计算机架构为人工智能提供了强大的硬件基础，也为智能程序的实现铺平了道路。

增加参考文献《面对面的办公室》

成就一生的会议

为便于阅读，本节（成就一生的会议）承接上节（1950-1970 年代：AI 的奠基与早期探索），先交代差异与共同点，再聚焦本节关键问题。

约翰·麦卡锡（John McCarthy）是 AI 奠基时期最关键的人物之一。他的思想和研究引领了 AI 这一全新领域的诞生和早期发展。1948 年，年轻的麦卡锡在普林斯顿大学听了**冯·诺依曼**关于自复制自动机的报告后，激发了他对“用计算机模拟人类智能”的强烈兴趣。1949 年，麦卡锡在普林斯顿数学系攻读博士学位时首次在论文中提出了“用计算机来模拟人类智能”的想法，尽管当时这一概念十分模糊，但已展现出 AI 作为独立研究领域的雏形。在冯·诺依曼的鼓励和支持下，麦卡锡将主要研究方向定为在机器上模拟人的智能，特别是将下棋问题作为重点课题之一。此后，为了减少计算机需要考虑的棋步，麦卡锡发明了 α - β 剪枝算法。这一方法有效地减少了计算量，至今仍是解决人工智能问题中一种常用的高效方法。麦卡锡和贝尔实验室的**克劳德·香农**（Claude Shannon）——“信息论”的创始人，在人工智能方面的若干深入探讨之后，他们萌生召开一次研讨会的共识。1956 年，在洛克菲勒基金会的一笔微薄的赞助下，他们邀请到当时哈佛大学的**马文·明斯基**（Marvin Minsky）和 IBM 工程师罗彻斯特等几位学者，在达特茅斯学院组织了这次夏季研讨会。在达特茅斯会议上，“人工智能”一词降临，备选名字有“自动机研究”、“复杂信息处理”、“机器智能”与“控制论”，与会者最终选择了“人工智能”（Artificial Intelligence），简洁而富有指向性。“……学习的每一方面或智力的任何其他功能，原则上都可以准确地描述，并由机器模拟。我们将尝试制造能够使用语言、提炼抽象概念的机器的方法，……如果一批优秀的科学家在一起研究一个夏天，那么这一领域中的一个或多个问题就能得到显著的推进。”这种大胆的设想不仅为 AI 的核心研究方向指明了道路，也传达出一种对机器智能的乐观信念——即智能行为是可以被精确模拟的。这次会议成为众多奠基者的灵感之源，确立了人工智能学科的基础框架，使“人工智能”这一领域获得科学界的认可，并为未来几十年的研究奠定了重要的学术基础。汇集了 10 位科学家，持续 2 个月的达

特茅斯会议是计算机史上的一座里程碑。

会议的主要参与者后来都在计算机科学和人工智能领域取得了杰出的成就。

- **约翰·麦卡锡 (John McCarthy)**：作为达特茅斯会议的主要发起人，麦卡锡不仅提出了“人工智能”这一术语，还在麻省理工学院 (MIT) 建立了第一个 AI 实验室，开发了 AI 的主要编程语言之一——Lisp，为 AI 的符号处理和逻辑推理奠定了基础。1971 年，麦卡锡因其在 AI 领域的杰出贡献而获得图灵奖。
- **马文·明斯基 (Marvin Minsky)**：人工智能和认知科学的先驱，达特茅斯会议的主要参与者之一。明斯基在 MIT 建立了著名的 AI 实验室，将人工智能的研究扩展到学习和认知的复杂领域。他于 1969 年获得图灵奖，彰显了其在 AI 理论和方法上的重要贡献。
- **克劳德·香农 (Claude Shannon)**：作为信息论的创始人，香农在达特茅斯会议上的参与为 AI 带来了信息处理的理论支持。他通过“信息熵”概念，为通信和数据处理打下了数学基础，推动了 AI 在数据传输和编码方面的进展，为机器学习和模式识别等 AI 技术的日后发展提供了理论依据。
- **艾伦·纽厄尔 (Allen Newell) 和赫伯特·西蒙 (Herbert Simon)**：这两位科学家不仅在达特茅斯会议上推动了 AI 的发展，还共同开发了早期的 AI 程序，如 Logic Theorist 和 General Problem Solver (GPS)，这些系统奠定了符号主义 AI 的基础。两人于 1975 年共同获得图灵奖，以表彰他们在逻辑推理和问题求解的 AI 系统上的创新。西蒙还因在经济学和认知科学的交叉研究中取得的杰出成果，于 1978 年获得诺贝尔经济学奖。
- **纳撒尼尔·罗切斯特 (Nathaniel Rochester)**：作为 IBM 计算机设计团队的一员，罗切斯特在达特茅斯会议上带来了对硬件和计算机体系结构的深刻理解，为 AI 的技术发展提供了重要支持。他为符号主义 AI 的实际应用提供了硬件基础，使得 AI 研究在之后的几十年中得以快速发展。

达特茅斯会议不仅是 AI 发展的里程碑，更是奠基者们各自成就的起点。他们在日后的研究和创新中，将符号处理、逻辑推理、信息论、经济学等跨学科思想融入 AI 之中，为这门新兴学科注入了多元化的基础。这次会议标志着人工智能领域的正式诞生，掀起了未来几十年 AI 研究的浪潮，因此，1956 年也被称为“AI 元年”。

2.2.1 达特茅斯会后早期黄金期：符号主义 AI

在达特茅斯会议之后，人工智能进入了早期探索阶段，其核心概念逐步成型。研究者们围绕“机器能否像人类一样思考”这一问题展开了深入探索，人工智能从学术设想走向实际研究，并取得了一系列早期的重要成果。1958 年，明斯基和麦卡锡在麻省理工学院建立了世界上第一个人工智能实验室——MIT AI Lab。同年，麦卡锡发明了 **Lisp** 编程语言。Lisp 的灵活性和符号处理能力使其成为 AI 研究的首要编程语言之一，尤其是在自然语言处理和知识表示等领域。20 世纪五六十年代，大多研究者都聚焦于自上而下的研究方向，**符号主义 AI (Symbolic AI)** 是这一时期的主流，即主张通过符号操作来建立人脑模型从而模拟智能。这一时期的 AI 研究主要聚焦于**符号处理**和**规则推理**，形成了一种“强 AI”乐观主义的思潮。科学家们设想，通过

逻辑推理和符号处理，机器能够像人类一样解决问题，甚至达到通用智能的水平。早在 1956 年，西蒙和纽厄尔预言“10 年之内，计算机将成为国际象棋世界冠军”。1966 年，西蒙再次乐观预言“20 年内，机器能完成人能做到的一切工作”。研究者们开发了早期的专家系统和逻辑推理系统，如阿伦·纽厄尔和赫伯特·西蒙等人研发的 **Logic Theorist** 和不依赖于具体领域的通用问题求解器 **General Problem Solver**，尝试模拟人类解决问题的逻辑推理过程，展示出机器进行逻辑推理的潜力。**机器定理证明**是这一时期最先取得重大突破的领域之一。

1956 年，**Logic Theorist** 由阿伦·纽厄尔和赫伯特·西蒙等人在兰德公司开发，被认为是首个具有“人工智能”特征的计算机程序，**Logic Theorist** 的目标是模拟人类的逻辑推理能力，能够自动证明逻辑定理。**Logic Theorist** 可独立证明罗素和怀特海所著《数学原理》第二章的 38 条定理，1963 年能证明该章的全部 52 条定理，甚至为其中一个定理上找到了比原作者给出的证明更为简洁的证明。这一成就引起轰动，显示出计算机在某些逻辑推理任务中的潜力。其它成果包括：

- 1958 年，华人数学家王浩王浩在 IBM704 上仅用 5 分钟便证明了《数学原理》有关命题演算的 220 条定理。
 - 1963 年，IBM 公司研制出平面几何的定理证明程序。
 - 1963 年，Slagle 发表了一个符号积分程序 **SAINT**，可自动计算输入函数的积分表达式。
- 4 年后推出的升级版 **SIN**，达到专家级水准。

这些为机器定理证明的“封闭”逻辑系统系统的成功为后续 AI 系统奠定了基础，但也暴露出符号 AI 系统明显的局限性——只能处理严格定义的逻辑问题，难以应对更复杂的、不确定的现实问题。



图 2.1: 手写数字识别

以手写字母识别 (如图2.1) 为例，手写字母在形态上具有高度的变化性和不确定性。不同人书写的“A”可能形态差异很大，甚至同一个人也会因速度、情绪等不同而写出不同的“A”。如果让 **Logic Theorist** 识别手写字母“A”和“B”，它会陷入困境，因为符号主义 AI 依赖预先设定的规则进行逻辑推理，要求字母形态稳定且符合固定定义。而现实中的手写字母形态多样，

难以满足这些硬性规则，因此，符号主义 AI 在这种不确定、变化多样的环境下表现不佳。虽然这些早期 AI 系统在模拟人类逻辑推理能力上取得了成功，能够解决简单的逻辑问题，但在面对复杂多变的现实问题时，它们的表现力和适应性明显不足，难以应对更广泛的智能任务。这一局限性表明，符号主义 AI 在处理现实中的不确定性问题时仍面临巨大挑战。

自我学习

为便于阅读，本节（自我学习）承接上节（达特茅斯会后早期黄金期：符号主义 AI），先交代差异与共同点，再聚焦本节关键问题。

与此同时，AI 研究者也开始探索机器学习的可能性，机器学习领域获得不少突破。1959 年，阿瑟·塞缪尔（Arthur Samuel）研制了一款跳棋程序，能够通过自我对弈提升棋艺。1962 年，它击败美国州冠军。这款程序被认为是早期**机器学习**的范例，展示了机器通过经验不断提高性能的潜力，标志着 AI 研究从预设规则向基于学习的方向发展。1956 年，Selfridge 研制出第一个字符识别程序，开辟了模式识别这一新领域。

1974-1980 年代初：第一次“AI 寒冬”

为便于阅读，本节（1974-1980 年代初：第一次“AI 寒冬”）承接上节（自我学习），先交代差异与共同点，再聚焦本节关键问题。

虽然早期的符号 AI 研究取得了显著进展，但随着研究的持续推进，符号 AI 的局限性日益显现。尽管符号 AI 擅长逻辑推理，且在特定任务上表现出色，但在应对现实世界的复杂性和不确定性问题时，特别在处理非结构化数据方面，表现出了明显的不足。1965 年，机器定理证明领域遇到了瓶颈。例如，计算机推了数十万步也无法证明“两个连续函数之和仍是连续函数”。萨缪尔的跳棋程序停留在了州冠军的水准，难以突破。更糟糕的问题出现在**机器翻译**领域，计算机在自然语言理解与翻译中表现得极其差劲。例如，翻译程序将英语句子“The spirit is willing but the flesh is weak.”（心有余而力不足）翻译成俄语，再回译成英语时，得到的句子竟变成了“The wine is good but the meat is spoiled.”（酒是好的，肉变质了）。这种笑话般的错误凸显出符号 AI 在处理语言模糊性方面的无力。1970 年代的乐观情绪（如通用智能）推动了大量 AI 项目的启动，然而，随着这些项目未能达最初的预期，尤其是在应对现实世界的复杂性方面暴露出显著局限，加上计算资源需求高，导致了随后 1974 年开始的第一次“AI 寒冬”——投资者和政府机构对 AI 研究的资金支持大幅削减，研究热情和社会关注下降，许多 AI 项目被取消，研究活动在 1974 年至 1980 年间大幅缩减，AI 研究陷入低谷期。尽管如此，低谷期的 AI 领域并非全无进展，反而通过知识工程和专家系统的兴起找到了新的突破口。

2.3 1980 年-1987 年：专家系统与知识工程的兴起

从 1980 年至 1987 年，AI 研究进入了一个新的阶段，以**专家系统**和**知识工程**的兴起为代表。美国计算机科学爱德华·费根鲍姆指出，传统的人工智能过于强调通用求解方法的作用，

忽略了具体的领域知识。因此，他提出了基于知识的专家系统。**知识工程**是利用人工智能技术来构建专家系统的一种方法。**专家系统**是基于符号逻辑的 AI 系统，通过将专家的知识编码为计算规则，帮助机器在特定领域中进行推理和决策，模仿领域专家解决问题的过程。

2.3.1 代表性专家系统

典型的专家系统包括医学诊断系统 **MYCIN** 和化学分析系统 **DENDRAL**，它们分别在医疗和化学领域取得了显著成效。通过嵌入领域专家的知识，专家系统在复杂任务中展现出较高的决策能力，推动了知识工程成为 AI 研究的重要分支。

- **DENDRAL**: 开发始于 1965 年，由斯坦福大学的爱德华·费根鲍姆 (Edward Feigenbaum) 和约书亚·莱德伯格 (Joshua Lederberg) 带领的团队创建。**DENDRAL** 则是一种用于化学分析的系统，能够根据分子结构推断化学物质。它被认为是第一个成功的专家系统，标志着知识工程的开端。
- **MYCIN**: 开发始于 1972 年，由斯坦福大学的布鲁斯·布坎南 (Bruce Buchanan) 和爱德华·肖特利夫 (Edward Shortliffe) 领导的团队创建。**MYCIN** 是一个医学专家系统，专注于诊断血液感染并推荐抗生素。它通过一系列规则模拟专家在诊断细菌感染时的决策过程，其规则系统和推理机制为后续专家系统的发展影响深远。

虽然 **DENDRAL** 和 **MYCIN** 的开发始于 1960 年代末至 1970 年初，但它们在 1970 年代后期到 1980 年代中期得到实际应用和广泛推广，成为知识工程的核心。这些专家系统的成功验证了知识工程的可行性。各种专家系统陆续涌现，尤其在医疗、化学分析、工程设计和商业决策等特定任务中，标志着 AI 技术的一个实用转变。研究者们从这些成功中看到了基于知识的 AI 应用的巨大潜力，开始着重于知识获取、知识表示和推理机制的系统性开发。这一时期成为专家系统和知识工程在实际应用中迅速扩展的高峰期。

2.3.2 知识表示与规则推理

专家系统的成功得益于对**知识表示**和**推理机制**的深入研究。**知识表示**是指如何在计算机中存储和处理专家的领域知识，而**规则推理**则是通过符号操作来进行逻辑推理的过程。在专家系统中，领域专家的知识被转换为计算机可以处理的规则集，从而模拟专家的判断。这种规则系统让计算机能够在特定领域中像人类专家一样做出高效的决策。

2.3.3 知识工程的广泛应用

DENDRAL 和 **MYCIN** 的成功验证了知识工程的可行性后，推动了更广泛的行业应用。知识工程通过获取和编码人类专家的知识，为 AI 系统提供了专业化的领域知识，并形成了软件产业的一个新分支。知识工程的兴起不仅推动了专家系统的广泛应用，也促成了 AI 技术在全球范围内的合作和竞争。受知识工程的激励，日本的第五代计算机计划、英国的阿尔维计划、

西欧的尤里卡计划、美国的星计划和中国的 863 计划陆续推出，尽管这些计划的目标并不完全针对 AI，但 AI 是其重要组成部分，成为提升国家科技竞争力的一部分。

2.3.4 新兴技术：模糊逻辑与贝叶斯网络

在这一时期，AI 领域还从知识工程中衍生出了新的学科分支，如**模糊逻辑**和**贝叶斯网络**。模糊逻辑提供了一种处理灰色区域的推理方式，使 AI 能够在信息不完全的情况下也能做出决策。贝叶斯网络则通过统计方法来处理不确定性，被广泛应用于诊断、预测和决策系统中。这些方法通过处理不确定性和模糊性，有效弥补了传统符号主义 AI 在处理复杂现实问题上的局限。

2.3.5 第二次“AI 寒冬”的反思与转型

20 世纪 80 年代中期，专家系统和知识工程在医疗诊断、工程设计、化学分析等狭窄领域表现出色，但伴着随大量实践的累积，其局限性也逐渐显现。**知识获取**成为了最大的瓶颈：专家系统高度依赖领域专家的知识输入和精确的规则，知识获取和维护成本高昂，系统缺乏灵活性和自适应性，难以应对多样和动态的现实需求。此外，硬件技术和计算资源的限制也不足以支撑更大规模的 AI 系统发展。随着专家系统的商业化受阻，AI 进入了第二次寒冬。1987 年至 1993 年之间，投资者和政府逐渐减少了对 AI 的资金支持，许多原本大规模投资的 AI 项目被迫中止。日本的“第五代计算机计划”因未能在智能计算取得预期成果而停滞，其他国家的大型 AI 项目也相继缩减，导全球范围内的 AI 研究热潮急速降温。在这次寒冬期内，AI 研究的资金大幅削减，许多研究人员转向其他领域。第二次的 AI 寒冬虽然带来了低谷，却促使 AI 领域反思技术路径，重新审视符号主义 AI 的局限性。研究者和行业逐渐认识到，仅凭符号规则和知识编码不足以实现真正的智能。知识工程和专家系统的应用展现了 AI 技术在特定领域的应用潜力，推动了 AI 从学术研究向实用化的转变。尽管专家系统存在明显局限，但这些实践经验和技术积累为之后的机器学习、神经网络等数据驱动方向的复兴提供了宝贵启示，推动 AI 研究进入了新的发展阶段。寒冬的沉寂期反而带来了新的视角，促使研究者在方法和技术上寻求突破和创新。这些反思和积累成为推动 AI 转型的重要动力，为现代 AI 的崛起，尤其是后续深度学习的发展，奠定了坚实的基础。

2.4 人工智能的分化

1987 年以后，人工智能分化为六大学科：

- 计算机视觉 (模式识别，图像处理等)
- 自然语言理解与交流 (语音识别、合成、对话等)
- 认知与推理 (各种物理和社会常识等)
- 机器学习 (各种统计建模、分析工具和计算方法)
- 机器人学 (机械、控制、设计、运动规划、任务规划等)

- 博弈与伦理 (多代理人 agents 的交互、对抗与合作、机器人社会融合等)

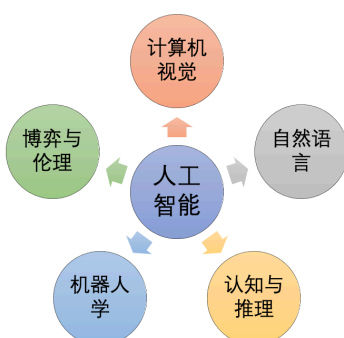


图 2.2: “1987 年后人工智能的分化”



图 2.3: “1987 年后人工智能的分化”



图 2.4: “1987 年后人工智能的分化”

可以看出来，人工智能在全面模拟人的智能，不仅是大脑智能，还包括眼睛智能，耳朵智能，手智能，口智能。

2.5 1990-2010 年代：联结主义和机器学习的兴起

20 世纪 80 年代，**机器学习**由原本的边缘分支成为关注的焦点。研究者发现，如果采用完全不同的世界观，即让知识通过自下而上的方式涌现，而不是让专家们自上而下地设计出来，机器学习的问题其实可以得到很好解决。自下而上的涌现智能的方案很早就提出过，但没有引起注意：一批人：可通过模拟大脑结构（神经网络）来实现 (联结学派) 另一批人：可从那些简单生物体与环境互动的模式中寻找答案 (行为学派) 20 世纪 80-90 年代，传统的人工智能符号学派、联结学派、行为学派三足鼎立。早期的学习算法开始在符号 AI 的基础上发展，成为 AI 从符号主义向数据驱动方法转型的基础。尽管 1987-1993 年期间的 AI“寒冬”导致研究资金减少，学术界对神经网络等方向的探索仍在继续。特别是在 1990 年代后期，联结主义和机

器学习的复兴为 AI 注入了新的活力。研究者们开始着力于数据驱动的学习算法，这些研究逐步成为现代 AI，尤其是深度学习的起点。进入 90 年代后，AI 研究逐渐从符号主义（基于符号的 AI）转向数据驱动的机器学习（**基于数据的 AI**）。在计算机能力提升和数据量爆发增长的推动下，机器学习逐渐成为 AI 研究的主流。通过统计学习和数据驱动的模式识别，AI 系统能够大数据进行训练，显著提升识别和决策的准确性。在这一阶段，**神经网络**的复兴尤为关键。虽然早期的神经网络（如感知机）在 1960 年代就已提出，但因计算能力受限，没有达到理想效果。1990 年代后，神经网络在计算能力提升的推动下重新焕发生机，显示出巨大潜力。**1997 年，IBM 的“深蓝”战胜国际象棋冠军卡斯帕罗夫**，标志着 AI 在某些特定任务上的重大突破，成为该领域的重要里程碑。

到了 2010 年代，**深度学习**技术的突破推动了 AI 的飞速发展。基于**卷积神经网络（CNN）**和**递归神经网络（RNN）**等深度学习模型的广泛应用，AI 在图像识别、语音识别、自然语言处理等多个领域取得了显著进展。2012 年，**AlexNet** 在 ImageNet 竞赛获胜，奠定了深度学习在图像识别领域的领先地位。2016 年，谷歌的 **AlphaGo** 战胜围棋世界冠军李世石，进一步展示了深度学习和强化学习在复杂问题处理上的强大能力，为 AI 新时代拉开了序幕。

2.5.1 2020 年代：大语言模型（LLM）与生成式人工智能（AIGC）的崛起

进入 2020 年代，人工智能在语言处理和内容生成领域迎来了崭新的突破。**大语言模型（LLM）**和**AI 生成内容（AIGC）**技术的兴起，标志着 AI 在自然语言理解、图像生成等跨模态任务上的重大进展。这一阶段的 AI 系统不仅限于执行特定任务，还展示出强大的语言生成和创意能力，推动了 AI 在内容生成、对话系统、设计创作等多元领域的应用。**大语言模型（LLM）**通过海量文本数据的训练和数十亿参数的深度优化，使得 AI 具备了对自然语言的复杂理解和生成能力。**GPT 系列、BERT**等模型的出现，让 AI 能够生成具有上下文连贯性和逻辑性的文本内容，实现更自然、更智能的人机互动。这些模型不仅能处理简单的文本任务，还能在医学、法律、科技等领域为人类提供辅助性建议。**LLM**的创新，尤其在教育、客服和专业问答等领域的广泛应用，使得 AI 能够更深入地理解和模拟人类语言的多样性和细腻性，逐步拓宽了人工智能的应用范围。与此同时，**AI 生成内容（AIGC）**技术使 AI 从信息处理者发展为创意生成者。**AIGC**让 AI 不仅可以理解和操作文本，还能够生成图像、视频、音乐等领域内容，为人类的创意需求提供助力。例如，基于扩散模型和生成对抗网络（GANs）的图像生成器能够创作出栩栩如生的图像，甚至具备了艺术创作的初步能力。**AIGC**技术使 AI 在广告、娱乐、设计和教育等多个领域得到了广泛应用，不仅缩短了内容创作的时间，还为人类的创作过程带来了新鲜的灵感和可能性。**LLM**和**AIGC**的兴起不仅丰富了 AI 的应用场景，更重要的是，它让 AI 在情感识别、语言生成和创意表达方面具备了更强的适应性，使得 AI 从单纯的“工具”逐渐演变为可以合作、创新的“伙伴”。AI 能够生成具有人性化特质的内容，从而在客服、医疗和心理咨询等对人类情感需求更高的领域中提供更加贴近人类需求的服务。2020 年代的 **LLM**和**AIGC**的进展，标志着 AI 从知识服务者走向创意合作者的转变。这一阶段的 AI 不仅展示了生成文本、图像等复杂内容的能力，也在理解人类表达和模拟创意思维上迈出

了一大步。LLM 和 AIGC 带来了前所未有的机遇，将继续推动 AI 技术在更多领域的应用与普及，引领人类进入一个充满创新和智慧的新时代。

人工智能从诞生至今，经历了一次又一次的繁荣与低谷，其发展历程大体上可以分为“推理期”，“知识期”和“学习期”。[25] 在人工智能的研究中，不同学派以各自的方法和理论基础探索智能的本质。这些学派的差异体现在如何定义智能、模拟智能的方法以及应用的领域中。三大主要学派包括**符号主义学派**、**联结主义学派**和**行为主义学派**，它们分别代表了对智能问题的三种主要思维方式。

2.5.2 符号主义学派 (Symbolicism)

符号主义学派：知识驱动的推理与表现

为便于阅读，本节（符号主义学派：知识驱动的推理与表现）承接上节（符号主义学派 (Symbolicism)），先交代差异与共同点，再聚焦本节关键问题。

符号主义学派，又称**逻辑主义**或**符号处理学派**，是 AI 发展的早期重要分支之一。符号主义学派的理论基础源于 20 世纪中期的认知科学和数理逻辑。符号主义学派主张，人类的知识和认知可以通过符号加以形式化，且问题解决可以通过符号操作（如逻辑推理）完成。因此，符号主义学派主要专注于建立规则和逻辑系统，尝试通过符号和规则来模拟人类的高级思维过程，包括推理、规划和知识表示。符号主义学派基于对问题的“高阶”解释，核心思想是智能行为能够通过逻辑规则和知识库来模拟，依赖于启发式搜索和基于规则的推理系统，试图用知识与经验的积累来填补信息空白。这种“If-then”式的推理方式使符号主义在规则明确的任务中表现出色，被广泛应用于医疗诊断、金融分析等领域的专家系统。

符号主义学派的成果与应用

为便于阅读，本节（符号主义学派的成果与应用）承接上节（符号主义学派：知识驱动的推理与表现），先交代差异与共同点，再聚焦本节关键问题。

符号主义学派的代表性成果包括早期的专家系统（如 MYCIN 和 DENDRAL）、逻辑定理证明系统（如 Logic Theorist）、推理系统和象征性规划系统。

- **人机对弈：“深蓝”战胜卡斯帕罗夫** 1997 年 5 月 11 日，IBM 的超级计算机“深蓝”在国际象棋比赛中战胜了国际象棋大师卡斯帕罗夫，标志着符号主义 AI 的一次里程碑事件。深蓝经过四位国际象棋特级大师的训练，储存了 100 年来几乎所有国际特级大师的开局和残局下法，利用启发式搜索可在 1 秒钟内计算两亿步棋。这种超强搜索能力，使得“深蓝”通过规则匹配和快速推理“险胜”人类选手，显示了符号主义在明确规则系统中的强大计算能力。
- **知识问答：Watson 战胜人类选手** 2011 年 2 月 14-16 日，在《危险》问答比赛中，IBM 的超级计算机 Watson 战胜人类选手。Watson 是个自然语言处理高手。它能够快速理解主持人的提问，并调用广泛的知识库回答问题。它的知识涵盖时事、历史、文学、科学、

艺术、流行文化、体育、地理、文字游戏等多个领域，甚至理解语言的隐含意思，比如字谜和双关语，甚至还装满诸如莎士比亚戏剧的独白。**Watson** 充分体现了符号主义 AI 在自然语言理解和知识问答中的突破性进展。

- **医疗应用：Watson Health** IBM Watson 还被应用于医疗领域，基于大量医学数据和患者数据，为药物研发和个性化健康管理提供支持。例如，在癌症免疫治疗方面，**Watson** 从数百万篇医学论文的 2500 万篇摘要、400 万条病人数据以及 1861 年以来的药物专利中提取信息，为个性化治疗提供方案。此外，在糖尿病管理上，**Watson** 能够对来自 1 万名匿名患者的电子健康记录、医保数据和其他公共数据进行实时分析和预测，提前 3-4 小时预测血糖波动，预测准确率达 75-86%，帮助患者有效管理健康。

符号主义 AI 的优势与局限

为便于阅读，本节（符号主义 AI 的优势与局限）承接上节（符号主义学派的成果与应用），先交代差异与共同点，再聚焦本节关键问题。

符号主义具有以下优势：

1. **可解释性**：符号主义的推理过程与人类的思维方式基本一致，因而具有很强的可解释性，使人们能够理解和追踪系统的推理过程。
2. **鲁棒性强**：符号规则系统在处理结构化、规则明确的问题时表现出色，具有较高的稳定性，尤其适用于特定领域内的任务。
3. **逻辑推理能力**：符号主义系统能够执行复杂的逻辑推理，适用于处理涉及高级认知的任务，如规划和知识表示等。

20 世纪 80 年代后，符号主义学派的发展逐渐减缓，主要原因在于其依赖的“演绎逻辑和语义描述”以及或“形式化方法”不可能为所有的对象建立模型。符号主义系统需要对问题抽象出一个精确数学意义上的解析式数学模型，通过构建理想的数学模型和逻辑规则来模拟智能行为。一旦模型建立，计算任务与能力就被唯一地确定。确定的算法无法表示现实世界问题所固有的测不准性和不完备性：世界是动态的、复杂的，无法用一个模型来表达。此外，图灵意义下的可计算问题都是可递归的，但可递归并不是解决问题的唯一途径。虽然符号处理可以应对某些递归性问题，但对于智能的全面理解和表现，符号主义学派的框架显然不够完整。

符号主义学派的主要局限性可归纳如下：

1. **知识依赖性**：符号主义系统依赖于大量准确、结构化的知识和规则，但人类知识难以全面准确地形式化，这种依赖性限制了系统在现实问题中的表现。
2. **缺乏灵活性**：符号主义系统只能应对预先定义的问题，难以适应动态、复杂和不确定的环境。这种限制使其在处理非结构化问题时表现不佳。
3. **条件与分枝问题**：在复杂的动态系统中，符号主义面临两大瓶颈：**条件问题 (Qualification Problem)**，即难以穷尽智能行为的所有先决条件；**分枝问题 (Ramification Problem)**，即无法枚举智能行为的所有隐含结果。符号主义系统因此难以应对复杂环境中的不确定性和隐含影响。

4. **应用领域狭窄**：尽管符号主义模型在逻辑推理和规则明确的任务中表现出色，但在感知、语言理解等模糊和动态性较强的任务中，由于对人工定义的规则和知识库的依赖，其适应性较差。

符号主义学派在 AI 发展的早期取得了显著成就，为人类理解和模拟智能行为提供了框架和方法。然而，随着应用需求的增加和技术环境的变化，符号主义的局限性逐渐显现，学界认识到，为应对复杂的动态环境，还需结合其他学派的思维方式。符号主义的贡献在于奠定了知识驱动的 AI 基础，展示了通过逻辑推理模拟人类智能的可能性，为之后联结主义与行为主义的发展开辟了道路。

2.5.3 联结学派/连接学派 (Connectionism)：从神经网络到深度学习

20 世纪 80 年代之后，随着计算能力的提升和神经科学的深入研究，人工智能的重心逐渐转向**联结主义学派**，也称**人工神经网络学派**。联结主义学派认为智能的产生并非依赖于符号和逻辑规则，而是源于大脑中成千上万个神经元的联结和计算。正如其支持者所言，“高级的智能行为是从大量神经网络的连接中自发出现的”。联结主义学派主张通过模仿大脑神经网络的工作原理来实现智能。大脑由一万亿个神经元细胞通过错综复杂的相互连接形成了复杂的认知和决策能力。特别关注如何通过数据训练来自动调整网络参数，使系统能够在数据中自发学习和优化。其核心思路是模仿神经元的工作方式，构建多层网络结构，调整神经元之间的连接权重，使得系统逐渐具备特定的识别和判断能力。例如，神经网络的训练过程就是通过不断调整权重，逐步优化输出结果的过程。不同于符号主义学派的知识驱动，联结主义的这一训练机制强调**数据驱动的学习**，即依靠大量的数据输入，使系统在学习自动调整网络参数，以形成稳定的模式识别和分类能力。联结主义学派的重要进展包括从早期的**感知机模型**到更复杂的**卷积神经网络 (CNN)**、**递归神经网络 (RNN)** 等模型，最终发展到现代**深度学习**模型。这些模型推动了 AI 在图像识别、自然语言处理、生成式 AI 等领域的广泛应用，成为现代 AI 的重要工具。这一学派的成就在近年来引发了广泛关注和思考。尽管联结主义学派推动了人工智能的飞跃式进展，但其局限性也不容忽视。首先，联结主义模型对大量数据的依赖较高，若数据不足或质量不佳，模型的表现可能会大幅下降。其次，随着神经网络层次增加，其计算复杂度也随之提高，导致训练和运行成本显著增加。其次，神经网络通常被视为“黑箱”，即难以解释其内部决策依据。这一“黑箱”问题主要源于以下几点：

- **多层非线性结构**：神经网络的层层变换过程使其难以还原具体决策路径，庞大的参数量也导致分析“哪些特征在决策中起了关键作用”变得困难。
- **局部特征依赖性**：模型往往基于局部特征组合来进行决策，而缺乏对整体决策过程的全局视角。例如，在图像分类中，模型可能依赖某些局部图案作出决策，使得预测依据难以解读。
- **数据驱动缺乏推理链**：联结主义模型倾向于基于统计特征进行预测，缺乏类似符号主义系统的逻辑推理链，不便于展示推理过程。

尽管存在可解释性方面的挑战，联结主义的理论和技術构建了现代神经网络的基石，极大地

推动了现代 AI 的发展。在下一章中，我们将进一步探讨联结主义的核心思想、模型结构及其应用实践。

2.5.4 行为主义学派 (Actionism)

行为主义学派源于 20 世纪中叶，其理论基础来自行为主义心理学和生物学对动物行为的研究，认为智能体可以通过与环境的交互获得经验，从而逐步学习最优的行为策略。行为主义学派主张智能的形成基于行动与环境反馈的相互作用，智能体在反复试错中调整自身策略，优化行为序列，以实现既定目标。行为主义学派的早期探索主要集中于模仿昆虫等低等生物的行为模式。简单生物无需复杂的大脑结构，也不会按照传统的方式进行复杂的知识表示和推理来执行智能行为，仅通过直接的环境交互就能实现适应性行为。例如，昆虫能够灵活移动、快速反应、躲避捕食者的攻击；小小的蚂蚁所组成的庞大蚁群展现出高度写作的群体智能。这些生物体的行为为行为主义学派提供了灵感。随着计算能力的提升，行为主义方法逐渐被应用于机器人控制、游戏策略等领域，特别适合在高不确定性和动态环境中构建智能体行为。行为主义学派的一个典型实现是**强化学习**（Reinforcement Learning, RL）算法，尤其是 Q 学习和深度强化学习（DRL）。这些方法通过奖励机制引导智能体在环境中逐步优化策略。例如，AlphaGo 挑战围棋世界冠军的成功便是强化学习和深度学习结合的成果。AlphaGo 在与环境的持续交互中，逐步学习复杂博弈中的最佳策略，展示了人工智能在自我优化和复杂决策方面的潜力。MIT 机器人专家 Rodney Brooks 设计的模仿昆虫的机器人正是这一学派的经典案例。通过四肢和关节的协调，这些机器人能够在复杂地形中灵活爬行，巧妙避开障碍物，无需复杂的大脑控制。类似地，波士顿动力公司（Boston Dynamics）开发的四足机器人在不同地形和环境中表现出高度的适应性和鲁棒性。这些机器人的行为不依赖于复杂的认知模型或高级符号推理，而主要依靠局部传感器数据、反馈控制和运动算法来实现稳定的运动与环境适应，在一定程度上可以被称为“无脑机器人设计”。这种设计理念类似于昆虫等简单生物的直接感知-行动环的反应式行为，展示了行为主义学派在机器人开发和运动控制中的实际价值。尽管行为主义学派取得了诸多进展，仍面临明显的局限性：

1. **高维度和复杂环境下的学习效率低**：行为主义学派在高维、复杂环境中往往需要大量交互才能收敛到合理策略，试错成本较高。
2. **奖励依赖**：行为主义模型依赖奖励反馈引导学习，当目标明确时效果良好，但在奖励稀疏或模糊的情况下，学习效果可能较差。
3. **难以实现高阶智能的自然涌现**：尽管行为学派在模仿昆虫和群体智能方面取得了一定成果，但对于实现更高层次智能行为的涌现机制，行为学派难以给出明确解释和实现路径，因此在复杂任务的应用中受到限制。

尽管行为学派在模拟高级智能方面仍有不足，其在机器人技术、自动驾驶等实际应用中展示了极强的适应性和应用潜力。

2.5.5 三大学派的比较

人工智能的发展经历了显著的转型：从依赖人工规则的符号主义学派，到借助经验和反馈的联结主义与行为主义学派，AI 的核心方法从“推理期”和“知识期”逐渐过渡到“学习期”。这种变化反映了 AI 在应对复杂问题时，从自上而下的规则构建转向自下而上的经验学习。符号主义学派强调人工设计规则和知识库来解决问题，而联结主义学派和行为主义学派则更关注如何通过数据驱动的方式从经验中学习知识。人工智能发展过程中，**符号主义学派、联结主义学派和行为主义学派**分别从不同角度探索智能的本质。每一学派都有其独特的理论基础、代表方法和应用领域，也各有特色。符号学派主要关注智能的逻辑和推理过程，模拟智能软件的工作方式；联结学派则通过模仿大脑神经元结构，试图在“硬件”层面上还原智能；而行为学派则基于简单生物体与环境互动的模式，从动态环境中找寻适应性智能。

1. **符号主义学派 (传统人工智能)**：以逻辑和符号操作为基础，符号主义学派侧重于通过规则和知识库模拟人类的逻辑推理能力。其典型方法包括基于规则的专家系统，主要应用于知识明确、规则清晰的领域，如医疗诊断和金融分析。符号主义的优势在于其强大的可解释性，推理过程清晰可见，便于理解。然而，符号主义在处理复杂、不确定的环境时表现有限，因为它依赖于结构化的知识库和固定规则，难以应对环境的动态变化。符号主义的知识设计是**自上而下**的，主要通过逻辑设计和专家知识进行知识建模，但人类的知识和经验往往难以精确描述，并且复杂的推理过程增加了计算难度。
2. **联结主义学派**：借鉴神经科学，联结主义学派通过构建人工神经网络来模拟大脑的学习和感知机制。其代表模型包括卷积神经网络 (CNN)、递归神经网络 (RNN) 等，强调通过数据驱动的学习，在大规模数据中自动调整参数，广泛应用于图像识别、语音识别和自然语言处理等领域。联结主义学派的优点在于其强大的数据处理能力，可以自适应复杂数据环境；联结主义的知识形成是**自下而上**的，通过模拟神经网络结构涌现知识。然而，其“黑箱”性质导致可解释性不足，且训练过程依赖大量数据和计算资源。
3. **行为主义学派**：基于行为主义心理学和生物学对动物行为的研究，行为主义学派强调通过智能体与环境的交互来优化行为策略，其典型实现是强化学习。行为主义方法更适合动态和不确定环境中的智能体训练，例如自动驾驶、机器人控制和游戏对抗。行为主义的特点是其试错式学习机制，使得智能体在持续反馈中逐步优化策略。然而，其高维度环境中的学习效率较低，且高度依赖明确的奖励信号，难以实现对高阶智能的自然涌现。

学派间的比较：

- **理论基础**：符号主义学派依赖符号和逻辑推理，经由专家自上而下基于逻辑来构建知识系统；联结主义学派模仿神经网络的结构，自下而上实现知识的涌现；行为主义学派则从智能体的行动和环境反馈中寻找最优策略，构建出适应性的智能。三者代表了不同的智能观：符号主义偏向于逻辑和推理，联结主义倾向于模式识别和学习，行为主义重视试错与适应。
- **适用领域**：符号主义适用于规则明确、知识体系完善的场景；联结主义适合海量数据驱动的模式识别任务；行为主义则在动态、非结构化的环境中表现突出，如自动驾驶和机

器人控制。

- **** 优劣势 ****：符号主义具有良好的可解释性，但灵活性有限；联结主义具备强大的感知与模式识别能力，但对数据依赖性高，且可解释性不足；行为主义适应性强，但在高维度和奖励稀疏的环境下学习效率较低，难以实现复杂智能的涌现。

三大学派的各自特长和局限使得现代人工智能研究趋向于融合不同方法。例如，将符号逻辑引入神经网络以提升可解释性，或将强化学习与深度学习结合以应对复杂策略任务。通过综合符号推理、神经网络和交互学习的优势，现代 AI 逐渐朝着多模态、多层次的智能系统发展，在更多领域实现了实际应用。

表 2.1: 三大学派的比较

学派	主要观点	优势	局限性
符号主义	符号和规则模拟人类智能	擅长结构化问题和逻辑推理	难以处理非结构化和动态环境
联结主义	神经网络模拟大脑联结	数据驱动，自动学习	需要大量数据，解释性不足
行为主义	交互和反馈强化行为学习	适用于策略优化和动态决策	需要大量交互，目标不明确时效率低

表 2.2: 人工智能主要学派

符号主义学派	联结主义学派	行为主义学派
认知即计算 • 起源于 2300 年前，更关注哲学、逻辑学和心理学，将学习视为逆向演绎 • 使用符号、规则和逻辑来表征知识和进行逻辑推理 常用算法：规则和决策树	认知即网络 • 起源于上世纪 40 年代，专注物理学和神经科学 • 使用概率矩阵和加权神经元来动态地识别、分类、归纳 常用算法：深度学习神经网络、反向传播	感知-行动 • 起源于上世纪 40 年代 • 在遗传学、进化生物学、控制论的基础上，生成变化，为特定目标获取其中最优解 常用算法：遗传算法、蚁群算法、强化学习

三大学派的方法论

2.6 小结：危机与机遇中的 AI 发展之旅

纵观人工智能的成长历程，它既是一场理解智能的探索，也是一场应对挑战的征途。AI 的发展如同蜿蜒的河流，经历过激荡的浪潮，也度过了平静的深潭。在数次寒冬的洗礼后，AI 不断焕发新的生机，展示出危机中的无限机遇。每一次技术突破和危机，都塑造了 AI 的未来方向。1. 达特茅斯会议后的开创期（1956 年 - 1970 年代初）：1956 年的达特茅斯会议首次提出

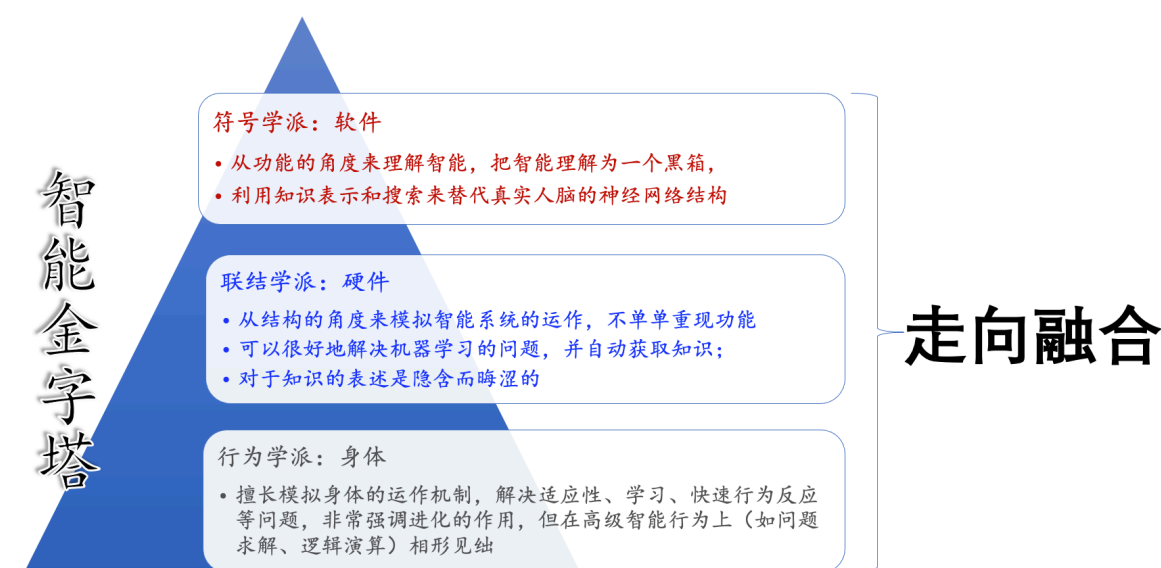


图 2.5: 智能金字塔

“人工智能”概念，开启了 AI 的黄金时代。研究者尝试用逻辑推理和符号操作来模仿智能，开发了 Logic Theorist 和 General Problem Solver 等符号主义 AI 的早期成果，虽为初探，却为智能探索奠定了基础。

2. 第一次 AI 寒冬（1974 年 - 1980 年代初）：由于符号主义 AI 无法解决现实问题的模糊性和不确定性，以及高昂的资源消耗，AI 研究在此期间陷入低谷，资金支持大幅削减。这场寒冬也促使研究者反思符号主义 AI 的局限性，重新思考智能的实现路径，为下一阶段的知识工程发展铺平了道路。

3. 专家系统和知识工程的兴起（1970 年代末 - 1980 年代中期）：第一次 AI 寒冬后，DENDRAL 和 MYCIN 等专家系统展示了 AI 在医疗、化学分析等领域的实际应用潜力，使 AI 逐渐从实验室走向商业应用，知识工程兴起。但由于专家系统过度依赖手动编码的知识和固定规则，逐渐暴露出适应性不足的局限。

4. 第二次 AI 寒冬（1980 年代末 - 1990 年代初）：专家系统的高昂维护成本和对环境的适应性限制使得全球的 AI 项目纷纷缩减以至于停滞，AI 再次进入低谷。但这场寒冬促使 AI 领域认识到，仅靠符号规则难以实现真正智能，推动了方法上的转型。

5. 联结主义和机器学习的崛起（1990 年代 - 2010 年左右）：以神经网络和数据驱动方法为基础，联结主义和机器学习的蓬勃发展，标志着 AI 从符号主义迈向数据驱动的新时代。机器学习逐渐成为主流方法，通过模式识别和大数据训练，使 AI 具备了在各种任务中自我改进的能力。1997 年，IBM 的“深蓝”战胜卡斯帕罗夫，这一里程碑事件展现了机器学习在复杂决策中的潜力，也宣告了数据驱动 AI 的初步胜利。

6. 深度学习的快速发展（2010 年代）：在计算能力提升和数据量增加的支持下，深度学习技术在图像识别、语音识别、自然语言处理等领域取得突破性进展，使 AI 再度迎来黄金期。2016 年，AlphaGo 战胜李世石，标志 AI 在复杂推理问题上的新高度。寒冬中的积累与反思，转化为深度学习这一数据驱动方法的强大推动力，成就了 AI 新时代的辉煌。

7. 大语言模型（LLM）与 AIGC 的崛起（2020 年代）：近年来，基于数十亿参数和海量数据训练的大规模语言模型（如 GPT 系列和 BERT）与 AI 生成内容（AIGC）技术迅速崛起。LLM 具备深度语言理解和生成能力，能进行逻辑连贯的文本生成，开创了更加自然的智能人机互动。AIGC 进一步拓展了 AI 的创作边界，使 AI 从信息回答者进化为创意合作伙伴，广泛

应用于创意内容生成、广告、教育和娱乐领域。在危机与机遇交织的 AI 发展史中，每一次危机都意味着新的机遇。两次寒冬带来低谷，但也促使 AI 领域在反思中寻找新方向，逐渐摆脱了对符号逻辑的单一依赖，向数据驱动和自适应学习的方向迈进。知识工程为 AI 的应用打开了大门，而机器学习和深度学习的崛起则奠定了现代 AI 的基础。今天，AI 的视野已从实验室扩展到生活的方方面面，从模仿人类思维到自主学习，从专注特定领域到应对复杂决策。AI 的历程是一部充满智慧与勇气的进化史，展现了智能的无限潜力。我们期待 AI 继续向前，为人类揭示智能的更多可能性，并引领我们在未知中发现更广阔的世界。

参考文献

- [1] “A Logical Calculus of Ideas Immanent in Nervous Activity”. In: *Bulletin of Mathematical Biophysics* 5 (1943), pp. 127–147.
- [2] Yoshua Bengio. “Practical recommendations for gradient-based training of deep architectures”. In: *Neural networks: Tricks of the trade: Second edition*. Springer, 2012, pp. 437–478.
- [3] George Cybenko. “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of control, signals and systems* 2.4 (1989), pp. 303–314.
- [4] Kunihiro Fukushima and Sei Miyake. “Neocognitron: A new algorithm for pattern recognition tolerant of deformations and shifts in position”. In: *Pattern recognition* 15.6 (1982), pp. 455–469.
- [5] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Multilayer feedforward networks are universal approximators”. In: *Neural Networks* 2.5 (1989), pp. 359–366.
- [6] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Universal approximation of an unknown mapping and its derivatives using multilayer feedforward networks”. In: *Neural networks* 3.5 (1990), pp. 551–560.
- [7] David H Hubel and Torsten N Wiesel. “Receptive fields and functional architecture of monkey striate cortex”. In: *The Journal of physiology* 195.1 (1968), pp. 215–243.
- [8] Han Jiawei, Kamber Micheline, and Pei Jian. *Data Mining: Concepts and Techniques*. -3rd. Morgan Kaufmann, 2011.
- [9] Fukushima K and Miyake S. “Neocognitron: A self-organizing neural network model for a mechanism of visual pattern recognition”. In: *Competition and cooperation in neural nets*. Springer, Berlin, Heidelberg, 1982, pp. 267–285.
- [10] Diederik P Kingma and Jimmy Ba. “Adam: A method for stochastic optimization”. In: *arXiv preprint arXiv:1412.6980* (2014).
- [11] Yann Le Cun, Boser Brian, and et al. Denker John S. “Handwritten digit recognition with a back-propagation network”. In: *Advances in neural information processing systems*. 1990, pp. 396–404.
- [12] Yann Le Cun et al. “Handwritten digit recognition: Applications of neural net chips and automatic learning”. In: *Neurocomputing: Algorithms, Architectures and Applications*. Springer. 1990, pp. 303–318.
- [13] Yann LeCun et al. “Backpropagation applied to handwritten zip code recognition”. In: *Neural computation* 1.4 (1989), pp. 541–551.

- [14] Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.
- [15] Minsky Marvin and A Papert Seymour. “Perceptrons”. In: *Cambridge, MA: MIT Press* 6.318–362 (1969), p. 7.
- [16] Warren S McCulloch and Walter Pitts. “A logical calculus of the ideas immanent in nervous activity”. In: *The bulletin of mathematical biophysics* 5 (1943), pp. 115–133.
- [17] Tom Mitchell. *Machine Learning*. McGraw-Hill Education, 1997.
- [18] Michael Nielsen. *Neural Networks and Deep Learning*. Accessed: 2025-03-03. 2015. URL: <http://neuralnetworksanddeeplearning.com/>.
- [19] Hastie T, Tibshirani R, and Friedman J. 统计学习基础 (*The Elements of Statistical Learning*). 北京: 世界图书出版公司, 2015.
- [20] Alan Turing. *Computing Machinery and Intelligence*. Oxford, 1950.
- [21] Vladimir Naumovich Vapnik. 统计学习理论的本质. 清华大学华夏出版社, 2000.
- [22] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).
- [23] 于剑. 机器学习—从公理到算法. 北京: 清华大学出版社, 2017.
- [24] [意] 翁贝托·博塔兹尼. 余婷婷译. 尖叫的数学—令人惊叹的数学之美. 湖南科学技术出版社, 2021.
- [25] 周志华. 机器学习. 清华大学出版社, 2016.
- [26] Vladimir N. Vapnik. 张学工译. 统计学习理论的本质. 北京: 清华大学出版社, 2000.
- [27] 张玉宏. 深度学习之美—AI 时代的数据处理与最佳实践. 电子工业出版社, 2018.
- [28] 张玉宏. 深度学习之美: AI 时代的数据处理与最佳实践. 电子工业出版社, 2018.
- [29] Tom Mitchell. 曾华军等译. 机器学习. 北京: 机械工业出版社, 2002.
- [30] 王珏, 周志华, 周傲英. 机器学习及其应用. Vol. 4. 清华大学出版社有限公司, 2006.

第二部分

学习的逻辑

第三章 人工智能的三种思维：从理论到应用的跨越

本章提要

□ 数学思维

□ 工程思维

□ 计算思维

发展数学理论是一回事, 实现它又是另一回事。

——约翰·冯·诺伊曼

来自应用, 回归应用: 从数学思维到算法思维到工程思维数学思维:

-理想情形 -存在性 -可构造性算法思维 -算法复杂度 -算法优化 -稳定性工程思维 -允许误差 -大规模问题 -鲁棒性 -经济效应举一些数值算法稳定性的反例. 无论何种学问, 构建理论是一回事, 将其付诸实现则是另一回事。**AI** 的研究同样如此。**AI** 研究的复杂性不仅源于理论的深度, 还在于将其应用于现实世界的挑战。解决一个 **AI** 问题往往要求从**数学思维、算法思维和工程思维**三个方面进行多角度思考。数学、算法和工程三种思维方式既互相联系, 又各有侧重, 从不同层面定义了 **AI** 的研究框架: 数学思维为理论提供框架, 算法思维将其具体化为可计算的实现方案, 工程思维则确保方案在现实中的可行性和应用价值。数学、算法和工程三种思维共同构成了全面理解和解决 **AI** 问题的整体方法论。本章将深入探讨这些思维方式, 理解它们如何在不同层面上塑造现代智能系统的实现路径。

3.1 数学思维：定义和可能性的框架

数学思维是 **AI** 研究的起点, 为定义、分析和解决智能与学习的基本问题提供了框架。通过数学思维, 研究者得以抽象出智能和学习的核心科学问题, 构建符合逻辑且可行的智能模型, 并进一步探索其解的存在性和特性。对于 **AI** 而言, 这不仅意味着找到一组解, 更是考量这些解在理想条件下的**存在性、唯一性和构造性**, 确保从逻辑上能够定义和证明智能的实现可能性。主要体现在以下几个方面:

- 理想情形**: 数学思维的“理想情形”, 是指研究者通常在简化、理想化的环境中建立模型, 以便能够从纯粹数学的角度分析智能系统的潜在行为和特性。例如,
 - 线性模型的假设**: 在早期的机器学习和回归分析中, 常假设数据间关系是线性的, 这使得问题的分析和求解可以通过线性代数完成, 虽然实际中数据往往具有非线性特性。
 - 凸优化假设**: 在神经网络训练中, 我们假设损失函数为凸函数来分析算法收敛性和解的稳定性, 因为凸函数可以保证找到优化问题存在全局最优解。然而实际中的损失函数往往是非凸的, 导致全局最优解难以找到, 但凸优化理论仍为非凸优化的启发式方法提供了基础。

- **无噪声数据假设**：在信号处理或统计学习中，我们通常假设数据没有噪声，即数据完全准确无误。这个理想情形让我们可以专注于分析算法的基本特性和效果。尽管现实数据几乎总是带有噪声，这一假设帮助我们更清楚地理解算法在“最优条件”下的性能。
- **完全信息假设**：在强化学习中，通常假设智能体可以完全观察到状态的所有信息（完全信息），这被称为马尔可夫决策过程（MDP）。这一理想假设使得我们能够推导出解决 MDP 的核心算法，如 Q-learning 和动态规划，从而允许智能体基于当前状态做出最优决策。尽管在实际应用中，智能体常只能观测到部分信息，信息往往是不完全的。因此需要用部分可观测马尔可夫决策过程（POMDP）来建模。
- **均匀分布假设**：在随机算法或统计推断中，均匀分布是理想化的假设之一。例如在蒙特卡洛模拟中，假设数据或样本来自均匀分布以简化概率计算。均匀分布假设常常使得复杂问题更易于处理，尽管在现实中，数据的分布通常不均匀，这一理想情形仍为其他分布下的理论分析提供了起点。

这些理想情形帮助研究者在数学框架下构建简化模型，使得智能系统的行为可以被有效分析。这些理想化的假设虽然不完全适用于现实中的复杂性，但为现实条件下的实际研究提供了启发。

2. **存在性问题**：在数学中，解的存在性是算法设计的前提，它告诉我们所研究的问题是否具备可解性。存在性解答了“问题是否有解”的疑问，是算法设计的理论依据。例如：
 - **支持向量机研究中的最优超平面**：在分类问题中，支持向量机通过优化理论证明最优超平面的存在性，确保能够找到一个分隔两类数据的最佳边界。
 - **多目标优化问题的帕累托前沿**：在多目标优化中，通过证明帕累托前沿的存在性，确定可以找到在各目标之间无明显优劣关系的解集。这种存在性证明在多目标场景下尤为关键。
 - **神经网络中的逼近定理**：神经网络的万能逼近定理表明，只要层数和神经元数量足够多，前馈神经网络就能够逼近任意连续函数。这一存在性定理确保了神经网络在理论上具备学习复杂映射关系的能力，为构建强大的模型提供了可能性。
3. **可构造性**：数学思维不仅关注解的存在，还注重解的具体实现可能性，即可构造性。这确保所设计的算法在理论上可行，且实际可实现。例如：
 - **梯度下降法**：在最优化问题中，存在最优解并不足够，还需有具体方法找到该解。梯度下降法是一种有效的数值方法，通过逐步调整参数接近最优解。可构造性使梯度下降能够成为实际优化问题的核心算法。
 - **数独问题的解构造**：存在性证明表明数独问题有解，但要找到这个解则需要特定的构造方法。回溯算法是一种常见的求解数独问题的方法，保证了解的具体实现可行性。
 - **矩阵分解的可实现性**：在主成分分析（PCA）中，矩阵的特征值和特征向量的存在性说明解是存在的，但通过特征分解构造出主成分的实际计算方法则体现了解的可构造性。这让 PCA 能够在实际中应用于数据降维。

可构造性这一思想是从数学思维向算法思维的自然过渡，使得理论上的解在现实中成为可计算、可操作的实际方法。

数学思维注重理论上的严谨性与可证明性。这确保 AI 能够在抽象层面上构建逻辑严密且可行的智能模型，并在理想条件下分析智能系统的行为，使得后续的计算实现与工程应用成为可能。

3.2 算法思维：从理论到可计算的路径

数学思维可以帮助我们抽象问题的本质特征，并提供理论基础，为智能系统的研究提供理论框架。而算法思维则是将这些理论转化为可计算、可执行的步骤与逻辑流程的关键。在 AI 研究中，算法思维至关重要，它将抽象的数学模型转化为可以实际执行的计算过程，并优化这些过程，使得它们不仅可行而且高效。算法思维是数学理论向现实转化的关键步骤，它不仅关注算法的准确性，还要考虑其效率、稳定性和资源利用，使得算法可以在有限计算资源下实际执行。算法思维主要体现在算法复杂度、优化和稳定性三个方面。

3.2.1 算法复杂度

复杂度分析是算法设计中的核心问题，尤其是在大数据和实时计算需求下。AI 模型往往面临大量计算，算法复杂度直接影响算法在处理大规模数据时的可行性和效率。算法复杂度通常以**时间复杂度**和**空间复杂度**来衡量。时间复杂度描述了算法在数据规模变化时所需的运行时间，而空间复杂度则描述了算法在执行过程中所需的存储空间。

- 1. 时间复杂度：**时间复杂度反映了算法执行过程中，随着输入数据规模增长所需的运行时间。例如，在图像识别任务中，卷积神经网络（CNN）逐层提取图像特征，处理的复杂度随着网络层数和输入图片的分辨率增长而增加。通常情况下，时间复杂度衡量使用 O 记号表示：如 $O(n)$ 、 $O(n^2)$ 等， $O(n)$ 代表算法的执行时间与输入数据量成正比，而 $O(n^2)$ 代表执行时间随输入量的平方增长。在实际应用中，优化时间复杂度是确保 AI 模型可以在合理时间内完成任务的前提，这在需要实时反应的系统中尤为重要。

- 图像处理任务中的时间复杂度：**在图像分类和物体识别任务中，模型需要处理数百万像素，提取特征用于分类任务。卷积神经网络（CNN）通过引入局部感受野和共享权重的机制，大大降低了计算复杂度，使得大规模数据集上的高效图像识别成为可能。CNN 的优化不仅使得算法适用于海量图片处理，也为自动驾驶中的实时物体识别提供了技术基础。随着数据量和图像分辨率的提升，CNN 的卷积层和池化层操作将耗费大量计算时间。如果模型的时间复杂度过高，将难以满足实时处理的需求，尤其是在自动驾驶等需要实时反馈的应用场景中。自动驾驶中的物体识别算法必须在极短时间内完成复杂场景的分析，确保车辆能够快速决策。通过降低模型的时间复杂度或采用轻量化网络结构（如 MobileNet），可以提升处理速度，保障应用在实时系统中的响应速度。

- **推荐系统中的时间复杂度：**推荐系统在处理用户和商品数据的海量组合时，常面临计算复杂度的挑战。例如，协同过滤算法在传统情况下计算复杂度较高，需要遍历大量数据。随着数据量增大，计算的时间复杂度可能达到 $O(n^2)$ 或更高，使得推荐系统的实时计算难以实现。为此，一些优化方法，如矩阵分解和基于图的推荐算法，在复杂度控制上做出了优化，使得算法可以在大型电商平台和社交媒体中提供实时推荐。
 - **大规模搜索算法：**搜索问题是经典的 AI 任务，比如导航系统的路径规划。传统的广度优先搜索（BFS）和深度优先搜索（DFS）在复杂度上表现不佳。A* 算法通过结合启发式搜索，在计算复杂度上得到了显著提升，确保算法能够在较短时间内完成最优路径的搜索。A* 算法应用于无人驾驶和地图导航中，使其能够快速找到从当前位置到目标的最优路径。
2. **空间复杂度：**空间复杂度描述了算法在运行中需要占用的内存资源，特别是在大规模数据处理时，空间复杂度的优化至关重要。在 AI 任务中，许多算法需要在处理时临时存储中间数据和模型参数，如果空间复杂度过高，将会占用大量存储资源，导致效率低下甚至无法执行。
- **深度学习中的空间复杂度：**在深度神经网络中，每一层都需要存储大量权重和中间结果（如激活值）。随着模型的层数和参数数量增加，空间复杂度也会急剧上升，通常呈 $O(n^2)$ 或更高的增长趋势。例如，在自然语言处理任务中，BERT 等预训练模型含有数十亿个参数，其空间复杂度极高，要求高配置的硬件资源支持。为了降低空间复杂度，研究者提出了模型剪枝、量化和蒸馏等方法，通过压缩模型参数来降低内存需求，使得这些模型可以在移动设备和嵌入式系统上有效运行。
 - **动态编程中的空间复杂度优化：**动态编程在许多 AI 算法中被广泛应用，比如在处理最短路径或最优策略的问题时。传统动态编程的空间复杂度往往较高，因为它存储了大量中间结果。为了解决这一问题，优化动态编程算法往往使用滚动数组等技巧，动态更新和覆盖原有存储，减少所需内存，达到空间复杂度的优化。例如，在自然语言处理中，使用滚动数组结构可以显著降低序列处理任务的空间需求，使模型更高效。

算法复杂度在 AI 算法设计中扮演着重要角色。通过分析和优化时间复杂度和空间复杂度，算法思维能够保障 AI 模型在大规模数据处理中的可行性和高效性。在图像识别、推荐系统、深度学习模型和动态编程等应用中，复杂度分析为算法的性能优化提供了有力的指导，使得 AI 能够在更大规模和更复杂的环境中得到广泛应用。优化是 AI 算法设计的核心，特别是在机器学习和深度学习中。优化算法不仅提升了效率，还使得 AI 模型在不同任务中的表现更加优良。算法优化直接关系到模型的精度和速度，通过减少算法运行时间或提升算法性能，确保模型更加高效、准确。在 AI 领域，优化问题无处不在。

- **深度神经网络训练：**反向传播（Backpropagation）算法是深度学习模型训练的基础，需要对神经网络中的每一个参数进行优化。传统的随机梯度下降（SGD）会更新数百万个参数以最小化损失函数，但收敛速度较慢。Adam 优化器、RMSprop 等改进算法通过自

适应调整学习率，显著提升了收敛效率并降低了陷入局部最优解的风险。优化算法不仅减少了训练时间，还提高了模型的泛化能力，使得深度学习可以在诸如语言翻译、图像分类等任务中取得更高的准确率。

- **强化学习中的策略优化**：在强化学习中，优化算法同样起着关键作用。比如 Q-learning 是一种经典的算法，用于离散环境中的策略优化。然而在连续和高维环境中，这种方法不适用，研究者采用了深度强化学习（如 DQN 和 PPO），通过神经网络来逼近 Q 值，使得强化学习可以在复杂的任务中表现出色。策略优化提升了 AI 系统在动态环境中的适应性，被广泛用于游戏 AI、机器人控制等领域。
- **机器学习中的模型压缩**：在移动设备和嵌入式系统中，为了在有限的硬件资源下实现 AI 应用，研究者开发了模型压缩技术，如剪枝（Pruning）和量化（Quantization）。这些方法通过减小模型规模或精度，显著减少了计算资源的需求。优化算法在资源有限的环境下保证了模型的性能，使得图像处理、语音识别等任务可以高效运行在移动端设备上。
- **强机器学习中的模型压缩**：在移动设备和嵌入式系统中，为了在有限的硬件资源下实现 AI 应用，研究者开发了模型压缩技术，如剪枝（Pruning）和量化（Quantization）。这些方法通过减小模型规模或精度，显著减少了计算资源的需求。优化算法在资源有限的环境下保证了模型的性能，使得图像处理、语音识别等任务可以高效运行在移动端设备上。

AI 算法的稳定性决定了 AI 模型在不确定环境下的可靠性和鲁棒性。稳定性分析关注算法在不同输入或噪声干扰下保持输出一致性的能力。AI 应用常在真实世界数据中遇到噪声和变化，因此稳定性在 AI 算法中的作用至关重要。

- **金融预测中的噪声处理**：在金融预测模型中，数据中常常充满噪声，市场波动的微小变化会对模型产生巨大影响。稳定性在此类模型中尤为重要，因为市场数据的噪声可能导致预测不准确。研究者们采用正则化技术（如 L2 正则化）来增加模型的鲁棒性，从而减少对噪声的敏感性，以提升金融预测的稳定性。
- **图像识别的抗扰性**：图像识别模型需要对噪声和输入扰动具备一定的抗扰性。例如，在自动驾驶应用中，摄像头捕获的图像可能受到光照、天气等因素的影响。通过使用数据增强技术，如随机旋转、剪切和颜色抖动，可以在训练时加入不同的场景变化，使模型在不同环境下表现稳定。抗扰性提升了算法的鲁棒性，保证了图像识别在多变环境下的性能。
- **自然语言处理中的错词纠正**：在自然语言处理（NLP）应用中，输入的文本数据可能存在拼写错误、口语化表述等问题。若模型的稳定性不佳，则可能因轻微的输入错误而产生误判。通过数据增强或错词处理技术，模型可以识别不同拼写变体并正常输出，提升模型在不完美数据上的表现。这种稳定性确保了语音助手和智能客服系统的回答更符合用户需求。

通过算法思维，AI 研究不仅将理论转化为可计算的流程，还推动了算法的创新与优化。算法思维为 AI 模型的实现提供了系统化路径，奠定了模型在现实环境中的计算基础。通过控制算法复杂度、运用优化技术以及设计稳定性保障，算法思维为 AI 模型在多样化应用中的高效执行提供了坚实基础。从图像识别、推荐系统到金融预测和自动驾驶，算法思维驱动了 AI 从理

论到实践的系统转化，确保了模型的高效性和可靠性，为其在实际场景中的应用奠定了可靠保障。

3.3 工程思维：从可计算到可用的实际落地

工程思维是 AI 从理论和算法走向现实应用的关键一步，是 AI 算法应用到实际中的最终环节。数学和算法的严谨性使 AI 模型具备了逻辑上的可行性和计算上的可操作性，而工程思维则进一步推动了 AI 的实际应用，使之能够在多变的环境中高效、可靠地运行，并满足实际应用中的多种约束。工程思维的核心在于将可计算的算法转化为可用的产品和服务，确保 AI 系统不仅具备计算能力，还在真实场景中展现出足够的适应性和鲁棒性。工程思维面向大规模可用的解决方案，所以它关注如何在复杂的现实环境中有效落地算法，确保模型在真实场景中不仅能运行，而且高效、可靠，并满足实际应用中的各种约束。AI 系统在面对真实世界时，不仅要完成预设任务，还要应对复杂多变的环境。工程思维因此强调对实际应用需求的适应，包括适度误差、大规模计算、鲁棒性、可扩展性、成本效益等方面，使得 AI 系统在应对现实复杂性时具有稳定性、扩展性和经济性。这一思维推动将理论模型和技术转化为可以普遍应用的大规模解决方案，真正满足不同领域的实际需求。

- **误差容忍与容错性：**在实际应用中，完全精确的计算通常既不必要也不经济，因此允许一定范围的误差成为工程优化的必然。在面对真实世界的的数据时，精确解往往不如近似解更具实际价值。

适当的误差容忍不仅降低了成本，还使得系统更加高效。为了提高速度和处理复杂问题的灵活性，工程思维鼓励在确保总体决策质量的前提下接受一定误差，这种做法使 AI 系统在处理非结构化数据时更加实用。例如，在**图像识别**任务中，模型可能因为噪声或光线的变化而产生轻微偏差，只要这种误差不影响整体决策，就可以在工程应用中接受。这种误差容忍的思想是图像识别模型能够更快速地处理图像，实现实时响应。在**自动驾驶**中，虽然系统要求高精度的物体识别，但在特定场景中允许适度误差，以提高系统的响应速度，使车辆能够迅速应对环境变化，从而兼顾精度和实时性。在**推荐系统**中，算法并不需要找到完美的推荐，只要提供与用户兴趣足够相关的内容即可。因此，推荐算法在计算上允许一定误差，以确保系统能在用户实时使用中快速响应，提供流畅的体验。同样地，语音识别系统在噪声、口音等因素影响下，难以保证绝对准确。语音识别通常接受小误差，使得语音识别在不牺牲用户体验的情况下维持高效的响应速度，避免因追求精度而导致反应速度过慢。举例：微信的语音识别 这种允许适度误差的工程思维在 AI 系统的设计中至关重要，它在确保足够精度的同时，使系统具备了适应性与高效性，在精度与实时性之间找到最佳平衡。

- **大规模问题的高效处理：**现实应用的规模往往远超实验室环境，在现实场景中，AI 系统常常面临大规模数据的处理需求。为了实现大规模应用中的高效响应，工程思维通过分布式计算和数据并行处理技术，将算法复杂度优化和资源合理调度结合在一起，使得系统能够在短时间内完成数据分析和结果返回，实现了大规模数据处理的高效性。

例如，在**搜索引擎**中，算法需快速处理数十亿条网页数据，以秒级响应用户搜索请求。为了实现这一点，工程思维通过分布式计算、并行处理等技术，使算法能够在极短时间内完成大规模数据分析。在**推荐系统**中，处理数百万用户和上亿个商品的海量数据是系统的常态。推荐系统需要在最短时间内生成个性化推荐结果，这不仅依赖于优化算法的设计，还需要使用分布式计算技术来支持大规模并行处理，将复杂任务拆解成小块，以应对海量数据处理。这不仅提升了系统的实时响应能力，还使得模型能够适应用户需求的动态变化。在**图像分类**和**自然语言处理**任务中，处理大型图像数据集（如 ImageNet）或长文本数据也面临相似的需求，单一计算节点的资源通常不足以应对这些任务。因此工程上通常采用分布式训练，将模型任务分配到多节点、多服务器环境下并行处理，以大幅提升系统的整体计算能力和响应效率。这种方法为图像处理、文本处理等大规模应用提供了计算基础，使得系统能够高效应对海量数据。

- **鲁棒性设计**：现实的 AI 应用面对的是动态、不可控的环境。工程思维要求 AI 系统具有鲁棒性，即在面对数据波动、异常输入、设备差异、环境干扰及复杂环境时，仍然能够保持稳定和可靠。鲁棒性是工程设计中的一个关键因素。以**自动驾驶系统**为例。自动驾驶系统必须在各种天气、复杂路况和突发交通状况中保持稳定运行，这对鲁棒性提出了极高要求。自动驾驶系统集成了多传感器（如摄像头、激光雷达、雷达）以提供冗余信息，即便某一传感器受干扰，其他传感器也能保障系统运行。此外，通过大量极端场景的仿真训练和场景测试，系统获得应对突发状况的能力，使其在毫秒级时间内完成环境识别、路径规划和避障决策。这种鲁棒性设计确保了自动驾驶在复杂环境中的安全性和可靠性。在**医疗 AI** 应用中，系统需要在面对患者数据的多样性和变化时保持高精度。不同医院的设备和数据采集方式各异，导致患者数据可能受到噪声或设备偏差的影响。为确保 AI 模型在多样化输入下的稳定性和准确性，工程思维通过鲁棒性测试和参数调整，使系统能够应对现实中的数据波动和异常情况。例如，通过鲁棒性设计，AI 模型能够针对噪声和设备差异进行优化，即便在不同医院或检测设备的条件下，依然能够提供高效、精确的结果，从而保障医疗 AI 系统在动态、复杂的现实环境中的可靠表现。**自动驾驶系统**是另一个典型案例。自动驾驶需要应对复杂的路况和多变的环境因素，包括雨天、夜晚以及突如其来的障碍物等。这就要求系统在极端条件下依然能够迅速做出可靠的决策。通过工程思维中的鲁棒性设计，系统在不同环境下均能稳定运行，从而保障了自动驾驶的安全性和可靠性。在**金融预测系统**中，市场数据波动剧烈，且受多种因素的影响。为了避免因市场剧烈波动导致的系统失效或错误决策，AI 模型必须具备足够的鲁棒性。工程思维在这里通过确保系统在市场波动中仍能保持一致性表现，从而使 AI 模型在极端经济条件下也能做出稳健的预测，提升了系统的可靠性和实用性。综上，通过对鲁棒性的重视和设计，工程思维确保了 AI 系统在复杂且不确定的环境中依然保持高效和稳定的表现，使得 AI 在实际应用中更加可靠和可用，能够在不同环境下正常运行，应对各种现实干扰。
- **经济效益与资源效率**：工程思维不仅关注 AI 系统的性能，还重视资源利用率和成本效益，以确保系统在真实应用中既高效又经济。例如，在**边缘设备**上的 AI 应用（如智能

家居设备)对计算和存储资源极其有限,需要设计简化版的模型,确保高效运行的同时控制成本。此外,通过低精度计算或量化方法,工程思维可以在性能和资源消耗之间取得平衡。例如,在**边缘设备**上(如智能家居或便携式健康监测设备)运行的**AI**应用,计算和存储资源非常有限。工程思维在此强调模型的轻量化,通过**低精度计算、模型量化和剪枝**等优化技术,降低功耗和存储需求,使得**AI**应用在受限资源条件下也能高效运行,从而实现规模化应用的经济可行性。在**医疗 AI**领域,成本效益同样至关重要。过高的系统成本会限制技术的普及,而过低的资源配置则可能影响性能,因此工程思维要求在性能和经济性之间找到合理平衡,使系统既具备商业价值,又具备可持续性。这种成本与资源的平衡,使得**AI**技术在多样化应用场景中得以普及,满足了不同行业对可靠性和经济性的需求。

工程思维让**AI**技术从实验室中走向真实场景,为其提供了落地所需的弹性和适应力,使得**AI**系统在多变的现实环境中稳定、可靠地运行,从而解决多样化的实际问题。通过允许适度误差、处理大规模数据、提升系统鲁棒性以及优化经济效应,**AI**系统实现了从可计算到可用的过渡,从理论模型转化为具有实用价值的技术系统,推动了**AI**技术在社会中的广泛应用。从自动驾驶、智慧医疗到工业自动化,工程思维赋予**AI**系统面对现实挑战的应变能力,确保其在各类场景中高效运作,让**AI**技术真正服务于社会需求,成为各行业不可或缺的技术支撑。

3.4 两个从数学思维到工程思维的例子

在本节中,我们将介绍推荐系统和数字图像处理,以此说明数学思维、算法思维和工程思维如何在**AI**系统设计和实现中层层递进、互相支撑。

3.4.1 数字图像处理：从理论到实际应用的三重思维

SVD 分解.....以数字图像处理为例,我们可以从数学思维、算法思维和工程思维三个方面来探讨其在**AI**系统中的实现过程。假设应用场景是一个**自动驾驶系统中的实时图像分类任务**,要求系统能识别交通信号、行人、车辆等关键元素。

3.4.1.1 数学思维：图像处理的理论基础

在数字图像处理任务中,数学思维定义了图像数据的处理方式以及背后的理论框架。数学思维关注图像的特性和转化,将复杂的视觉任务抽象为特征提取和分类问题。

图像表示与矩阵运算

为便于阅读,本节(图像表示与矩阵运算)承接上节(数学思维:图像处理的理论基础),先交代差异与共同点,再聚焦本节关键问题。

图像在数学上被表示为矩阵，每个像素点对应一个矩阵元素。数学思维通过线性代数和矩阵变换帮助我们定义图像的基础处理方式。例如，通过卷积运算提取图像的边缘、角点和纹理特征，这些特征对于图像分类至关重要。

卷积操作的存在性与稳定性

为便于阅读，本节（卷积操作的存在性与稳定性）承接上节（图像表示与矩阵运算），先交代差异与共同点，再聚焦本节关键问题。

卷积神经网络（CNN）使用了大量的卷积运算以提取图像特征。数学思维帮助我们证明卷积核在一定条件下能够提取稳定、有效的特征。例如，边缘检测的卷积核在任意位置都可以应用于图像，并且能够稳健地捕捉到物体边缘，这使得卷积在图像特征提取中的应用成为可能。

概率建模与分类

为便于阅读，本节（概率建模与分类）承接上节（卷积操作的存在性与稳定性），先交代差异与共同点，再聚焦本节关键问题。

在图像分类任务中，我们通常使用概率建模来判断图像所属的类别。数学思维通过概率论确保分类器的准确性和一致性。例如，使用最大似然估计（MLE）来判断像素特征与目标类别的相关性，为图像分类奠定了理论基础。

3.4.1.2 算法思维：从理论模型到具体算法

在算法思维层面，我们将数学模型转化为具体的图像处理算法，以确保其在复杂的数据中具有可计算性、效率和鲁棒性。

时间复杂度

为便于阅读，本节（时间复杂度）承接上节（算法思维：从理论模型到具体算法），先交代差异与共同点，再聚焦本节关键问题。

图像处理任务通常需要在短时间内完成大量计算，时间复杂度成为关键问题。例如，CNN中的卷积操作通过共享权重来降低计算复杂度，使得算法能够在合理时间内处理大量图像数据。这一优化使得CNN在自动驾驶中的实时图像处理成为可能。

算法优化

为便于阅读，本节（算法优化）承接上节（时间复杂度），先交代差异与共同点，再聚焦本节关键问题。

为了处理大量图像数据并确保分类的准确性，算法思维引入了优化技术。比如，在训练 CNN 模型时，采用随机梯度下降（SGD）等优化算法不断调整参数，使模型的特征提取能力不断提高，从而达到更好的分类效果。这种优化不仅提高了分类的精度，还提升了模型的计算效率。

算法稳定性

为便于阅读，本节（算法稳定性）承接上节（算法优化），先交代差异与共同点，再聚焦本节关键问题。

图像处理系统在遇到不同光照、天气等条件变化时仍需保持稳定性能。算法思维通过数据增强和正则化技术，提高系统在不同场景下的稳定性。比如在自动驾驶中，对图像进行不同角度、亮度的调整，使模型更具有鲁棒性，避免因光照变化而影响识别效果。

3.4.1.3 工程思维：图像处理的实际落地

工程思维关注如何将图像处理算法部署在自动驾驶系统中，使得模型在真实环境中高效、可靠地工作。

允许误差

在自动驾驶中，图像识别算法需要高精度，但并不需要绝对的像素级精确度。工程思维允许对图像处理任务设定一定的误差范围，以提升系统响应速度。比如，在车辆检测中，模型只需定位到车辆的大致位置，无需完全精确到每个边缘点。这种误差容忍确保了图像处理算法的实时性，提升了系统的整体性能。

为便于阅读，本节（允许误差）承接上节（工程思维：图像处理的实际落地），先交代差异与共同点，再聚焦本节关键问题。

大规模计算

为便于阅读，本节（大规模计算）承接上节（允许误差），先交代差异与共同点，再聚焦本节关键问题。

自动驾驶系统通常会接收和处理多个摄像头的的数据，这对系统的计算能力提出了挑战。工程思维通过分布式计算和边缘计算等技术，确保系统能够实时处理海量图像数据。例如，通过将计算任务分配到不同的边缘节点，可以快速处理多个摄像头的图像，从而实现实时监控和决策。

鲁棒性

为便于阅读，本节（鲁棒性）承接上节（大规模计算），先交代差异与共同点，再聚焦本节关键问题。

自动驾驶面临多变的环境，比如雨天、雾天、夜间等不同光照和天气条件。工程思维在图像处理系统的设计中加入了鲁棒性考虑，确保系统在多样化条件下依然可靠。通过预训练不同环境下的图像数据，模型在遇到特殊情况时能够适应并作出正确的响应，避免误识别或漏识别情况的发生。

经济效应

为便于阅读，本节（经济效应）承接上节（鲁棒性），先交代差异与共同点，再聚焦本节关键问题。

在自动驾驶的硬件中运行图像处理算法需要考虑成本。工程思维通过模型压缩、量化等方法，将算法转化为轻量化的模型，确保其在有限的硬件资源上也能高效运行。例如，在车载设备中，轻量化的模型降低了存储和能耗需求，为规模化生产提供了经济性支持。

3.4.2 总结

在数字图像处理任务中，**数学思维**为图像处理定义了理论框架，从矩阵表示到概率建模，为图像分类和特征提取提供了基础；**算法思维**则进一步将这些理论转化为可行的计算步骤和优化策略，提高了系统的可计算性和执行效率；**工程思维**确保图像处理算法能够应对复杂的现实场景，确保在成本受限、环境多变的情况下依然具备稳定性和经济性。

3.4.2.1 数学思维：推荐系统的理论建模

推荐系统的核心任务是为用户找到他们可能喜欢的内容（例如商品、电影、音乐等），这是一个典型的预测问题。数学思维帮助我们在抽象层面上定义这个问题，通过对用户行为的建模找到推荐算法的理论基础。常见的数学方法包括矩阵分解和向量空间模型：

矩阵分解

为便于阅读，本节（矩阵分解）承接上节（数学思维：推荐系统的理论建模），先交代差异与共同点，再聚焦本节关键问题。

通过构建用户与项目的评分矩阵，推荐问题可以被表示为矩阵分解问题。数学思维定义了这一评分矩阵的特性和结构，并通过奇异值分解（SVD）等技术探索其潜在特征。**存在性**问题在此处确保了该矩阵可以被有效分解，以提取出每个用户和物品的潜在特征向量。

相似性度量

为便于阅读，本节（相似性度量）承接上节（矩阵分解），先交代差异与共同点，再聚焦本节关键问题。

推荐系统依赖于用户和项目之间的相似性来生成推荐。例如，余弦相似性、皮尔逊相关系数等方法用于衡量不同用户或物品的相似性。数学思维在此为定义相似性度量提供了理论依据，确保系统在计算相似度时具备合理性。

概率模型

为便于阅读，本节（概率模型）承接上节（相似性度量），先交代差异与共同点，再聚焦本节关键问题。

例如在基于概率图模型的推荐系统中，我们可以使用贝叶斯模型，预测用户对未接触项目的评分概率。这种模型确保系统具备合理的推荐结果的概率解释。

在算法思维的层面上，我们将推荐系统的数学模型转化为实际可行的算法，并对算法复杂度、优化和稳定性进行考量。

算法复杂度

为便于阅读，本节（算法复杂度）承接上节（概率模型），先交代差异与共同点，再聚焦本节关键问题。

推荐系统需要处理大量用户和项目的数据，计算复杂度成为一个关键问题。矩阵分解的直接计算复杂度很高，算法思维通过优化，如使用**随机梯度下降**和**分布式计算**，来减少复杂度。这些优化确保了算法在大规模数据处理时仍能在合理时间内完成任务。

优化问题

为便于阅读，本节（优化问题）承接上节（算法复杂度），先交代差异与共同点，再聚焦本节关键问题。

矩阵分解问题需要优化用户和项目的特征向量，使得系统能够拟合已有的评分数据。常见的优化算法包括**随机梯度下降**和 Adam 优化器等，这些方法可以在迭代中不断调整参数，以获得更好的推荐精度。

算法的稳定性

为便于阅读，本节（算法的稳定性）承接上节（优化问题），先交代差异与共同点，再聚焦本节关键问题。

推荐系统的算法需要在用户评分数据存在一定噪声的情况下依然有效。算法思维通过**正则化**来提升系统在不同数据分布下的稳定性，避免推荐结果过度依赖特定用户的偏好数据。

3.4.2.2 工程思维：推荐系统的实际部署

在工程思维的层面，我们需要考虑如何将理论和算法转化为能在大规模生产环境中稳定、可靠运行的系统。

允许误差

为便于阅读，本节（允许误差）承接上节（工程思维：推荐系统的实际部署），先交代差异与共同点，再聚焦本节关键问题。

在推荐系统中，我们通常不需要找到用户的“完美”推荐，而是提供大致符合用户兴趣的推荐内容。例如，推荐系统可以允许一定的计算误差以提升处理速度，使得系统能够满足实时响应的需求。

大规模数据处理

为便于阅读，本节（大规模数据处理）承接上节（允许误差），先交代差异与共同点，再聚焦本节关键问题。

推荐系统需要处理成千上万的用户和上亿个项目。为此，工程思维通过分布式系统（如 Hadoop、Spark）和云计算技术支持大规模数据的实时处理。模型可以在多个节点上并行运行，大大提升系统处理数据的速度。

鲁棒性

为便于阅读，本节（鲁棒性）承接上节（大规模数据处理），先交代差异与共同点，再聚焦本节关键问题。

推荐系统需要在不同用户行为（如异常评分、短期偏好变化）或数据源变化（如新商品、新用户）下保持稳定的推荐效果。工程思维通过设计数据预处理和动态更新机制，确保系统在面对这些变化时依然具有稳定的性能。

经济效应

为便于阅读，本节（经济效应）承接上节（鲁棒性），先交代差异与共同点，再聚焦本节关键问题。

推荐系统的存储和计算资源消耗通常很大。在工程思维的优化下，系统可以采用模型压缩、权重量化等技术，在确保准确率的同时减少资源消耗。通过云服务进行按需分配的计算资源，也降低了运行成本，确保系统在大规模商业应用中具有经济可行性。

在推荐系统的开发中，**数学思维**定义了推荐任务的核心问题和方法，构建了系统的理论基础。接下来，**算法思维**将这些数学模型转化为实际可计算的步骤，使得系统不仅能给出准确的推荐结果，还能高效、稳定地执行。最后，**工程思维**关注将这一算法落地应用，使得推荐

系统能够在真实场景中处理海量数据，满足用户的实时需求，并在资源有限的环境下实现经济性。

3.5 小结：三种思维的交融与驱动

从数学思维、算法思维再到工程思维，AI 的发展从理论走向了实践，每一步都在推动人工智能的进步。**数学思维**赋予了 AI 坚实的理论基础，为探索智能的可能性提供了方向；**算法思维**则将数学转化为具体计算方法，使得智能从可定义走向可实现；**工程思维**则是实现 AI 的最终一步，通过考虑环境适应性、成本和规模等因素，推动了 AI 的落地应用。三种思维的交互塑造了 AI 的现代图景，让我们在创新的路上不断前行。这三种思维的融合不仅是技术路径的演进，更体现了 AI 发展中的哲学思辨与应用智慧，成为推动 AI 走向未来的重要动力。从理论的存在性到计算过程的优化，再到应用的落地，这些思维贯穿了 AI 技术的整个发展脉络。

参考文献

- [1] “A Logical Calculus of Ideas Immanent in Nervous Activity”. In: *Bulletin of Mathematical Biophysics* 5 (1943), pp. 127–147.
- [2] Yoshua Bengio. “Practical recommendations for gradient-based training of deep architectures”. In: *Neural networks: Tricks of the trade: Second edition*. Springer, 2012, pp. 437–478.
- [3] George Cybenko. “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of control, signals and systems* 2.4 (1989), pp. 303–314.
- [4] Kunihiro Fukushima and Sei Miyake. “Neocognitron: A new algorithm for pattern recognition tolerant of deformations and shifts in position”. In: *Pattern recognition* 15.6 (1982), pp. 455–469.
- [5] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Multilayer feedforward networks are universal approximators”. In: *Neural Networks* 2.5 (1989), pp. 359–366.
- [6] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Universal approximation of an unknown mapping and its derivatives using multilayer feedforward networks”. In: *Neural networks* 3.5 (1990), pp. 551–560.
- [7] David H Hubel and Torsten N Wiesel. “Receptive fields and functional architecture of monkey striate cortex”. In: *The Journal of physiology* 195.1 (1968), pp. 215–243.
- [8] Han Jiawei, Kamber Micheline, and Pei Jian. *Data Mining: Concepts and Techniques*. -3rd. Morgan Kaufmann, 2011.
- [9] Fukushima K and Miyake S. “Neocognitron: A self-organizing neural network model for a mechanism of visual pattern recognition”. In: *Competition and cooperation in neural nets*. Springer, Berlin, Heidelberg, 1982, pp. 267–285.
- [10] Diederik P Kingma and Jimmy Ba. “Adam: A method for stochastic optimization”. In: *arXiv preprint arXiv:1412.6980* (2014).
- [11] Yann Le Cun, Boser Brian, and et al. Denker John S. “Handwritten digit recognition with a back-propagation network”. In: *Advances in neural information processing systems*. 1990, pp. 396–404.
- [12] Yann Le Cun et al. “Handwritten digit recognition: Applications of neural net chips and automatic learning”. In: *Neurocomputing: Algorithms, Architectures and Applications*. Springer. 1990, pp. 303–318.
- [13] Yann LeCun et al. “Backpropagation applied to handwritten zip code recognition”. In: *Neural computation* 1.4 (1989), pp. 541–551.

- [14] Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.
- [15] Minsky Marvin and A Papert Seymour. “Perceptrons”. In: *Cambridge, MA: MIT Press* 6.318–362 (1969), p. 7.
- [16] Warren S McCulloch and Walter Pitts. “A logical calculus of the ideas immanent in nervous activity”. In: *The bulletin of mathematical biophysics* 5 (1943), pp. 115–133.
- [17] Tom Mitchell. *Machine Learning*. McGraw-Hill Education, 1997.
- [18] Michael Nielsen. *Neural Networks and Deep Learning*. Accessed: 2025-03-03. 2015. URL: <http://neuralnetworksanddeeplearning.com/>.
- [19] Hastie T, Tibshirani R, and Friedman J. 统计学习基础 (*The Elements of Statistical Learning*). 北京: 世界图书出版公司, 2015.
- [20] Alan Turing. *Computing Machinery and Intelligence*. Oxford, 1950.
- [21] Vladimir Naumovich Vapnik. 统计学习理论的本质. 清华大学华夏出版社, 2000.
- [22] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).
- [23] 于剑. 机器学习—从公理到算法. 北京: 清华大学出版社, 2017.
- [24] [意] 翁贝托·博塔兹尼. 余婷婷译. 尖叫的数学—令人惊叹的数学之美. 湖南科学技术出版社, 2021.
- [25] 周志华. 机器学习. 清华大学出版社, 2016.
- [26] Vladimir N. Vapnik. 张学工译. 统计学习理论的本质. 北京: 清华大学出版社, 2000.
- [27] 张玉宏. 深度学习之美—AI 时代的数据处理与最佳实践. 电子工业出版社, 2018.
- [28] 张玉宏. 深度学习之美: AI 时代的数据处理与最佳实践. 电子工业出版社, 2018.
- [29] Tom Mitchell. 曾华军等译. 机器学习. 北京: 机械工业出版社, 2002.
- [30] 王珏, 周志华, 周傲英. 机器学习及其应用. Vol. 4. 清华大学出版社有限公司, 2006.

第四章 人工智能的数学工具

内容提要

- 微积分在机器学习中的核心应用
- 线性代数与高维数据处理
- 概率统计在不确定性建模中的作用
- 优化理论与算法设计的数学基础
- 信息论与数据压缩的数学原理

微积分 - 导数和偏导数：用于计算函数的变化率，广泛应用于梯度下降算法和神经网络的训练。- 链式法则：用于反向传播算法，以计算神经网络中的梯度。- 梯度和 Hessian 矩阵：用于优化问题，特别是在高维空间中。- 积分和多重积分：用于概率计算和期望值计算，如在贝叶斯推断中。概率与统计 - 随机变量和概率分布：用于建模和分析随机现象，如正态分布、泊松分布。- 贝叶斯定理：用于更新概率分布，广泛应用于贝叶斯网络和贝叶斯推断。- 最大似然估计 (MLE) 和最大后验估计 (MAP)：用于参数估计和模型选择。

- 马尔可夫链和马尔可夫决策过程 (MDP)：用于建模和解决决策过程，特别是在强化学习中。- 蒙特卡罗方法：用于数值积分和概率分布的近似计算。优化理论 - 凸优化：研究凸函数和凸集合的优化问题，如 LASSO 和支持向量机 (SVM)。- 非凸优化：处理复杂的优化问题，如深度学习中的神经网络训练。- 梯度下降和随机梯度下降：用于迭代优化问题。- 线性规划和整数规划：用于资源分配和组合优化问题。- 拉格朗日乘数法：用于含约束条件的优化问题。图论 - 图的表示：用于建模关系和网络，如邻接矩阵和邻接表。- 最短路径算法：如 Dijkstra 算法，用于路径规划和网络分析。- 图匹配和图划分：用于社交网络分析、图像分割和聚类。- 网络流和最大流问题：用于优化运输和流量问题。信息论 - 熵和互信息：用于衡量信息量和依赖关系，在特征选择和通信中应用。- 编码理论：研究数据压缩和错误检测/纠正码，如香农编码和汉明码。- 卡尔曼滤波：用于动态系统的状态估计和信号处理。计算理论 - 图灵机和计算复杂性：研究计算问题的复杂性和可计算性，如 P 与 NP 问题。- NP 完全性：研究问题的求解难度和算法的效率，判定问题是否具有多项式时间复杂度解。- 自动机理论：研究有限状态机和形式语言，应用于编译器和正则表达式。离散数学 - 组合数学：用于分析和解决离散结构的问题，如组合优化、排列和组合。- 布尔代数：用于逻辑推理和数理逻辑，特别是在逻辑编程和电路设计中。- 图论和网络流：用于优化和分析网络结构。动态系统与微分方程 - 常微分方程 (ODEs)：用于建模连续时间的动态系统，如物理模拟和控制理论。- 偏微分方程 (PDEs)：用于建模空间和时间的连续变化，如图像处理和物理场模拟。- 分岔理论和混沌理论：研究动态系统的稳定性和非线性行为。几何与拓扑 - 微分几何：用于分析曲面和形状，如在计算机视觉和图像处理中的应用。- 拓扑数据分析 (TDA)：用于数据的形状分析和降维，应用于高维数据的可视化。- 计算几何：研究几何对象的算法和数据结构，用于计算机图形学和机器人学。变分法与计算几何 - 变分法：用于优化问题的解，如在图像处理、路径规划和物理模拟中的应用。- 计算几何：研究几何对象的算法和数据结构，用于图形学、计算机视觉和 CAD 系统。数值分析 - 数值微分和积分：用于逼近和求解微分方程和积分方程。- 线性代数的数值方

法：如 QR 分解、LU 分解，用于求解线性方程组。- 优化的数值方法：如牛顿法、共轭梯度法，用于高效求解优化问题。笛卡尔几何与代数几何 - 笛卡尔几何：用于解析几何问题，如直线和曲线的表示与计算。- 代数几何：研究多项式方程的解集结构，用于计算机代数系统和图形学。逻辑与集合论 - 一阶逻辑和高阶逻辑：用于形式化推理和验证。- 集合论：研究集合的性质和操作，是数学的基础理论之一。

4.1 预备知识

要学习深度学习，首先需要先掌握一些基本技能。所有机器学习方法都涉及从数据中提取信息。因此，我们先学习一些关于数据的实用技能，包括存储、操作和预处理数据。机器学习通常需要处理大型数据集。我们可以将某些数据集视为一个表，其中表的行对应样本，列对应属性。线性代数为人们提供了一些用来处理表格数据的方法。我们不会太深究细节，而是将重点放在矩阵运算的基本原理及其实现上。深度学习是关于优化的学习。对于一个带有参数的模型，我们想要找到其中能拟合数据的最好模型。在算法的每个步骤中，决定以何种方式调整参数需要一点微积分知识。本章将简要介绍这些知识。幸运的是，‘autograd’包会自动计算微分，本章也将介绍它。机器学习还涉及如何做出预测：给定观察到的信息，某些未知属性可能的值是多少？要在不确定的情况下进行严格的推断，我们需要借用概率语言。最后，官方文档提供了本书之外的大量描述和示例。在本章的结尾，我们将展示如何在官方文档中查找所需信息。本书对读者数学基础无过分要求，只要可以正确理解深度学习所需的数学知识即可。但这并不意味着本书中不涉及数学方面的内容，本章会快速介绍一些基本且常用的数学知识，以便读者能够理解书中的大部分数学内容。如果读者想要深入理解全部数学内容，可以进一步学习本书数学附录中给出的数学基础知识。

-2.1. 数据操作

4.2 浅谈图像处理：从矩阵到深度学习

目录摘要 ABSTRACT 第 1 章引言 1 1.1 图像识别如何作用于我们的生活？ 1 1.2 追根溯源——其中的数学方法 1 1.3 报告主体结构与主要内容 1 第 2 章从矩阵到深度学习 2 2.1 矩阵 2 2.1.1 矩阵的起源 2 2.1.2 矩阵的发展 2 2.1.3 矩阵的基本操作 2 2.2 传统图像处理 5 2.2.1 基础知识——图像矩阵化 5 2.2.2 传统图像处理的起源 10 2.2.3 传统图像处理的发展 12 2.2.4 传统图像处理的应用 19 2.3 深度学习与图像处理——三种模型 20 2.3.1 起源 20 2.3.2 单层感知器模型 21 2.3.3 多层感知器模型 21 2.3.4 卷积神经网络模型 22 2.4 深度学习在图像处理上的发展——以目标检测为例 26 2.4.1 起源 26 2.4.2 R-CNN 27 2.4.3 SPP-Net 30 2.4.4 Fast R-CNN 31 2.4.5 Faster R-CNN 32 2.4.6 FPN 34 2.4.7 单阶段算法与两阶段算法 35 2.4.8 总结 35 2.4.9 深度学习图像处理举例 35 第 3 章体会启发与结语 38 3.1 体会与启发 38 3.2 结语 39 参考文献 40 致谢 41 摘要 本论文从代数学矩阵开始，讲述了将图像矩阵（数字）化的重要意义，以及如何使用矩

阵分析与变换的方法来分析并修改图像，并介绍了传统图像处理技术的建立与发展过程，若干重要算法以及一些弊端，最后将图像处理技术指向了深度学习与神经网络模型，引入人工智能协助以达到更先进的技术水平，并展示了一个具体实例。

4.3 第 1 章引言

4.3.1 1.1 图像识别如何作用于我们的生活？

图像识别是计算机视觉和人工智能领域的重要组成部分，其终极目标是使计算机具有分析和理解图像内容的能力。图像识别是一个综合性的问题，涵盖图像匹配、图像分类、图像检索、人脸检测、行人检测等技术，并在互联网搜索引擎、自动驾驶、医学分析、遥感分析等领域具有广泛的应用价值。具体例子：最近，由于疫情封锁，室内运动成了当下人们主要的运动方式，一款用于督促人们运动的 app 应运而生，那就是天天跳绳，而它就是通过识别人的肢体动作图像来对人的运动做出判断。这样的例子数不胜数。

4.3.2 1.2 追根溯源—其中的数学方法

将图像进行数字（矩阵）化后，我们就可以使用对于矩阵的分析方法来分析图像了，由矩阵的一些基本运算即可达到修改图像的目的，例如可以使用仿射变换来对图像做平移、放大、旋转、剪切等操作。更可以引入更加深入的算法和数学知识来进行一些更为先进的图像处理方式，例如使用高等数学中的卷积概念进行图像分析，使用 Canny 算法实现边缘检测，以及借许多数学知识与编程思维使用深度学习技术训练，达到使用人工智能协助图像分析的目的。

4.3.3 1.3 报告主体结构与主要内容

本报告从矩阵知识引入，介绍了其起源（2.1.1）、发展（2.1.2）、基础知识（2.1.3），这一部分由王晨阳同学负责。然后，我们探究了其在传统图像处理中的应用—图像矩阵化（2.2），层层深入，揭示了现代图像处理中矩阵的作用，这一部分由谭诚同学负责。在传统图像处理（2.2）中我们探究了其方法（2.2.1），起源（2.2.2），发展（2.2.3）及应用（2.2.4），这一部分由汪坤灿同学负责。在深度学习与图像处理中，我们介绍了若干种深度学习的神经网络模型，以及若干个深度学习的算法，这一部分由史晓磊同学负责。最后，我们使用 Python-OpenCV 库以及调用了 Github 上一个已经训练好的模型，完成了一次对图片中人脸的识别检测，这一部分由谭诚同学负责。小组中，我们各自安排撰写各自的文档、ppt，以及录制视频中的各个部分。最后由谭诚同学整合、排版，以及剪辑。

去年的某一天，一张利用 AI 修复的林徽因的照片冲上了热搜。可以看出，右边由 AI 修复的照片清晰度有了巨大的提升，尽管仍有些模糊，但是对于照片的关键部分—人脸已经有了巨大的提升。可以看出后者非常像网红脸。那么人工智能是如何修复照片的呢？那就是对大量人脸数据的特征进行提取。但是对这么大量的数据如何分析呢？如上图，在电路中，该图

是最简单的一个电路模型。其输出端 Y 的值即为输入端 Y 的值，其值有两种，0 和 1。因此，我们可以通俗的认为，0 代表否，1 代表是。因此，对于一个特征的有无，0 代表无，1 代表有。这样就可以用数字记录一个事物的特征。同样的，利用一串数组，可以记录一系列的特征。那么利用矩阵，就可以记录大量的特征。本报告将涉及数学概念中的“矩阵 (Matrix)”。

4.3.4 2.1 矩阵

在数学中，矩阵 (Matrix) 是一个按照长方阵列排列的复数或实数集合 [1]，最早来自于方程组的系数及常数所构成的方阵。这一概念由 19 世纪英国数学家凯利首先提出。作为解决线性方程的工具，矩阵也有不短的历史。成书最早在东汉前期的《九章算术》中，用分离系数法表示线性方程组，得到了其增广矩阵。在消元过程中，使用的把某行乘以某一非零实数、从某行中减去另一行等运算技巧，相当于矩阵的初等变换。但那时并没有现今理解的矩阵概念，虽然它与现有的矩阵形式上相同，但在当时只是作为线性方程组的标准表示与处理方式。

2.1.1 矩阵的起源 作为解决线性方程的工具，矩阵也有不短的历史。成书最迟在东汉前期的《九章算术》中，用分离系数法表示线性方程组，得到了其增广矩阵。在消元过程中，使用的把某行乘以某一非零实数、从某行中减去另一行等运算技巧，相当于矩阵的初等变换。但那时并没有现今理解的矩阵概念，虽然它与现有的矩阵形式上相同，但在当时只是作为线性方程组的标准表示与处理方式。矩阵正式作为数学中的研究对象出现，则是在行列式的研究发展起来后。逻辑上，矩阵的概念先于行列式，但在实际的历史上则恰好相反。日本数学家关孝和 (1683 年) 与微积分的发现者之一戈特弗里德·威廉·莱布尼茨 (1693 年) 近乎同时地独立建立了行列式论。其后行列式作为解线性方程组的工具逐步发展。1750 年，加布里尔·克拉默发现了克莱姆法则。矩阵最早出现在我国的九章算术中，在九章算术方程篇中，刘徽就给出过“方程”的定义，不过他所定义的方程，指的是由线性方程得到的数码矩阵，其本质就是线性方程组的增广矩阵，不仅如此，他还提出过解线性方程组的两种方法，即利用该方阵进行“直除”和“互乘”运算，“直除”就是将某行的同一倍数加到另一行上，“互乘”则是以一个不为零的数乘该方阵的某一行上，这里已经涉及到了矩阵初等变换的知识。

2.1.2 矩阵的发展 1801 年，德国数学家高斯首次将线性方程的全部系数放在一起，并提出了高斯消元法（这种方法在今天的计算机解线性方程组的问题中仍在使用），给出了矩阵的雏形；1844 年，德国数学家爱森斯坦首次讨论了变换（也就是今天的“矩阵”）的乘积，为此后的数学家打开了一条新路。1850 年，英国数学家西尔维斯特首先使用矩阵一词；1858 年，英国数学家凯莱发表《关于矩阵理论的研究报告》。他首先将矩阵作为一个独立的数学对象加以研究，并在这个主题上首先发表了一系列文章，因而被认为是矩阵论的创立者，他给出了现在通用的一系列定义，如两矩阵相等、零矩阵、单位矩阵、两矩阵的和、一个数与一个矩阵的数量积、两个矩阵的积、矩阵的逆、转置矩阵等，并且凯莱还注意到矩阵的乘法是可结合的，但一般不可交换，且 $m \times n$ 矩阵只能用 $n \times k$ 矩阵去右乘。1854 年，法国数学家埃米尔特使用了“正交矩阵”这一术语，但他的正式定义直到 1878 年才由德国数学家费罗贝尼乌斯发表。1879 年，费罗贝尼乌斯引入秩的概念。至此，矩阵的体系基本上完全建立起来了。矩阵的现代概念在 19 世纪逐渐形成。1800 年

代，高斯和威廉·若尔当建立了高斯—若尔当消去法。1844年，德国数学家费迪南·艾森斯坦（F.Eisenstein）讨论了“变换”（矩阵）及其乘积。1850年，英国数学家詹姆斯·约瑟夫·西尔维斯特（James Joseph Sylvester）首先使用矩阵一词。英国数学家凯利被公认为矩阵论的奠基人。他开始将矩阵作为独立的数学对象研究时，许多与矩阵有关的性质已经在行列式的研究中被发现了，这也使得凯利认为矩阵的引进是十分自然的。他说：“我决然不是通过四元数而获得矩阵概念的；它或是直接从行列式的概念而来，或是作为一个表达线性方程组的方便方法而来的。”他从1858年开始，发表了《矩阵论的研究报告》等一系列关于矩阵的专门论文，研究了矩阵的运算律、矩阵的逆以及转置和特征多项式方程。凯利还提出了凯莱-哈密尔顿定理，并验证了 3×3 矩阵的情况，又说进一步的证明是不必要的。哈密尔顿证明了 4×4 矩阵的情况，而一般情况下的证明是德国数学家弗罗贝尼乌斯（F.G.Frohenius）于1898年给出的。

1854年时法国数学家埃尔米特（C.Hermite）使用了“正交矩阵”这一术语，但他的正式定义直到1878年才由费罗贝尼乌斯发表。1879年，费罗贝尼乌斯引入矩阵秩的概念。至此，矩阵的体系基本上建立起来了。无限维矩阵的研究始于1884年。庞加莱在两篇不严谨地使用了无限维矩阵和行列式理论的文章后开始了对这一方面的专门研究。1906年，希尔伯特引入无限二次型（相当于无限维矩阵）对积分方程进行研究，极大地促进了无限维矩阵的研究。在此基础上，施密茨、赫林格和特普利茨发展出算子理论，而无限维矩阵成为了研究函数空间算子的有力工具。

4.3.5 矩阵的应用现状

矩阵在机器学习和深度学习上面的应用，有以下几个。1. 在将数据集应用于机器学习模型之前，需要对数据集做预处理，比如给数据去噪、降维，这样子可以减少计算资源和提高模型预测准确度。常用的去噪降维的方法是主成分分析（PCA），里面就用到了矩阵，具体步骤为（1）求解数据集的协方差矩阵的特征值和特征向量，（2）根据特征值从大到小排序（3）选取前面几个特征值对应的特征向量作为新的坐标基向量。（4）将所有数据集映射到这个新的坐标系中，从而得到降噪降维之后的数据集了。在第（3）步中只选取前面最大的几个特征值所对应的特征向量，是因为小特征值所蕴含的信息量比较小，它们很可能是一些噪声数据。2. 另一个在机器学习中的应用是奇异值分解，奇异值分解的公式为 $M=U\Sigma V$ ，在 Netflix Prize 电影推荐系统中，根据用户对电影的评分来建模的推荐系统使用到了奇异值分解的思想。3. 在深度学习模型的框架中，数据是以矩阵的形式组织的，比如一次输入 64 张彩色图片，输入数据是 $64 \times 3 \times 32 \times 32$ 维度的。这样子一方面可以简化表示，另一方面可以提高计算速度，特别是 GPU 能够针对这个进行优化，如果不以矩阵的形式表示数据，那么 GPU 的优势就没发挥出来。矩阵是人工智能基础范畴中不可或缺的一部分。必备的数学知识是理解人工智能不可或缺的要害，所有的人工智能技术归根到底都建立在数学模型之上，而这些数学模型又都离不开线性代数的理论框架。线性代数不仅仅是人工智能的基础，更是现代数学和以现代数学作为主要分析方法的众多学科的基础。从量子力学到图像处理都离不开向量和矩阵的使用。而在向量和矩阵背后，线性代数的核心意义在于提供了一种看待世界的抽象视角：万事万物都

可以被抽象成某些特征的组合，并在由预置规则定义的框架之下以静态和动态的方式加以观察。线性代数是 AI 专家必须掌握的知识。线性代数是一套非常通用的思想和工具，可以应用于各个领域。如果你只想把人工智能和机器学习的工具当作一个黑匣子，那么你只需要足够的数学计算就可以确定你的问题是否符合模型使用。如果你想提出新想法，线性代数则是你必须要学习的东西。并不是说你需要学习有关数学的所有知识，这样会耽搁于此，失去研究其他更重要的东西（如微积分/统计）的动力。

线性代数是应用数学领域，这是人工智能专家离不开的。如果你不精通这个领域，你永远不会成为一个好的 AI 专家。正如斯凯勒·斯皮克曼所说：“线性代数是 21 世纪的数学。”线性代数有助于产生新的想法，这就是为什么它是人工智能科学家和研究人员必须学习的东西。他们可以用标量、向量、张量、矩阵、集合和序列、拓扑、博弈论、图论、函数、线性变换、特征值和特征向量的概念来建立模型。向量矩阵理论在科幻电影中，你通常会看到，通过执行一些类似于神经系统的计算结构，产生了一个神经网络，它生成神经元之间的连接，以匹配人类大脑的推理方式。将矩阵的概念引入到神经网络的研究中神经网络可以通过形成三层人工神经元来实现非线性假设：1. 输入层 2. 隐藏层 3. 输出层特征向量搜索引擎排名的科学是建立在数学科学的基础上的。页面排名，这正是谷歌作为一家基于数学视角的公司的基础。页面排名是一种算法，最初由拉里·佩奇和谢尔盖·布林在他们的研究论文《大型超文本 Web 搜索引擎的剖析》中提出。在这一巨大突破的背后，使用了数百年来众所周知的特征值和特征向量的基本概念。

机器人爬虫程序首先检索网页，然后通过分配页面排名值对其进行索引和编目。每个页面的可信度取决于该页面的链接数。

2.1.3.1 加减法对应位置相加减。例如：

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} + \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$$

需要注意的是，这两个矩阵必须行列数均相同。2.1.3.2 矩阵数乘矩阵的所有元素都乘上这个数即可。

$$k \cdot \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & k \\ k & 0 \end{pmatrix}$$

2.1.3.3 矩阵乘法两个矩阵的乘法仅当第一个矩阵 A 的列数和另一个矩阵 B 的行数相等时才能定义。如果矩阵 A 是 $m \times n$ 矩阵（m 行 n 列），矩阵 B 是 $n \times k$ 矩阵（n 行 k 列），则它们的乘积 C 是一个 $m \times k$ 矩阵。对于矩阵 C 的每个元素，它如下计算：

$$c_{i,j} = a_{i,1}b_{1,j} + a_{i,2}b_{2,j} + \cdots + a_{i,n}b_{n,j} = \sum_{k=1}^n a_{i,k}b_{k,j}$$

容易看出，单位矩阵乘任何矩阵，不论左乘还是右乘，都不会改变矩阵。这在传统的线性代

数教材中已经被证明很多次了，这里不再赘述。2.1.3.4 矩阵转置将每个元素的行数和列数互换即可。

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}^T = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

2.1.3.5 共轭矩阵共轭是复数域的概念，只是把矩阵中每个元素取其共轭的复数即可。

$$A = \begin{pmatrix} 3+i & 5 \\ 2-2i & i \end{pmatrix}, \quad \overline{A} = \begin{pmatrix} 3-i & 5 \\ 2+2i & -i \end{pmatrix}$$

2.1.3.6 矩阵的特征值及特征向量设 A 是 n 阶方阵，如果数 λ 和 n 维非零列向量 x 使关系式 $Ax=\lambda x$ 成立，那么这样的数 λ 称为矩阵 A 特征值，非零向量 x 称为 A 的对应于特征值 λ 的特征向量。对于矩阵的特征值及特征向量，在传统线性代数教材中已经有了足够充分的解释，并对其性质和计算方法也有了足够多的介绍，这里不再赘述了。

矩阵的特征值和特征向量可以揭示线性变换的深层特性，我们稍后会展开介绍其具体应用，并简述其实现原理。2.1.3.7 线性变换在数学上，如果满足下式，那么映射 $F(a)$ 就是线性的： $F(a+b)=F(a)+F(b)$ 以及 $F(ka)=kF(a)$ 如果映射 F 保持了基本运算：加法和数量乘，那么就可以称该映射为线性的。在这种情况下，将两个向量相加然后再进行变换得到的结果和先分别进行变换再将变换后的向量相加得到的结果相同。同样，将一个向量数量乘再进行变换和先进行变换再数量乘的结果也是一样的。这个线性变换的定义有两条重要的引理：(1) 映射 $F(a)=aM$ ，当 M 为任意方阵时，此映射是一个线性变换，这是因为： $F(a+b)=(a+b)M=aM+bM=F(a)+F(b)$ $F(ka)=(ka)M=k(aM)=kF(a)$ (2) 零向量的任意线性变换的结果仍然是零向量。(如果 $F(0)=a$, $a \neq 0$ 。那么 F 不可能是线性变换。因为 $F(k0)=a$, 但 $F(k0) \neq kF(0)$ ，因此线性变换不会导致平移（原点位置上不会变化）。2.1.3.8 矩阵的仿射变换仿射变换是指线性变换后接着平移。因此仿射变换的集合是线性变换的超集，任何线性变换都是仿射变换，但不是所有仿射变换都是线性变换。任何具有形式 $v'=vM+b$ 的变换都是仿射变换。2.1.3.9 矩阵的可逆变换如果存在一个逆变换可以“撤销”原变换，那么该变换是可逆的。换句话说，如果存在逆变换 G ，使得 $G(F(a))=a$ ，对于任意 a ，映射 $F(a)$ 是可逆的。存在非仿射变换的可逆变换，但暂不考虑它们。现在，我们集中精力于检测一个仿射变换是否可逆。一个仿射变换就是一个线性变换加上平移，显然，可以用相反的量“撤销”平移部分，所以问题变为一个线性变换是否可逆。显然，除了投影以外，其他变换都能“撤销”。当物体被投影时，某一维有用的信息被抛弃了，而这些信息是不可能恢复的。因此，所有基本变换除了投影都是可逆的。因为任意线性变换都能表达为矩阵，所以求逆变换等价于求矩阵的逆。如果矩阵是奇异的，则变换不可逆；可逆矩阵的行列式不为 0。2.1.3.10 矩阵的等角变换如果变换前后两向量夹角的大小和方向都不改变，该变换是等角的。只有平移，旋转和均匀缩放是等角变换。等角变换将会保持比例不变，镜像并不是等角变换，因为尽管两向量夹角的大小不变，但夹角的方向改变了。所有等角变换都是仿射和可逆的。2.1.3.11 矩阵的正交变换术语“正交”用来描述具有某种性质的矩阵。正交变换的基本思想是轴保持互相垂直，而且不进行缩放变换。平移、旋转和镜像仅是仅有的正交变换。长度、角度、面积和体积都保持不变。(尽管如此，但因为镜像变换被认为是正交变换，所以一定要密切注意角度、面积和体积的准确定义)。正交矩阵的行列

式为 1 或者负 1，所有正交矩阵都是仿射和可逆的。

2.2 传统图像处理

2.2.1 基础知识—图像矩阵化

2.2.1.1 灰度 [9][10][11]

在电子计算机领域中，灰度（Gray scale）数字图像是每个像素只有一个采样颜色的图像。这类图像通常显示为从最暗黑色到最亮的白色的灰度，尽管理论上这个采样可以是任何颜色的不同深浅，甚至可以是不同亮度上的不同颜色。灰度图像与黑白图像不同，在计算机图像领域中黑白图像只有黑白两种颜色，灰度图像在黑色与白色之间还有许多级的颜色深度。但是，在数字图像领域之外，“黑白图像”也表示“灰度图像”，例如灰度的照片通常叫做“黑白照片”。在一些关于数字图像的文章中单色图像等同于灰度图像，在另外一些文章中又等同于黑白图像。[12]

图 2.2.1.1.1 原始彩色图片与灰度图灰度图像经常是在单个电磁波频谱如可见光内测量每个像素的亮度得到的。用于显示的灰度图像通常用每个采样像素 8 bits 的非线性尺度来保存，这样可以有 256 种灰度（8bits 就是 2 的 8 次方 =256）。这种精度刚刚能够避免可见的条带有损，并且非常易于编程。在医学图像与遥感图像这些技术应用中经常采用更多的级数以充分利用每个采样 10 或 12 bits 的传感器精度，并且避免计算时的近似误差。在这样的应用领域流行使用 16 bits 即 65536 个组合（或 65536 种颜色）。[12]

[13] 2.2.1.2 RGB 颜色系统 [7]

三原色光模式（RGB color model），又称 RGB 颜色模型或红绿蓝颜色模型，是一种加色模型，将红（Red）、绿（Green）、蓝（Blue）三原色的色光以不同的比例相加，以合成产生各种色彩光。RGB 颜色模型的主要目的是在电子系统中检测，表示和显示图像，比如电视和电脑，利用大脑强制视觉生理模糊化（失焦），将红绿蓝三原色子像素合成为一色彩像素，产生感知色彩（其实此真彩色并非加色法所产生的合成色彩，原因为该三原色光从来没有重叠在一起，祇是人类为了“想”看到色彩，大脑强制眼睛失焦而形成。红绿蓝三色模型在传统摄影中也有应用。在电子时代之前，基于人类对颜色的感知，RGB 颜色模型已经有了坚实的理论支撑。RGB 是一种依赖于设备的颜色空间：不同设备对特定 RGB 值的检测和重现都不一样，因为颜色物质（荧光剂或者染料）和它们对红、绿和蓝的单独响应水平随着制造商的不同而不同，甚至是同样的设备不同的时间也不同。[7]

图 2.2.1.1.1 RGB 颜色空间示意图一个颜色显示的描述是由三个数值控制的，他分别为 R、G、B。当三个数值为最大（255）时，显示为白色，当三个数值最小（0）时，显示为黑色。数值表示可以使用以下几种不同的方式：从 0 到 1 之间可用的浮点数来表示、从 0 % 到 100 % 用百分比表示，或者使用 0 到 255 之间的整数以八位数字表示，通常表示为十进制和十六进制的数值。[7]

2.2.1.3 Gamma 校正 [8][11]

伽马校正（Gamma correction）又叫伽马非线性化（gamma nonlinearity）、伽马编码（gamma encoding）或是就只单纯叫伽马（gamma）。是用来针对影片或是影像系统里对于光线的辉度（luminance）或是三色刺激值（tristimulus values）所进行非线性的运算或反运算。最简单的例子中，伽马校正由下幂定律公式所定义的： $V_{out} = AV_i n^\gamma$ 其中 A 是一个常量，输入和输出的值都为非负实数值。一般地来说在 A=1 的通常情况下，输入输出的值的范围都是在 0 到 1 之间。伽马值 $\gamma < 1$ 的情况有时被称作编码伽马值（encoding gamma），而执行这个编码运算所使用上述幂定律的过程也叫做伽马压缩（gamma compression）；相对地，伽马值 $\gamma > 1$ 的情况有时也被称作解码伽马值（decoding gamma），而执行这个解码运算所使用上述幂定律的过程也叫做“伽马展开（gamma expansion）”。[8][11]

RGB 值与功率并非简单的线性关系，而是幂函数关系，这个函数的指数称为 Gamma 值，一般为 2.2，而这个换

算过程，称为 **Gamma** 校正。为什么显示器要 **Gamma** 校正呢？因为人眼对亮度的感知和物理功率不是成正比的，而是幂函数的关系，这个函数的指数通常为 2.2，称为 **Gamma** 值。所以 **RGB** 中的灰度值，为了考虑到较小的存储范围 (0 255) 和较平衡的亮暗部比例，所以需要进行 **Gamma** 校正，而不是直接对应功率值，因此 **RGB** 值 **RGB** 颜色值不能简单直接相加，而是必须用 2.2 次方换算成物理光功率后才能进行下一步计算。由于 **Gamma** 校正，在计算机显示设备上的颜色输出的强度通常不是直接正比于在图像文件中 **R**, **G** 和 **B** 值。就是说，即使值 0.5 非常接近于 0 到 1.0（完全强度）的一半，计算机显示器在显示 (0.5, 0.5, 0.5) 时候的光强度通常（在标准 2.2-gamma CRT/LCD 上）是在显示 (1.0, 1.0, 1.0) 时候的大约 22%，而不是 50%。图 2.2.1.3.1 **Gamma** 校正为图像进行伽马编码的目的是用来对人类视觉的特性进行补偿，从而根据人类对光线或者黑白的感知，最大化地利用表示黑白的数据位或带宽。在通常的照明（既不是漆黑一片，也不是令人目眩的明亮）的情况下，人类的视觉大体有伽马或者是幂函数的性质。如果不将图像进行伽马编码，那么数据位或者带宽的利用就会分布不均匀——会有过多的数据位或者带宽用来表示人类根本无法察觉到的差异，而用于表示人类非常敏感的视觉感知范围的数据位或者带宽又会不足。图像的伽马编码并不是必须的（甚至有的时候会适得其反），浮点数格式的颜色值已经提供了一部分对数曲线的线性估计。[8][11] 2.2.1.4 图像的数字（矩阵）化位图 [14][15]（英语：**Bitmap**，台湾称为点阵图），又称栅格图（**Raster graphics**），是使用像素阵列（**Pixel-array/Dot-matrix** 点阵）来表示的图像。我们这篇课题所涉及的图像，都是在指位图，而不是矢量图。我们先来考虑灰度图在 **OpenCV** 中如何存储。**OpenCV** 是一个跨平台计算机视觉和机器学习软件库，我们稍后会介绍它。这里我们会用简单的代码来读取一个图片并观察它的输出结果，来推断出图像是如何数字化的。**C++** 代码以及我们需要读取的“1.jpg”图片：“1.jpg”图片（一个方格示意为一个像素）输出结果是：事实上，这也就是矩阵：

$$\begin{pmatrix} 0 & 255 & 0 \\ 255 & 0 & 255 \\ 0 & 255 & 0 \end{pmatrix}$$

结合我们上面介绍过的 **RGB** 与灰度的知识，不难发现出：由于灰度仅由一个大小为 0 255 的八位无符号整数表示，所以每个像素点只需要用一个数来表示就可以了（黑色的灰度是 0，白色的灰度是 255）。接下来，我们再来考虑 **RGB** 色彩系统下的彩色图像的数字（矩阵）化。事实上，这也就是矩阵：

$$\begin{pmatrix} 0 & 0 & 255 \\ 0 & 255 & 0 \\ 255 & 0 & 0 \end{pmatrix} \begin{pmatrix} 255 & 0 & 0 \\ 0 & 0 & 255 \\ 0 & 255 & 0 \end{pmatrix} \begin{pmatrix} 0 & 0 & 255 \\ 0 & 255 & 0 \\ 255 & 0 & 0 \end{pmatrix}$$

我们在“1.jpg”中预设的三种颜色都是三原色，也就是：以 (R,G,B) 三元组表示一个颜色的话，红色为 (255,0,0)，绿色为 (0,255,0)，蓝色为 (0,0,255)。而观察我们的矩阵，这是一个 3x3 像素的图像，而这个矩阵却是一个 3x9 矩阵，结合我们上面介绍的 **RGB** 颜色系统，可以得知一个颜色被三个矩阵元素表示，且这三个元素分别就是 **RGB** 三个值中的某个值。然而，我们图像中第一行的颜色顺序为“红-蓝-红”，按照 (R,G,B) 三元组表示，它们应该为“(255,0,0) - (0,0,255) - (255,0,0)”，然而矩阵中却是“(0,0,255) - (255,0,0) - (0,0,255)”，这说明实际在 **OpenCV** 中存储时

的顺序为 (B,G,R)。到这里为止,所有的位图都可以如上数字化了。在 RGB 颜色系统中,我们还可以分析一个图像中各原色通道的占比,并用灰度图输出表示:例如,上图橙子果肉的颜色用 (R,G,B) 三元组表示大概为 (168,102,0),在 red 通道图中它最明亮,在 green 通道图中其次,在 blue 通道图中最暗。在灰度中,255 为白色,0 为黑色,这表示出了图像中各个位置的原色比例。

2.2.2 传统图像处理的起源 [1] 数字图像处理,是现代信息处理形式的代表,具有画面处理程序一体化、图像处理清晰度高等特性。较之传统的图像处理,如胶片剪辑、胶片冲洗等,数字图像处理利用计算机对图像进行加工与处理,满足了图片数字化、可数码操作的需要。1921 年 [1],人们通过海底电缆将图像由伦敦传输至纽约,标志着数字图像处理系统的诞生。首个数字图像处理系统名为“巴特兰”(Bartlane),该系统使用 5 个不同的灰度级来编码图像。而到了 1929 年,“巴特兰”编码图像的这一能力已经扩展到了 15 级。图 2.2.1 1921 年经“巴特兰”系统处理的数字图片 20 世纪 50 年代,人们开始对数字图像处理技术进行系统的研究。1964 年,美国加州的喷气推进实验室使用数字计算机处理了“徘徊者 7 号”月球探测器发回的宇宙照片。图 2.2.3 “徘徊者 7 号”月球探测器该事件为数字图像处理的一个重要里程碑,标志着第三代计算机问世后数字图像处理概念开始得到运用。此后数字图像处理技术发展迅速并于不久之后形成一门独立的学科,初步建立了起来。图 2.2.4 “徘徊者 7 号”月球探测器传回的照片 2.2.3 传统图像处理的发展在数字图像处理成为一门独立的学科之后的三四十年,随着各领域对图像处理的要求越来越高,数字图像处理技术得到了更加深入、广泛和迅速的发展。国内外学者在图像处理的各个方面取得了令人瞩目的成果……伴随着数字图像处理的不断发展 [2],各种处理技术层出不穷如图像数字化、图像变换、图像增强、图像恢复、图像数据压缩、图像边缘检测、图像分割、图像特征分析、图像配准、图像融合、图像分类、图像识别、基于内容的图像检索、图像数字水印等……下面将为大家以传统数字图像处理技术中的图像的增强、图像的滤波、图像的边缘检测的典型算法为例,为大家展示下数字图像处理技术的魅力。

2.2.3.1 图像增强算法 [2] 图像增强算法用于增强对所需信息的辨别能力但同时也可能损失一些其他信息。因此在实际的应用中要对图像进行多种变换,然后再从中选取效果较好的一个。下面我们将以图像增强算法中的灰度变换为例来解释说明图像增强的一种实现。图 2.2.5 利用图像增强算法突出水中鱼类灰度变换:灰度变换是指根据某种目标条件来确立变换关系逐然后逐点改变图像中每一个像素的灰度值。[3] 让我们假设原图像像素的灰度值 $D=f(x,y)$,处理后图像像素的灰度值记为 $D'=g(x,y)$,则灰度增强可表示为: $D'=T(D)$ 其中 D 和 D' 都在图像的灰度范围之内。函数 $T(D)$ 称为灰度变换函数,它描述了输入灰度值和输出灰度值之间的转换关系。当我们确定了灰度变换函数后,一个具体的灰度增强方法便确立了。根据 $T(D)$ 表达式的不同将灰度变换分为线性变换和非线性变换。其中的线性变换如下所示:若 $g(x,y)=T[f(x,y)]$ 是一个线性或分段线性的单值函数例如:对某个定义域内的灰度 $g(x,y)=T[f(x,y)]=a*f(x,y)+b$,则由它确定的灰度变换称为灰度线性变换简称线性变换。式中参数 a 为线性函数的斜率 b 为线性函数的在 y 轴的截距。线性灰度变换具有算法较易实现,使用方便等特性。但是其可扩展性较差。一般地,我们更多会使用非线性灰度变换。图 2.2.6 对钙化点的医学影像使用线性灰度变换处理前后对比图相对线性变换,当映射关系为非线性函数时,该变换显然为非线性变换。常用的非线性变换有对数变换和指数变换。其中的对数变换

如右所示： $g(x,y)=c*\ln[f(x,y)+1]$ 公式中的 c 是为了调整曲线的位置和形状而引入的参数。由于对数函数的自身特性，当我们想要对图像的低灰度区进行较大的扩展而对高灰度区进行压缩时可采取此变换。此处的灰度对数变换能使图像灰度分布与人的视觉特性相匹配。其中的指数变换如右所示： $g(x,y)=b^{c*[f(x,y)-a]-1}$ 公式中 a 、 b 、 c 是为了调整曲线的位置和形状。由于指数函数的特性，故该变换能对图像的高灰度区域给予较大的扩展。综上，我们可以得出，当我们关注的灰度区不同时，我们可以选用不同的灰度变换来达到理想的效果。

2.2.3.2 图像滤波算法众所周知，由于外部或内部的各种原因，数字图像不可避免地会有一些噪声 [4]，如拍摄图片时光照不足会产生噪点，扫描图片时我们也无法得到百分之百不含杂质的图像。这便对后续的图像处理造成了影响。因此，图像去噪是数字图像处理工作中一个必不可少的部分，对于后续的各项其他处理都有重要意义。应用图像滤波算法去噪已被应用于各项研究，涉及到了医疗勘探、雷达、文字等方面。发展至今，优化滤波算法，改进滤波算法的设计仍具有重要的意义。下面我们将以滤波算法中的中值滤波为例来说明图像滤波算法的一种实现。

图 2.2.8 利用一种图像滤波算法来去除桥梁探伤图片的噪声

常规中值滤波：不同于均值滤波中对像素矩阵中 (x,y) 邻域进行平均化的处理，中值滤波是一种非线性的处理方法。[5] 常规中值滤波首先用 3×3 大小的滑动窗口在彩色图像上自左向右、自上向下移动，设 S_{xy} 代表以点 (x,y) 为中心、大小是 3×3 的矩形图像邻域像素值（窗口）的一组坐标：再对 S_{xy} 中 9 个点的像素值 (R,G,B) 分别进行排序，一般使用冒泡排序法，得到三个序列 R_{ij}, G_{ij}, B_{ij} ，以 R_{ij} 为例：其中， $f_1 \leq f_2 \leq f_3 \leq \dots \leq f_9$ 。之后，使用序列 R_{ij} 的中间值替代原窗口中心点 (x,y) 的 R 值 $f(x,y)$ 。G 值、B 值操作过程相同，分别对 (R,G,B) 值完成滤波处理后，点 (x,y) 处的中值滤波处理完成，窗口移动到下一个像素点。

图 2.2.9 常规中值滤波算法流程图

图 2.2.10 利用常规中值滤波算法对加入脉冲噪声后的 Lena 图像处理前后对比图

通过对比图可以看到，常规中值滤波算法处理后虽然去除了大部分噪声但是同时也损失了不少图像细节，如 Lena 图中其脸颊和肩膀部分都损失了较多细节，通俗来说，图像某些部分变得较为模糊，仍有很大的改进空间。

自适应中值滤波：自适应中值滤波算法是常规中值滤波算法的改进算法，在进行中值滤波处理之前，它会先判断采样点是否是噪声点，如果是噪声点，则进行滤波处理，如果不是噪声，则跳过当前采样点，移动到下一个采样点。和常规中值滤波算法的前半部分相同。（右图为自适应中值滤波算法流程图）

首先用 3×3 大小的滑动窗口在彩色图像上自左向右、自上向下移动，设 S_{xy} 代表以点 (x,y) 为中心、大小是 3×3 的矩形图像邻域像素值（窗口）的一组坐标：其中， $f_1 \leq f_2 \leq f_3 \leq \dots \leq f_9$ 。当排序后，中心点 (x,y) 的 R 值 $f(x,y)$ 位于向量 R_{ij} （已排序）的两端时，中心点 (x,y) 可能是一个噪声点。如果它同时满足 $f(x,y) \leq T_{min}$ 或 $f(x,y) \geq T_{max}$ ，则中心点 (x,y) 判定为噪声点。参数 T_{min} 和 T_{max} 根据情况设定，一般 T_{min} 取 40， T_{max} 取 190 比较合适。判定为噪声后，使用序列 R_{ij} 的中间值 f_5 替代原窗口中心点 (x,y) 的值 $f(x,y)$ ，完成点 (x,y) 处的中值滤波处理，然后，窗口移动到下一个像素点，如此循环操作。可以看到自适应中值滤波算法在去除噪声的同时，图像细节损失较少。在大多数情况下，自适应中值滤波算法的处理效果是要优于常规中值滤波的。

图 2.2.11 利用自适应中值滤波算法对加入脉冲噪声后的 Lena 图像处理前后对比图

图 2.2.12 自适应中值滤波算法流程图

2.2.3.3 图像边缘检测算法

图像边缘是图像的基本特征之一，它包含对人类视觉和机器视觉有价值的物体图像信息。例如在利用

计算机锐化图像时便有效利用了图像的边缘信息。图像边缘检测算法是图像处理领域中一种重要的预处理技术，其广泛地应用于轮廓、特征的抽取和纹理分析等领域。(而其中较晚出现的)Canny 边缘检测算法是针对局部区域或整幅图像确定一个门限 [6]，对图像进行分割，从而提取出边缘，下面我们将以 Canny 算法为例来说明图像边缘检测算法的一种实现。图 2.2.12 应用一种边缘检测算法检测 Lena 图像 Canny 边缘检测算法：Canny 边缘检测算法总共有四个步骤，分别是：I. 采用高斯滤波的方法对原始图像作平滑处理 II. 梯度幅值与方向的计算 III. 对梯度幅值的非极大值抑制 IV. 双阈值检测和连接边缘。I. 采用高斯滤波的方法对原始图像作平滑处理：高斯函数如上所示其中，其中 σ 为高斯滤波的参数， σ 的选择会直接影响滤波效果。图像平滑化的处理步骤如下：(1) 设置一个模板，在该模板内，求出所有像素灰度的加权平均值；(2) 将该值赋给该模版中心像素点的灰度值；(3) 按照同样的方法，扫描图像中的每个像素点，再进行加权平均计算。II. 梯度幅值与方向的计算：首先，对每个像素点求偏导数再利用下面两个公式得到像素的梯度幅值 $M(i,j)$ 和方向 $\theta(i,j)$ 。III. 对梯度幅值的非极大值抑制：由于梯度幅值的极大值附近会产生屋脊带，故非极大值抑制的原理是通过细化幅值图中的屋脊带，更好地确定边缘的位置非极大值抑制过程为，将梯度幅值矩阵 $M(i,j)$ 内的每一个点都作为中心像素点，与其周围 8 个方向邻域中沿该中心点梯度方向上的两个邻域的梯度幅值进行比较。若中心像素点的幅值小于这两个方向上邻域的梯度幅值，则该中心像素点边缘标志位的值为 0。经过非极大值抑制，不仅用一个像素的宽度替代了梯度幅值矩阵的屋脊带宽度，还保留了屋脊的梯度幅值。经过上述三个步骤的处理后，我们已经得到了一个边缘检测的粗粒度结果，其包含有较多的伪边缘。故仍需要进一步处理。IV. 双阈值检测和连接边缘：(1) 设定两个阈值，即一个高阈值、一个低阈值，将梯度幅值小于低阈值的像素点所对应的灰度值视为 0；(2) 将低阈值提取出的图像设为 A，将梯度幅值大于高阈值的图像设为 B，由于 A 为通过低阈值提取得到的图像，故边缘的连续性较好，但会含有较多的伪边缘，B 为梯度幅值大于高阈值的图像，故其不包含伪边缘，但是其边缘的连续性不佳；(3) 以图像 B 为基础，图像 A 为补充，通过递归追踪法获得边缘信息。至此，Canny 算法的处理步骤便完成了。图 2.2.13 利用 Canny 算法对两幅图片进行边缘检测的结果 2.2.3.4 传统图像处理算法存在的缺陷在上述的种种算法中无论是灰度变换中线性或是非线性变换中公式参数的选择还是自适应中值滤波算法中 T_{max} 和 T_{min} 的选择抑或是 Canny 边缘检测算法中双阈值的选择都具有一定的主观性。相应地，根据主观选择的不同，所导出的结果并不相同。倘若数值选择不当，则有可能会有一个糟糕的实现表现。那么，怎样去选择这些数值，怎样根据不同的实际情况去调换数值便是一个值得考虑的问题，如果一味地按照固定的参数去应用，结果可能并不尽如人意。’ 2.2.4 传统图像处理的应用随着数字图像处理技术的不断发展，目前，数字图像处理的应用越来越广泛，它已经与诸多领域如遥感、公共安全、工业检测、医疗诊断、军事等深度融合，下面我们将举例说明传统图像处理的应用。遥感：如探索者月球图像、勇气号火星图像、再如农作物产量计算、农作物病情防治、山林防火防灾等图 2.2.4.1 火星外表面图像传回公共安全如：银行防盗、“天网工程”，极大保障了公民的财产和人生安全。工业检测如：上文在滤波算法提到的桥梁探伤、或是道路网裂、龟裂等公路路面破损图像识别。医疗诊断如：上文在 Canny 边缘识别算法中提到的细胞显微成像边缘识别、又如 CT 技术、癌细胞识别、钙化点识别等总结：

传统图像处理自六十年代诞生以来，经过五十余年的不断发展，业已形成一门庞大的知识体系。其中包含了诸多算法如图像数字化、图像变换、图像增强、图像恢复、图像数据压缩、图像边缘检测、图像分割、图像特征分析、图像配准、图像融合、图像分类、图像识别、基于内容的图像检索、图形数字水印……，其广泛地应用于各种场景如地理信息科学、摄影摄像技术、航空航天、医学领域等，推动了现今社会的进步和发展。但同时，我们也应该注意到虽然传统数字图像处理可以解决诸多问题，然算法中参数的选择仍然依赖工程师的主观判断，可移植性仍有较大的进步空间。而人工智能思维的出现或许能够填补这一漏洞，使数字图像处理技术得到进一步发展。

4.4 体会启发与结语

我负责的部分是图像的数字矩阵化。在接触此次课题之前，我对矩阵的认知是相当局限的，仅仅由线性代数的基本课程了解到它对于线性方程组与一些坐标系变换的意义，并没有体会到它如何应用于我们实际的生活中，更何况是电子计算机领域呢。通过本次课题的学习，虽然过程稍有艰辛，但我学会了不少知识，体会到了数学思维是如何作用于人类发展历史上的。在本次课题的撰写与学习过程中，我使用了 C++ 和 Python 语言搭载 OpenCV 库来实操了一番，还是非常有趣的经历，也期待自己未来能够掌握更多的技术栈。当然，我需要更多不断地学习，需要学习更多软件工程方面的知识，才可以领会到更多的思想。——谭诚我个人写的是传统数字图像处理的起源、发展和应用。经过这次的小组作业学习。我对数字图像处理及其发展的脉络和多样的应用场景有了大致的了解。同时也引发了个人对于数字图像处理和计算机视觉的兴趣。在和小组成员的交流沟通中，我也得到了很多锻炼和启发。虽然前后查阅资料、组织语言、制作演示文稿和其他内容的部分是不易的，但当看到小组成员汇报的结果，还是觉得颇为值得。在简要了解完传统数字图像处理之后，发觉其仍存在一定程度上的缺陷，也尝试着去利用老师上课所教授的内容去思考，是否可以用人工智能的思维去改善现有的算法，不仅拓宽了知识面，也锻炼了自己的思维，感受到大有裨益。数字图像处理是丰富多彩、颇具魅力的，其应用领域的宽广也让我切实感到技术落地的魅力，大大增强了自我不断学习相关知识、努力探索学科领域的驱动力。提升了个人对于软件工程学科的认识水平，对自我是一次全方位的发展。——汪坤灿写完神经网络与目标检测的发展史以后，最大的收获是了解了一个自己从未涉足过的领域是如何发展的。从上世纪的摸索，到最近十年间的快速发展，无数新理论、新模型、新思想在不断诞生。对于非人工智能研究者而言，人工智能的发展看起来似乎是一蹴而就的事情。但是，仅仅在目标检测这一小小的领域，就经历了一个模型从诞生到不断完善每一个部分的过程，甚至直到现在还在不断发展。因此人工智能不该被当作一个概念而热炒，似乎只要高举支持人工智能旗帜就能顺应人工智能的浪潮，而更应该了解其发展历史与具体内涵。在写神经网络与目标检测两段内容之前，我可能都不知道机器学习和深度学习有什么区别和联系，这个写作过程也相当于给自己扫了个盲。我想这份经历也可以沿用到人工智能的其他分支，乃至其他领域的学科上。除此以外，在目标检测模型的发展方面，我的内容是按照问题—解决方案—新问题的思路来撰写的，通过这种方式来理解

每个看似不同的模型之间的内在联系，是一种线性连贯的解决问题方式，我想这可能也是一种比较大的启发。——史晓磊致谢自确定开题以来，小组中的每个成员都花费了大把时间来学习新知识。面对晦涩难懂的学术论文，以及眼花缭乱的公式与知识，每个成员的努力与付出都值得被铭记，每个成员的心血与成功都值得被尊重与感谢。

同时，我们也很感谢华东师范大学软件工程学院的吴敏老师，她为我们带来了半个学期的精彩淋漓的《人工智能的数学思维》课程。很感谢她为我们介绍了近代以来的数学知识发展以及对应电子计算机领域所诞生的新计算机科学，让我们了解到了数学思维与计算机科学的重要性。最后，我们也非常感谢华东师范大学软件工程学院 (East China Normal University, Software Engineering Institute, ECNU-SEI) 开设此门课程。互不相容的模型组成的森林比其中每一棵树都要睿智。

4.5 次线性算法

到这里为止，我非常感兴趣的还是那些有可能获得置信度的算法的性质。我同样确定，要寻找最优预测算法，囤积海量数据是必不可少的。然而，并不是所有数据都有价值。就像在沙漠深处安装的监视摄像头传来的视频流那样，实际上绝大部分收集来的数据毫无意义。问题在于，在这个大数据的时代，单单读取这些没有意义的数据来验证它们，真的没有意义，所需的时间就已经不切实际了——我就是这样说服自己不去查阅收到的全部邮件，这种做法没问题的！为了解决这个问题，理论计算机学家将注意力投向了所谓的次线性算法。这些算法的特点就是在不花时间读取所有数据的情况下，仍然能够提取数据集里最核心的内容。也就是说，这些算法能够在读取输入数据时“一目十行”。这种算法在历史上最典型的例子就是谷歌的算法。在谷歌上搜索几乎瞬间就能得到结果，然而，谷歌的数据库包含了互联网的数百亿网页中相当大的一部分。对于谷歌来说，在向用户返回结果之前探索整个数据库根本不在考虑范围之内，因为这样做要花上几天时间！因此，谷歌的搜索算法必须是次线性的。谷歌用到的技巧跟图书馆和词典一样，就是预先对数据库排序和整理，迅速完成对数据库的查阅。比如说，词典将单词按照字母顺序整理，就能让使用者很快知道希望查找的单词应该会在什么地方。更厉害的是，利用字母顺序，给定词典中的一页以及要查找的单词，使用者就能知道这个单词应该在词典中这一页的前面还是后面。一般来说，在（按照全序关系）排序过的数据库中查找数据非常迅速。所谓的二分算法（大概就是你用词典查单词的时候用的方法）的计算时间是数据库大小的对数。然而谷歌、图书馆管理员和词典使用的算法都需要预先进行大量的计算。要将所有网页整理并排序，除非对其全部访问，否则不可能做到。即使如此，我们能否在既不检索整个数据集，也不预先对其进行处理的前提下，将有用的信息提取出来呢？令人惊讶的是，对于某些有用的信息以及某些数据来说，答案是肯定的。傅里叶变换就属于这种情况。在 19 世纪初，约瑟夫·傅里叶等人就曾研究过傅里叶变换，它是一种对声音、图像、视频以及股票走势等信号的描述方式进行变换的方法 [8]。我们在这里就只讨论声音信号。声音可以通过鼓膜附近气压的变化来描述。但还有另一种等价的描述方式，它对于音乐的谱写来说特别简单，那就是利用组成声音的频率来描述这个声音。傅里叶变换就像一个双语词典，

能够将声音的振动变化描述转换为频率描述。这样的翻译有着很多用途，一个原因是要改善声音的话，调整频率通常比调整振动更有效，另一个原因就是声音通常只由为数不多的几个频率组成。

实际上，傅里叶变换已经占据了我们的日常生活，据理查德·巴拉纽克教授所言，就连我们的计算机和电话每天都会进行数十亿次傅里叶变换。在每次听音乐、查看数字化图像，或者观看视频的时候，我们都在让机器进行傅里叶变换。因此，计算傅里叶变换的算法如果有任何加速的可能性，都会在程序计算或者计算所需硬件的方面带来数十亿欧元的收益，更不用说用户节省下来的等待时间了。在 1964 年，詹姆斯·库利和约翰·图基就完成了一项壮举，大幅缩短了(离散)傅里叶变换的计算时间。尽管最简单的算法需要的时间与数据量的平方成正比，但库利和图基的算法，又叫快速傅里叶变换，可以在接近线性的时间内完成⁹。也就是说，要对一百万字节(1MB)的数据进行计算的话，快速傅里叶变换只需要数百万次运算(对于现代计算机来说，这相当于几毫秒的计算)，而不是朴素算法所需的 100 万乘以 100 万次运算(差不多 1 分钟的计算)。⁹ 也就是说离散傅里叶变换所需的时间复杂度从 $O(n^2)$ 变成了 $O(n\log n)$ 。然而在大数据的时代，快速傅里叶变换还是太慢了，尤其在处理上十亿字节(GB)甚至上万亿字节(TB)的数据时更是如此。我们能不能让傅里叶变换变得比现在更快？在 2012 年，哈桑尼耶、因迪克、卡塔比和普赖斯[9]发现，答案是肯定的。也可以说他们提出了一个巧妙的算法，可以对信号最主要的个频率进行近似计算，而需要的时间大概在乘以数据量的对数这个数量级¹⁰。重点在于，这个叫作稀疏傅里叶变换的精妙算法的运行时间是次线性的，也就是说它几乎忽略了整个待处理的信号，却能得知这个信号大概是什么。¹⁰ 它的复杂度实际上是 $O(k\log^2 n)$ 。思考的多种模式虽然次线性算法在计算机科学中仍不是主流，但我们的大脑似乎喜欢使用这种算法。谁又未曾一目十行读完书面材料，漫不经心地听别人讨论，或者吃饭时根本没有在意食物的味道呢？奇怪的是，有时候文章中的某些词语、讨论中的某些话题或者食物中的某些味道会吸引我们的注意力。当这种情况出现的时候，我们似乎突然就切换到了思考状态。我们会开始使用更缓慢、更精确的算法来仔细分析文章的含义、讨论的深意与本地特色菜的独特味道。我在这里描述的，正是诺贝尔经济学奖得主丹尼尔·卡内曼的杰作《思考，快与慢》的核心。卡内曼区分了两套思考系统：系统一和系统二。系统一就像稀疏傅里叶变换，非常迅速、有效、勤奋，但有可能出现严重错误；系统二就像精确傅里叶变换，很慢且需要大量能量，懒惰但更正确。据卡内曼所说，我们通常会将自己等同于系统二，而且几乎不会意识到系统一的存在。然而，在绝大部分时间内主导的都是系统一，而这会让我们经常犯错。试试这个：球拍和球的价格一共是 1.1 欧元，球拍的价格比球高 1 欧元，那么球的价格是多少？很可能你立刻就看到了答案，但这个答案是错的。根据卡内曼所说，如果这种情况出现，那是因为系统一匆匆忙忙给出了第一个想到的答案；只有在之后，也许是当你看到这里的时候，你的系统二才会开始质疑系统一。

我们可能会认为卡内曼希望说服我们放弃系统一，尽可能经常让系统二运作。并非完全如此。毕竟即使系统二得出的结果更正确，但对于大脑来说，使用系统二更疲劳。思考有其代价。然而，正如所有演员、钢琴家和冲浪者都知道的那样，我们不可能一一思考自己的所有行为与动作。我们最终必须能够自然且自发地做出这些动作，无须大量有意识的思考。要

做到无须长久思考就能做出合格的动作，演员、钢琴家和冲浪者用的是同样一套方法：他们的系统二会迫使系统一学习所需的动作。对于那些希望学习或者授课的人来说，这可能是最重要的意见。与其说学习是将信息塞进脑中，不如说是让系统二来教导系统一，使得系统一发现一条捷径，能够迅速解决那些系统二已经知道怎么解决，但要花上许多时间和能量的问题。也就是说，学习的根本就是发现又快又（足够）好的捷径。

4.6 迈进后严谨阶段!

数学的学习似乎也是如此进行的。在今天，如果有人问我一个关于加法、指数或者图灵机的问题，那么对我来说，不通过系统二就能回答这些问题也并非毫无可能。这是因为，长年以来我的系统二已经成功教会系统一如何在不花费脑力劳动的前提下回答这些问题！我的大脑中储存着大量捷径，能用于解决大量数学问题，即使是那些对于年轻学生来说略感困难的问题。某些人把这种能力称为数学能力，还有些人把它称为数学直觉。我更愿意把它称为系统二通过努力发现的高效捷径。但这不是系统一的数学训练中最重要方面。数学家、菲尔兹奖获得者陶哲轩在他的博客中 [10] 将数学学习分为三个阶段，分别叫作前严谨阶段、严谨阶段和后严谨阶段。简单来说，学生首先会开始摆弄数字和数学概念，而不会忧心于自己执行的那些代数操作的有效性。然后，随着时间流逝以及接受的数学教育越来越多，严谨性的时刻就会到来，而且很快会转变为纯粹主义：一切都应该用形式化的语言做出解释。最后，后严谨阶段属于研究者，他们花上大部分时间构建不同的启发式论证，用来得出定理证明的大体轮廓，其间会暂时舍弃之前学到的形式体系。关键在于，第二个阶段似乎是到达第三个阶段的必经之路。陶哲轩的说法可以用系统一和系统二的语言来理解。前严谨阶段对应的是系统一和系统二都没有学会严谨性的情况。当系统二发现严谨性及其重要性时，严谨阶段就开始了。在这个阶段中，系统二会教导系统一，让它意识到自身直觉的局限性。但更重要的是，系统一会由此学会测量自身的置信度，这样就能够更好地做出决定，判断自己能解决问题，还是需要向系统二求助。只有经过这种困难的学习过程，系统一才能成为系统二的完美协助者，而不会在重要的时候拖后腿。因此，成为一名优秀的数学家，基本上相当于让系统一足以在这方面胜任——但要达到这一状态，严谨阶段必不可少。话虽如此，即使到了后严谨阶段，系统二仍会不断教导系统一，让它能够进步。这就是在莱姆基·奥利弗和孙达拉拉詹发表了关于素数最后一位数字的研究结果之后，所有数论学家的大脑中发生的事情。这个发现惊动了数论学家的系统一，它关于素数的直觉基于素数定理，而这一捷径预言相邻素数的最后一位数字大多是独立的。在这个情况下，这个捷径出错了。自此之后，数论学家可能进行了某种近似贝叶斯推断，以减少素数定理这一捷径在相邻素数的情况中应用的置信度。丹尼尔·卡内曼的双系统思考模型对于实用贝叶斯主义者来说的确很有趣。当然，这个模型是错的。但毕竟“所有模型都是错的”。然而卡内曼的模型似乎很有用——即使第 17 章暗示还存在负责创造性思考过程的第三个系统。在实践中，尤其是在面对大数据的时候，大部分用到的算法大概是迅速的启发式算法，甚至是次线性的。然而拥有更缓慢但更正确的算法仍然是必要的。此外，如果我们拥有数个更缓慢、但我们知道足够正确的算法，那么我们就可以用这

些算法来训练那些快速的启发式算法。假如我能打个赌，(作为合格的贝叶斯主义者，我喜欢打赌!) 我会说这就是未来人工智能的模样。

线性代数 - 矩阵和向量运算：在数据表示和转换中使用，如矩阵乘法、转置和逆矩阵。 - 特征值和特征向量：用于数据降维和特征选择，如主成分分析 (PCA)。 - 奇异值分解 (SVD)：用于数据压缩、降噪和推荐系统。 - 矩阵分解技术：如非负矩阵分解 (NMF) 用于主题建模。

参考文献

- [1] “A Logical Calculus of Ideas Immanent in Nervous Activity”. In: *Bulletin of Mathematical Biophysics* 5 (1943), pp. 127–147.
- [2] Yoshua Bengio. “Practical recommendations for gradient-based training of deep architectures”. In: *Neural networks: Tricks of the trade: Second edition*. Springer, 2012, pp. 437–478.
- [3] George Cybenko. “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of control, signals and systems* 2.4 (1989), pp. 303–314.
- [4] Kunihiro Fukushima and Sei Miyake. “Neocognitron: A new algorithm for pattern recognition tolerant of deformations and shifts in position”. In: *Pattern recognition* 15.6 (1982), pp. 455–469.
- [5] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Multilayer feedforward networks are universal approximators”. In: *Neural Networks* 2.5 (1989), pp. 359–366.
- [6] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Universal approximation of an unknown mapping and its derivatives using multilayer feedforward networks”. In: *Neural networks* 3.5 (1990), pp. 551–560.
- [7] David H Hubel and Torsten N Wiesel. “Receptive fields and functional architecture of monkey striate cortex”. In: *The Journal of physiology* 195.1 (1968), pp. 215–243.
- [8] Han Jiawei, Kamber Micheline, and Pei Jian. *Data Mining: Concepts and Techniques*. -3rd. Morgan Kaufmann, 2011.
- [9] Fukushima K and Miyake S. “Neocognitron: A self-organizing neural network model for a mechanism of visual pattern recognition”. In: *Competition and cooperation in neural nets*. Springer, Berlin, Heidelberg, 1982, pp. 267–285.
- [10] Diederik P Kingma and Jimmy Ba. “Adam: A method for stochastic optimization”. In: *arXiv preprint arXiv:1412.6980* (2014).
- [11] Yann Le Cun, Boser Brian, and et al. Denker John S. “Handwritten digit recognition with a back-propagation network”. In: *Advances in neural information processing systems*. 1990, pp. 396–404.
- [12] Yann Le Cun et al. “Handwritten digit recognition: Applications of neural net chips and automatic learning”. In: *Neurocomputing: Algorithms, Architectures and Applications*. Springer. 1990, pp. 303–318.
- [13] Yann LeCun et al. “Backpropagation applied to handwritten zip code recognition”. In: *Neural computation* 1.4 (1989), pp. 541–551.

- [14] Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.
- [15] Minsky Marvin and A Papert Seymour. “Perceptrons”. In: *Cambridge, MA: MIT Press* 6.318–362 (1969), p. 7.
- [16] Warren S McCulloch and Walter Pitts. “A logical calculus of the ideas immanent in nervous activity”. In: *The bulletin of mathematical biophysics* 5 (1943), pp. 115–133.
- [17] Tom Mitchell. *Machine Learning*. McGraw-Hill Education, 1997.
- [18] Michael Nielsen. *Neural Networks and Deep Learning*. Accessed: 2025-03-03. 2015. URL: <http://neuralnetworksanddeeplearning.com/>.
- [19] Hastie T, Tibshirani R, and Friedman J. 统计学习基础 (*The Elements of Statistical Learning*). 北京: 世界图书出版公司, 2015.
- [20] Alan Turing. *Computing Machinery and Intelligence*. Oxford, 1950.
- [21] Vladimir Naumovich Vapnik. 统计学习理论的本质. 清华大学华夏出版社, 2000.
- [22] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).
- [23] 于剑. 机器学习—从公理到算法. 北京: 清华大学出版社, 2017.
- [24] [意] 翁贝托·博塔兹尼. 余婷婷译. 尖叫的数学—令人惊叹的数学之美. 湖南科学技术出版社, 2021.
- [25] 周志华. 机器学习. 清华大学出版社, 2016.
- [26] Vladimir N. Vapnik. 张学工译. 统计学习理论的本质. 北京: 清华大学出版社, 2000.
- [27] 张玉宏. 深度学习之美—AI 时代的数据处理与最佳实践. 电子工业出版社, 2018.
- [28] 张玉宏. 深度学习之美: AI 时代的数据处理与最佳实践. 电子工业出版社, 2018.
- [29] Tom Mitchell. 曾华军等译. 机器学习. 北京: 机械工业出版社, 2002.
- [30] 王珏, 周志华, 周傲英. 机器学习及其应用. Vol. 4. 清华大学出版社有限公司, 2006.

第五章 从传统机器学习到深度学习

“如果我们想创造智能机器，模仿大脑是最自然的路径。”

— 麦卡洛克 (Warren McCulloch)

在信息爆炸的时代，数据已然成为新的“石油”，而机器学习的使命，就是从这片数据海洋中提炼模式，挖掘其中的潜在规律。近年来，深度学习作为机器学习的一个重要分支，在语音识别、图像分类、自然语言处理、自动驾驶等领域取得了突破性进展，展现了惊人的应用价值。深度学习的成功秘诀是什么？简单来说，它依赖于三个关键因素：

- **大规模数据**：数据越多，模型的学习效果越好。就像一个见多识广的老侦探，比新手更容易识破复杂案件的真相。
- **高性能计算**：没有强大的计算资源，深度学习的庞大网络就像一辆缺少燃料的赛车，跑不起来。
- **深层神经网络的强大学习能力**：深度学习之所以“深”，正是因为它能层层递进，从细节推演整体，像一位善于归纳的学者，从海量信息中自动萃取关键特征。

5.1 从机器学习到深度学习

在传统机器学习中，模型学习前需要经过**人工特征工程 (Feature Engineering)**，由专家手动提取如颜色、边缘、形状等特征，再交由分类器进行学习。这种方法虽然有效，但存在两个主要问题：

- **高度依赖人为经验**：特征选取的好坏直接影响模型效果，且不同任务需要不同的特征工程，耗时耗力。
- **难以扩展**：面对海量数据和复杂任务，人工构造特征的方式就像是用算盘计算大数据，效率低下，根本跟不上时代的发展。

而**深度学习的最大突破**在于它利用**多层神经网络**自动从数据中学习特征，从低级模式（如边缘、纹理）逐步构建高级概念（如猫、狗），跳过传统机器学习中繁琐的特征工程步骤，直接从高维复杂数据（如图像、语音、文本）中提炼信息，大幅提升模型的泛化能力。

5.1.1 从人工筛选到自动归纳

机器学习的发展可以类比于人类识别世界的方式。传统机器学习像一位依赖手册的侦探，需要有人告诉它该找什么特征，比如“猫的耳朵是尖的，狗的嘴巴较大”。而深度学习像一位神探，通过自己观察和分析，自动归纳出哪些特征最关键，不再依赖人类给出固定规则。在传统机器学习中，特征提取和分类为是两个独立的步骤，需要人为拆分任务。而随着数据量的爆炸增长，人工特征提取的效率开始捉襟见肘。相比之下，以卷积神经网络 (CNN) 为代表的深度学习提供了另一种“端到端 (end-to-end)”的学习范式。这里的“end-to-end”（端到端）是指，

输入原始数据（始端），输出最终目标（末端），不再人为拆解子问题，如图 5.1 所示。深度学习

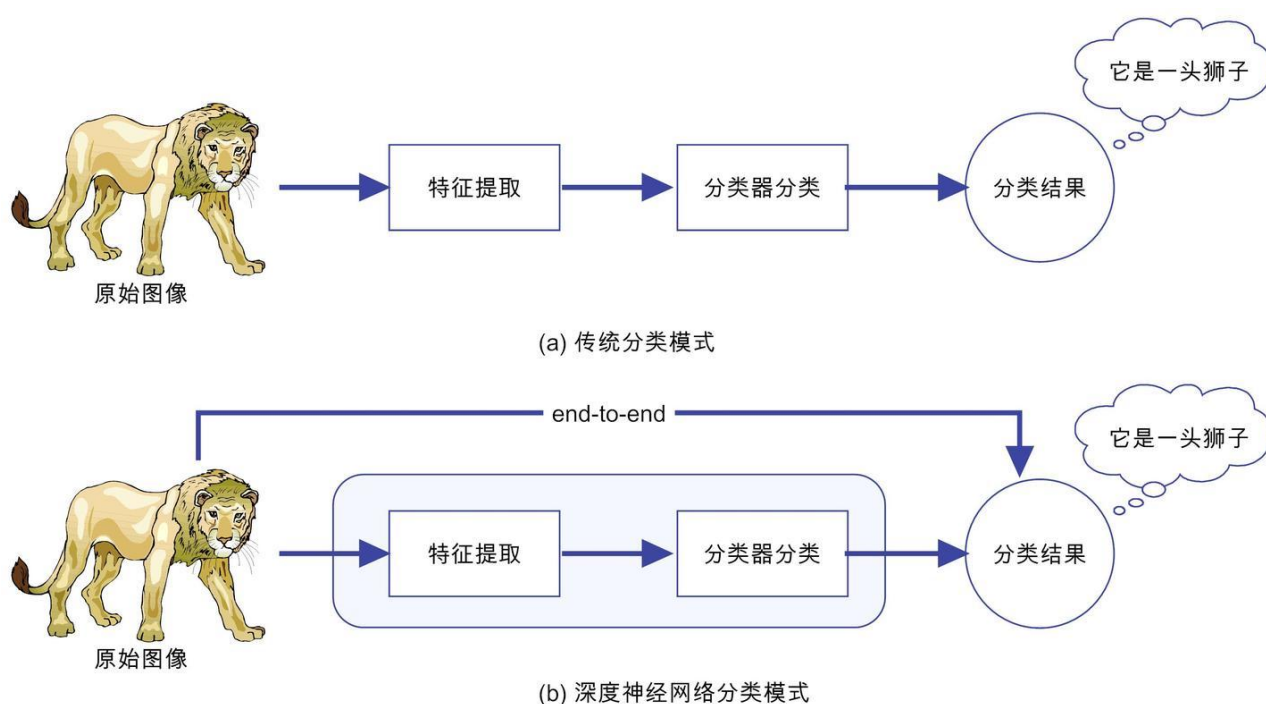


图 5.1: 传统模式分类与深度神经网络的区别与联系

习通过端到端的训练，让模型从原始数据直接学习映射到最终任务目标，避免了繁琐的手工特征工程，使得模型更具泛化能力和适应性。

5.1.2 从“人工扶持”到“自主学习”

5.1.3 像侦探一样找线索

想象你是一名侦探，手里有一堆线索（数据）。传统机器学习的方式类似于人工筛选线索——工程师手动挑选最关键的特征，例如，在图像识别任务中，他们可能会精心挑选边缘、颜色、形状等特征，然后交给模型去学习这些特征之间的关系。但问题在于，人类选特征的方式太主观了！如果人为定义的规则不够全面，机器在面对新的情况时就会“失灵”——比如，一只耳朵有缺口的猫，传统模型可能无法正确识别。深度学习换了一种思路——让机器自己找线索。神经网络通过多层学习，逐级提炼特征，就像一个经验丰富的侦探，从零碎的线索中推理出整个案件。换个角度理解，

- 传统机器学习：手工挑选特征，就像老师在考试前划重点，选对了才能考高分。
- 深度学习：让学生自己归纳做题方法，不用依赖老师划重点，学得更透彻。

5.1.4 学骑自行车的启示

还记得小时候学骑自行车的经历吗？一开始，你可能需要家长扶着，甚至装上辅助轮。经过反复练习，你终于找到了平衡，开始稳稳地骑行，甚至可以轻松应对各种路况。这和人工

智能的成长路径如出一辙：传统机器学习需要人工扶持（特征工程），就像学骑车时家长在旁边扶着。深度学习则学会了自己掌握平衡，逐渐适应各种复杂任务。以“猫”和“狗”的图像分类任务为例，在传统机器学习中，工程师需要告诉计算机：“看耳朵形状、毛发颜色、体型大小”来区分猫和狗。而当深度学习见过成千上万张猫和狗的照片后，会自己总结出哪些特征最有区分度，比如：喵星人的耳朵更尖，汪星人的嘴巴更大，甚至可能发现狗通常比猫更爱笑（很多狗的嘴角上扬，看起来像在微笑）。

5.1.5 深度学习的“洋葱式思考”

深度学习，顾名思义，就是一个“更深”的神经网络。其核心在于**层级式特征表示**——从浅层特征（如图像中的边缘）逐步学习到深层语义特征（如图片中的物体类别）。这一特性使深度学习在处理高维复杂数据时具有独特的优势。想象一下，当你观察一张猫的照片时，会先从局部细节感知，再整合形成完整认知。你的认知过程大致如下：

- 你先注意到边缘和线条（“哦，这里有个轮廓”）。
- 然后，你识别出形状和局部特征（“这像是耳朵，这里是眼睛”）。
- 最后，你得出结论：“这是一只猫！”

深度学习的工作原理也与此类似——底层学会边缘检测，中层识别局部结构，高层最终判断物体类别：

- **第一层（基础感知）**：提取最基础的边缘、线条、颜色等特征，类似于刚学会画画时画的“毛毛虫线”。
- **第二层（模式识别）**：组合这些特征，形成更具结构性的局部信息，比如眼睛、鼻子、耳朵。
- **第三层（高阶概念）**：综合所有特征，最早判断整张图片的类别，比如“猫”或“狗”。

5.1.6 深度学习为何如此强大？

5.1.6.1 超强泛化能力

深度学习的神奇之处不仅仅是能学会找特征，更在于它的“自适应”能力。试想一下，一个人如果学习新技能，每次都需要从头来过，那未免太低效了。而深度学习可以借助**迁移学习（Transfer Learning）**，让一个训练好的模型迅速适应新的任务。比如，一个训练好的猫狗识别模型，只需稍作调整，就能用于汽车品牌识别；又如，一个普通照片分类模型，可以扩展用于医学影像分析，识别癌细胞。这种“举一反三”的能力，让深度学习成为人工智能领域里的“最强大脑”。

5.1.6.2 数据越多，模型越强

传统机器学习面对大规模数据时太大时容易崩溃，而深度学习则表现出数据越多，模型越强的特性。例如，早期语音识别系统需要大量手工调试，而深度学习的语音识别模型，只

要有足够数据，就能自动优化，越来越精准。传统图像分类在数百万张图片上表现一般，而深度学习在百万、甚至亿级别数据集上仍能不断提升。深度学习的崛起，不仅仅是技术的进步，更是人工智能发展史上的一次范式变革。它让计算机从“被动学习”走向“主动探索”，让模型不仅能“做对题”，更能“学会思考”。

5.2 生物神经元的启发：模拟大脑

深度学习的历史发展，可以看作是一场不断逼近“智能化”的旅程。从最早的生物神经元研究，到人工神经网络的兴起，再到如今大型神经网络的惊人表现，这条道路充满了挑战与突破。理解其发展脉络，不仅能帮助我们更清楚地认识深度学习的本质，也能洞察人工智能未来的可能性。如果说深度学习是一座摩天大厦，那么它的地基便是人工神经网络（Artificial Neural Networks, ANN）。在本节中，我们将从深度学习的生物学启发出发，揭示它从理论探索到实际应用的关键突破，如何从最早的单个神经元模型进化到如今的超大规模人工智能系统。

5.2.1 深度学习的起源：向人脑学习

深度学习出现并非一蹴而就，而是沿着模仿人类智能的路径，一步步演化而来。想想看，人类是怎么变聪明的？无非是观察世界、提炼规律、然后找到解决问题的方法。“人法地，地法天，天法道，道法自然。”老子在《道德经》中所阐述的哲思，形象地体现了人类技术发展的灵感来源：效法自然。自然界的规律为人类提供了无穷无尽的灵感：从蝙蝠声呐导航的研究中，人们获得了发明雷达的灵感；从鱼鳔的启迪下，人们设计了潜水艇……同样，要让“人造物”具备类似人类的智能，模仿人类显然是最直接的路径。自古以来，人们便认识到智能是大脑生理活动的产物，科学家因此将研究目光投向了大脑。研究大脑的工作原理，用数学的方法模拟这种能力，成为了实现人工智能的理想途径。

5.2.2 智能如何产生：生物神经元的启发

人工智能的灵感来源于大脑的工作机制，而深度学习更是直接借鉴了生物神经元的信息处理方式。理解人工神经网络（Artificial Neural Network, ANN）的发展，需要先从其生物学原型——大脑的基本单元神经元（Neuron）说起。19 世纪末，西班牙科学家 Santiago Ramón y Cajal 通过显微镜观察到，大脑由独立的神经元构成，并提出了神经元学说（Neuron Doctrine），奠定了现代神经科学的基础。这一发现首次揭示了大脑的信息处理方式，使得后来的科学家得以研究神经元之间的信号传递机制。人类大脑是一个高度复杂的生物信息处理系统，由约 1000 亿个神经元（ 10^{11} neurons）组成，这些神经元通过突触（Synapse）相互连接，形成一个庞大的神经网络，使我们能进行感知、学习、记忆、思考和决策。整个大脑的突触连接数高达 10^{14} ，远超过现代计算机芯片的晶体管数量。尽管单个神经元的结构简单，但其功能强大。它由以下三个主要部分组成（如图 5.3 所示）：

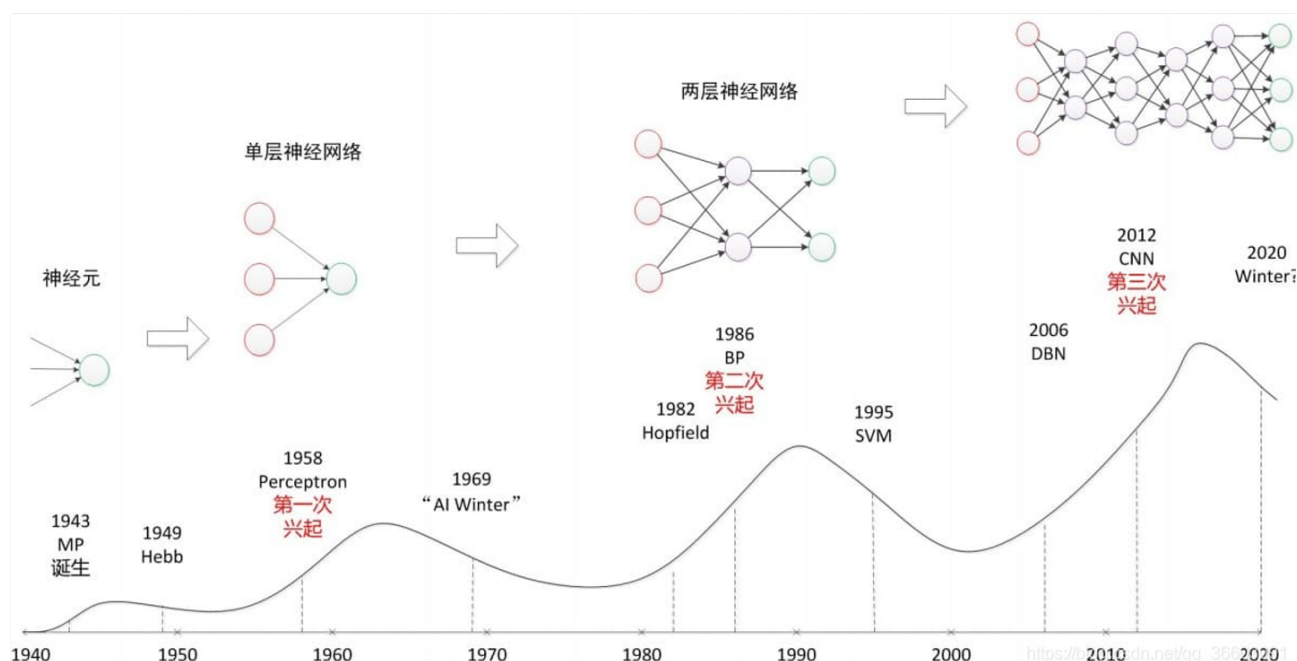


图 5.2: 人工神经网络发展史

1. **细胞体 (Soma)**：神经元的核心部分，整合来自多个突触的输入信号，并决定是否产生新的神经冲动。
2. **树突 (Dendrite)**：接收其他神经元通过突触传递来的电信号，相当于“天线”。
3. **轴突 (Axon)**：将信号传输到其他神经元，末端连接突触，形成信息传递链条。

生物神经元通过电信号和化学信号协同作用进行信息传递。尽管不像计算机使用二进制 (0 和 1) 进行编码，但其运行方式却具有某种“数字化”特征：神经元要么处于静息状态，要么在特定条件下触发信号。当神经元处于静息状态 (Resting State) 时，细胞膜内外的离子浓度保持稳定，细胞膜电位约为 -70mV 左右。此时，神经元处于抑制状态，不会主动传递信号。树突接收输入信号后，信号在细胞体中进行累积，当累积信号超过一定阈值时，神经元会被激活。“兴奋”神经元会通过轴突释放神经递质 (Neurotransmitter)，这些化学物质会改变下游神经元的膜电位。如果这种改变足够显著，受体神经元也会被激活，继续传输信号。这种信号的层层传播，犹如涟漪扩散，如图 5.3 与图 5.4 所示。具体来说，神经元的信息处理过程可分为以下五个阶段：

1. **静息状态 (Resting Potential)：信息存储阶段** 在未受到足够刺激时，神经元处于静息状态，细胞膜内外的电位保持稳定 (约 -70mV)。这一阶段类似于一座水闸——水流虽然存在，但尚未被释放。
2. **信号接收 (Input from Dendrites)：信息采集阶段** 神经元的树突 (Dendrites) 类似于雷达天线，从突触接收来自其他神经元的输入信号。输入信号可以是兴奋性 (Excitatory) 或抑制性 (Inhibitory) 信号，前者增加细胞内电位，使其更接近触发点，后者则降低电位，使其远离触发点。
3. **动作电位触发 (Action Potential Firing)：信号决策阶段** 如果输入信号的总和超过阈值 (Threshold, 约 -55mV)，神经元触发动作电位，形成“全或无” (All-or-Nothing) 的反应——要

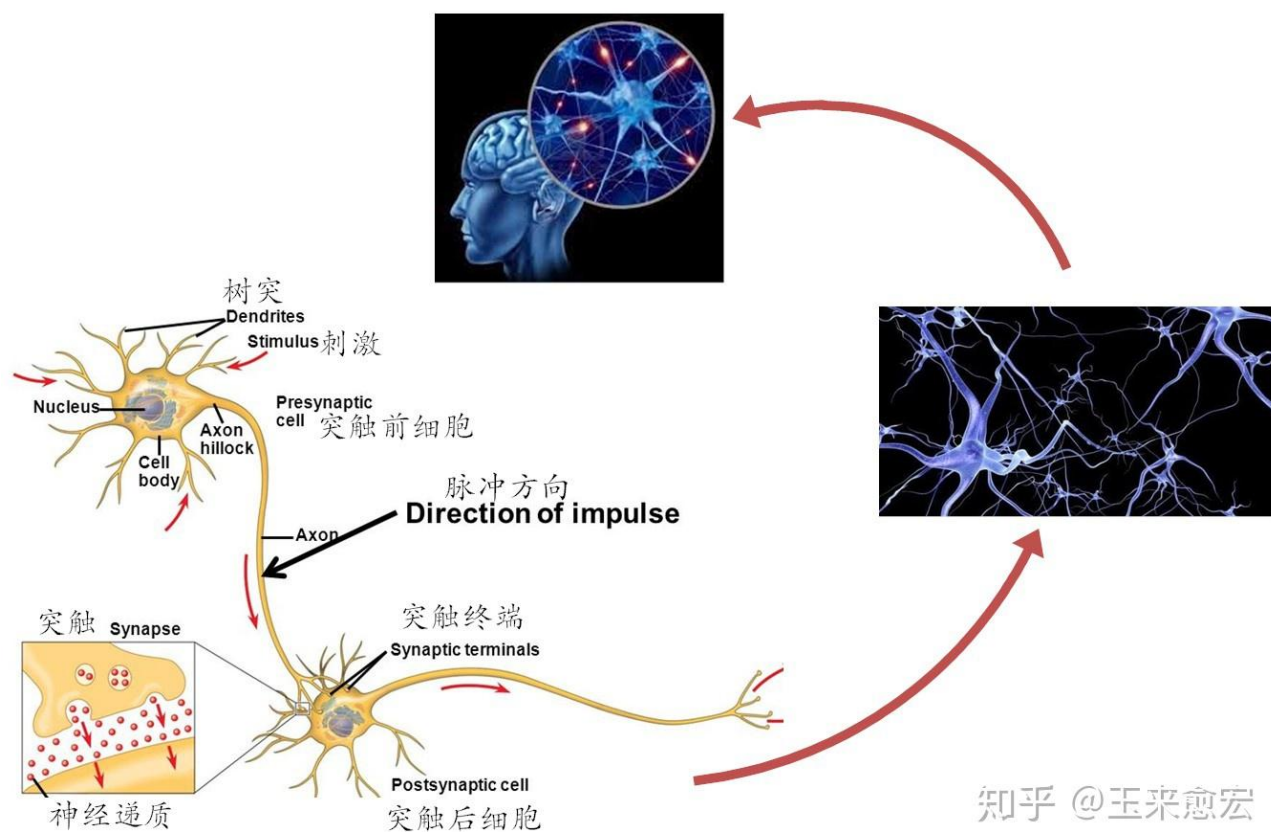


图 5.3: 大脑神经细胞的工作流程

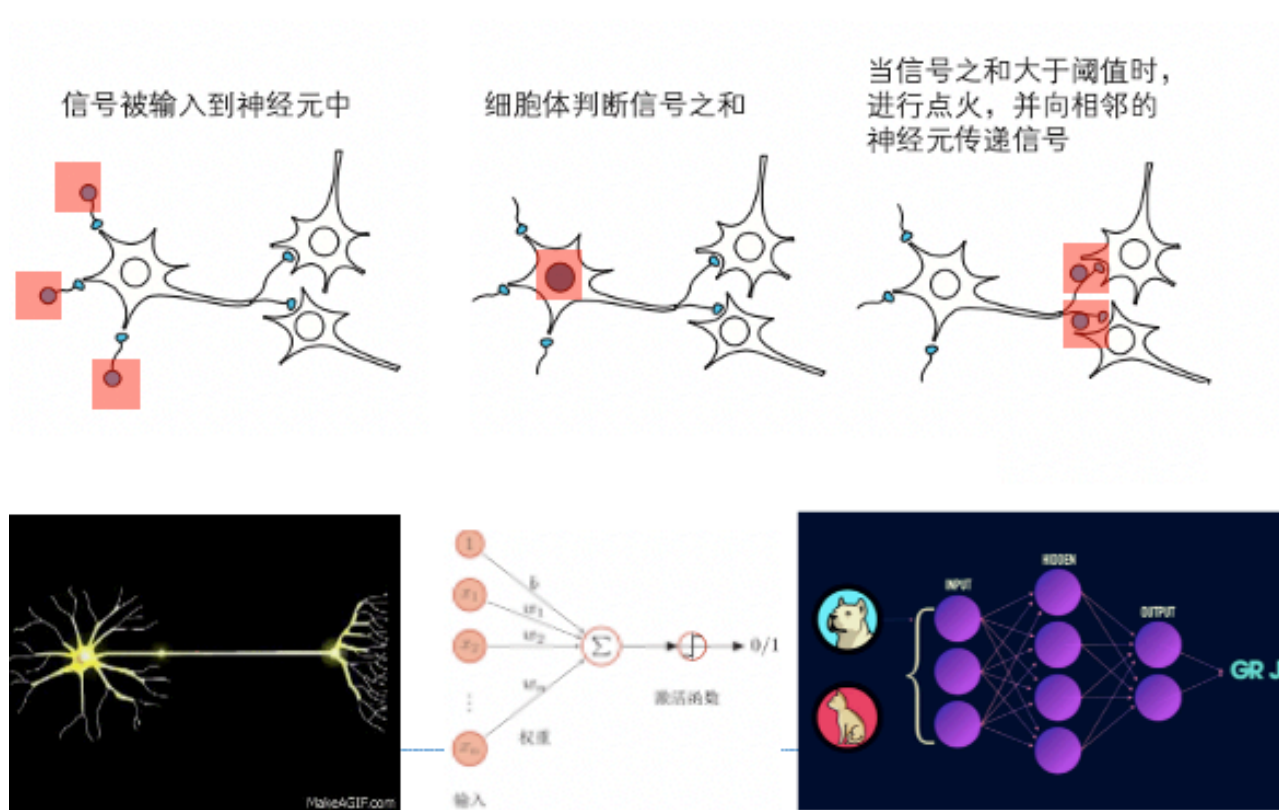


图 5.4: 生物神经元与人工神经元

么完全激活，要么什么都不做，类似于逻辑门（AND/OR）。

4. **信号传导 (Propagation along the Axon): 信息高速传输阶段** 触发的动作电位沿着轴突传播，速度可达 100 m/s。轴突通过髓鞘 (Myelin Sheath) 进行加速，类似于数据传输中的光纤加速。
5. **突触传递 (Synaptic Transmission): 信息共享阶段** 信号到达轴突末端后，神经元不会直接“接触”下一个神经元，而是通过神经递质 (Neurotransmitter) 进行化学信号传递。这些化学物质穿过突触间隙 (Synaptic Cleft)，作用于下一个神经元的树突，引发新的信号传递。

5.2.3 生物神经元计算模型的启示

大脑的强大计算能力源于“神经网络、非线性激活、权重调整”等特性。虽然神经元的信号传递机制依赖于简单的电-化学反应，但其高度并行、动态可塑的连接方式使其成为迄今为止最强大的信息处理系统。人工神经网络 (ANN) 的设计正是受此启发，深度学习的核心概念——信息传播、非线性激活、权重调整——均可在生物神经系统中找到生物学对应。

神经元的连接与信息传播

为便于阅读，本节（神经元的连接与信息传播）承接上节（生物神经元计算模型的启示），先交代差异与共同点，再聚焦本节关键问题。

单个神经元的功能虽然简单，但当成千上万个神经元“抱团”工作时，它们能够完成感知、学习和推理等复杂认知任务。这种网络化的连接模式直接启发了人工神经网络的层级结构，即多层神经元通过权重连接，使信息逐层提取，从低级特征（如边缘、颜色）逐步抽象为高级概念（如人脸、语义）。

非线性激活：从生物神经元到深度学习

为便于阅读，本节（非线性激活：从生物神经元到深度学习）承接上节（神经元的连接与信息传播），先交代差异与共同点，再聚焦本节关键问题。

神经元不会线性地响应所有输入，而是具有“全或无” (All-or-Nothing) 特性，即只有当输入信号超过一定阈值时，才会触发神经冲动。这一特性启发了深度学习中激活函数 (Activation Function) 的设计，使得人工神经网络能够学习复杂的非线性映射。

突触可塑性：权重调整的生物学基础

为便于阅读，本节（突触可塑性：权重调整的生物学基础）承接上节（非线性激活：从生物神经元到深度学习），先交代差异与共同点，再聚焦本节关键问题。

生物神经网络的学习能力来源于突触可塑性 (Synaptic Plasticity)，即神经元之间的连接强度会随着经验不断颁布。心理学家唐纳德·赫布 (Donald Hebb) 在 1940 年代提出的赫布学

习 (Hebbian Learning)。是这一现象的核心原理。其核心思想可概括为：“Fires together, wires together.” — 同时激活的神经元，其连接会变强。突触可塑性主要表现为长期增强 (LTP) 和长期抑制 (LTD) 两种形式，即“用进废退”原则：

- **长期增强 (Long-Term Potentiation, LTP)**：— 突触连接随着使用频率的增加而增强频繁同时激活的神经元，其突触连接会变强，以提高未来的信息传递效率，使大脑更容易记住信息。这类似于“熟能生巧”：使用越频繁的神经通路，传输效率越高。在深度学习中，这类似于梯度下降调整权重，使网络收敛。
- **长期抑制 (Long-Term Depression, LTD)**：— 突触连接随着使用减少而削弱如果某个突触长期得不到激活，它的连接会逐渐变弱，甚至被修剪掉，大脑会“遗忘”无用的信息。这类似于“用进废退”：不常使用的神经通路会被削弱，以优化大脑资源分配。在深度学习中，这类似于权重衰减 (Weight Decay)，防止过拟合，提高泛化能力。

突触可塑性的动态调整机制，为人工神经网络的权重更新提供了生物学依据，并直接启发了误差反向传播 (Backpropagation) 算法的设计。这种生物学习机制揭示了神经网络训练的基本逻辑：

学习 = 权重调整， 记忆 = 增强连接， 遗忘 = 连接削弱

神经科学的研究结果为人工神经网络的构建提供了重要启示，主要包括：

1. **神经元的层级结构启发了深度学习的多层神经网络**，即将简单的低层特征逐步组合成更复杂的高级特征。
2. **神经元的激活类似于非线性函数**，这促使人工神经网络引入 Sigmoid、ReLU、Softmax 等激活函数。
3. **突触可塑性启发了神经网络的权重调整机制**，对应于梯度下降 (Gradient Descent) 和误差反向传播 (Backpropagation)。

表 5.1 清晰地展示了生物神经网络与人工神经网络的相似性和差异。人工神经网络作为对生物神经网络的一种数学抽象，但仍存在诸多差距，例如能耗、学习速度、适应性和泛化能力等方面。生物大脑的学习能力为人工神经网络的构建提供了关键灵感，而深度学习的进化，在数学和计算机科学的加持下，让“智能”逐渐变得可计算、可训练、可优化。

5.3 M-P 模型：人工神经网络的起点

20 世纪 40 年代，计算机科学家开始思考如何用数学模型模拟生物神经元的行为。1943 年，神经生理学家沃伦·麦克洛克 (Warren McCulloch) 和数学家沃尔特·皮茨 (Walter Pitts) 发表了《神经活动中思想内在性的逻辑演算》(A Logical Calculus of Ideas Immanent in Nervous Activity) [16] 一文，被称作“神经网络天下第一文”。这篇文章中的 M-P 模型 (McCulloch-Pitts Model) 是人工神经网络的第一个数学化模型，被誉为神经网络研究的奠基之作。不仅奠定了人工神经网络的数学基础，还对计算机科学、控制论和存储程序计算机的发展产生了深远影响。不同于弗洛伊德的心理学模型，M-P 神经元模型是第一个将计算用于大脑的应用，同时

表 5.1: 从生物大脑到人工智能

特性	生物神经网络	人工神经网络
计算单元	神经元 (Neuron)	人工神经元 (Perceptron)
连接方式	突触 (Synapse)	权重 (Weight)
信号传递	电信号 + 化学信号 (神经递质传递)	数值加权求和 (矩阵运算)
信号类型	脉冲式 (All-or-Nothing, 动作电位)	连续值或离散值 (激活函数处理)
激活机制	非线性激活 (超过阈值触发)	非线性激活 (ReLU、Sigmoid、Softmax 等)
学习机制	突触增强 (LTP) 或削弱 (LTD)	误差反向传播 (Backpropagation)
优化方式	通过经验和环境自适应调整	通过梯度下降优化
学习时间	秒、分钟、甚至数年 (生物学习较慢但长期有效)	毫秒到小时 (训练速度快, 可快速调整权重)
优化方式	通过经验和环境自适应调整	通过梯度下降优化
能量效率	高效, 消耗极少能量 仅约 20W, 大脑功耗极低)	能耗较高 (深度学习需要大量计算资源和电力)
并行处理能力	超强并行 (10^{11} 个神经元并行计算)	受限的并行性 (GPU 并行计算, 但远不及大脑)
信息存储方式	分布式存储 (信息存储在突触连接中)	参数存储在权重矩阵
自适应能力	高度自适应, 可终身学习	受限的适应性 (需大量数据训练, 无法像大脑终身学习)
泛化能力	强 (可以举一反三, 适应多种环境)	有限 (依赖数据集, 可能存在过拟合问题)
容错性	高 (单个神经元损坏不会影响整个大脑)	低 (单个权重异常可能影响整个模型)
可解释性	难以解释 (大脑的学习机制仍未完全理解)	部分可解释 (可以通过梯度分析、可视化等方法理解)

也衍生出一个著名的论断“本质上，大脑不过就是一个信息处理器。——计算神经学”“M-P 神经元模型”，是对生物大脑的极度简化，却成功地给我们提供了基本原理的证明。使得关于“思想”的理解，在某种程度上变得更加具有可解释性，而不必笼罩一层弗洛伊德式的神秘主义，然后在自我与本我之间牵扯不清。于是，麦克洛克曾向一群研究哲学的学生骄傲地宣布：“在科学史上，我们首次知道了我们是怎么知道的（For the first time in the history of science, we know how we know）。”ss

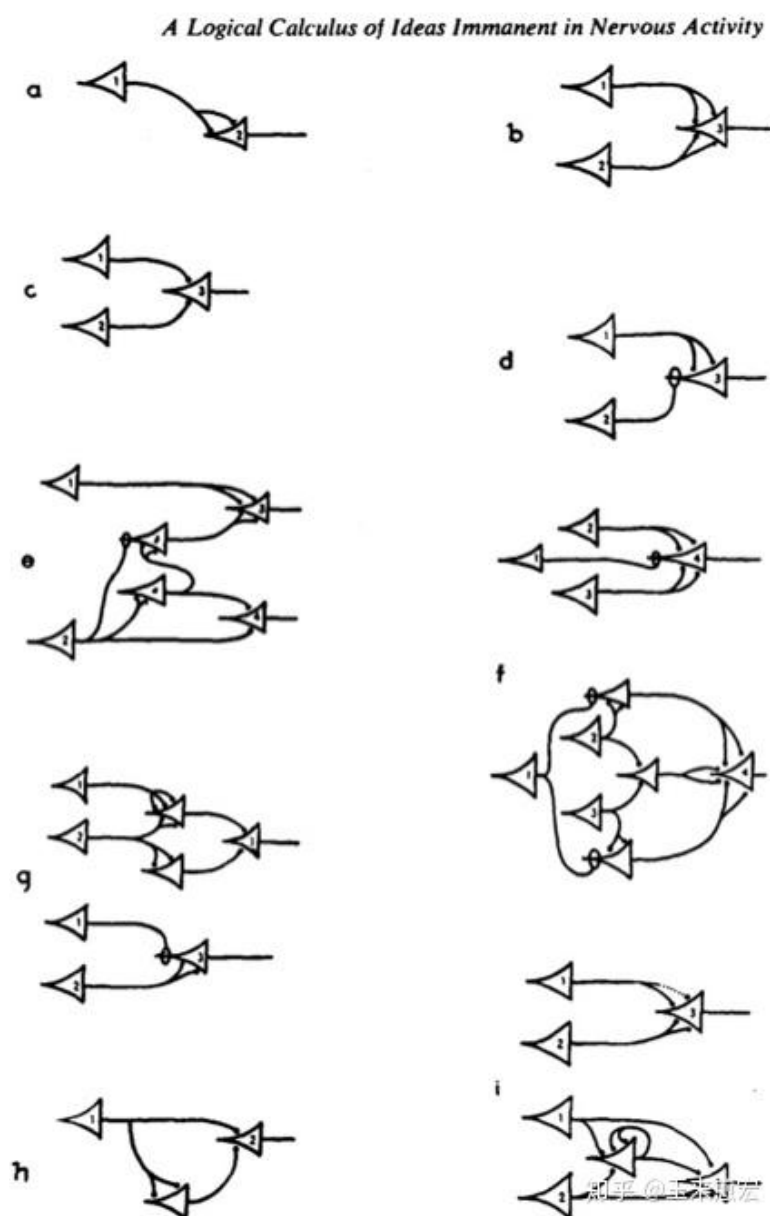


图 5.5: 神经网络天下第一文

5.3.1 M-P 模型的基本结构

M-P 模型的核心思想是，将生物神经元的工作方式简化为数学模型，用二值逻辑描述信息传递的过程。其基本计算流程如下（如图 5.6）：

1. 输入 (Inputs, x_i): 来自其他神经元的二值输入 (0 或 1)。
2. 权重 (Weights, w_i): 每个输入的影响程度 (可以是正数或负数)
3. 加权求和 (Weighted Sum, S): 对所有输入按权重进行求和:

$$S = \sum_{i=1}^n w_i x_i$$

4. 阈值 (Threshold, θ): 决定神经元是否激活的临界值。
5. 激活函数 (Activation Function): 使用阶跃函数 (Step Function) 进行二值化:

$$y = \begin{cases} 1 & \text{if } S \geq \theta \\ 0 & \text{if } S < \theta \end{cases}$$

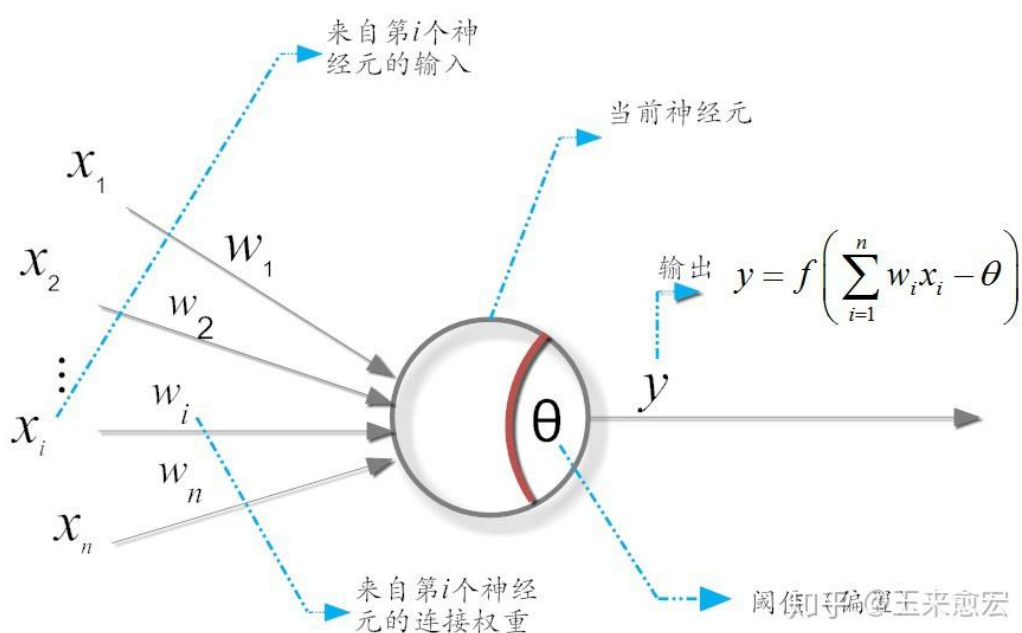
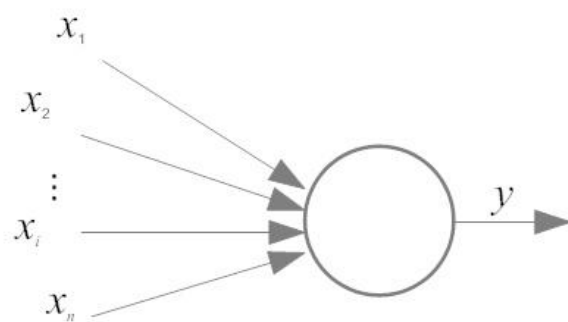


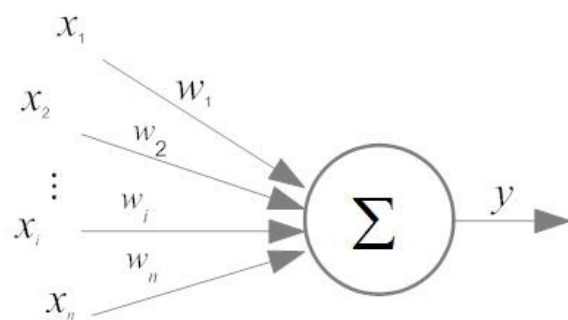
图 5.6: M-P 神经元模型

麦卡洛克和皮茨的神经元模型还可以被描述图 5.7 所示的样子。图 5.7 的描述与现在神经网络中的表示几乎没什么差别了，图 5.7(a) 表示神经元从外界获得输入 $x_1, x_2, \dots, x_n$ (箭头模拟表示突触)，传给神经元，神经元的输出是 y 。图 5.7(b) 突触根据不同的性质以及强度，赋予一个不同的权重，然后，神经元的输入现在升级为一个加权和。图 5.7(c) 表示的是，加权和并不是“赤裸裸”地直接输入，而是阈值函数（亦称激活函数）将“全（1）或无（0）”法则，引入到神经元结构中，这个法则模拟的是神经元激活或抑制。**M-P** 神经元模型的提出，意味着我们可以把大脑视为计算机的存在，由此奠定了人工神经网络的数学基础：

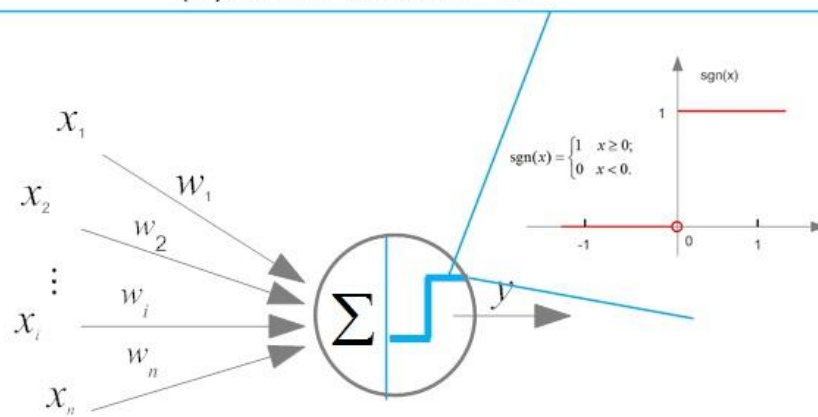
- 神经元的状态可以用二值 (0 和 1) 表示：兴奋为 1，不兴奋为 0。
- 二值化输入和输出：所有输入和输出只能取 0 或 1，模拟生物神经元的“全或无”特性。
- 神经元可以被数学化表示—输入信号经过加权求和，并通过一个阈值函数决定是否激活输出。
- 逻辑运算可以用神经元实现—基本的布尔逻辑运算（如 AND、OR、NOT）都可以用神经元的连接方式模拟。



(a) 获取输入



(b) 对输入加权求和



(c) 对加权值“激活”输出

知乎 @玉来愈宏

图 5.7: M-P 模型的另一种描述

5.3.2 M-P 模型如何实现逻辑运算

由于 M-P 模型的输出是 0 或 1，它可以直接模拟布尔逻辑运算。以下是它实现基本逻辑门的示例：

1. AND 逻辑门

为便于阅读，本节（**1. AND 逻辑门**）承接上节（M-P 模型如何实现逻辑运算），先交代差异与共同点，再聚焦本节关键问题。

表 5.2: AND 逻辑门

x_1	x_2	输出 y
0	0	0
0	1	0
1	0	0
1	1	1

设定：

- $w_1 = 1, w_2 = 1$
- $\theta = 2$

计算方式：

$$S = x_1 \cdot 1 + x_2 \cdot 1$$

如果 $S \geq 2$, 输出 1，否则输出 0。

2. OR 逻辑门

为便于阅读，本节（**2. OR 逻辑门**）承接上节（**1. AND 逻辑门**），先交代差异与共同点，再聚焦本节关键问题。

表 5.3: OR 逻辑门

x_1	x_2	输出 y
0	0	0
0	1	1
1	0	1
1	1	1

3. NOT 逻辑门

为便于阅读，本节（**3. NOT 逻辑门**）承接上节（**2. OR 逻辑门**），先交代差异与共同点，再聚焦本节关键问题。

设定：

表 5.4: NOT 逻辑门

输入 x	输出 y
0	1
1	0

- $w = -1$
- $\theta = -0.5$

4. 异或 XOR 逻辑门

为便于阅读，本节（4. 异或 XOR 逻辑门）承接上节（3. NOT 逻辑门），先交代差异与共同点，再聚焦本节关键问题。

表 5.5: 异或 XOR 逻辑门

x_1	x_2	输出 y
0	0	0
0	1	1
1	0	1
1	1	0

设定:

- $\theta = -0.5$

M-P 模型的成功展示了简单的神经元可以通过不同的权重和阈值，实现基本的逻辑推理能力。尽管 M-P 模型奠定了人工神经网络的数学基础，但它存在以下主要局限性：

1. 无法解决非线性问题：XOR（异或）问题无法用单个 M-P 神经元表示，因为 XOR 不是线性可分的。
2. 缺乏学习能力：权重和阈值需要人工设定，无法通过数据训练自动优化。
3. 输入与输出的二值化限制：M-P 模型仅支持二值输入和输出，无法表示和处理连续值或概率性信息。

由于这些局限性，M-P 模型虽然奠定了人工神经网络的基础，但不能用于复杂的学习任务。M-P 模型是人工神经网络发展的第一步，它的提出不仅奠定了人工神经网络的数学基础，更在多个学科产生了深远影响。其价值主要体现在以下三个方面：

1. 将神经科学引入数学建模，使人工神经网络成为一个可计算的系统。M-P 模型首次将生物神经元的计算机制用数学模型描述，使神经科学与逻辑学相结合。这一突破性思路影响至今，迄今，许多研究者仍然认为，计算机智能的发展终将回归对神经科学和大脑的研究。
2. 影响计算机科学，推动存储程序计算机与控制论的发展 M-P 模型为现代计算机科学提供了由神经元构成的神经网络实现的逻辑计算的借鉴，另一方面，也为现代存储程序计算机的设计提供了早期理论参考，M-P 模型为图灵机和现代的存储程序计算机之间，搭建了一座难以忽略的桥梁，并深刻影响了控制论、信息论的兴起。

3. 激发神经网络学习的发展，奠定深度学习的理论框架 M-P 模型的局限性在于无法自动学习，但它促使后续研究者不断探索如何让神经网络具备自学习能力。从皮茨开始，研究者们致力于设计可调节参数的神经网络，最终催生了感知机（Perceptron）、多层感知机（MLP）、BP 算法、卷积神经网络（CNN）、循环神经网络（RNN）、深度信念网络（DBN）等诸多技术。它们都是致力于在不同应用场景下，自动快速找到网络的连接权值。如今，神经网络学习（包括深度学习）已经占据了人工智能领域的半壁江山，M-P 模型可谓居功至伟。

M-P 模型，不仅是人工神经网络的起点，更是人类探索智能的里程碑。从 Cajal 的神经元学说到麦克洛克-皮茨的数学模型之间，科学史实现了一次重要的飞跃：人类不仅能够观察神经元的行为，还能够用精确的数学语言描述和预测这些行为。这种从定性到定量的转变，不仅使认知神经科学和脑科学脱离了玄学的边缘，更为人工智能的崛起提供了坚实的理论框架。

小故事：数学天才遇上神经科学家

麦克洛克的人生，走的是一条妥妥的精英科学家路线。他出生于美国东海岸的贵族家庭，从小接受精英教育。在耶鲁大学研究哲学和心理学，随后在哥伦比亚大学取得心理学硕士和医学博士学位。他的研究领域横跨神经生理学、精神病学和计算理论。但麦克洛克的内心更向往哲学，希望探索知识的终极意义。当时，弗洛伊德的精神分析学派如日中天，而麦克洛克坚持认为，种种精神活动，包括神秘工作及精神失常，都来自于神经元的机械式反应。他更倾向于用数学和生物学解释大脑的运作方式。相比之下，皮茨是个非典型天才——他没有上过正式大学，但他的数学才能让众多科学家惊叹。14 岁时，他便自学掌握罗素和怀特海的《数学原理》（*Principia Mathematica*）靠，这是一本许多数学家都难以完全掌握的巨著。16 岁时，他因家境贫寒流浪到芝加哥，偶然遇到了数学家鲁道夫·卡恩（Rudolf Carnap），并成为他的学生。18 岁时，他遇到了 42 岁的麦克洛克。皮茨精通逻辑推理和计算理论，而麦克洛克正急需一个数学天才来帮助他用数学形式化描述大脑的逻辑，两人一拍即合。两人一拍即合，皮茨的数学能力完美补足了麦克洛克的神经科学知识。罗素等人在书中论述说，“所有的数学法则，都可自下而上地用无可辩驳的基本逻辑来建立（all of mathematics could be built from the ground up using basic, indisputable logic）”。而在底层、基础的逻辑，即为是与非两种判定。然后，通过对这两个逻辑判定进行一系列的组合操作，例如，合取（conjunction）实现“与（and）”操作、析取（disjunction）实现“或（or）”操作、取反（negation）实现“否（not）”操作，这样就可以一砖一瓦地构建起复杂的数学大厦。受《数学原理》的启发，麦克洛克当时正尝试尝试用莱布尼茨的二进制逻辑来构建一个大脑模型。他推测：神经元的工作机制可能类似于逻辑门电路，接受多个输入，通过阈值决定是否激活，进而执行“与”（AND）、“或”（OR）、“非”（NOT）等逻辑运算。这一思想为后来的人工神经网络奠定了理论基础。然而，大脑的工作方式比数学逻辑更复杂。神经元网络可能形成环状连接，即最后一个神经元的输出又成为第一个神经元的输入，这导致了逻辑悖论——“结果”成了“原因”。麦克洛克一度被这个难题困住，而皮茨巧妙地通过模数法（Modulo Mathematics）解决了这个问题，把最后一个输出巧妙地变成了第一个输

入。就像时钟的数字循环一样，12 点的下一个小时不是 13，而是 1。从人的反应消耗能量来看，这一设置非常智能。神经元细胞的二元工作状态（即激发或静默），让麦克洛克联想到了莱布尼茨二进制逻辑单元以及罗素的数学构建论断。大脑的工作机制复杂而神秘，但其底层逻辑，是不是也构建于神经元的简单输出呢？于是，麦克洛克猜想，**神经元的工作机制很可能类似于逻辑门电路，它接受多个输入，然后产生单一的输出。通过改变神经元的激发阈值以及神经元之间的连接程度，就可以让它执行“与”“或”“非”等功能。**当时，麦克洛克刚读了一篇英国数学家阿兰·图灵（Alan Turing）的论文。麦克洛克认为，大脑的工作机理很可能也是这样的一种机器，它用编码在神经网络里的逻辑来完成计算。如果神经元可用逻辑规则连接起来，这样就构建了结构更为复杂但功能更加强大的思维链（Chains of Thoughts）。这种方式与《数学原理》将简单命题链（Chains of Propositions）连接起来，以塑造更加复杂的数学定理，是一致的。当麦克洛克向一群哲学学生宣布这一发现时，他骄傲地说：“在科学史上，我们首次知道了我们是怎么知道的。”（For the first time in the history of science, we know how we know.）他们的研究受到阿兰·图灵（Alan Turing）的启发，尝试用逻辑演算模拟大脑的计算能力。最终，他们成功建立了 M-P 神经元模型（McCulloch-Pitts Model），这是人工神经网络的第一个数学模型，开启了用数学理解智能的时代。几十年后，他们的思想依然影响着现代深度学习，其中的逻辑连接启发了人工神经网络，环状结构则成为了循环神经网络（Recurrent Neural Network, RNN）的重要概念。更多精彩内容，可以参见 [28] 中的《牛掰扫地僧：M-P 模型背后的那些人和事》。

5.4 感知机：第一个可训练的神经网络，神经网络的“Hello World”

感知机（Perceptrons），受启发于生物神经元，它是一切神经网络学习的起点。感知机学习，就是神经网络学习的“Hello World”，是很多从事计算机行业的人“光荣与梦想”开始的地方。1957 年，康奈尔大学心理学教授弗兰克·罗森布拉特（Frank Rosenblatt）在 M-P 模型基础上提出了“感知机”（Perceptron）的概念。这是第一个能够自动调整权重、进行学习的人工神经网络。所谓的感知机，其实是一个由两层神经元构成的网络结构，它在输入层接收外界的输入，通过激活函数（含阈值）的变换，把信号传送至输出层，因此它也称之为“阈值逻辑单元（threshold logic unit）”。感知机的提出标志着人工智能进入了可训练学习的阶段，因此，它被称为神经网络的“Hello World”。感知机后来成为许多神经网络的基础，但它的理论基础依然建立于皮茨等人提出来的“M-P 神经元模型”。罗森布拉特提出的感知机模型如图 5.8 所示。对比图 5.7 和图 5.8，感知机模型可以看作一个带有反馈机制的 M-P 神经元模型。感知机虽然简单，但已初具神经网络的必备要素。在前面我们也提到，所有“有监督”的学习，在某种程度上，都是分类（classification）学习算法。而感知机就是有监督的学习，因此它也是一种分类算法。

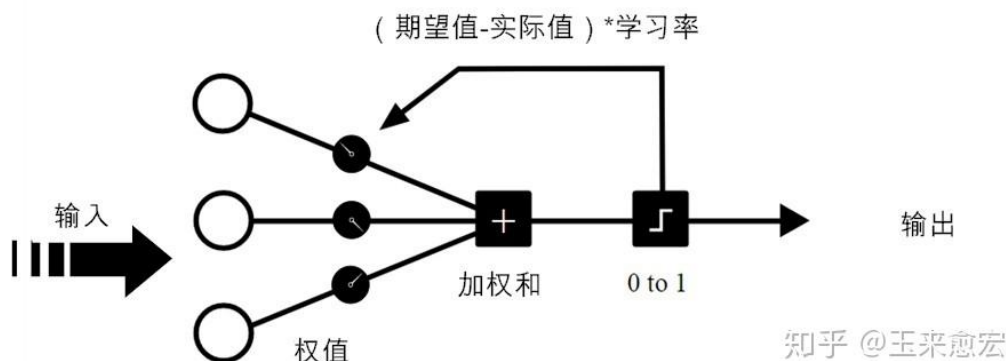


图 5.8: 罗森布拉特提出的感知机模型

5.4.1 感知机的工作原理

每一次练习都是以左面的输入神经元为开端，先给每个输入神经元都赋上随机的权重，然后计算它们的加权输入之和。如果加权求和为负数，则预测结果为 0，否则，预测结果为 1（这里的 0 或 1，对应于图片的“左”或“右”，在本质上，感知机实现的就是一个二分类）。如果预测是正确的，则无须修正权重；如果预测有误，则用学习率（Learning Rate）乘以差错（期望值与实际值之间的差值），来对应地调整权重，如图 5.8 所示。学着，学着，这部感知机就能“感知”出最佳的连接权值。然后，对于一个全新的图片，在没有任何人工干预的情况下，它能“独立”判定出图片标识为左还是为右。这个过程，不就和小孩的学习差不多吗？儿童成长的第一阶段（从出生到 2 岁，相当于婴儿期）就是感知阶段。此阶段的儿童主要靠感觉和动作探索周围的世界，逐渐形成物体永存性观念，慢慢就有了自己对事物的独立判断。感知机的数学表示为：

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right) \quad (5.1)$$

其中， w_i 为权重， x_i 为输入信号， θ 为偏置， σ 为激活函数（如阶跃函数）。除了神经元的权重连接之外，神经元内部还有一个施加于自身的特殊权值—偏置（bias），即(5.1)中的 θ 。偏置决定了神经元的连接加权求和得有多大，才能让激发变得有意义。感知机的目标是通过训练学习输入和输出之间的映射关系。它采用简单的更新规则优化权重：

$$w_i \leftarrow w_i + \eta(y_{\text{target}} - y_{\text{actual}})x_i$$

其中， x_i 为输入， η 为学习率， y_{target} 为期望的真实输出， y_{actual} 为模型的当前输出值。具体地，感知机的运行包括以下三个步骤：

1. 输入数据加权求和：每个输入 x_i 乘以相应的权重 w_i ，然后求和

$$S = \sum_{i=1}^n w_i x_i - \theta$$

其中 θ 是阈值。非齐次项 $-\theta$ 通常用 b 表示并被称作偏置（bias）。于是有

$$S = \sum_{i=1}^n w_i x_i + b \quad (5.2)$$

2. 通过激活函数决定输出类别：计算的结果 S 通过阶跃函数（step function）处理：

$$y = \text{sgn}(S) = \begin{cases} 1 & \text{if } S \geq 0 \\ 0 & \text{if } S < 0 \end{cases} \quad (5.3)$$

这样，感知机实现了一个二分类任务。

3. 使用误差反馈训练（感知机学习规则）：如果预测（分类）错误，则触发学习规则。我们应用如下的感知机权重更新规则：

$$w_i \leftarrow w_i + \eta(y_{\text{target}} - y_{\text{actual}})x_i, \quad (5.4)$$

$$\theta \leftarrow \theta + \eta(y_{\text{target}} - y_{\text{actual}}) \times 1 = \theta + \eta(y_{\text{target}} - y_{\text{actual}}) \quad (5.5)$$

其中 η 是学习率， y_{target} 为期望输出的真实值， y_{actual} 为感知机的当前输出值。不断迭代，直到感知机能够在给定的训练集上实现正确分类。预测“误差”就是整个网络中权值和阈值的调整动力。“知错能改，善莫大焉。”很显然，如果为 0，即没有误差可言，那么新、旧权值和阈值都是一样的，网络就稳定可用了。感知机参数学习的更新过程感知机的负反馈纠偏机制

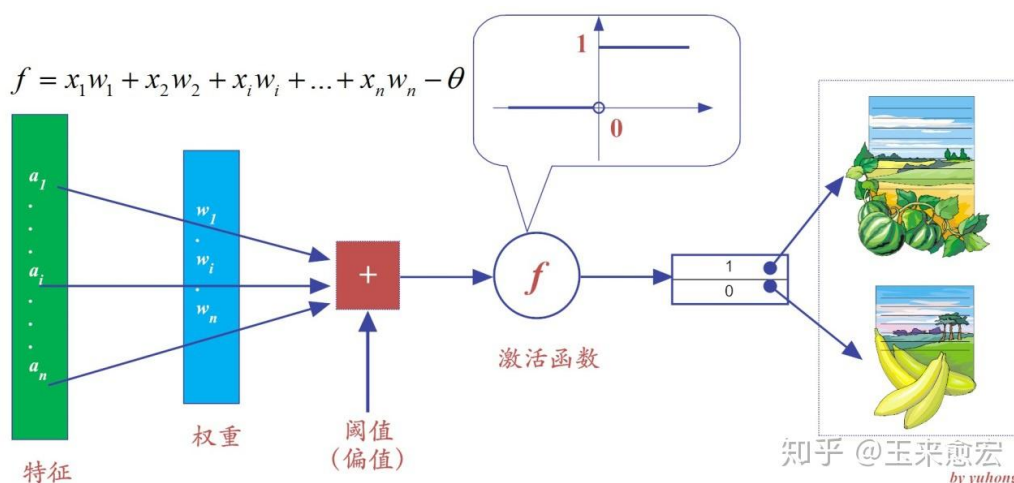


图 5.9: 感知机学习算法示意图

放到机器学习领域，这句话显然属于“监督学习”的范畴。因为“知错”，就表明事先已有了事物的评判标准，如果你的行为不符合（或说偏离）这些标准，那么就要根据“偏离的程度”，来“改善”自己的行为。下面，我们就根据这个思想来制订感知机的学习规则。从前面的讨论中，我们已经知道，感知机学习属于“有监督学习”（即分类算法）。感知机有明确的结果导向性。这有点类似于“不管白猫黑猫，抓住老鼠就是好猫”的说法，不管是什么样的学习规则，能达到良好的分类目的，就是好的学习规则。我们知道，对象本身的特征值，一旦训练样本确定下来就不会变化，可视为常数。因此，所谓的神经网络的学习规则，就是调整神经元之间的连接权值和神经元内部阈值的规则（这个结论对于深度学习而言，依然是适用的）。

5.4.2 感知机示例：“西瓜 vs 香蕉” 分类

在本节中，我们列举一个区分“西瓜和香蕉”的经典案例，来看看如何训练感知机，使其能够区分西瓜和香蕉。为了简单起见，假定西瓜和香蕉都仅有基于视觉刺激最容易得到的两个特征 (feature)：颜色 (绿色或黄色) 和形状 (圆形和弯形)，其它特征暂不考虑。设特征 x_1 取值为 1 若颜色为绿色，反之取值为 -1 若颜色为黄色；同样，特征 x_2 取值为 1 若形状为圆形，反之取值为 -1 若形状为弯曲状。特征的取值如表 5.6 所示。假设感知机的输出数字化，如果输出为“1”则代表分类为“西瓜”，而输出为“0”代表分类为“香蕉”。假设初始权重为 $w_1 = 1, w_2 = -1, b = 1$ 。接下来，我们根据感知机的参数更新流程逐步获得能将西瓜和香蕉分类正确的感知机。(1)

表 5.6: 西瓜与香蕉的特征值表

品类	颜色 (x_1)	形状 (x_2)
西瓜	1 (绿色)	1 (圆形)
香蕉	-1 (黄色)	-1 (弯形)

计算西瓜的分类。对于西瓜，特征 $x_1 = x_2 = 1$ 。于是决策函数

$$S = w_1x_1 + w_2x_2 + b = 1 \times 1 + (-1) \times 1 + 1 = 1 > 0$$

根据公式 (5.3)，输出 $y_{\text{actual}} = 1$ ，即感知机实际输出“西瓜”，分类正确。同样，计算香蕉的分类。对于香蕉，特征 $x_1 = x_2 = -1$ 。于是决策函数

$$S = w_1x_1 + w_2x_2 + b = 1 \times (-1) + (-1) \times (-1) + 1 = 1 > 0$$

根据公式 (5.3)，输出 $y_{\text{actual}} = 1$ ，即感知机实际输出“西瓜”，分类错误。(2) 计算预测误差：

$$\epsilon = y_{\text{target}} - y_{\text{actual}} = 0 - 1 = -1$$

(3) 由于香蕉的分类错误，需要纠偏。而纠偏，就需要利用公式 (5.4) 所示的学习规则更新网络的权重和阈值：

$$\begin{aligned} w_i &\leftarrow w_i + \eta \epsilon x_i, \quad i = 1, 2, \\ b &\leftarrow b + \eta \epsilon. \end{aligned}$$

为了简化计算，假设学习率 $\eta = 1$ 。于是，

$$\begin{aligned} w_1 &\leftarrow w_1 + 1 \times (-1) \times (-1) = 1 + 1 = 2, \\ w_2 &\leftarrow w_2 + 1 \times (-1) \times (-1) = -1 + 1 = 0, \\ b &\leftarrow 1 + 1 \times (-1) = 0 \end{aligned}$$

在新一轮的网络参数（即权值、阈值）重新学习后，我们再次输入香蕉的属性值，测试一下，看看它能否正确判定：

$$S = w_1x_1 + w_2x_2 + b = 2 \times (-1) + 0 \times (-1) + 0 = -2$$

再经过激活函数（阶跃函数）处理后，实际输出结果 $y_{\text{actual}} = 0$ ，分类为香蕉，预测正确！我们知道，一个对象的类别判定正确，不能算好。于是，在判定香蕉正确后，我们还要尝试在这

样的网络参数下，看看西瓜的判定是否正确。对于西瓜， $x_1 = x_2 = 1$ ，于是决策函数

$$S = w_1x_1 + w_2x_2 + b = 2 \times 1 + 0 \times 1 + 0 = 2$$

类似的，经过激活函数（阶跃函数）处理后，实际输出结果 $y_{\text{actual}} = 1$ ，分类为西瓜，判定正确。在这个示例里，仅仅经过一轮“试错法”（Try-Error），感知机学会了如何区分西瓜和香蕉。但这是一个“Hello World”版本的神经网络呢！事实上，在有监督的学习规则中，我们需要根据输出与期望值的“落差”，经过多轮重试，反复调整神经网络的权值，直至这个“落差”收敛到能够忍受的范围之内，训练才告结束（示意图 5.9）。这里的“试错”其实就是感知机的学习过程。感知机的学习过程可以看作一个“自我修正”过程，其调整方向由误差 $y_{\text{target}} - y_{\text{actual}}$ 确定，而调整幅度则取决于输入特征 x_i ，特征值较大者调整幅度也更大。具体的权重调整规则如下：

- 如果分类正确，则无需调整；
- 如果分类错误，则根据误差进行调整：
 - 若样本应判定为 1，但误判为 0，则增加相关特征的权重，使其更倾向于 1。
 - 若样本应判定为 0，但误判为 1，则减少相关特征的权重，使其更倾向于 0。

这种误差反馈机制，使得感知机能够逐步优化自身的分类能力，最终找到合适的决策边界。感知机虽然是一个简单的神经网络模型，但提出了几个关键概念：

1. **可学习权重** 相较于 M-P 模型的固定权重，感知机能够通过数据训练自动调整权重，从而具备了“学习”能力。
2. **误差反馈训练** 通过感知机学习规则（Perceptron Learning Rule），感知机可以根据分类错误调整权重，改进分类能力。这一机制成为现代神经网络梯度下降算法的早期雏形。
3. **误差反馈训练：感知机学习规则（Perceptron Learning Rule）** 使其能利用分类误差调整权重，这一机制成为现代神经网络梯度下降算法的早期雏形。

5.4.3 感知机的几何意义

数学上，感知机的决策边界实际上是一个超平面：

$$S = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = 0$$

这一超平面将数据空间划分为两个区域：超平面一侧的数据点被分类为 1，而另一侧的数据点被分类为 0。在二维空间中，超平面是一条直线，而在三维空间，它是一个平面，在更高维度（如 100 维或 1000 维）中，超平面难以直观可视化。在训练过程中，感知机不断调整超平面的位置和方向，直到所有训练数据都被正确分类。具体地，对每个误分类的样本，调整权重，使其朝向正确分类方向移动，其中调整偏置 b 相当于平移决策边界。如果数据是线性可分的，感知机算法会在有限步内收敛，最终找到一个能够正确分类所有样本的超平面。

$$\begin{aligned}f(\vec{x}) &= x_1 w_1 + x_2 w_2 + \dots x_i w_i + \dots + x_n w_n - \theta \\&= x_0 w_0 + x_1 w_1 + \dots + x_i w_i + \dots + x_n w_n \\&= \vec{x} \cdot \vec{w}\end{aligned}$$

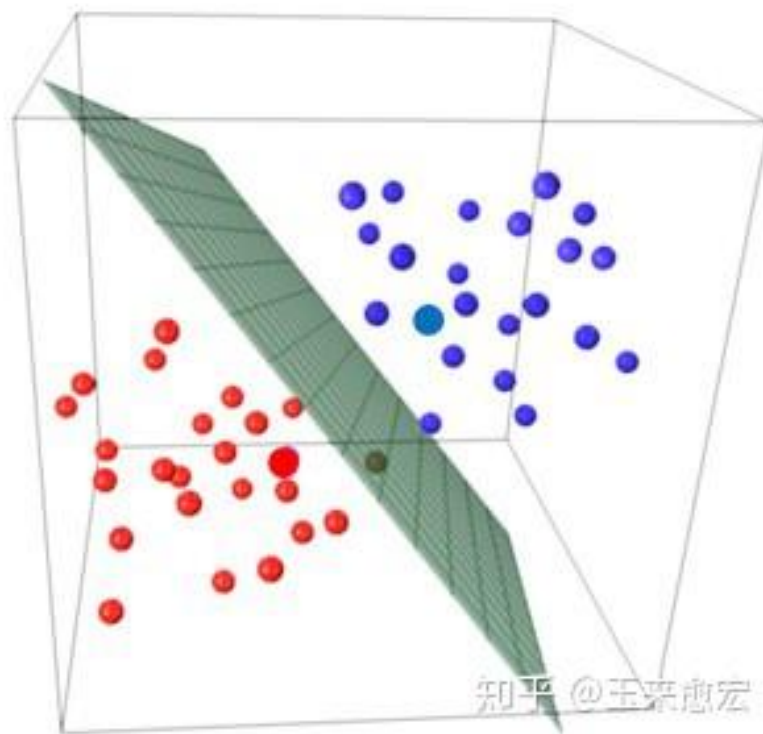


图 5.10: 感知机的超平面

5.4.4 感知机的局限性：XOR 问题与神经网络的停滞

尽管感知机的提出标志着人工神经网络的初步形成，但它存在一个重大局限性：无法解决线性不可分问题（如 XOR）。感知机能够完美解决线性可分问题（如 AND 和 OR），因为它们的决策边界可以用一条直线分割，但，其决策边界是非线性的，如图所示：

$$\text{XOR (异或) 逻辑: } \begin{cases} 0 \oplus 0 = 0 \\ 0 \oplus 1 = 1 \\ 1 \oplus 0 = 1 \\ 1 \oplus 1 = 0 \end{cases} \quad (5.6)$$

XOR（异或）的数据分布无法用一条直线分开，因此单层感知机无法拟合这种逻辑关系。1969 年，马文·明斯基 (Marvin Minsky) 和西摩·帕珀特 (Seymour Papert) 在《感知机 (Perceptrons)》[15] 一书中指出：“单层感知机无法学习 XOR 逻辑，因为 XOR 是线性不可分的，而感知机仅能处理线性可分数据。”由于当时尚未找到有效的解决方法，这一发现对神经网络的研究造成了沉重打击，直接导致了 1970-1980 年代神经网络研究的停滞。直到 1980 年代，多层神经网络 (MLP) 和反向传播算法 (Backpropagation) 的提出，才打破了这一局限。感知机的局限性催生了多层神经网络 (MLP) 的发展，推动了更复杂神经网络的研究。

5.4.5 感知机的实际应用

尽管感知机是一个相对简单的神经网络模型，但它仍然可以用于一些基础的分类任务，如：应用 1：图像分类感知机可以用于简单的二分类任务，如判断一张图片是“猫”还是“狗”。

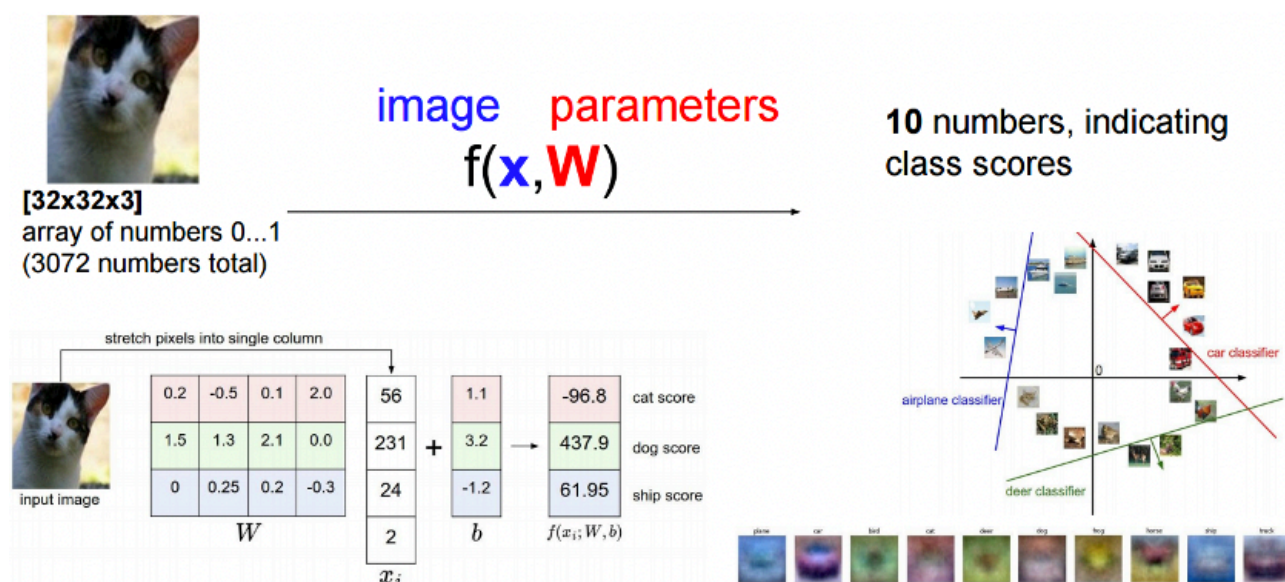


图 5.11: 图像分类

应用 2：文本分类在自然语言处理（NLP）中，感知机可用于垃圾邮件分类或情感分析。例如，输入邮件内容中的关键词（如“免费”“中奖”“折扣”等），可以判定是否为垃圾邮件。

根据文本内容来判断文本的相应类别

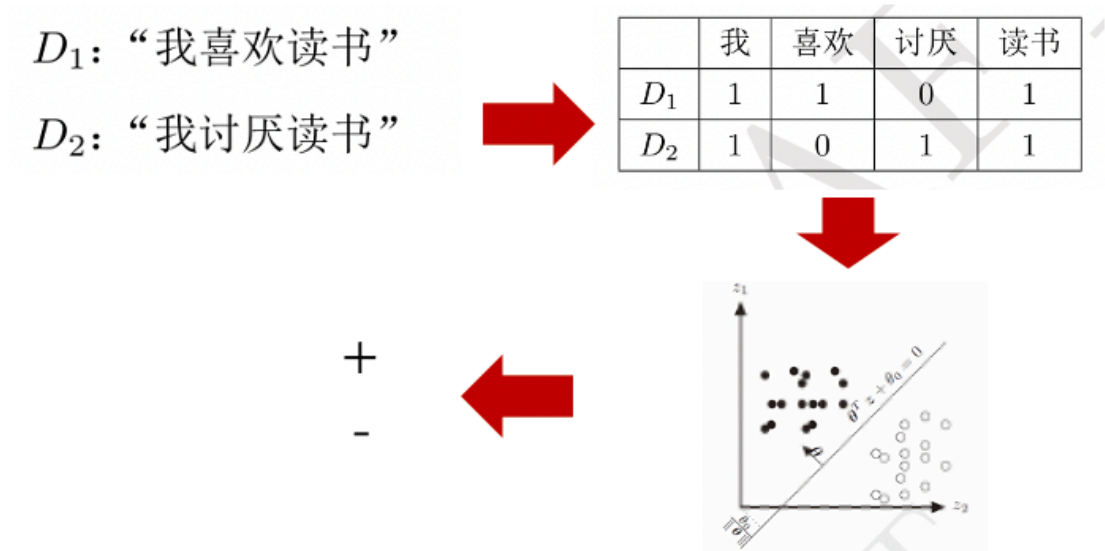


图 5.12: 文本分类

5.4.6 感知机的历史影响

感知机的提出不仅是人工神经网络的开端，更是机器学习迈向自主学习的第一步。在人工神经网络领域中，感知机也被指为单层的人工神经网络，以区别于更复杂的多层感知机（Multilayer Perceptron, MLP）。作为最简单的前馈神经网络，感知机虽然结构简单，但它首次实现了参数自动更新，使机器具备了“学习”能力。这一突破使其能够通过训练调整权重，解决一定复杂度的分类问题。感知机的另一个重大贡献是误差反馈机制，它提出了基于分类误差调整权重的学习规则，为后来的梯度下降（Gradient Descent）等优化算法提供了启发。这一机制使得机器不仅能执行计算，还能根据错误不断优化自身，为现代深度学习奠定了重要基础。然而，感知机的线性分类特性限制了其能力，使其无法处理非线性可分问题（如异或 XOR 问题）。这一缺陷导致单层感知机逐渐被更强大的多层神经网络（MLP）所取代。然而，感知机仍然是人工神经网络演化的起点，其核心思想催生了更强大的深度学习技术，并影响了现代人工智能的发展。后续神经网络的发展奠定了坚实的理论基础。更多细节，请参见（第 12 章：感知机是如何工作的？，[28]）

小故事：感知机的传奇人物：弗兰克·罗森布拉特

弗兰克·罗森布拉特（Frank Rosenblatt）不仅是一位杰出的科学家，更是一位充满探索精神的“折腾王”。白天他在实验室研究蝙蝠的大脑，夜晚则在自家后山搭建天文台，试图与外星生命交流。他的研究跨越神经科学、心理学和计算机科学，为智能学习理论奠定了基础。1949年，唐纳德·赫布（Donald Hebb, 1904—1985）在《行为的组织》一书中，提出了**神经心理学理论**，认为**神经网络的学习过程发生在神经元之间的突触部位，突触的连接强度会随着突触前后神经元的活动而变化，变化的幅度与两个神经元之间的活性和成正比**。这一假说为人工神经网络的发展提供了生物学依据。受到赫布假说的启发，1958年，30岁的罗森布拉特提出了感知机（Perceptron），在工程上模拟实现了赫布假说。这是第一种能够自我学习的人工神经网络。在罗森布拉特的实验中，感知机能够学习识别 50 组图片的左右方向，这是机器视觉和模式识别的早期尝试。这一突破立即引发了学术界和媒体的广泛关注。1958年7月8日，《纽约时报》在头版刊登了关于感知机的报道，称其为：“**一个能够行走、拥有视觉、能够写作、能自我复制，且有自我意识的电子计算机的雏形**”。当时，距离标志着人工智能学科诞生的达特茅斯（Dartmouth）会议，仅仅过去两年，人工智能仍处于萌芽阶段。然而，媒体的热情远远超出了学术界的预期，《纽约时报》非常乐观地估计，“再花上 10 万美元，一年以后，上述构想就能实现可期。那时，机器能够识别人类，还能把人们演讲的内容即时地翻译成另一种语言或记录下来。”这在当时看似天方夜谭，但六十多年后的今天，这些愿景几乎都已成真。罗森布拉特本人也信心满满，几天后，他亲自撰写了一篇文章——《Electronic 'Brain' Teaches Itself》（能自学的电脑），进一步向公众介绍他的研究。他表示：“感知机将成为第一个能够感知、识别、适应环境的非生命智能体。”在某种程度上，我们如今所熟知的“计算机（Computer）”一词在人工智能语境下的广泛传播，或许也与他的宣传密切相关。尽管感知机本身存在诸多局限性，例如无法解决线性不可分问题（如 XOR 问题），但它开启了机器学习自主学习的可能性，并催生了多层感知机（MLP）和深度学习的发展。科技预言家凯文·凯利（Kevin Kelly）在《必然》中提出，未来科技的 12 大进化趋势之一是“知化（Cognifying）”——让一切事物具备认知能力。凯文·凯利认为，人工智能的本质就是“知化”，即硬件问题软件化。回看罗森布拉特的感知机，正是“知化”理念的雏形。虽然感知机并未直接实现《纽约时报》所预言的所有功能，但它确实点燃了人类对智能机器的期待。六十多年后的今天，计算机视觉、语音识别、自动翻译、智能写作早已成为现实，而神经网络，正是这些技术背后的核心支撑。罗森布拉特的大胆畅想，放在今天来看，依然不过时。

5.5 多层感知机（1980s 复兴）：突破单层感知机的局限

5.5.1 增加隐藏层：解决线性不可分问题

尽管感知机的提出奠定了神经网络的基础，但其最大局限在于无法解决线性不可分问题，例如 XOR（异或）问题。研究者们发现，单层感知机只能学习线性可分数据，要解决 XOR 这

样的非线性问题，必须在输入层与输出层之间增加隐藏层 (Hidden Layer)，提升网络的表达能力。这一改进催生了**多层感知机 (MLP, Multilayer Perceptron)**，即深度神经网络的前身。1974 年，Geoffrey Hinton 提出“多则不同”——将多个感知机连接成一个分层网络，从而可处理非线性可分问题。MLP 的结构：

- **输入层**：接收数据并传递给网络
- **隐藏层**：逐层提取数据中的复杂特征
- **输出层**：根据学习到的特征进行最终分类

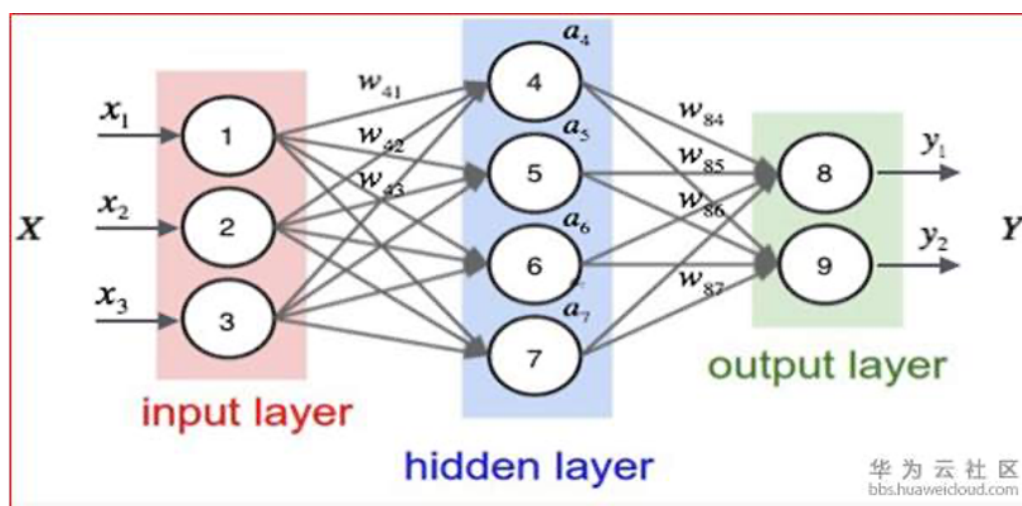


图 5.13: 多层感知机网络

5.5.2 两层感知机如何解决“异或”问题

多层感知机 (MLP) 之所以能够解决 XOR (异或) 问题，关键在于隐藏层的引入。以下是一个两层感知机 (输入层 + 隐藏层 + 输出层) 实现 XOR 的示例。对于 XOR 逻辑 5.6 假设

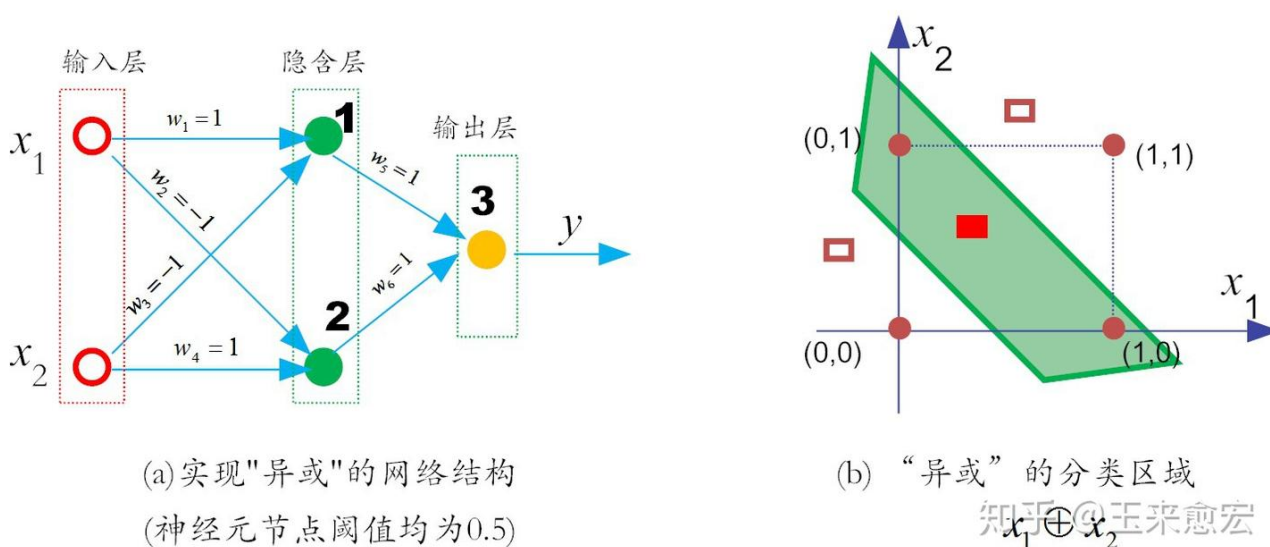


图 5.14: 可解决“异或”问题的两层感知机

在如图 5.14 所示的神经元 (即实心圆), 其激活函数依然是阶跃函数 (即 sgn 函数), 它的输出规则非常简单: 当 $x \geq 0$ 时, $f(x)$ 输出为 1, 否则输出 0。假设神经元 x_1 对隐藏层节点 1 和 2 的权重分别为 $w_1 = 1$ 和 $w_2 = -1$, 且神经元 x_2 对隐藏层的节点 1 和 2 的权重分别为 $w_3 = -1$ 和 $w_4 = 1$, 偏置 $b = -0.5$ 。假设神经元 x_1 和 x_2 具有相同的输入, 例如 $x_1 = x_2 = 1$ 。此时, 隐藏层的神经元 1 的输出为:

$$f_1 = w_1x_1 + w_2x_2 + b = \text{sgn}(1 \cdot 1 + 1 \cdot (-1) - 0.5) = \text{sgn}(-0.5) = 0.$$

类似地, 对于隐藏层的神经元 2, 有

$$f_2 = w_1x_1 + w_2x_2 + b = \text{sgn}(1 \cdot (-1) + 1 \cdot 1 - 0.5) = \text{sgn}(-0.5) = 0.$$

现在到了输出层, 对于输出神经元 3 而言, 这时 f_1 和 f_2 是它的输入, 于是有:

$$y = f_3 = w_1f_1 + w_2f_2 + b = \text{sgn}(0 \cdot 1 + 0 \cdot (-1) - 0.5) = \text{sgn}(-0.5) = 0.$$

因此, 当 x_1 和 x_2 同为 1 时, 神经网络输出 0。同理, 可以验证, 当神经元 x_1 和 x_2 的输入同为 0 时的情况, 这个简单的两层感知机输出为 0。现在假设神经元 x_1 和 x_2 的输入不相同, 不妨设 $x_1 = 1$ 而 $x_2 = 0$ 。因此, 对于隐藏层的神经元 1, 有

$$f_1 = \text{sgn}(w_1x_1 + w_2x_2 + b) = \text{sgn}(1 \times 1 + (-1) \times 0 - 0.5) = \text{sgn}(0.5) = 1,$$

而对于隐藏层的神经元 2, 有

$$f_2 = \text{sgn}(w_3x_1 + w_4x_2 + b) = \text{sgn}((-1) \times 1 + 1 \times 0 - 0.5) = \text{sgn}(-1.5) = 0$$

然后, 对于输出层的输出神经元 3 而言, 这时 f_1 和 f_2 是它的输入, 于是有:

$$y = f_3 = \text{sgn}(w_5f_1 + w_6f_2 + b) = \text{sgn}(1 \times 1 + 0 \times 0 - 0.5) = \text{sgn}(0.5) = 1$$

不失一般性, 由于 x_1 和 x_2 的地位是可以互换的。因此有, 当 x_1 和 x_2 取值不同时, 感知机输出为 1。由上面的分析可知, 如图 5.14 所示的两层感知机实现了“异或”功能。需要说明的是, 能够完成“异或”功能的网络权重也不是唯一的。以上神经网络中的权值和阈值是我们事先给定的, 而实际上, 它们是需要神经网络自己通过反复地“试错”学习而来。如前文所述, 想解决“异或”问题, 就需要让网络复杂起来。这是因为, 复杂的网络, 表征能力就比较强。按照这个思路, 可以在输入层和输出层之间, 添加一层神经元, 将其称之为隐含层 (hidden layer, 亦有文献简称为“隐藏层”、“隐层”, 后文不再区分这三个称谓)。

5.5.3 真正的突破: 反向传播 (Backpropagation)

虽然多层感知机 (MLP) 的结构在 1970 年代已被提出, 但如何高效地训练多层感知机仍然是一个难题。由于 MLP 采用隐藏层, 误差不像单层感知机那样可以直接计算和修正。这一难题直到 1986 年才被突破, 杰弗里·辛顿 (Geoffrey Hinton) 等人提出了反向传播算法 (Backpropagation, BP), 通过梯度下降来优化权重, 使多层神经网络的训练成为可能。反向传播 (BP) 的本质是通过梯度下降 (Gradient Descent) 来优化神经网络的权重, 使网络的输出误差逐步减小。其核心步骤包括:

1. **前向传播 (Forward Propagation)** 输入数据经过网络, 依次通过输入层 → 隐藏层 → 输出层, 得到预测值 $\hat{y}$ 。
2. **计算输出误差 (Loss Computation)** 计算预测值 $\hat{y}$ 与真实值 y 之间的误差。常用的误差函数 (损失函数) 包括

$$\text{均方误差 (MSE): } L = \frac{1}{2} (y - \hat{y})^2$$

和

$$\text{交叉熵损失 (Cross Entropy Loss): } L = - \sum y_i \log \hat{y}_i$$

3. **误差反向传播 (Backward Propagation)** 误差从输出层向隐藏层反向传播, 通过链式法则 (Chain Rule) 计算各层的梯度:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w}$$

这一过程涉及梯度计算, 确保误差能够层层传递。

4. **更新权重 (Weight Update)** 采用梯度下降法 (Gradient Descent), 调整网络参数:

$$w^{(t+1)} = w^{(t)} - \eta \frac{\partial L}{\partial w}$$

其中 η 是学习率 (Learning Rate), 决定了更新步长。

反向传播算法的提出, 解决了多层感知机 (MLP) 难以训练的问题, 使得深度学习的学习成为可能。它的意义主要体现在以下几个方面:

- **突破多层网络训练难题** 反向传播提供了一种高效的方法, 使误差可以通过链式法则逐层传递, 从而优化整个网络的权重。它解决了多层神经网络的梯度如何传播这一核心问题, 使得 MLP 训练变得可行。
- **为深度学习奠定基础** 反向传播不仅使 MLP 可训练, 还成为深度学习的技术基石。随着数据量的增长和计算能力的提升, BP 算法被广泛应用于训练卷积神经网络 (CNN)、循环神经网络 (RNN) 等更复杂的网络结构, 推动了深度学习的发展。
- **结合现代优化方法, 提高训练效率** 传统的 BP 算法采用梯度下降更新权重, 但存在收敛慢、易陷入局部最优的问题。后续研究在 BP 的基础上, 发展出动量梯度下降 (Momentum)、自适应优化算法 (如 Adam、RMSprop) 等方法, 使神经网络训练更加高效和稳定。

尽管反向传播极大推动了神经网络的发展, 但它仍然存在一些局限性, 主要包括以下几点:

- **梯度消失 (Vanishing Gradient) 问题** 在深层网络中, 反向传播过程中, 梯度在层层传递时可能逐渐衰减, 导致靠近输入层的权重更新几乎停滞, 使得网络难以学习深层特征。这一问题在使用 Sigmoid 或 Tanh 作为激活函数时尤为严重。为了解决这个问题, 研究者提出了 ReLU (修正线性单元) 激活函数, 以及 Batch Normalization (批归一化) 技术, 有效缓解了梯度消失的问题。
- **计算复杂度较高** 反向传播涉及大量矩阵运算, 尤其在训练大型神经网络时, 计算成本非常高。BP 训练过程需要对整个网络的参数进行多次更新, 因此计算时间较长, 训练效率受到硬件性能的限制。近年来, GPU (图形处理单元) 加速、TPU (张量处理单元)、分布式计算的发展, 在一定程度上提高了 BP 训练的效率。

- **容易陷入局部最优** 由于 BP 依赖梯度下降优化，如果损失函数具有多个局部极小值，网络可能会陷入局部最优而非全局最优。此外，在高维空间中，可能会出现鞍点 (Saddle Point)，即梯度为零但并非全局最优的情况。为应对这一问题，研究者提出了随机梯度下降 (SGD)、Adam、自适应学习率方法，这些优化算法在一定程度上提升了 BP 训练的稳定性和收敛速度。
- **需要大量数据和长时间训练** BP 训练需要大量的标注数据，并且需要进行多轮迭代以确保网络收敛。当数据量不足时，神经网络容易过拟合 (Overfitting)，导致泛化能力下降。因此，研究者们引入了数据增强 (Data Augmentation)、正则化 (Regularization) 和 dropout (随机失活) 等技术，以提升模型的泛化能力。

尽管存在这些局限，反向传播仍然是现代深度学习的核心训练方法。随着计算资源的提升以及优化算法的改进，BP 仍然在深度神经网络训练中发挥着关键作用，并推动人工智能向更复杂、更强大的方向发展。

5.5.4 反向传播的影响：神经网络的真正复兴

多层感知机 (MLP) 通过隐藏层解决了单层感知机无法拟合非线性数据 (如 XOR) 的问题，而反向传播算法赋予 MLP 可训练性，使得 MLP 真正可用。这标志着现代深度学习的真正起点，并推动了人工智能进入新的发展阶段。随着计算能力和数据规模的增长，多层感知机不断演化，并催生了多个关键的深度神经网络架构：用于图像分类和计算机视觉任务的卷积神经网络 (CNN)，用于自然语言处理 (NLP) 和时间序列分析的循环神经网络 (RNN)，用于自动驾驶、AlphaGo 等智能决策系统的深度强化学习 (Deep Reinforcement Learning)，以及近两年用于大规模 NLP 任务的 Transformer 模型，如 ChatGPT、Google Bard 等。值得注意的是，早在 1965 年，乌克兰数学家亚历克赛·伊瓦赫年科 (Alexey Ivakhnenko) 就已提出深层网络的概念，但由于缺乏有效的训练方法，未能得到广泛关注。相比之下，杰弗里·辛顿 (Geoffrey Hinton) 凭借反向传播算法让深度学习重获新生，因此被尊称为“深度学习教父 (Godfather of Deep Learning)”。有趣的是，多层感知机的概念甚至比明斯基指出“感知机仅能处理线性可分数据”的《感知机》一书 (1969 年) 更早被提出。但由于缺乏有效的训练方法，它在很长时间内未能受到足够重视。直到 1975 年——离伊瓦赫年科提出多层神经网络概念已达 10 年之久了，感知机的“异或难题”才被理论界彻底解决。到了 1980 年代，随着反向传播算法的引入，神经网络研究才重新进入黄金时代。见微知著可以看到，科学技术的发展，从来都不是线性地一蹴而就，而是呈现螺旋上升的！

5.6 前馈神经网络 (Forward Neural Networks): 从简单到复杂

随着多层感知机 (MLP) 的成功，计算机科学家们开始探索更通用的神经网络模型，以解决更加复杂的计算问题。前馈神经网络 (Forward Neural Networks, FNN) 作为最基础的人工神经网络架构，是感知机的自然扩展，它通过**多层结构**和**非线性激活函数**克服了单层感知

机的局限，赋予了神经网络更强的表达能力。前馈神经网络的本质可以理解为大量神经元之间的有向连接，其核心包括三大要素：**神经元的激活规则**决定输入如何被映射为输出，**网络的拓扑结构**决定神经元之间的连接关系，**学习算法**决定如何通过数据训练来优化神经网络参数。在本节中，我们将系统介绍前馈神经网络的基本概念、数学理论、主要类型，并探讨神经网络如何从简单的感知机逐步演变为更深层的神经网络（DNN）。

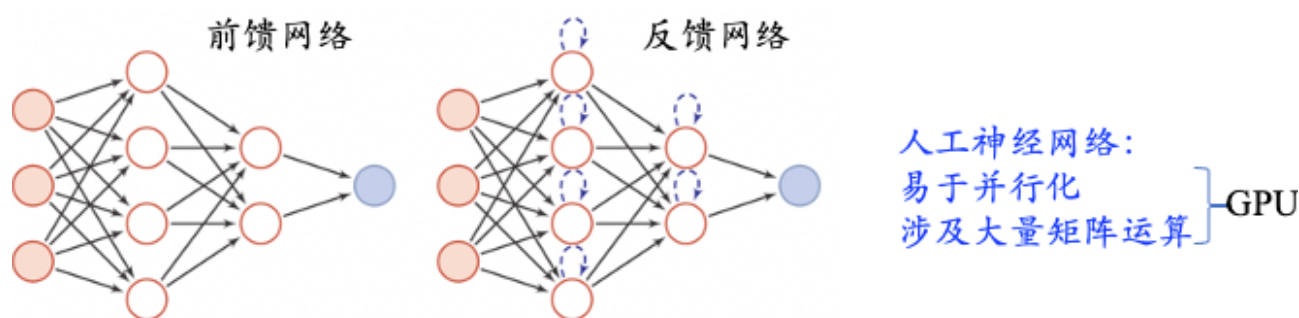


图 5.15

5.6.1 前馈神经网络的基本结构

人工神经网络（ANN）通常由多个神经元（Neuron）层叠组成，每一层执行不同的计算任务。一个典型的前馈神经网络由三部分构成：

1. **输入层（Input Layer）** 负责接收数据，将特征输入到网络中。
2. **隐藏层（Hidden Layer）** 用于提取数据的深层特征，并通过非线性变换增强网络的表达能力。
3. **输出层（Output Layer）** 根据隐藏层学习到的特征，进行最终的分类或回归预测。

随着隐藏层的增加，神经网络的学习能力和表达能力大幅提升，使其能够处理更复杂的模式识别和预测任务。

5.7 深度学习（2006 及之后）：突破梯度消失，实现多层神经网络的高效训练

针对前馈神经网络，输入层和输出层设计比较直观。针对神经网络，我们拆开想想 **What** 和 **How** 的问题：1) 神经：即神经元，什么是神经元（**What**）？2) 网络：即连接权重和偏置，它们是怎么连接的（**How**）？对于计算机而言，它能计算的就是一些数值。而数值的意义是人赋予的。法国著名数学家、哲学家笛卡尔曾在它的著作《谈谈方法》提到：“我想，所以我是”。文雅一点的翻译就是“我思故我在”。套用在神经元的认知上，也是恰当的。这世上本没有什么人工神经元，你（人）认为它是，它就是了。也就是说，数值的逻辑意义，是人给的。比如说，假如我们尝试判断一张手写数字图片上面是否写着数字“2”。很自然地，我们可以把图片的每一个像素的灰度值，作为网络的输入。在输入层，每个像素都是一个数值（如果是彩色

图, 则是 3 通道的数组), 而把包容这个数值的容器视作一个神经元。输入层神经元个数的设计: 如果图片的维度是 16×16 , 那么我们输入层神经元就可以设计为 256 个 (也就是说, 输入层是一个包括 256 个灰度值的向量), 每个神经元接受的输入值, 就是归一化处理之后的灰度值。0 代表白色像素, 黑色像素代表 1, 灰度像素的值介于 0 到 1 之间。也就是说, 输入向量的维度 (像素个数) 要和输入层神经元个数相同。而对输出层而言, 它的神经元个数和输入神经元的个数, 是没有对应关系的, 而是和待分事物类别有一定的相关性。比如说, 对于图 5.16 所示的案例, 我们的任务是识别手写数字, 而数字有 0-9 共 10 类。所以, 如果在输出层采用 Softmax 回归函数, 它的输出神经元数量仅为 10 个, 分别对应是数字“0-9”的分类概率。最终的分类结果, 择其大者而判之。比如说, 如果判定为“2”的概率 (比如说 80%), 远远大于其他数字, 那么整个神经网络的最终判定, 就是手写图片中的数字是“2”, 而非其它数字。隐含

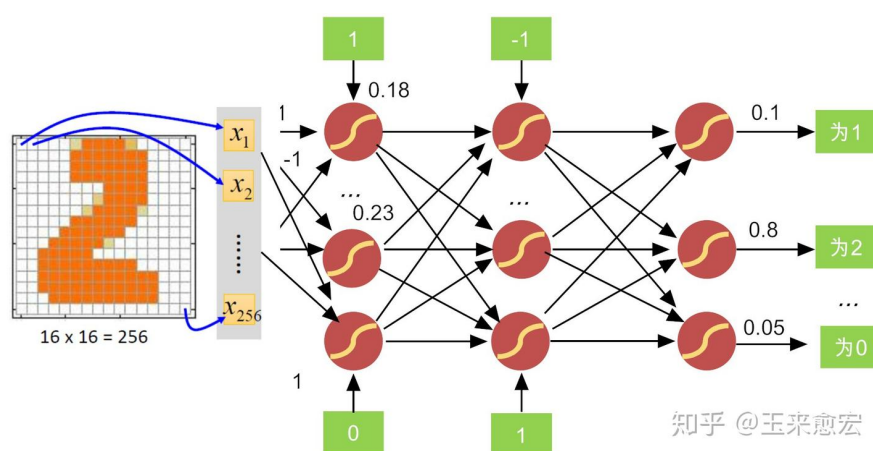


图 5.16: 前馈神经网络的输出

层神经元个数的设计: 相比于神经网络输入与输出层设计的明了直观, 前馈神经网络的隐含层设计, 可就没有那么简单了。说好听点, 它是一门艺术, 依赖于“工匠”的打磨。说不好听点, 它就是一个体力活, 需要不断地折腾“试错”。我们可以把隐含层暂定为一个黑箱, 它负责输入和输出之间的非线性映射变化, 具体功能有点“说不清、道不明” (这是神经网络的理论短板所在)。隐含层的层数不固定, 每层的神经元个数也不固定, 它们都属于“超参数”, 是人们根据实际情况不断调整选取的。神经元之间是如何连接的。事实上, 它同样是人赋予意义的数值参数。我们把神经元与神经元之间的影响程度叫作为权值 (weight)。权重的大小就是连接的强弱。它“告诉”下一层相邻神经元更应该关注那些像素图案。神经网络结构的设计目的在于, 让神经网络以“更佳”的性能来学习。而这里的所谓“学习”, 就是找到合适的权重和偏置, 让损失函数的值达到最小。神经元 (Neuron) 是神经网络的基本计算单元, 其核心计算包括:

1. 计算输入加权和 每个神经元接收多个输入信号, 进行加权求和:

$$z = \sum w_i x_i + b$$

其中, x_i 是输入, w_i 是对应的权重 (weight), b 是偏置项 (bias)。这里, 权重 w_i 代表神经元与神经元之间的影响程度。权重的大小就是连接的强弱。它“告诉”下一层相邻神经元更应该关注那些像素图案。

2. 通过激活函数 (Activation Function) 进行非线性变换 对加权和 z 施加激活函数的变换:

$$y = f(z)$$

其中 $f(x)$ 是**激活函数**, 决定神经元的输出是否被激活。

5.7.0.1 激活函数 (Activation Function)

激活函数的核心作用在于引入非线性, 使神经网络能够学习复杂模式。如果神经网络没有激活函数, 每一层的计算仅仅是线性变换的叠加, 最终整个网络仍然是线性映射。这意味着无论增加多少隐藏层, 网络的表达能力仍然受限。因此, 非线性激活函数是神经网络突破线性限制、学习复杂函数的关键。在所有激活函数中, 阶跃函数 (Step Function) 和 ReLU (Rectified Linear Unit) 函数的截断能力最强, 它们的作用类似“开关”, 决定神经元是否被激活。而其他激活函数可以被视为它们的“软化”版本, 即将非连续的折线变为平滑曲线, 以改善网络的学习性能。图 5.17 展示了几种常见的激活函数:

• 阶跃函数 (Step Function)

$$y = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

阶跃函数是最简单的激活函数, 在早期的 (单层) 感知机中使用, 决定神经元是否被激活。但它仅能处理线性可分问题, 无法进行复杂学习。此外, 由于阶跃函数非连续不可导, 无法用于梯度下降优化, 因此在现代神经网络中较少使用。

• Sigmoid 函数

$$f(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid 函数的输出值在 (0,1) 之间, 能够将输入映射到概率空间, 适用于二分类任务。但它的梯度在极端值时趋于 0, 容易导致梯度消失问题, 训练深度网络时效果较差。

• Tanh (双曲正切) 函数:

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Tanh 正切函数输出范围为 (-1,1), 相比 Sigmoid 更适合归一化数据。它的梯度在 0 附近较大, 收敛速度比 Sigmoid 快, 但仍然容易出现梯度消失问题。

• ReLU (修正线性单元, Rectified Linear Unit):

$$f(x) = \max(0, x)$$

ReLU 函数是目前深度神经网络中最常用的激活函数, 它计算简单, 不会饱和, 能够有效缓解梯度消失问题。然而, 它在负半轴上的梯度恒为 0, 可能导致“神经元死亡” (Dead Neuron) 现象, 即某些神经元的输出永远为 0, 不再更新参数。

• Leaky ReLU (带泄漏的 ReLU)

$$y = \begin{cases} x & \text{if } x \geq 0 \\ \alpha x & \text{if } x < 0 \end{cases}$$

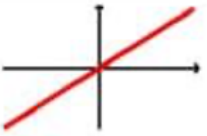
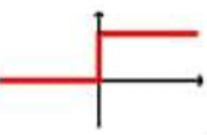
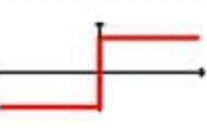
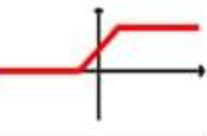
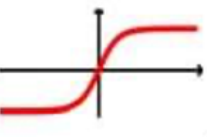
Activation Function	Equation	Example	1D Graph
Linear	$\phi(z) = z$	Adaline, linear regression	
Unit Step (Heaviside Function)	$\phi(z) = \begin{cases} 0 & z < 0 \\ 0.5 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Sign (signum)	$\phi(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Piece-wise Linear	$\phi(z) = \begin{cases} 0 & z \leq -\frac{1}{2} \\ z + \frac{1}{2} & -\frac{1}{2} \leq z \leq \frac{1}{2} \\ 1 & z \geq \frac{1}{2} \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multilayer NN	
Hyperbolic Tangent (tanh)	$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multilayer NN, RNNs	
ReLU	$\phi(z) = \begin{cases} 0 & z < 0 \\ z & z > 0 \end{cases}$	Multilayer NN, CNNs	

图 5.17: 激活函数

Leaky ReLU 在负半轴引入了一个小的非零斜率 α (通常取 0.01), 使得负值区域时神经元在仍然能接收梯度更新, 减少神经元死亡问题。

• Softmax 函数

$$f(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$$

Softmax 函数适用于多分类任务, 能够将输出归一化的概率分布, 使得所有类别的概率总和为 1, 便于模型进行分类决策。

激活函数在神经网络中起着至关重要的作用, 它决定了网络的非线性映射能力, 影响梯度传播的稳定性以及计算效率。激活函数的设计需要平衡以下几个关键因素:

1. **可导性 (Differentiability)**: 由于梯度下降法 (Gradient Descent) 用于优化神经网络, 需要对激活函数求导, 因此激活函数应在定义域内保持可微。早期感知机 (Perceptron) 使用的阶跃函数 (Step Function) 仅能输出 0 或 1, 不连续不可导, 无法用于梯度优化。
2. **梯度稳定性 (Gradient Stability)**: 选择适当的激活函数能够避免梯度消失 (Vanishing Gradient) 或梯度爆炸 (Exploding Gradient) 问题, 从而确保深度网络能够有效训练。例如, Sigmoid 和 Tanh 函数在极端值区域会导致梯度接近零, 影响网络学习, 而 ReLU 函数及其变体在深度学习中表现更优。
3. **计算效率 (Computational Efficiency)**: 计算简单的激活函数能够加速网络的训练。ReLU 及其变体 (Leaky ReLU、PReLU) 计算量较小, 相比 Sigmoid 和 Tanh 具有更好的数值稳定性, 成为深度神经网络的主流选择。

选择合适的激活函数对于训练稳定性、收敛速度和最终模型的性能有着至关重要的影响。不同任务需要选择合适的激活函数, 以确保网络能够高效学习并避免梯度消失或梯度爆炸等问题。在实际应用中, 对于回归任务, 常用 ReLU 或 Tanh 作为隐藏层激活函数, 因为它们能够输出连续值; 对于二分类任务, 通常使用 Sigmoid 作为输出层激活函数, 因为其输出值在 (0, 1) 之间, 可以解释为概率; 对于多分类任务, Softmax 是最优选择, 能够将输出归一化为概率分布; 深度神经网络的隐藏层通常使用 ReLU 或 Leaky ReLU, 以避免梯度消失, 提高训练效率。神经网络的学习过程本质上是一个参数优化问题, 目标是不断调整网络的权重参数, 使输出尽可能接近真实值。本节, 我们以监督学习 (Supervised Learning) 为例, 详细描述神经网络的学习过程。在无监督学习 (Unsupervised Learning) 和半监督学习 (Semi-Supervised Learning) 中, 神经网络的训练方式有所不同, 我们将在后续章节详细讨论它们的学习范式。在监督学习任务中, 神经网络的学习过程通常由前向传播 (Forward Propagation)、误差计算 (Loss Computation)、反向传播 (Backpropagation) 和权重更新 (Weight Update) 四个阶段构成。

1. **前向传播 (Forward Propagation)**: 数据在网络中向前逐层传递, 经过加权求和和激活函数变换等神经元计算, 生成输出预测值。
2. **计算损失 (Loss Computation)**: 评估预测结果与真实标签之间的误差, 使用损失函数进行衡量。
3. **误差反向传播 (Backpropagation)**: 误差从输出层向隐藏层反向传播, 计算各层权重对损失的梯度。

4. **权重更新 (Weight Update)**：通过优化算法（如梯度下降）调整网络权重，使模型在下次迭代中表现得更好。

接下来，我们详细探讨这四个阶段。

前向传播 (Forward Propagation)

为便于阅读，本节（前向传播 (Forward Propagation)）承接上节（激活函数 (Activation Function)），先交代差异与共同点，再聚焦本节关键问题。

神经网络的前向传播是神经网络的计算流程，输入数据从输入层开始，逐层流向隐藏层，最终到达输出层。不失一般性，我们假设神经网络仅有一个隐藏层。假设神经网络的输入层有 m 个特征，表示为 $X = (x_1, x_2, \dots, x_m)$ ，隐藏层包含 p 个神经元，表示为 $H = (h_1, h_2, \dots, h_p)$ ，输出层包含 n 个神经元，表示为 $Y = (y_1, y_2, \dots, y_n)$ 。神经网络的权重矩阵由两部分组成，分别是输入层到隐藏层的权重矩阵 $W = (W_{ij})_{m \times p}$ 和隐藏层到输出层的权重矩阵 $V = (V_{jk})_{p \times n}$ ，其中 W_{ij} 为连接输入 x_i 和隐藏层神经元 h_j ， V_{jk} 为连接隐藏层 h_j 和输出神经元 y_k ， b_j 和 c_k 分别为隐藏层神经元 h_j 和输出层神经元 y_k 的偏置项。在前向传播过程中，输入层的数据 X 首先与权重 W 进行加权求和，并添加偏置项 b_j ，然后通过激活函数 $f(x)$ 进行非线性变换，得到隐藏层的输出：

$$h_j = f \left(\sum_{i=1}^m W_{ij} x_i + b_j \right), \quad j = 1, 2, \dots, p$$

其中 $f(x)$ 是隐藏层的**激活函数**，如 ReLU 或 Sigmoid。接下来，隐藏层的输出继续传递到输出层。每个输出神经元同样对隐藏层的输出进行加权求和，并添加偏置项 c_k ，然后通过输出层的激活函数 $g(x)$ 计算最终的预测值：

$$\hat{y}_k = g \left(\sum_{j=1}^p V_{jk} h_j + c_k \right), \quad k = 1, 2, \dots, n$$

其中 $g(x)$ 是输出层的激活函数，常见的选择包括 Sigmoid（用于二分类任务）和 Softmax（用于多分类任务）。网络的预测输出 $\hat{Y} = (\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n)$ 作为模型的输出，与真实标签 $Y = (y_1, y_2, \dots, y_n)$ 进行比较，以计算损失，从而指导下一步的优化过程。在前向传播过程中，神经网络根据输入数据计算出预测值 $\hat{Y} = (\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n)$ ，但由于模型的初始权重是随机设定的，预测值 $\hat{Y}$ 可能与真实值 $Y = (y_1, y_2, \dots, y_n)$ 存在较大的误差。为了衡量这种误差，我们需要定义损失函数 (Loss Function)，以量化预测结果与真实标签之间的偏差，指导后续的权重更新。损失函数 L 的目标是最小化预测值 $\hat{Y}$ 和真实值 Y 之间的误差。不同的任务类型适用于不同的损失函数。在监督学习中，最常见的任务是回归 (Regression) 和分类 (Classification)，它们通常采用不同的损失函数来度量误差。对于回归任务，目标是预测一个连续的数值，因此损失函数通常采用均方误差 (Mean Squared Error, MSE)：

$$L = \frac{1}{2} \sum_{k=1}^K (y_k - \hat{y}_k)^2,$$

其中 y_k 是真实值， $\hat{y}_k$ 是预测值。均方误差衡量了预测值 $\hat{y}_k$ 与真实值 y_k 之间的平方差，并对

所有输出求和后取平均。平方的作用是让误差对大偏差更加敏感，同时保持导数的连续性，使得优化更稳定。除了均方误差，有时也会使用均方根误差（Root Mean Squared Error, RMSE）：

$$L = \sqrt{\frac{1}{K} \sum_{k=1}^K (y_k - \hat{y}_k)^2}$$

或者平均绝对误差（Mean Absolute Error, MAE）：

$$L = \frac{1}{K} \sum_{k=1}^K (y_k - \hat{y}_k|$$

其中，MAE 计算的是预测值与真实值之间的绝对误差，鲁棒性较强，但对大误差的敏感度低于 MSE。而在分类任务中，输出层通常是一个概率分布，我们希望模型预测出的概率尽可能接近真实类别的概率。为此，常采用交叉熵损失（Cross-Entropy Loss），它能够衡量两个概率分布之间的差异，并通过放大低概率类别的损失来加速模型的收敛。此外，交叉熵的梯度更适合优化，使得模型在训练过程中能够更稳定地更新权重。对于二分类问题，交叉熵损失定义如下：

$$L = - \sum_{k=1}^K y_k \log \hat{y}_k + (1 - y_k) \log(1 - \hat{y}_k).$$

其中， $\hat{y}_k$ 是模型预测的类别概率， y_k 是真实类别标签（0 或 1）。交叉熵损失能够度量模型在概率预测上的不确定性，当预测值 $\hat{y}_k$ 越接近真实值 y_k ，损失越小。对于多分类任务，交叉熵损失的形式推广为：

$$L = - \sum_{k=1}^K y_k \log \hat{y}_k.$$

在该公式中， y_k 是独热编码（one-hot encoding）形式的真实标签， $\hat{y}_k$ 是模型预测的 Softmax 输出概率。该损失能够衡量整个分类任务的误差，使得模型趋向于正确分类样本。通过计算损失函数 L ，我们得到了模型的当前误差。下一步，我们将利用反向传播（Backpropagation）算法，计算损失函数关于网络权重的梯度，以指导权重的更新，优化模型的预测能力。

误差反向传播（Backpropagation）

为便于阅读，本节（误差反向传播（Backpropagation））承接上节（前向传播（Forward Propagation）），先交代差异与共同点，再聚焦本节关键问题。

在计算损失后，神经网络需要调整参数，以使预测结果更接近真实值。这一过程依赖于**误差反向传播（Backpropagation, BP）**，它是一种基于**梯度下降（Gradient Descent）**的优化方法，用于计算损失函数对每个权重的梯度，并逐层更新网络参数，以最小化预测误差。梯度下降的核心思想是**沿着损失函数下降最快的方向调整参数**，从而优化模型性能。具体来说，它首先计算损失对输出层的影响，然后逐层反向传播误差信息，计算各层参数对最终误差的贡献，并据此调整权重，使所有参数都能基于它们对整体误差的影响进行合理优化。误差反向传播是一种高效的梯度计算方法，它通过**链式法则（Chain Rule）**计算误差在网络中的传播方式，使得误差能够从输出层逐层反向传播至输入层，并高效计算每个权重的调整量。具体

而言,

1. **计算输出层的误差** 由于损失函数 L 依赖于网络的输出 $\hat{y}_k$, 首先计算输出层的误差:

$$\delta_k = (y_k - \hat{y}_k) g'(\hat{y}_k)$$

其中, y_k 是真实标签, $\hat{y}_k$ 是网络的预测值, $g'(\hat{y}_k)$ 是输出层激活函数的导数, 用于衡量输出层的梯度变化。直观来看, 如果预测值 $\hat{y}_k$ 远离真实值 y_k , 那么误差项 δ_k 会较大, 导致较大的梯度, 从而驱动参数向减少误差的方向调整; 反之, 如果预测已经接近真实值, 则误差项较小, 参数的更新幅度也会相应减小。

2. **计算隐藏层的误差项** 由于隐藏层的输出直接影响输出层, 因此需要计算隐藏层对最终损失的贡献, 并将误差从输出层反向传递到隐藏层:

$$\delta_j = \left(\sum_k V_{jk} \delta_k \right) f'(h_j)$$

其中, V_{jk} 是隐藏层到输出层的权重矩阵, δ_k 是从输出层传播回来的误差, $f'(h_j)$ 是隐藏层激活函数的导数, 衡量隐藏层的输出变化对最终误差的影响。这一计算的直观含义是, 如果某个隐藏层神经元对多个输出神经元都有较大的权重连接 (即 V_{jk} 较大), 那么它对误差的贡献就更显著, 因此它的梯度应该相应增大。

权重更新 (Weight Update)

为便于阅读, 本节(权重更新(Weight Update))承接上节(误差反向传播(Backpropagation)), 先交代差异与共同点, 再聚焦本节关键问题。

基于上一步计算出的梯度, 我们利用**梯度下降算法 (Gradient Descent)** 或其变体方法 (如 Adam, RMSprop) 来更新权重, 使得网络在下次迭代时能够产生更准确的预测。在误差反向传播到隐藏层之后, 计算每个权重参数对损失函数的影响, 并利用梯度下降来更新权重:

$$\begin{aligned} \frac{\partial L}{\partial V_{jk}} &= \delta_k h_j \\ \frac{\partial L}{\partial W_{ij}} &= \delta_j x_i \end{aligned}$$

其中, h_j 是隐藏层的激活值, x_i 是输入层的输入数据。这意味着, 隐藏层到输出层的权重更新量取决于隐藏层的输出 h_j 以及输出层的误差项 δ_k , 而输入层到隐藏层的权重更新量取决于输入数据 x_i 以及隐藏层的误差项 δ_j 。通过这些步骤, 神经网络可以计算出如何调整每个权重, 以最小化预测误差。在传统梯度下降方法中, 使用的是基本的权重更新规则:

$$\begin{aligned} V_{jk}^{(t+1)} &= V_{jk}^{(t)} + \eta \delta_k h_j \\ W_{ij}^{(t+1)} &= W_{ij}^{(t)} + \eta \delta_j x_i \end{aligned}$$

其中 η 是学习率 (learning rate), 决定了更新步长, V_{jk} 是隐藏层到输出层的权重, W_{ij} 是输入层到隐藏层的权重, δ_k 和 δ_j 分别是输出层和隐藏层的误差项。学习率 η 是神经网络训练中的关键超参数, 决定了参数的更新步长。如果学习率过大, 更新步长过长, 可能导致梯度剧烈震荡或无法收敛; 反之, 如果学习率过小, 更新步长过短, 可能导致收敛速度慢或陷入局部

最优。通常，我们会使用学习率衰减（Learning Rate Decay）技术，在训练过程中逐步降低学习率，以保证更稳定的收敛。

梯度下降方法及其变体

为便于阅读，本节（梯度下降方法及其变体）承接上节（权重更新（Weight Update）），先交代差异与共同点，再聚焦本节关键问题。

在神经网络的训练过程中，梯度下降（Gradient Descent）是一种核心优化方法，其目标是 최소화 损失函数 L 以优化模型参数 θ 。梯度下降的更新规则如下：

$$\theta^{(t+1)} = \theta^{(t)} - \eta \frac{\partial L}{\partial \theta} \quad (5.7)$$

其中， η 为学习率（learning rate），决定了每次参数更新的步长。不同的梯度下降策略影响计算效率、收敛速度和最终的优化性能。根据计算梯度所使用的数据量不同，梯度下降方法主要分为三种：批量梯度下降（BGD）、随机梯度下降（SGD）和小批量梯度下降（Mini-batch SGD）。

批量梯度下降（Batch Gradient Descent, BGD） 批量梯度下降在每次迭代时计算整个训练集上的损失函数平均梯度，并基于该平均梯度更新参数：

$$\theta^{(t+1)} = \theta^{(t)} - \eta \frac{1}{N} \sum_{i=1}^N \frac{\partial L_i}{\partial \theta} \quad (5.8)$$

其中， N 是训练样本的总数， $\frac{\partial L_i}{\partial \theta}$ 是第 i 个样本的梯度。这种方法保证了梯度更新的稳定性，优化路径不会受随机噪声影响。然而，它需要在每次更新时计算整个训练集的梯度，在数据量较大时，计算开销较高，并且当数据集包含大量冗余信息时，计算效率较低。因此，尽管批量梯度下降具有全局稳定的优化方向，但它不适用于大规模数据集。

随机梯度下降（Stochastic Gradient Descent, SGD） 随机梯度下降在每次更新时仅使用一个训练样本计算梯度：

$$\theta^{(t+1)} = \theta^{(t)} - \eta \frac{\partial L_i}{\partial \theta} \quad (5.9)$$

其中， L_i 是训练集中某个随机抽取的样本的损失。随机梯度下降使得参数能够更快地进行更新，提高了计算效率，并避免了因冗余数据而导致的计算浪费。然而，由于每次更新仅依赖单个样本的梯度，随机梯度下降会导致优化路径的剧烈波动，可能使参数在损失函数的局部最优点附近振荡，而难以稳定收敛。因此，在实际应用中，随机梯度下降需要结合学习率衰减或其他优化方法来提高收敛效果。

小批量梯度下降（Mini-batch SGD） 小批量梯度下降在每次迭代时在一个小批量（Mini-batch）样本来计算梯度：

$$\theta^{(t+1)} = \theta^{(t)} - \eta \frac{1}{m} \sum_{i=1}^m \frac{\partial L_i}{\partial \theta} \quad (5.10)$$

其中， m 是小批量的大小，通常设置为 32、64 或 128。小批量梯度下降结合了批量梯度下降的稳定性和随机梯度下降的计算效率，既能提高计算速度，又能减少梯度更新的波动，使训练更加稳定。由于其在大规模数据集上的良好适应性，目前小批量梯度下降是深度学习中最常用的优化方法。除了基本的梯度下降方法，研究者们还提出了一些自适应优化算法，它们能够动态调整学习率，以提高收敛速度和训练稳定性。这些方法在深度学习中得到了广泛应用。

动量法 (Momentum) 动量法 (Momentum) 在梯度下降的基础上引入一个历史梯度的累积项，使得参数更新方向更加稳定，并减少震荡。其更新公式如下：

$$v^{(t+1)} = \beta v^{(t)} + (1 - \beta) \frac{\partial L}{\partial \theta} \quad (5.11)$$

$$\theta^{(t+1)} = \theta^{(t)} - \eta v^{(t+1)} \quad (5.12)$$

其中， $v^{(t)}$ 代表梯度的指数加权平均， β 是动量因子，通常取 0.9。当梯度更新方向相对一致时，动量法可以加速收敛；而当梯度方向变化较大时，它能平滑优化路径，减少震荡。

RMSprop (Root Mean Square Propagation) RMSprop 通过指数加权平均的方式动态调整学习率，以适应不同参数的梯度变化。其核心思想是对梯度的二阶矩进行指数平滑：

$$s^{(t+1)} = \beta s^{(t)} + (1 - \beta) \left(\frac{\partial L}{\partial \theta} \right)^2 \quad (5.13)$$

$$\theta^{(t+1)} = \theta^{(t)} - \frac{\eta}{\sqrt{s^{(t+1)} + \epsilon}} \frac{\partial L}{\partial \theta} \quad (5.14)$$

其中， ϵ 是一个小数值（如 10^{-8} ），用于避免分母为零的情况。RMSprop 适用于非平稳目标函数，特别是在时间序列任务中表现优越，如循环神经网络 (RNN) 的训练。

Adam (Adaptive Moment Estimation) Adam 结合了 Momentum 和 RMSprop 方法，能够自适应调整每个参数的学习率，是目前最常用的优化算法之一。其更新规则如下：

$$m^{(t+1)} = \beta_1 m^{(t)} + (1 - \beta_1) \frac{\partial L}{\partial \theta} \quad (5.15)$$

$$s^{(t+1)} = \beta_2 s^{(t)} + (1 - \beta_2) \left(\frac{\partial L}{\partial \theta} \right)^2 \quad (5.16)$$

$$\hat{m}^{(t+1)} = \frac{m^{(t+1)}}{1 - \beta_1^t}, \quad \hat{s}^{(t+1)} = \frac{s^{(t+1)}}{1 - \beta_2^t} \quad (5.17)$$

$$\theta^{(t+1)} = \theta^{(t)} - \frac{\eta}{\sqrt{\hat{s}^{(t+1)} + \epsilon}} \hat{m}^{(t+1)} \quad (5.18)$$

其中， β_1 和 β_2 分别控制梯度一阶矩和二阶矩的衰减速率，默认值通常为 0.9 和 0.999。关于这些自适应优化方法的更详细介绍，请参考 [2, 10]：

5.7.1 神经网络的数学灵魂：通用逼近定理（Universal Approximation Theorem）

神经网络的核心目标是**学习并逼近复杂函数**，无论是图像识别、语音处理，还是预测模型，最终都可以抽象为一个数学问题：**找到一个合适的函数，使其能够精准地映射输入到输出**。然而，神经网络是否具备逼近任何函数的能力？通用逼近定理（Universal Approximation Theorem, UAT）正是对这个问题的回答。从数学角度来看，单纯的线性变换是无法表达复杂模式的。若神经网络不包含激活函数，那么无论增加多少层，其整体仍然是线性变换的堆叠。而线性变换的组合仍然是线性变换，因此，这种神经网络最终只能学习一个线性映射，无法解决非线性问题。这意味着，单纯的加深网络并不会提升模型的表达能力。为了使神经网络能够逼近复杂的模式，必须引入非线性激活函数。通用逼近定理（Universal Approximation Theorem, UAT）由 Cybenko（1989）和 Hornik 等人（1989）[3, 6]”提出，描述如下：“任何具有至少一个非线性激活函数的单隐藏层前馈神经网络，只要隐藏层神经元的数量足够，可以以任意精度近似任何定义在实数域上的有界闭集函数。”换句话说，**单隐藏层神经网络非常强大，理论上可以逼近任何连续函数**。只要隐藏层的神经元足够多，并调整合适的权重，总能找到一个网络，使其输出无限接近于目标函数。这一理论为神经网络的表达能力提供了数学保证，也解释了为什么深度学习能够在各种复杂任务中大放异彩。

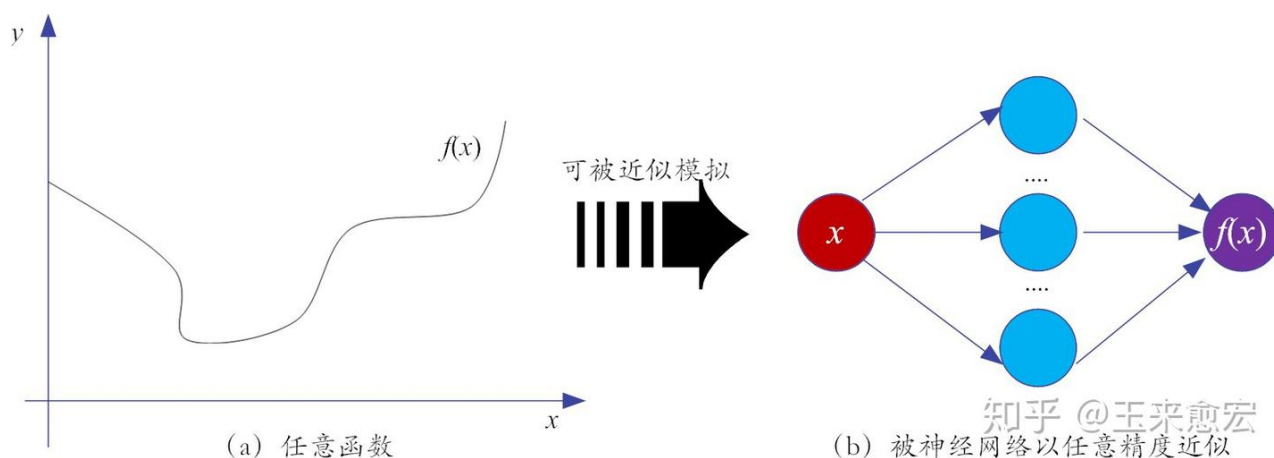


图 5.18: 通用逼近定理

通用逼近定理的直观理解

为便于阅读，本节（通用逼近定理的直观理解）承接上节（神经网络的数学灵魂：通用逼近定理（Universal Approximation Theorem）），先交代差异与共同点，再聚焦本节关键问题。

通用逼近定理听起来近乎神奇，因此也常被称为“万能逼近定理”。它的数学证明基于神经网络的分段逼近（piecewise approximation）思想。直观上，任何复杂的非线性函数都可以拆解为多个局部线性片段，而神经网络通过激活函数的非线性变换，能够灵活地拼接这些片段，从而形成复杂的全局映射。激活函数是神经网络的“非线性源泉”。在没有激活函数的情况

下，神经网络的每一层只是对输入数据进行线性变换，无论如何堆叠隐藏层，最终只能拟合线性关系，无法有效表达非线性函数。因此，激活函数的引入至关重要。正如激活函数的名字“activation”所言，激活函数的作用可以理解为提供局部的“打开-关闭”机制，使得神经网络可以在不同的输入区域内学习不同的映射关系。以 ReLU（修正线性单元）为例，输入 $x < 0$ 时，输出 $y = 0$ ，相当于“截断（关闭）”功能；而输入 $x > 0$ 时，输出 $y = x$ ，相当于“导通（打开）”功能。而 Sigmoid/Tanh 等激活函数具有“挤压”性质，能够将输入映射到特定区间，从而增强神经网络的非线性表达能力。通过这些激活函数，神经网络可以在不同输入区域内应用不同的线性变换，最终在全局上形成非线性映射，逼近任意目标函数。¹

5.8 人工神经网络的主要类型

在上一节中，我们介绍了前馈神经网络（FNN），它是最基础的人工神经网络结构，也是多层感知机（MLP）的扩展形式。FNN 通过前向传播计算输出，并使用误差反向传播（Back-propagation）进行训练，最终调整权重参数，使得网络能够逼近目标函数。然而，前馈神经网络并不能解决所有任务。随着研究的深入，科学家们提出了多种不同结构的神经网络，以适应不同的数据特性和应用场景。根据**拓扑结构**和**信息传播方式**，人工神经网络可以大致分为以下几类：

前馈神经网络（Feedforward Neural Networks, FNN） 前馈神经网络（Feedforward Neural Networks, FNN）前馈神经网络是**最基础**的神经网络结构，它由输入层、隐藏层和输出层组成。数据在网络中按照单向流动的方式传播，并不会在层之间形成循环连接。FNN 适用于静态数据处理，例如图像分类和回归任务。由于其结构相对简单，训练相对稳定，因此在许多传统机器学习任务中得到了广泛应用。多层感知机（MLP）是 FNN 的典型代表，它通过增加隐藏层的数量，使得网络能够学习更加复杂的非线性映射。然而，由于缺乏循环连接，前馈神经网络难以处理时间序列数据，这种局限性促使研究者提出了递归神经网络（RNN）来建模时间相关的信息。

循环神经网络（RNN） 与前馈神经网络不同，递归神经网络允许信息在网络中循环传播，使得当前时刻的输出可以受到之前时刻输入的影响。因此，RNN 适用于处理时间序列数据，例如语音、文本和金融数据预测。它能够利用隐藏状态来存储过去的信息，并结合当前输入进行计算，从而建模序列数据中的时间依赖性。然而，传统 RNN 在长序列任务中容易遭遇梯度消失或梯度爆炸问题，导致网络难以学习较长时间跨度的信息。为了缓解这一问题，研究者提出了**长短时记忆网络（LSTM）**和**门控循环单元（GRU）**，它们通过设计门机制来控制信息

¹如果将神经网络的逼近能力与数学分析中的方法进行类比，它的核心思想与傅里叶级数逼近定理极为相似。傅里叶级数通过一系列正弦波的叠加来逼近任何复杂的周期函数，而神经网络通过多个非线性激活单元的组合，来逼近任意复杂的连续函数。换句话说，神经网络可以看作是“数据驱动的傅里叶级数”，这正是它能够广泛应用于图像识别、自然语言处理等复杂任务的数学基础。

的流动，从而有效提升了模型的长期依赖学习能力。RNN 及其变种广泛应用于机器翻译、语音识别和文本生成等任务。

卷积神经网络 (CNN) 卷积神经网络专门用于处理图像和其他网格状数据，它通过**局部连接**和**权重共享**方式减少计算量，并提高模型对位移、旋转和缩放变化的学习能力。CNN 由**卷积层**、**池化层**和**全连接层**组成，其中卷积层通过滑动窗口的方式提取局部特征，池化层进一步降低计算复杂度并提高模型的泛化能力。CNN 在计算机视觉领域取得了革命性的突破，成为目标检测、人脸识别和医学影像分析等任务中的核心模型。随着研究的深入，CNN 逐步发展出 ResNet、DenseNet 和 MobileNet 等变体，使得网络在深度增加的同时，仍然能够保持较高的训练效率和泛化能力。

自编码器 (Autoencoder, AE) 自编码器是一种**无监督学习**的神经网络，其目标是学习输入数据的低维表示，并尽可能保留关键信息。该网络由**编码器 (Encoder)**和**解码器 (Decoder)**组成，编码器负责将输入数据映射到低维表示，而解码器则尝试从低维表示中重建原始输入。自编码器可以用于**数据降维**、**特征提取**和**异常检测**，并且在生成模型和数据压缩等领域具有广泛的应用。变分自编码器 (VAE) 是自编码器的一种扩展形式，它在编码过程中引入了概率分布的约束，从而使得生成的数据更加多样化和稳定。

生成对抗网络 (Generative Adversarial Networks, GAN) 生成对抗网络由**生成器 (Generator)**和**判别器 (Discriminator)**组成，二者通过对抗训练进行优化，最终使得生成器能够生成逼真的合成数据，而判别器则不断提高对真实数据和生成数据的辨别能力。GAN 的核心思想在于让生成器和判别器进行博弈，使得生成器最终可以生成难以区分的高质量数据。GAN 在图像生成、风格迁移和数据增强等领域取得了突破性的成果，被广泛应用于 AI 艺术、DeepFake 和医学图像合成等任务。然而，由于 GAN 训练的不稳定性和模式崩溃 (mode collapse) 问题，研究者提出了 WGAN、StyleGAN 和 BigGAN 等改进模型，以提升训练的稳定性和生成数据的质量。

其他神经网络架构 除了上述主要类型的神经网络外，研究者还提出了适用于不同数据结构和任务的特殊网络架构。例如，**变分自编码器 (VAE)** 结合了概率生成建模的思想，适用于生成任务；**图神经网络 (Graph Neural Networks, GNN)** 能够处理**非欧几里得数据**，如社交网络和分子结构；**Transformer** 采用**自注意力机制 (Self-Attention)**，能够高效建模长序列数据，在自然语言处理 (NLP) 任务中表现优异，并成为 BERT 和 GPT 等预训练模型的核心结构。人工神经网络的不同架构适用于不同的数据类型和任务。例如，前馈神经网络适用于静态数据分析，而递归神经网络更适合时间序列建模；卷积神经网络是计算机视觉任务的核心，而生成对抗网络和自编码器主要用于数据生成和特征学习。下表总结了各类神经网络的主要特点及其应用场景：

表 5.7: 神经网络的不同类型及适用范围

神经网络类型	主要特点	适用任务
前馈神经网络 (FNN)	结构简单, 适用于静态数据	图像分类、回归任务
递归神经网络 (RNN)	适用于时序数据, 具有记忆能力 (Synapse)	语音识别、机器翻译
卷积神经网络 (CNN)	适用于图像处理, 具有局部特征提取能力	目标检测、医学影像
自编码器 (AE)	无监督学习, 数据降维	图像降噪、异常检测
生成对抗网络 (GAN)	生成式模型, 可合成数据	AI 艺术、DeepFake
Transformer	自注意力机制, 处理长序列任务	NLP、机器翻译)

5.9 通用逼近定理的局限性：为何我们需要深度学习？

通用逼近定理证明了，单隐藏层神经网络（即浅层网络），只要神经元足够多，理论上可以逼近任何连续函数。但**理论上的可行性，并不意味着工程上的最优解**。在实际应用中，浅层神经网络面临着一系列工程挑战，尤其是在计算开销、泛化能力，优化稳定性方面，使得浅层网络难以胜任复杂任务。

网络规模可能过大 为了达到高精度逼近，单隐藏层神经网络往往需要指数级增长的神经元数量，导致计算和存储成本急剧上升。这类似于“用大量小直线逼近曲线”——虽然可行，但效率极低。当时的计算机算力有限，大规模浅层网络几乎难以训练。例如，为了在 $\mathbb{R}^n$ 将逼近误差控制在 $\frac{1}{N}$ ，可能需要 $O(n^N)$ 个神经元，这在实践中几乎不可行。

训练过程面临优化困难 神经网络的权重优化是一个高度非凸的优化问题，可能存在多个局部最优，使得梯度下降的收敛过程变得困难。此外，浅层网络的结构限制了其分层特征提取能力，更依赖于大规模参数的调整，这不仅增加了计算复杂度，还容易陷入局部最优，限制了模型对复杂模式的有效学习。

泛化能力较弱 尽管浅层网络具备强大的逼近能力，但它们往往容易**过拟合训练数据**，导致泛化能力不足。换句话说，网络可能会“记住”训练样本的细节，而不是学习其潜在模式，因此在未见数据上表现较差。特别是在图像、语音、自然语言等高维复杂数据任务中，浅层网络的表现远不及更深的结构。

仅适用于连续函数 通用逼近定理仅适用于连续函数，而对于非连续函数或具有剧烈跳变的目标函数(如陡峭跃迁的信号、非连续决策边界等)，浅层网络难以有效拟合。因此，在许多实际应用中，必须对数据进行平滑化或特征变换，以增强神经网络的学习能力。言及浅层网络的局限性时，生成对抗网络 (GAN) 发明者 Ian Goodfellow 曾指出: “*A feedforward network with a single layer is sufficient to represent any function, but the layer may be infeasibly large and may fail to learn and generalize correctly.*” (仅含有一层的前馈网络，的确足以有效地表示任何函数，但是，这样的网络结构可能会格外庞大，进而无法正确地学习和泛化。) 实际上，通用逼近定理早在 1989 年就已提出，但直到 2006 年深度学习的崛起，神经网络才真正迎来突破性发展。这

一事实也验证了 Goodfellow 的判断：虽然单层网络足以逼近任何函数，但深度结构的学习效率更高，泛化能力更强，因而在实际应用中更具优势。

5.10 从浅层网络到深度学习

单隐藏层网络的局限性，促使研究者转向深度神经网络（Deep Neural Networks, DNN）。与其通过增加神经元数量来扩展浅层网络的容量，不如增加网络的深度，让多个隐藏层分工合作，逐层提取数据的抽象特征，从而提升学习效率和泛化能力。深度学习（Deep Learning）便是沿着这一思路发展而来的，这里的“深”即意味着**多个隐藏层（‘Deep’ means many hidden layers）**，即通过逐层提取特征，使模型具备更强的表达能力。简单来说，深度学习就是“具有多个隐藏层的神经网络学习。”近年来，深度神经网络的发展带来了卷积神经网络（CNN）、循环神经网络（RNN）、变换器（Transformer）等突破性架构，这些模型在计算机视觉、自然语言处理、强化学习等领域取得了前所未有的成就。相比于单隐藏层网络，深度神经网络在计算效率、泛化能力和特征提取方面更具优势。更深的网络结构可以通过逐层抽取特征，在更紧凑的参数规模下实现更高效的学习。这一层次化的特征表示方式，使得神经网络能够在更少的计算资源下学习更复杂的模式，同时减少训练所需的计算需求，从而提升训练效率。浅层网络的局限性推动了深度学习的崛起，但深度学习的发展远不止“增加网络层数”这么简单。深度学习的兴起不仅仅依赖于更深的网络架构，还受益于新型网络架构的探索、优化算法的改进、大规模数据集的可用性，以及 GPU 等计算资源的飞速发展。

5.11 小结

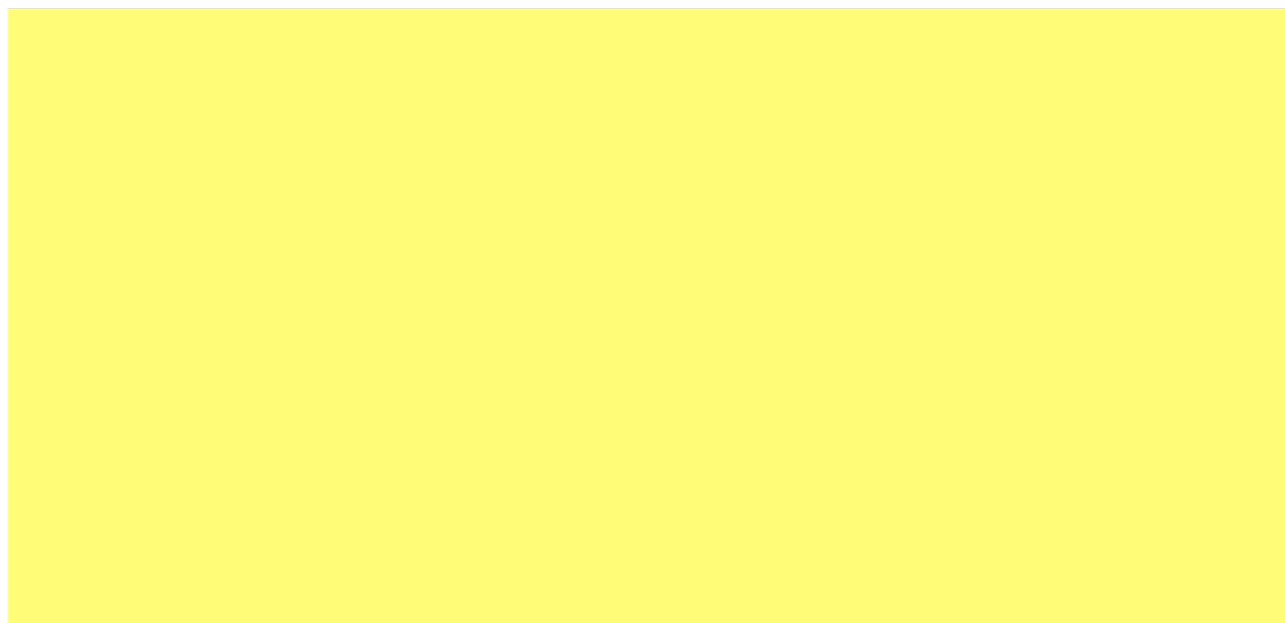


图 5.19: 认知世界：我们对自身了解多少？

科学的本质之一，是回答“我们如何认知世界”这一根本性问题。人工神经网络的诞生和发展为这一追问提供了技术上体现。它不仅赋予机器模拟学习的能力，也促使我们重新审视自身的认知过程。人工智能的探索，某种程度上也是映射人类如何理解思维与智能的一面镜子。人工神经网络的发展历程，从对生物神经元的模拟，到数学模型的构建，再到复杂网络结构的设计，充分体现了“效法自然”的思想。正如自然规律启发了无数科学技术的发明，人工神经网络的原理同样根植于对生物大脑的观察与理解。人类大脑通过感知、记忆、推理感知世界，而人工神经网络则通过分层结构模仿这一机制，并通过神经元的连接模式和权重调整，逐步实现了从简单模式识别到复杂认知任务的跨越。神经网络的出现，也促使我们重新思考“学习”这一概念。传统的计算机系统依赖规则和指令，而神经网络的独特之处在于，它能够通过数据训练，自主调整内部结构，以适应新的环境。这种能力使得它不仅能执行预设任务，更能在不断变化的条件下学习和进化。这种自适应性，正是智能最重要的特征之一。然而，我们仍然处在理解智能的早期阶段。尽管人工神经网络已在许多领域取得突破，但它与真正的智能仍有本质区别。我们仍在追问：机器能理解因果关系吗？它能进行抽象推理吗？它能像人类一样创造性地思考吗？这些问题仍然没有确切答案。但可以确定的是，神经网络的发展，已经为我们打开了一扇新的大门，使得探索智能的旅程变得更加清晰。在下一章，我们将深入探讨深度学习的基本原理、核心架构、优化方法以及实际应用，揭示深度神经网络如何突破传统浅层网络的局限，使人工智能迈向新的高度。

参考文献

- [1] “A Logical Calculus of Ideas Immanent in Nervous Activity”. In: *Bulletin of Mathematical Biophysics* 5 (1943), pp. 127–147.
- [2] Yoshua Bengio. “Practical recommendations for gradient-based training of deep architectures”. In: *Neural networks: Tricks of the trade: Second edition*. Springer, 2012, pp. 437–478.
- [3] George Cybenko. “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of control, signals and systems* 2.4 (1989), pp. 303–314.
- [4] Kunihiro Fukushima and Sei Miyake. “Neocognitron: A new algorithm for pattern recognition tolerant of deformations and shifts in position”. In: *Pattern recognition* 15.6 (1982), pp. 455–469.
- [5] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Multilayer feedforward networks are universal approximators”. In: *Neural Networks* 2.5 (1989), pp. 359–366.
- [6] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Universal approximation of an unknown mapping and its derivatives using multilayer feedforward networks”. In: *Neural networks* 3.5 (1990), pp. 551–560.
- [7] David H Hubel and Torsten N Wiesel. “Receptive fields and functional architecture of monkey striate cortex”. In: *The Journal of physiology* 195.1 (1968), pp. 215–243.
- [8] Han Jiawei, Kamber Micheline, and Pei Jian. *Data Mining: Concepts and Techniques*. -3rd. Morgan Kaufmann, 2011.
- [9] Fukushima K and Miyake S. “Neocognitron: A self-organizing neural network model for a mechanism of visual pattern recognition”. In: *Competition and cooperation in neural nets*. Springer, Berlin, Heidelberg, 1982, pp. 267–285.
- [10] Diederik P Kingma and Jimmy Ba. “Adam: A method for stochastic optimization”. In: *arXiv preprint arXiv:1412.6980* (2014).
- [11] Yann Le Cun, Boser Brian, and et al. Denker John S. “Handwritten digit recognition with a back-propagation network”. In: *Advances in neural information processing systems*. 1990, pp. 396–404.
- [12] Yann Le Cun et al. “Handwritten digit recognition: Applications of neural net chips and automatic learning”. In: *Neurocomputing: Algorithms, Architectures and Applications*. Springer. 1990, pp. 303–318.
- [13] Yann LeCun et al. “Backpropagation applied to handwritten zip code recognition”. In: *Neural computation* 1.4 (1989), pp. 541–551.

- [14] Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.
- [15] Minsky Marvin and A Papert Seymour. “Perceptrons”. In: *Cambridge, MA: MIT Press* 6.318–362 (1969), p. 7.
- [16] Warren S McCulloch and Walter Pitts. “A logical calculus of the ideas immanent in nervous activity”. In: *The bulletin of mathematical biophysics* 5 (1943), pp. 115–133.
- [17] Tom Mitchell. *Machine Learning*. McGraw-Hill Education, 1997.
- [18] Michael Nielsen. *Neural Networks and Deep Learning*. Accessed: 2025-03-03. 2015. URL: <http://neuralnetworksanddeeplearning.com/>.
- [19] Hastie T, Tibshirani R, and Friedman J. 统计学习基础 (*The Elements of Statistical Learning*). 北京: 世界图书出版公司, 2015.
- [20] Alan Turing. *Computing Machinery and Intelligence*. Oxford, 1950.
- [21] Vladimir Naumovich Vapnik. 统计学习理论的本质. 清华大学华夏出版社, 2000.
- [22] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).
- [23] 于剑. 机器学习—从公理到算法. 北京: 清华大学出版社, 2017.
- [24] [意] 翁贝托·博塔兹尼. 余婷婷译. 尖叫的数学—令人惊叹的数学之美. 湖南科学技术出版社, 2021.
- [25] 周志华. 机器学习. 清华大学出版社, 2016.
- [26] Vladimir N. Vapnik. 张学工译. 统计学习理论的本质. 北京: 清华大学出版社, 2000.
- [27] 张玉宏. 深度学习之美—AI 时代的数据处理与最佳实践. 电子工业出版社, 2018.
- [28] 张玉宏. 深度学习之美: AI 时代的数据处理与最佳实践. 电子工业出版社, 2018.
- [29] Tom Mitchell. 曾华军等译. 机器学习. 北京: 机械工业出版社, 2002.
- [30] 王珏, 周志华, 周傲英. 机器学习及其应用. Vol. 4. 清华大学出版社有限公司, 2006.

第六章 深度学习，真的行

深度学习的崛起，不仅推动了人工智能的发展，也彻底改变了我们对计算系统的认知。过去几十年，人工智能经历了多次兴衰，传统机器学习方法在特征工程和模型表现上遇到了瓶颈，而深度学习的出现突破了这一限制。本章将探讨深度学习的基本原理、核心架构、优化方法及实际应用，揭示深度神经网络如何突破传统浅层网络的局限，并成为现代人工智能的关键技术。接下来，我们将正式迈入深度学习的世界。

6.1 引言—深度学习：从理论走向现实

人工神经网络的概念早在 20 世纪 40 年代便已提出，并在 20 世纪 80-90 年代经历了一次快速发展，然而，受限于计算能力、优化方法和数据规模，早期神经网络在实际应用中未能展现出真正的潜力。直到 2010 年之后，随着深度学习的崛起，人工智能才迎来了真正的变革，并在多个领域实现了突破性进展。深度学习的成功并非偶然，而是多个因素共同推动的结果。其中，其中最关键的因素包括大规模数据的可获取性、计算资源的飞跃和优化算法的突破。首先，**海量数据的可获取性是深度学习发展的关键基础**。神经网络的学习能力在很大程度上取决于数据的规模和质量。过去，由于数据获取困难，神经网络的潜力难以展现。如今，随着互联网、社交媒体和智能设备的普及，文本、图像、语音等数据呈指数级增长，使得神经网络能够通过大规模数据训练，提升模型的泛化能力。其次，**计算资源的飞跃为深度学习的落地提供了重要技术支撑**。早期的计算机性能有限，训练大型神经网络往往需要耗费大量时间甚至难以实现。而如今，图形处理单元（GPU）的并行计算能力大幅提升，使得大规模神经网络的训练得以在可接受的时间内完成。更进一步，近年来 TPU（Tensor Processing Unit）等专用深度学习芯片的出现，使得更深、更复杂的神经网络成为可能，极大地推动了深度学习的实际应用。第三，**优化算法的持续突破推动了深度学习的成熟**。过去，浅层神经网络受限于梯度消失、过拟合、局部最优等问题，使得训练深层网络极为困难。而近年来，一系列关键技术的提出，如改进的反向传播、批量归一化（Batch Normalization）、残差连接（ResNet）和自适应优化方法（如 Adam、RMSprop）等，使得训练上百层的深度模型成为可能。正是这些因素的共同作用，使得深度学习从学术研究走向现实应用，并在计算机视觉、自然语言处理、医学影像分析、自动驾驶等多个领域展现出惊人的能力，成为推动人工智能变革的核心动力，不断拓展着 AI 的边界。相比早期的浅层神经网络，深度学习的成功并不仅仅是“增加了网络层数”那么简单，更是因为能够自动学习层次化特征、摆脱对人工特征工程的依赖，并通过优化方法解决训练难题。这些突破，使得深度学习在短短十几年间，从实验室走向大规模应用，并在多个领域引发了一场技术革命。

6.2 深度学习的方法论

深度学习 (Deep Learning) 之所以能在多个领域取得突破, 关键在于其**端到端特征学习能力和层次化信息表达方式**。与传统神经网络依赖手工特征工程不同, 深度神经网络能够通过多层结构自动学习数据的层级特征, 从低级模式 (如边缘、纹理) 到高级语义信息 (如物体类别、语言语境), 逐层构建更丰富、更具表达力的特征表示。这种方法不仅提高了模型的泛化能力, 还减少了对领域知识的依赖, 使其能够适应更广泛的任务, 如图像识别、语音处理、自然语言理解等。深度学习的核心突破体现在以下几个方面: **端到端特征学习和优化方法的改进**。本节将围绕这些关键概念展开分析, 以深入理解深度学习如何突破传统神经网络的局限, 并在实际应用中展现卓越的学习能力。

6.2.1 端到端 (end-to-end) 学习: 自动特征学习的突破

深度学习的核心突破在于其**端到端学习 (End-to-End Learning)**, 即神经网络直接从输入数据学习到最终的预测结果, 而无需人工干预。例如, 基于深度学习的图像识别系统, 输入端是图片的像素数据, 而输出端直接就是或猫或狗的判定。这个端到端就是: 像素判定。又例如, 自动驾驶系统, 输入的是前置摄像头的视频信号 (其实也就是像素), 而输出的直接就是控制车辆行驶指令 (方向盘的旋转角度)。端对端学习不仅摆脱了手工特征工程的限制, 还使得模型能够更高效地适应不同任务, 提高泛化能力。此外, 深度学习实现了**表征学习 (Representation Learning)**, 即模型能够自主学习数据的层级特征, 而无需人工定义特征, 使得不同任务间的迁移更加容易。在早期的神经网络模型中, 研究者通常需要依据领域知识手动设计特征, 再将这些特征输入模型进行训练。例如, 在图像识别任务中, 研究者往往会先提取边缘、颜色直方图、纹理等信息, 然后使用机器学习模型进行分类。在语音识别任务中, 传统方法通常依赖于特定的特征提取步骤, 如梅尔频谱 (Mel-spectrogram) 或 MFCC (梅尔倒谱系数), 然后再利用统计模型进行语音识别。然而, 这种方法存在明显的局限性。首先, 它高度依赖人工经验, 研究者需要针对不同任务手工设计特征, 这不仅增加了开发成本, 还可能导致特征工程成为限制模型性能的瓶颈。其次, 手工特征往往是低维的, 难以捕捉数据中的复杂模式, 使得模型在面对高维、非结构化数据时表现受限。此外, 不同任务所需的特征类型可能截然不同, 导致研究者在每个新任务中都需要重新设计特征, 使得模型的泛化能力受限。与此相对, 深度学习采用端到端的学习方式, 使得神经网络能够直接从原始数据中自动学习特征, 而无需人为干预。这种学习方式让模型可以自行决定哪些特征最为重要, 从而避免了手工特征提取过程中可能出现的信息丢失或人为偏差。更重要的是, 深度学习能够通过表征学习逐层构建高层语义信息, 提升对复杂数据的理解能力。在图像识别任务中, 深度神经网络的不同层能够学习到不同层次的特征。例如, 底层神经元会识别基本的边缘、角点和颜色变化等低级特征, 中间层神经元则能够提取更复杂的局部结构, 如纹理、形状和特定模式 (如狗的耳朵、人脸的轮廓), 而高层神经元则可以提取更抽象的语义信息, 如完整的对象类别 (如汽车、人、猫), 甚至具体的身份 (如某个特定人的脸)。这种层次化的特征学习方式, 使得深度学习在处

理高维数据时展现出比浅层网络更强的表达能力和泛化能力。端到端学习的优势不仅体现在特征学习的自动化上，还在于其提高了整个模型的训练效率和性能。由于神经网络可以同时优化多个计算阶段，端到端学习允许模型以全局最优的方式进行训练，从而提升整体系统的性能。此外，这种方式还减少了人为调整特征的成本，使得神经网络可以适应更广泛的应用场景，并在不同任务之间更容易进行迁移学习。以语音识别任务为例，传统方法通常需要经历多个独立的步骤，包括特征提取、声学建模、语言建模和最终预测，每个步骤都需要单独设计并优化。而在端到端的深度学习方法中，整个流程被整合到一个统一的神经网络中，该网络能够直接接收原始语音信号，并最终输出文本。这种方法不仅减少了人工干预的环节，还提高了模型的适应性，使其能够更好地处理不同语言环境和语音风格的变化。综上所述，端到端学习是深度学习相较于传统神经网络最显著的优势之一。它通过数据驱动的方式自动学习特征，减少了对手工特征工程的依赖，提高了模型的泛化能力，并使得神经网络在复杂任务中能够高效地学习和适应不同的数据模式。

以手写数字识别（MNIST）为例，我们来看看深度神经网络如何学习不同层次的特征。MNIST 数据集包含 0 到 9 的手写数字，目标是让模型识别输入的图片是哪一个数字。如果使用传统方法，研究者首先提取手工特征，如计算像素值的直方图、轮廓信息、霍夫变换等，然后再将这些信息输入支持向量机（SVM）或 K 近邻（KNN）等分类器进行分类。而使用深度神经网络（如卷积神经网络 CNN），模型会自动学习特征。第一层卷积（低级特征）识别简单的线条、边缘、拐角，如“7”的横线和竖线部分。中间层（中级特征）学习更复杂的形状，如圆弧、封闭区域，能够区分“8”和“3”的不同。高层（高级特征）最终网络学习到完整的数字概念，能够基于整体形状、笔画的分布来识别输入的数字。

6.2.2 优化方法的改进：让深度网络真正可训练

深度学习的成功不仅依赖于网络结构的改进，还得益于优化方法的突破。尽管神经网络的基本理论早已提出，但由于训练困难、梯度不稳定和计算成本高昂，早期的神经网络难以扩展到更深层的结构。而近年来的优化方法，如**反向传播改进**、**梯度稳定技术**、**自适应优化算法**等，使得深度学习成为现实。反向传播算法是神经网络训练的核心机制。它利用链式法则（Chain Rule）计算误差对每个权重的梯度，并通过**梯度下降（Gradient Descent）**优化神经网络的权重，以最小化损失函数。在训练过程中，误差信息从输出层反向传播到输入层，每一层的权重根据梯度信息进行更新，使得模型不断优化。在浅层神经网络中，因为只有一个隐藏层，反向传播非常容易进行。然而，随着网络深度的增加，**梯度消失（Vanishing Gradient）**和**梯度爆炸（Exploding Gradient）**逐渐成为训练深层网络的主要挑战，使得优化变得更加困难。因此，仅仅依靠传统的反向传播还不足以训练深度神经网络，必须引入更多优化策略来增强训练的稳定性和效率。

6.2.2.1 梯度稳定性问题及解决方案

在深度网络的训练过程中，梯度可能会在反向传播时迅速衰减或增长过快，分别导致**梯度消失 (Vanishing Gradient)** 或**梯度爆炸 (Exploding Gradient)** 问题，使得前层的参数几乎无法更新。这些问题会导致模型学习受阻，甚至无法有效训练。然而，随着网络深度的增加，梯度信号可能会在传播过程中逐渐减弱或放大，导致问题，使得训练深度网络变得极为困难。

6.2.2.2 梯度消失问题 (Vanishing Gradient)

梯度消失通常出现在使用 Sigmoid 或 Tanh 激活函数的深层网络中。由于这些激活函数的梯度在接近极值时趋于零，导致前几层的参数更新变得极为缓慢，使得模型难以学习长期依赖关系。为了解决这一问题，研究者提出了新的激活函数—**ReLU (Rectified Linear Unit)** 及其变体，如 Leaky ReLU、PReLU、GELU 等。相比 Sigmoid 和 Tanh，ReLU 的计算简单，并且在正区间保持恒定梯度，使得梯度不会衰减过快，从而大大缓解了梯度消失问题。然而，即使使用 ReLU 作为激活函数，随着深度的增加，深层网络仍然可能面临梯度消失问题。为了进一步提升训练深度网络的可行性，研究者提出了一系列优化策略：

- **批量归一化 (Batch Normalization, BN)**：在每一层对激活值进行归一化，使得数据分布保持稳定，从而提升训练的稳定性和收敛速度。
- **残差连接 (Residual Connection, ResNet)**：ResNet 结构通过在层间引入跳跃连接 (Skip Connections)，允许梯度直接跨层传播，减少信息的损失，使得数百层的神经网络也能稳定训练。
- **层归一化 (Layer Normalization, LN) 与权重初始化方法**：优化权重初始化（如 Xavier 初始化、He 初始化）可以减少梯度消失的概率，而层归一化 (LayerNorm) 进一步提升深度模型的训练稳定性，特别适用于 Transformer 架构。

这些优化方法的提出，使得训练 50 层、100 层甚至更深的神经网络成为可能，也为大规模深度学习模型（如 GPT、BERT）奠定了基础。

6.2.2.3 梯度爆炸问题 (Exploding Gradient)

梯度爆炸通常发生在深度神经网络或循环神经网络 (RNN) 中。当误差在反向传播过程中逐层回传时，梯度可能以指数级增长，导致参数剧烈更新，使训练过程不稳定，甚至出现参数溢出 (NaN 值)，影响模型收敛。为解决这一问题，**梯度裁剪 (Gradient Clipping)** 是最直接的方法，它通过设定梯度的最大范数，当梯度超过阈值时进行缩放，从而防止梯度过大导致训练不稳定。这种方法特别适用于 RNN，防止长序列训练过程中梯度指数级增长。此外，**合适的权重初始化** 也能有效缓解梯度爆炸，如 Xavier 初始化适用于 Sigmoid/Tanh 激活函数，而 He 初始化更适用于 ReLU 激活函数，确保初始权重分布合理，防止梯度在初始阶段过大或过小。**优化学习率** 也是关键手段。学习率过高会导致参数剧烈波动，引发梯度爆炸。采用**学习率调度 (Learning Rate Scheduling)**，如指数衰减或循环学习率，避免初始学习率过高导致

梯度爆炸，确保训练稳定。

6.2.2.4 自适应优化算法 (Adaptive Optimization Algorithms)

传统的梯度下降 (SGD, Stochastic Gradient Descent) 在深度神经网络的训练中虽然有效，但它依赖于一个固定的学习率，在训练不同阶段或面对不同数据分布时，难以始终保持最佳收敛速度。若学习率过高，模型可能无法收敛或出现梯度爆炸；若学习率过低，则收敛速度缓慢，甚至陷入局部最优。因此，研究者们提出了一系列自适应优化算法，使得深度神经网络在训练过程中能够动态调整学习率，提高训练效率和稳定性。Adam (Adaptive Moment Estimation) 是目前最常用的自适应优化方法之一。它结合了动量梯度下降 (Momentum) 和 RMSprop 的优点，在计算梯度的一阶动量 (期望值) 和二阶动量 (方差估计) 后，自适应地调整每个参数的学习率。Adam 的优势在于，它在初始阶段可以使用较大的有效步长加速收敛，同时在后期自适应地降低步长，避免震荡，使得训练更加稳定。RMSprop (Root Mean Square Propagation) 则主要用于解决梯度更新过程中参数的变化幅度不均的问题。它通过对梯度平方的指数加权平均进行归一化，从而减少参数更新的不均匀性，使得模型可以更平稳地收敛。RMSprop 在训练循环神经网络 (RNN) 时尤为有效，能够防止梯度消失或爆炸，提高长期依赖问题的建模能力。此外，Adagrad (Adaptive Gradient Algorithm) 也是一种自适应优化方法。它会对每个参数的梯度累积求和，使得更新频率较高的参数学习率逐渐降低，而对更新较少的参数保持相对较高的学习率。尽管 Adagrad 在稀疏特征问题上表现良好，但由于学习率不断减小，训练后期容易出现收敛停滞的问题。相比之下，AdamW 作为 Adam 的变种，引入了权重衰减 (Weight Decay) 机制，能够更好地控制 L2 正则化的影响，提高模型的泛化能力。通过这些自适应优化算法，深度神经网络能够更高效、稳定地训练，减少对人工调整学习率的依赖，同时在复杂任务中实现更快的收敛和更优的泛化性能。

6.2.3 正则化：防止过拟合，提高泛化能力

在深度学习模型的训练过程中，**过拟合 (Overfitting)** 是一个常见的问题。当模型在训练数据上表现良好，但在新数据上的性能下降时，说明模型可能学习到了训练数据中的噪声或不具备泛化能力的模式。**正则化 (Regularization)** 方法的核心目标是提高模型的泛化能力，使其在未见过的数据上也能保持良好的预测效果。正则化的实现方式包括 L1 和 L2 正则化、Dropout、批量归一化、数据增强、早停等策略的综合运用。

L1 和 L2 正则化：约束权重，防止模型过复杂

在神经网络训练过程中，模型的复杂度通常由其参数 (权重和偏置) 决定。如果模型参数过大，可能会导致过拟合。L1 和 L2 正则化 (也称为权重衰减) 通过对模型的权重施加约束，防止其变得过于复杂。**L1 正则化 (Lasso)** 通过添加权重的绝对值惩罚项，使得部分权重趋于零，从而实现特征选择，使模型更加稀疏。

$$L_{L1} = \lambda \sum |w_i|$$

而 **L2 正则化 (Ridge)** 则通过添加权重的平方惩罚项，鼓励较小的权重值，使模型更加平滑，避免对个别特征过度依赖。

$$L_{L2} = \lambda \sum w_i^2$$

在神经网络中，L2 正则化（也称为**权重衰减 (Weight Decay)**）更常用，因为它能有效降低模型的复杂度，同时保留所有特征。

Dropout：随机丢弃神经元，增强模型鲁棒性

为便于阅读，本节（Dropout：随机丢弃神经元，增强模型鲁棒性）承接上节（正则化：防止过拟合，提高泛化能力），先交代差异与共同点，再聚焦本节关键问题。

Dropout 是深度学习中特有的一种正则化方法，它在每次训练时，**随机“丢弃”**一定比例的神经元（即将其输出置为零），使得网络不会过度依赖特定神经元，从而提高模型的泛化能力。**Dropout** 机制的作用类似于训练多个不同的子网络并进行集成：

- 训练时，每个 mini-batch 训练时随机关闭部分神经元，使得模型学习到更加通用的特征；
- 测试时，使用完整的神经网络，并对 Dropout 进行均值归一化，以保证预测的稳定性。

Dropout 通常用于全连接层，对于卷积神经网络 (CNN)，一般使用批量归一化 (Batch Normalization) 作为替代方案。

批量归一化 (Batch Normalization)：稳定训练，提高泛化能力

为便于阅读，本节（批量归一化 (Batch Normalization)：稳定训练，提高泛化能力）承接上节（Dropout：随机丢弃神经元，增强模型鲁棒性），先交代差异与共同点，再聚焦本节关键问题。

批量归一化 (Batch Normalization, BN) 通过对每一层的激活值进行归一化，使得数据分布更稳定，从而加速训练并降低对初始参数的敏感性。批量归一化计算每个 mini-batch 内的均值和方差，并对激活值进行标准化：

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

其中， μ_B 和 σ_B^2 分别是 mini-batch 的均值和方差， ϵ 是防止除零错误的一个小常数。批量归一化具有如下优势：

- **加速收敛**：减少梯度消失，使得深度网络能够稳定训练；
- **减少对学习率的依赖**：在较大学习率下仍能训练稳定；
- **一定程度上起到正则化作用**，减少对 Dropout 的需求。

BN 适用于深度神经网络的卷积层和全连接层，并广泛用于 CNN 结构中。

数据增强 (Data Augmentation): 扩大数据集, 提升泛化能力

为便于阅读, 本节(数据增强 (Data Augmentation): 扩大数据集, 提升泛化能力)承接上节(批量归一化 (Batch Normalization): 稳定训练, 提高泛化能力), 先交代差异与共同点, 再聚焦本节关键问题。

数据增强是一种间接的正则化方法, 它通过对训练数据进行变换, 扩展数据集规模, 从而减少模型对数据分布的依赖, 提高泛化能力。常见的数据增强技术包括:

- **图像数据增强**: 旋转、翻转、缩放、裁剪、颜色抖动等;
- **文本数据增强**: 同义词替换、随机删除、回译 (Back-Translation);
- **语音数据增强**: 加入噪声、改变语速、时间遮掩等。

数据增强能有效提升模型的鲁棒性, 特别是在数据量有限的情况下, 是提高模型性能的重要手段。

早停 (Early Stopping): 防止过拟合

为便于阅读, 本节(早停 (Early Stopping): 防止过拟合)承接上节(数据增强 (Data Augmentation): 扩大数据集, 提升泛化能力), 先交代差异与共同点, 再聚焦本节关键问题。

早停 (Early Stopping) 是一种简单有效的正则化方法。它的核心思想是: 在训练过程中, 持续监测验证集上的损失; 当验证集损失开始上升时, 停止训练, 防止模型过度拟合训练数据。早停的实施通常结合学习率衰减 (Learning Rate Decay), 即在训练后期逐步降低学习率, 以进一步提升模型的稳定性。

正则化的选择与综合使用

为便于阅读, 本节(正则化的选择与综合使用)承接上节(早停 (Early Stopping): 防止过拟合), 先交代差异与共同点, 再聚焦本节关键问题。

在实际应用中, 不同的正则化方法通常结合使用。

- **L2 正则化 + Dropout**: 适用于全连接层, 防止权重过大, 同时增强网络鲁棒性;
- **批量归一化 (BN)**: 适用于深度网络中的每一层, 加速训练并减少 Dropout 需求;
- **数据增强**: 适用于计算机视觉、自然语言处理等领域, 以扩展数据集规模;
- **早停**: 防止训练过长导致模型过拟合。

综上, 正则化方法在深度学习中起着至关重要的作用, 它不仅能防止模型过拟合, 还能提高泛化能力, 使得深度神经网络能够更稳健地应用于各种复杂任务。

6.2.4 优化与泛化的平衡

6.3 全连接神经网络：深度学习的“原型”与局限性

在探索深度学习为何“真得行”之前，我们先从最早期、最基础的架构谈起——**全连接神经网络 (Fully Connected Neural Network)**，也称为**多层感知机 (MLP)**。它是深度学习的原型，是最直接、最纯粹的“函数逼近器”。从数学视角来看，全连接神经网络本质上是一个函数逼近器，它通过层层堆叠的映射逐步实现数据的高维变换。在每一层，输入首先经历一个仿射变换（线性变换叠加偏置），再经过非线性激活函数进行转换，最终产生下一层的表示。我们先看看第 i 的表达式。设第 i 层有 n_i 个神经元，则该层输出表示为一个向量：

$$h^{(i)} \in \mathbb{R}^{n_i}, \quad h^{(i)} = [h_1^{(i)}, h_2^{(i)}, \dots, h_{n_i}^{(i)}]^T$$

其中， $h_j^{(i)}$ 是第 i 层第 j 个神经元的输出。

神经网络的前向传播可以用以下公式表示

$$h^{(i)} = f^{(i)} (W^{(i-1)} h^{(i-1)} + b^{(i-1)})$$

其中， $W^{(i-1)} \in \mathbb{R}^{n_i \times n_{i-1}}$ 是第 $i-1$ 层到第 i 层的权重矩阵； $b^{(i-1)} \in \mathbb{R}^{n_i}$ 是偏置项； $f^{(i)}(\cdot)$ 是逐元素施加的非线性激活函数，如 **ReLU**、**Sigmoid** 或 **tanh**； $h^{(i-1)} \in \mathbb{R}^{n_{i-1}}$ 为第 $i-1$ 层的输出向量。

现假设网络共有 L 层，则从输入到输出完整的前向传播过程可递归表示为

$$h^{(1)} = f^{(1)} (W^{(0)} x + b^{(0)})$$

$$h^{(2)} = f^{(2)} (W^{(1)} h^{(1)} + b^{(1)})$$

⋮

$$h^{(L)} = f^{(L)} (W^{(L-1)} h^{(L-1)} + b^{(L-1)})$$

$$y = W^{(L)} h^{(L)} + b^{(L)}$$

其中， x 为原始输入数据； $W^{(l)}$ 为第 l 层的权重矩阵； $b^{(l)}$ 为第 l 层的偏置项； $f(\cdot)$ 为非线性激活函数（如 **Sigmoid**、**ReLU**）； $h^{(l)}$ 为第 l 层的特征表示； y 为网络最终的输出结果。所谓的“全连接”神经网络，是指每个神经元都与上一层的所有神经元完全相连，就像是每个节点都与其他节点“握手”。网络结构无任何稀疏性，也没有位置或顺序的偏好。这使得它在建模结构灵活性上表现强大。在全连接网络中，每一层的激活函数可以一致，也可以因层而异。在大多数经典的全连接神经网络结构中（例如传统的多层感知机 **MLP**），各隐藏层通常采用相同的激活函数（早期网络多用 **Sigmoid** 或 **Tanh**，现代网络更常用 **ReLU** 或其变种 **Leaky ReLU**、**GELU** 等）。输出层的激活函数根据任务不同而定，例如多分类任务使用 **softmax**，二分类任务使用 **sigmoid**，回归任务则无需激活，直接使用线性函数。

中间层的高维映射：为何越“深”越有表现力？“在高维空间里，复杂问题变得更容易解决。”这不仅是支持向量机中“核技巧”（Kernel Trick）背后的核心思想，也深刻揭示了深度神经网络具有强大表达能力的数学本质。在全连接神经网络中，所谓“高维映射”并非指最终输出的维度，而是指输入在经过前几层被逐层“展开”到更高维度的特征空间中。具体而言，输入数据 $x \in \mathbb{R}^{d_{in}}$ 过一系列仿射变换与非线性激活，映射到多个中间的高维特征空间 $h^{(i)} \in \mathbb{R}^{d_i}$ ，其中通常 $d_i > d_{in}$ 。这种升维操作是深度学习增强表达能力的关键所在。其核心思想与支持向量机中的“核技巧”如出一辙：通过非线性映射，将原始输入投射到一个更高维的特征空间，使得在低维中难以区分的数据，在高维空间中可能变得线性可分，进而简化决策边界的学习。不同之处在于，支持向量机通常采用预定义的核函数进行一次性映射，而神经网络则是通过多层可训练的多层非线性变换，动态构建出越来越抽象、越来越强表达能力的特征空间。每一层都相当于一个非线性变换器，这种“逐层嵌套 + 非线性激活”的结构在数学上构成了一个复杂的高阶复合函数，使模型具有强大的函数逼近能力。简而言之，深度神经网络的多层结构，就

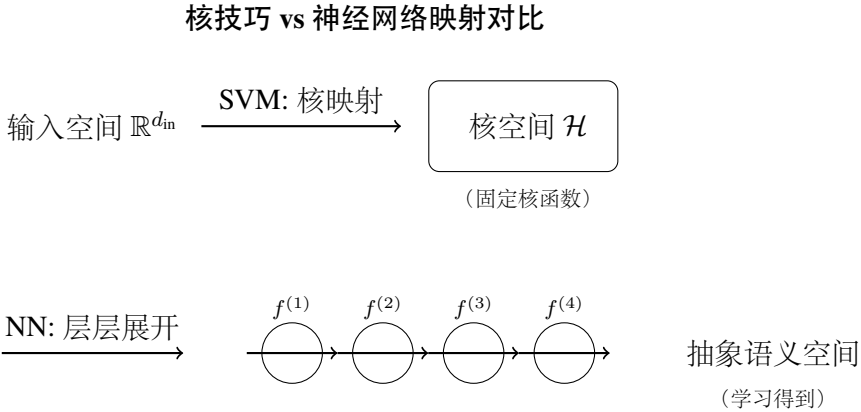


图 6.1: 核技巧（SVM）与深度神经网络非线性高维映射的对比

像是一个可以自动学习的“高维嵌入序列”，不断将数据“展开”到更适合建模和判别的空间中。这正是深度学习在面对复杂任务时展现出惊人表现力的根源所在。中间层的高维映射带来了多方面的数学优势：

- **增强非线性可分性：**许多复杂模式在低维中无法用简单边界区分，而在高维特征空间中变得更易于区分，分类性能得以提升；
- **提升特征组合的自由度：**高维空间允许模型组合出更多、更复杂的特征交互关系，有利于模型拟合复杂函数，模型的表达力得以增强；
- **提升函数逼近能力：**深度模型叠加的非线性结构使其能够更好地拟合复杂的目标分布。

随着网络层数的加深，这种高维映射逐层构建出越来越抽象的特征表示：底层可能识别边缘或角点，中层形成局部结构如形状或纹理，高层则可能对具体的语义概念（如“猫耳朵”或“车轮”）作出响应。每一层的映射都在为模型提供一个更“好理解世界”的坐标系。因此，所谓“越深越有表现力”，从数学本质上看，是网络不断扩大其特征空间与函数族的覆盖范围，增强了对复杂任务的适应能力，也为深度学习成为“通用建模器”奠定了坚实的理论基础。正如人类认知是从感官感知逐步上升到抽象思维，深层神经网络也通过层层嵌套的高维变换，构建出一种可计算、可训练、可泛化的认知系统。

局限性与转向 虽然全连接神经网络在理论上具备近似任意函数的能力，但在实践中，存在着几个关键瓶颈：

- **参数数量激增** 由于所有神经元两两相连，全连接网络的参数规模随着数据维度迅速膨胀，计算和存储成本高。
- **缺乏结构性假设** 全连接神经网络在处理图像或语言等结构化数据时存在局限。图像具有空间结构，语言和语音则是时间序列，而全连接网络需要将输入“扁平化”，打散了原有的空间或时序关系。由于它对所有输入维度一视同仁，既难以捕捉局部特征，也难以保留上下文依赖，因此不适合建模具有结构的信息。
- **梯度消失与难以训练深层结构** 当网络层数变深时，传统的全连接神经网络面临梯度消失或梯度爆炸问题，深层网络往往难以有效训练，阻碍有效学习。

这些局限促使研究者探索更高效、更具结构化的模型架构。由此诞生了卷积神经网络（CNN）、循环神经网络（RNN）和 Transformer 等一系列重要的深度学习架构。它们不再满足于单纯地“堆叠层数”，而是在结构设计中引入了明确的数学假设和认知启发，使深度学习模型更好地适应图像、语言、序列等不同类型的数据。可以说，CNN、RNN 和 Transformer 的出现，正是在全连接神经网络这一“原型”基础上的演化成果。它们是深度学习数学思维不断深入、抽象结构不断优化的具体体现。接下来，让我们依次看看这些更高效、更强大的神经网络架构，是如何让深度学习真正“行”起来的。

6.4 神经网络的不同架构：从认知方式到计算方式

深度学习的强大不仅体现在其学习能力，还在于它对世界的“认知方式”——不同类型的数据决定了不同的学习策略，而神经网络的架构正是这种策略的具体实现。我们的大脑在感知世界时，不会用同一种方式去理解视觉、听觉和语言。视觉处理需要分层解析物体的形状和纹理，而语言理解则需要把信息组织在时间轴上，建立语义联系。那么，计算机如何模拟人的这些认知方式？答案就在神经网络架构的多样性。机器学习中有个著名的“没有免费的午餐”定理（No Free Lunch Theorem）。它指出，没有一个万能的算法可以在所有问题上都表现最佳。深度学习虽然强大，但并非在所有任务上都能取得理想效果，选择合适的模型至关重要。不同的神经网络架构，实际上就是对不同数据模式的最佳适配。如果让一个完全连接的神经网络去处理图像，每个像素都作为独立的输入，它不仅需要记住庞大的模式空间，而且无法利用图像的空间结构，计算量巨大，泛化能力低。同样，如果用同样的方法去处理语音或文本数据，模型将难以理解单词和句子之间的顺序关系。因此，神经网络的演化不仅是数学上的优化，更是认知策略的进化——让计算机能够以最适合的方式去理解不同类型的信息。深度学习发展至今，已经形成了多种不同的网络架构，每一种都针对特定类型的数据和任务进行了优化。其中，卷积神经网络（CNN）通过局部感受野和权重共享机制，极大提升了计算效率，使计算机能够高效地识别图像特征。循环神经网络（RNN）及其变种 LSTM 和 GRU 通过引入时间序列依赖性，使得模型能够记住前后关系，适用于语音识别、机器翻译等任务。而 Transformer 彻底颠覆了序列学习的传统模式，利用自注意力机制（Self-Attention）突破了 RNN

的长程依赖问题，并在自然语言处理、计算机视觉等领域表现卓越。接下来，让我们从三个最具代表性的神经网络架构—卷积神经网络（CNN）、循环神经网络（RNN）和 Transformer 来看，深度学习是如何根据数据的特性，发展出不同的计算模式。

6.4.1 卷积神经网络（CNN）：如何高效处理图像？

在探讨卷积神经网络（CNN）之前，我们先进行一个有趣的实验。请观察以下三幅点描画 6.2。



图 6.2: 仅凭轮廓，就能做出判断

尽管这三幅画缺乏细节，但我们仍然能轻松判断出画中人物的性别与年龄，推测从左到右分别是男性、女性、老年男性。为什么我们能做到这一点？明明这些点并没有提供完整的细节信息，但我们的视觉系统依然能够识别出轮廓，推测出结构，甚至自动补全缺失的信息。这正是人类视觉系统的一大特性：**先感知整体，再关注局部**。我们的大脑并不会像相机一样简单记录所有像素，而是先捕捉轮廓和形状，再细化辨识细节，逐步构建对物体的整体认知。这种层级感知能力，使得我们能够快速识别目标，而不需要逐像素分析整个场景。这一认知模式在日常生活中随处可见。例如，即使你的朋友戴着口罩和头盔，你仍然能在一瞬间认出他。而计算机视觉，即便经过复杂的深度学习训练，在处理戴口罩的人脸识别任务时，仍远远达不到人类这种近乎直觉般的默契程度。人类不仅依赖五官的细节特征，还会利用头部轮廓、身形、步态，甚至熟悉的服饰风格来做出判断，这种整体与局部信息的结合，使得我们在极端遮挡条件下仍能精准识别。这一认知模式源自于人类进化过程中形成的生存本能。设想远古时期，一位原始人在森林中狩猎时，必须迅速判断远处的影子是猛兽、猎物还是同类，进而决定逃跑躲避，还是准备狩猎。此时，过度关注细节不仅耗费能量，还可错失反应时机，导致危险。因此，人类视觉系统进化出了“Global First”（先整体、后局部）的策略—先快速提取大致轮廓，形成整体认知，再细化分析局部特征。

6.4.1.1 从人类视觉到人工智能：CNN 如何“看”世界

让计算机像人类一样“看”世界，一直是计算机视觉领域的核心挑战。早期的神经网络，如多层感知机（MLP），采用全连接结构，在处理图像时，需要将二维图像展平为一维向量，并让所有像素点与神经元进行全连接计算。这种方式不仅破坏了图像的空间结构，使模型难以理解边缘、形状等局部信息，同时也导致参数量激增。例如，对于一张 1000×1000 像素的彩色图像，每个像素包含 RGB 三个通道，全连接神经网络需要处理 300 万 ($1000 \times 1000 \times 3$) 个像素的全连接，参数量巨大，训练难度指数级增长，因而难以扩展到复杂的计算机视觉任务。

相比之下，卷积神经网络（Convolutional Neural Network, CNN）提出了一种更符合人类视觉系统的方式。CNN 并不是试图记住所有像素点，而是模仿人类视觉的分层感知策略，从局部特征逐渐构建整体认知。正如我们在看一幅点描画时，不会立即关注所有点，而是先感知整体轮廓（如头形和脸形），再逐步识别细节（如眼、嘴的位置），CNN 也是先提取图像的低级特征，如边缘、纹理，再逐步组合形成高级语义信息，从而实现更高效的图像理解。

6.4.1.2 人类视觉的分层感知机制

数学直观：卷积如何看世界卷积神经网络的灵感，源自神经科学家对视觉系统的研究。1962 年，美国神经科学家大卫·休伯尔（David Hubel）和托斯坦·威泽尔（Torsten Wiesel）在研究猫的视觉皮层时发现，大脑皮层的神经元对视野中的特定区域敏感，它们不会响应整幅图像，而只在特定方向的边缘或形状出现时产生响应。这一现象被称作**局部感受野 (receptive field)** [7]。换句话说，视觉皮层的神经元只对视野中的某一小块区域敏感，而不是直接处理整个图像。他们还进一步发现，大脑视觉皮层是分级、分层处理的。在大脑的初级视觉皮层中，简单细胞（simple cell）主要负责检测基本边缘和方向信息，复杂细胞（complex cell）则能够综合多个简单细胞的输出，识别更复杂的局部形状，而超复杂细胞（hyper-complex cell）则进一步整合高级语义信息，形成完整的对象认知。如图 6.3 所示，人类的视觉感知是一个从低级到高级的层级化过程，从视网膜的光感受器记录像素信息，到初级皮层识别基本边缘、方向和颜色等，再到中级视觉皮层（V2-V4）组合这些低级特征形成更复杂的局部形状，最后高级视觉皮层（如颞叶 IT 皮层）整合完整的物体认知，例如区分人脸、动物或交通工具。对于不同类别的物体，视觉系统也是按照这种逐层分级的方式进行感知。从图 6.4 可以看出，最底层的特征基本上是类似的，主要是各种边缘，越往上越能提取出此类物体的一些特征（轮子、眼睛、躯干等），到最上层，不同的高级特征最终组合成相应的图像，从而能够让人类准确的区分不同的物体。因为这项研究，休伯尔和维塞尔在 1981 年被授予诺贝尔医学奖。他们的发现启发了计算机科学家，人工神经网络可以摒弃全连接方式，而采用局部连接的方式，从而极大提升计算效率。受此启发，日本科学家福岛邦彦（Kunihiko Fukushima）在 1980 年提出了神经认知机（Neocognitron）模型 [4]，这可以看作是卷积神经网络的雏形如图 6.5 所示。神经认知机引入了局部连接（Local Connectivity）和权值共享（Weight Sharing）机制，使得神经网络可以逐层提取图像的不同层次特征，并极大减少计算参数的规模。这一思想成为后来的 CNN 设计的重要基础。然而，Neocognitron 仍存在一些问题，尤其是训练方法的局限性，使

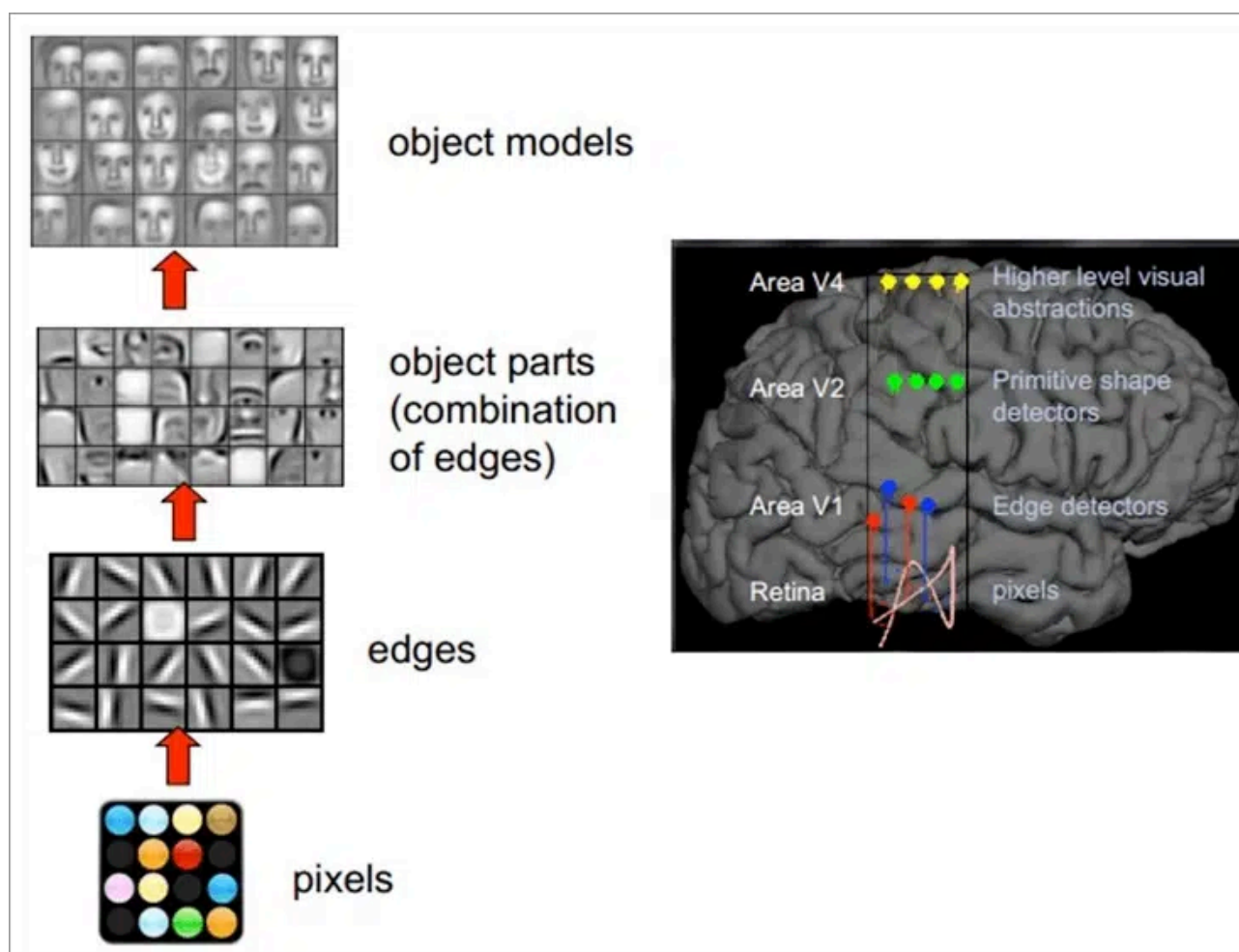


图 6.3: 人类的视觉原理

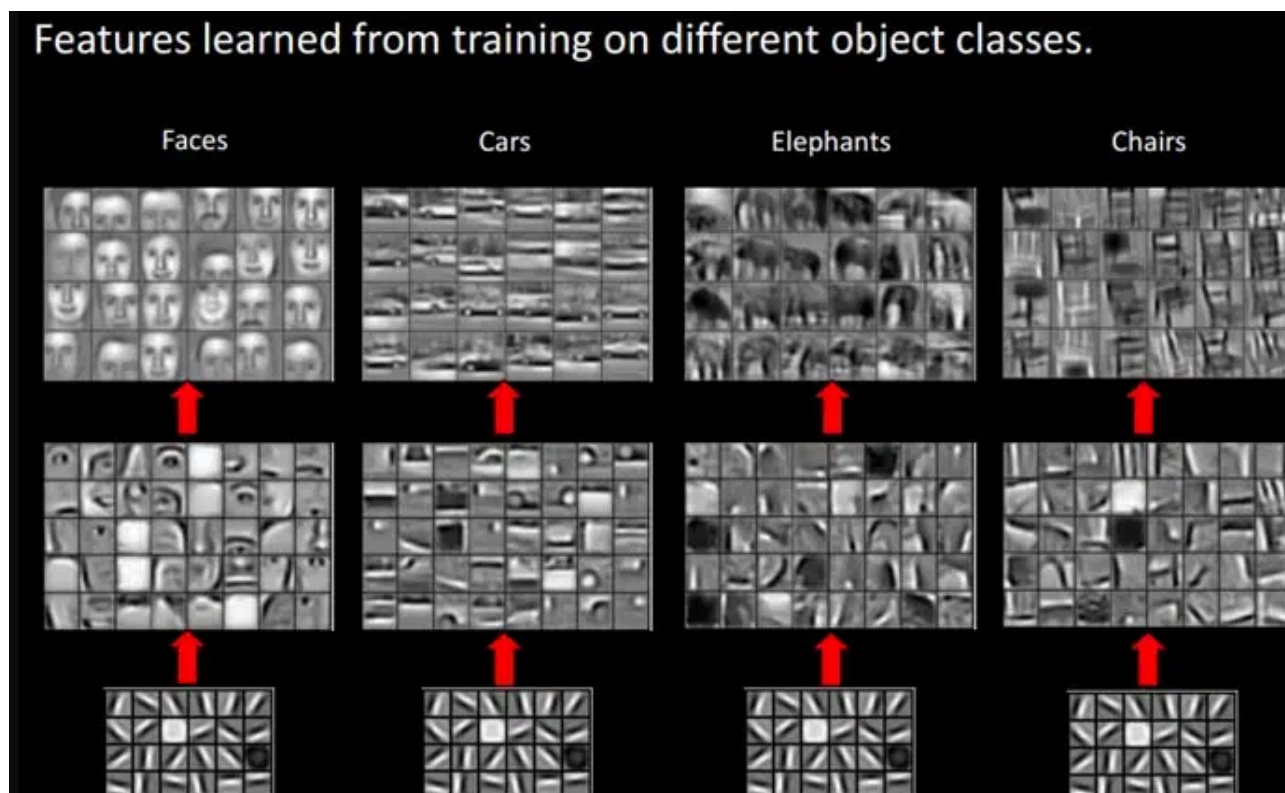


图 6.4: 视觉分层

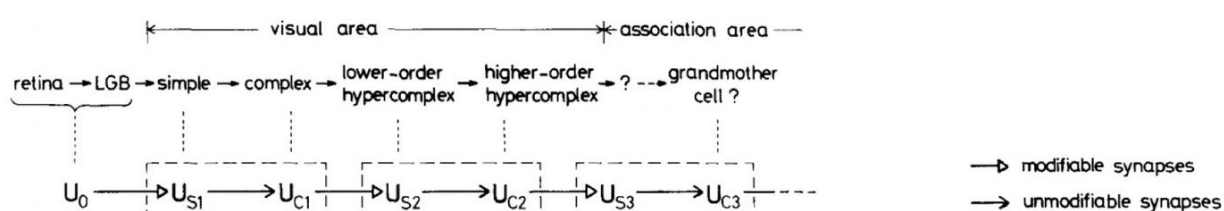


Fig. 1. Correspondence between the hierarchy model by Hubel and Wiesel, and the neural network of the neocognitron

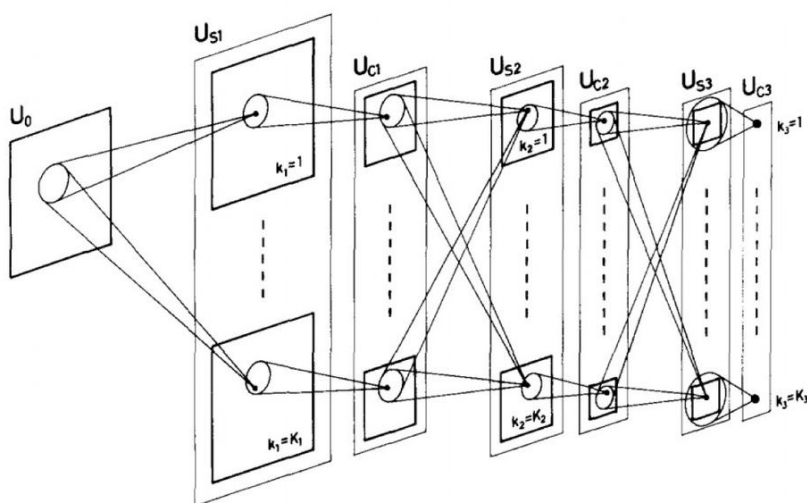


Fig. 2. Schematic diagram illustrating the interconnections between layers in the neocognitron

图 6.5: 福岛邦彦的神经认知机论文

其在实践中难以推广。直到 1990 年，法国计算机科学家 Yann LeCun 在 AT&T 贝尔实验室工作时，将有监督的反向传播（BP）算法应用于福岛邦彦的架构，奠定了现代 CNN 的基础 [12]。LeCun 等人提出的 CNN 结构相比于传统的图像处理算法，最大突破在于避免了繁琐的人工特征提取工作，也就是说，CNN 能够直接从原始图像出发，经过非常少的预处理，就能从图像中找出视觉规律，进而完成识别分类任务。其实这就是端到端（end-end）的含义。LeCun 将自己的研究命名为 LeNet，几经版本更新，最终形成了 LeNet-5。在当时，这一架构主要用于字符识别任务，例如读取邮政编码、手写数字等，并取得了前所未有的成功。在 MNIST 手写数字识别任务中，LeNet-5 的错误率降低到了 5% 左右，这一突破性成果使得 CNN 在学术界和工业界迅速流行起来。尽管 LeCun 在 1990 年代提出了 CNN，但这一技术在当时并未引起广泛关注。CNN 的发展受到了当时计算资源和数据规模的限制。由于当时缺乏大规模训练数据，以及计算能力的不足，CNN 的训练过程极为缓慢，性能也未能达到工业应用水平。因此，在 1990 年代末，支持向量机（SVM）等方法一度成为主流，CNN 研究进入了低潮期。然而，随着大数据和高性能计算的发展，深度学习迎来了爆发期。正如吴恩达所说，深度学习的突破就像发射火箭，需要计算能力作为“发动机”，大数据作为“燃料”。随着 GPU 计算的普及，CNN 训练的速度大幅提升，而海量图像数据的可用性，使得 CNN 能够在复杂任务上展现出超越传统方法的性能。2012 年，AlexNet 在 ImageNet 图像分类竞赛中击败传统计算机视觉方法，将错误率降低到了 15.3%，远超之前的最佳成绩（26.2%）。这标志着 CNN 的真正崛起。此后，VGGNet、GoogLeNet、ResNet 等一系列 CNN 变种相继问世，使得深度学习在计算机视觉领域取得了革命性进展。

6.4.1.3 CNN 的关键结构及其数学原理

LeCun 提出的 LeNet-5 在推进深度学习的发展上起到了重要作用，其结构如图 6.6 所示。LeNet-5 的设计精髓体现在三个核心操作和三个关键概念之中。核心操作包括**卷积（Convolution）**、**池化（Pooling）**和**非线性激活（ReLU）**，而关键概念涉及**局部感受野（Local Receptive Field）**、**权值共享（Weight Sharing）**和**降采样（Subsampling）**。这些计算机制不仅提升了模型的计算效率，同时也增强了泛化能力。尽管 LeNet-5 取得了成功，但其主要应用仍局限于手写数字识别等较简单的任务。在更复杂的计算机视觉任务中，如物体检测、图像分割和自然场景理解，模型需要更强的特征提取能力。为了适应这些需求，研究者们对 LeNet-5 进行了扩展和优化，进一步发展出更深层次的**卷积神经网络（CNN）**结构。卷积神经网络继承了 LeNet-5 的基本思想，并在网络深度、参数优化和计算效率方面做出了诸多改进。其核心架构主要由**卷积层（Convolution Layer）**、**池化层（Pooling Layer）**和**全连接层（Fully Connected Layer）**组成（如图 6.7 所示），每个部分承担特定的计算任务，共同完成对图像的层次化建模。这些改进使得 CNN 能够在计算机视觉领域取得突破性进展，并广泛应用于图像分类、目标检测和语义分割等任务。

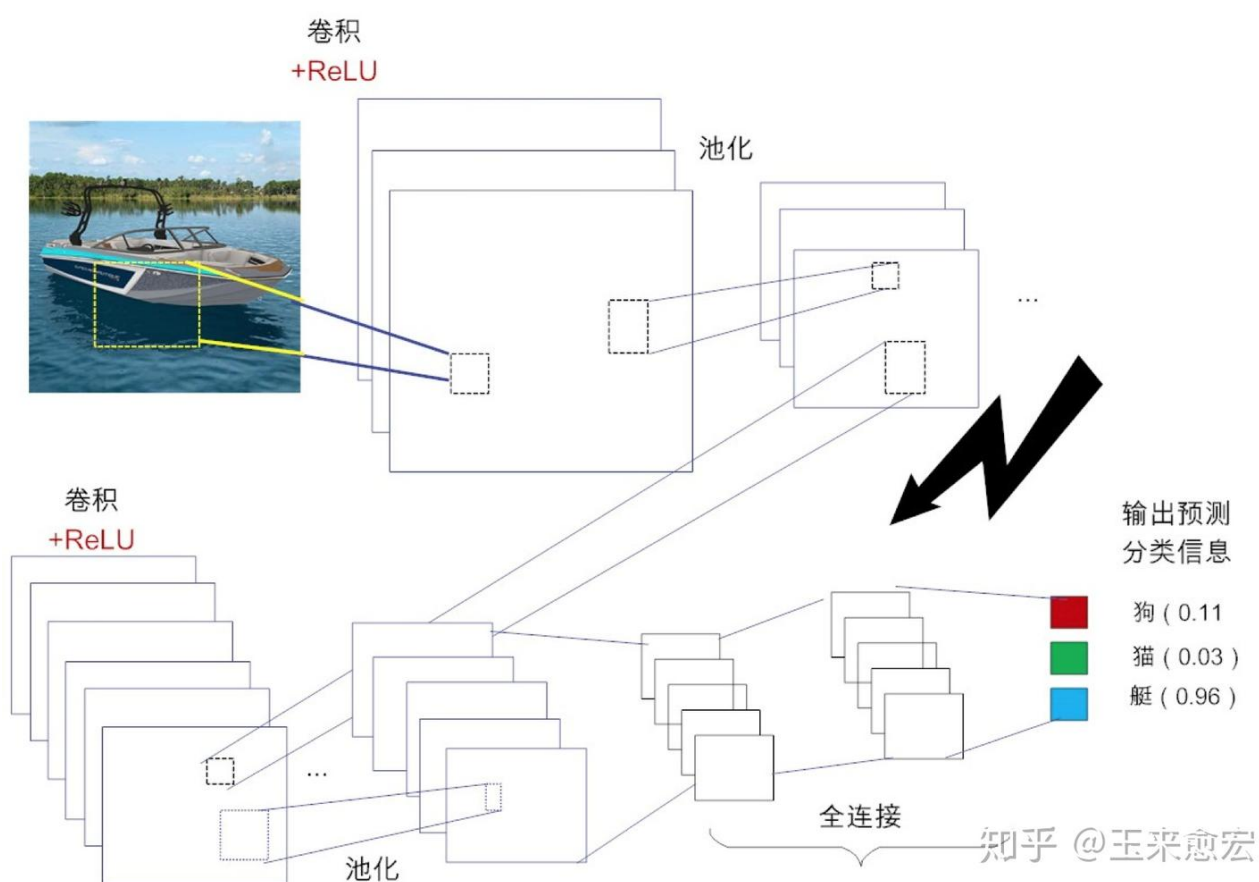


图 6.6: CNN 的基本结构

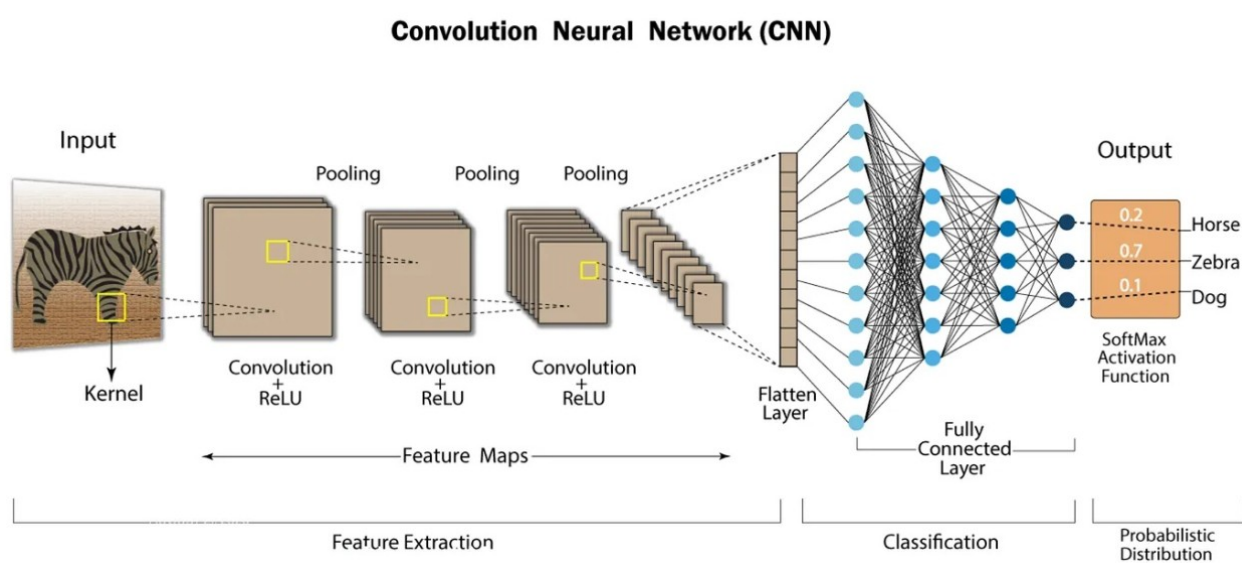


图 6.7: 卷积神经网络 (CNN)

卷积层 (Convolution Layer)：从局部到全局的特征提取

为便于阅读，本节（卷积层 (Convolution Layer)：从局部到全局的特征提取）承接上节（CNN 的关键结构及其数学原理），先交代差异与共同点，再聚焦本节关键问题。

在传统的全连接神经网络（如 MLP）中，每个神经元都与所有输入像素相连，导致参数数量庞大，同时丢失了图像的空间信息，使得模型难以有效提取局部特征。相比之下，卷积神经网络（CNN）采用了一种更加高效的方式。它通过局部感受野（Local Receptive Field）限制神经元的连接范围，并利用权重共享（Weight Sharing）机制在整个图像区域复用卷积核，从而大幅降低计算复杂度，并提高模型的泛化能力。

局部感受野：如何高效提取特征？ 局部感受野（Local Receptive Field）是指，每个神经元只关注输入数据的一小块区域，而非整个输入。这一机制使得 CNN 通过分层学习的方式，从低级的边缘和纹理特征，逐步学习到高级的物体形状和语义信息。数学上，局部感受野通过卷积运算来实现，其计算公式如下：

$$y_{i,j} = \sum_{m,n} W_{m,n} \cdot x_{i+m,j+n} + b,$$

其中， $W_{m,n}$ 代表卷积核（滤波器，Filter）的权重矩阵， $x_{i+m,j+n}$ 代表图像的局部像素值，而 $y_{i,j}$ 代表卷积运算后的输出特征图（Feature Map）。在图像处理中，卷积核可以看作是一个小窗口（如 3×3 或 5×5 ），在整个图像上滑动，对图像的局部区域加权求和，从而获得特征图（Feature Map）。如果某个图像块与此卷积核卷积出的值大，则认为此图像块十分接近于此卷积核。

每个卷积核都相当于一个“特征检测器”，用于识别特定的模式，如边缘、角点、曲线等。不同的卷积核会响应不同的特征，如检测边缘的卷积核可能对亮暗变化敏感，检测形状的卷积核会捕捉到特定的几何结构。图 6.8 中给出 25 种卷积核。当多个卷积层堆叠在一起时，CNN 能够逐步从简单的局部特征构建出万总的全局模式。例如，低层卷积（前几层）学习边缘、角点等基础特征，中间层卷积提取纹理和形状等局部模式，高层卷积（深层）则捕捉物体结构，识别人脸、动物、车辆等语义信息。

权重共享 (Weight Sharing)：减少参数，提高泛化能力 CNN 的另一个关键特性是**权重共享 (Weight Sharing)**，即在整个图像区域内使用相同的卷积核权重，而不是为每个输入像素单独学习一个权重。这一机制基于**平稳假设 (Stationary Assumption)**，即假设图像的局部特征在不同位置具有相似的模式，因此可以共享权重。例如，在人脸识别任务中，无论一只眼睛出现在图像的左侧还是右侧，模型都应该能够识别它的特征，而不需要为不同位置的眼睛学习单独的参数。数学上，权重共享可以表示为

$$f(x) = f(T(x))$$

其中， $T(x)$ 表示对输入数据的平移操作，而 $f(x)$ 代表 CNN 通过卷积操作提取的特征。这一特性不仅减少了参数数量，降低了计算成本，还提高了模型的泛化能力，使得 CNN 对平移、

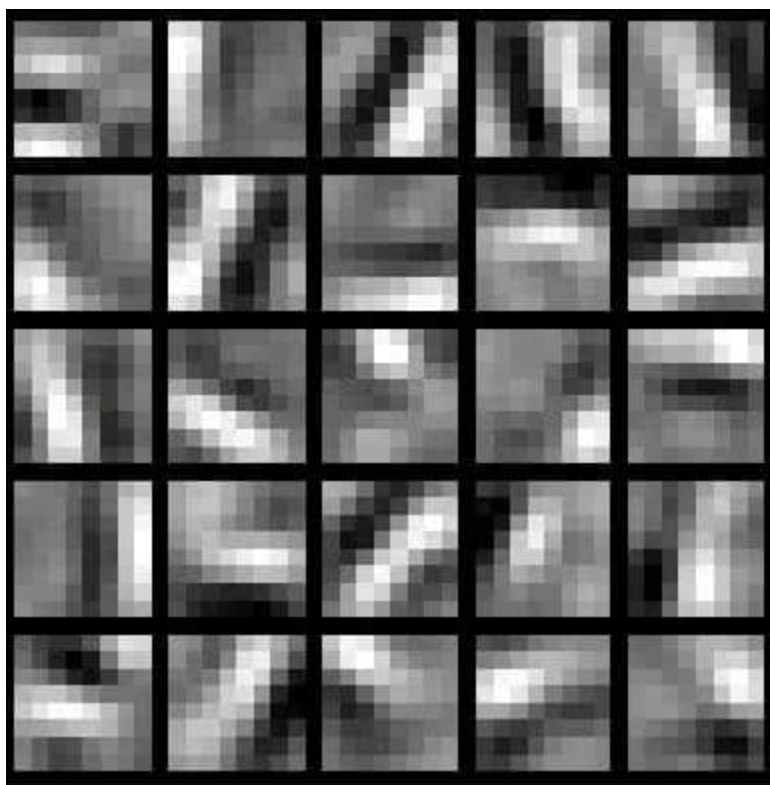


图 6.8: 不同的卷积核

旋转等变换具有鲁棒性。

池化层 (Pooling Layer): 降维与增强不变性

为便于阅读，本节（池化层 (Pooling Layer): 降维与增强不变性）承接上节（卷积层 (Convolution Layer): 从局部到全局的特征提取），先交代差异与共同点，再聚焦本节关键问题。

即使经过卷积操作之后，由于卷积核比较小，输出的图像仍然很大，所以为了降低数据维度，就进行下采样。池化层的主要作用是**对卷积层提取的特征进行降维，降低计算复杂度，避免过拟合，同时增强模型对变形、旋转、缩放等变化的适应能力**。它相当于一种信息筛选机制，帮助 CNN 只关注最重要的特征，避免模型对微小变化过度敏感。池化操作相当于在一个固定窗口（如 2×2 或 3×3 内，选择某个值作为代表，从而减少特征图的尺寸。常见的池化操作包括最大池化 (Max Pooling) 和平均池化 (Average Pooling)。最大池化 (Max Pooling) 的计算方式是从池化窗口内选取最大值

$$y_{ij} = \max_{i,j \in R} x_{i+m,j+n}$$

其中 $x_{i+m,j+n}$ 是输入特征图中的像素值， R 是池化窗口的范围（例如 2×2 或 3×3 ）， y_{ij} 是池化后的输出值。最大池化可以有效提取最显著的特征，提高模型对形状、边缘等局部信息的识别能力，并增强模型对小尺度变换的鲁棒性。

平均池化的计算方式是计算池化窗口内所有元素的均值

$$y_{ij} = \frac{1}{R} \sum_{(m,n) \in R} \max x_{i+m, j+n}$$

其中 $|R|$ 表示池化窗口的大小（即包含的元素个数，如 $2 \times 2 = 4$ ），其余符号与最大池化相同。平均池化可以平滑特征图，使得特征分布更加均匀，适用于需要降噪或提取整体信息的任务。池化的数学思想类似于**统计学中的降维方法**，即通过降采样减少数据维度，同时保留关键信息。例如，在图像识别任务中，池化操作有助于保留轮廓信息，使得模型即使面对噪声或轻微变形，仍然可以识别出目标对象。

全连接层（Fully Connected Layer, FC）：从特征到分类—输出结果

为便于阅读，本节（全连接层（Fully Connected Layer, FC）：从特征到分类—输出结果）承接上节（池化层（Pooling Layer）：降维与增强不变性），先交代差异与共同点，再聚焦本节关键问题。

经过多层卷积和池化后，CNN 提取到的特征仍然是二维的特征图。为了将这些特征用于分类或回归任务，需要通过全连接层将特征映射到具体的类别。全连接层的数学形式为

$$h = W \cdot x + b$$

其中， x 是二维特征图展开后的特征向量， W 是权重矩阵， b 是偏置项， h 是输出。全连接层在 CNN 中扮演着最终决策者的角色。前面的卷积层和池化层负责提取特征，而全连接层则通过学习这些特征之间的权重关系，将其映射到具体的分类结果。例如，在图像分类任务中，最终的输出可能是 10 维（对于 MNIST 数据集）或 1000 维（对于 ImageNet 数据集），其中每个维度对应一个类别。当然，数据只有在经过卷积层和池化层降维过，全连接层才能“跑得动”，不然数据量太大，计算成本高，效率低下。然而，全连接层的参数数量较多，计算量较大，因此许多现代 CNN 结构，如 GoogLeNet 和 ResNet，都会使用全局平均池化（Global Average Pooling, GAP）来替代全连接层，以减少参数量并提高泛化能力。

当然，典型的 CNN 并非只是上面提到的 3 层结构，而是多层结构，例如 LeNet-5 的结构就包括卷积层 → 池化层 → 卷积层 → 池化层 → 卷积层 → 全连接层，如下图所示：

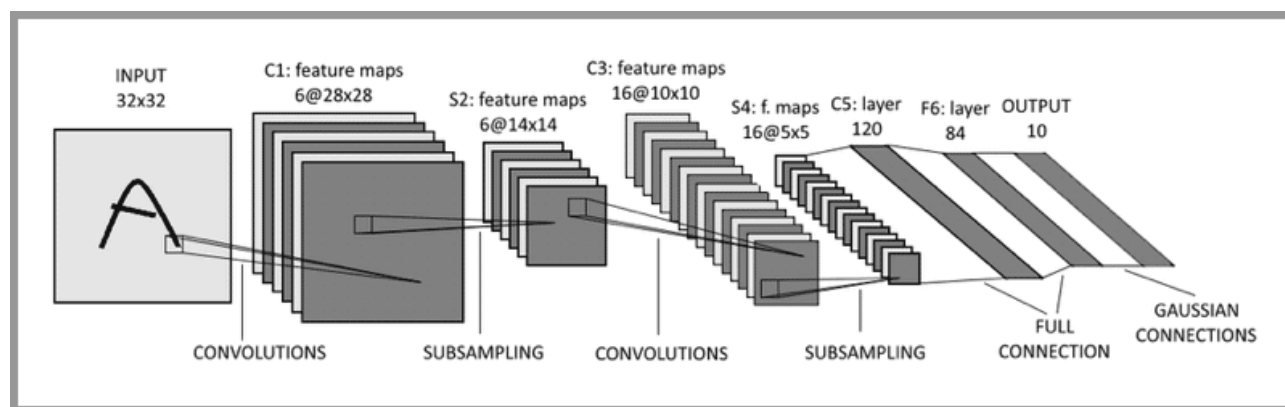


图 6.9: LeNet-5

示例

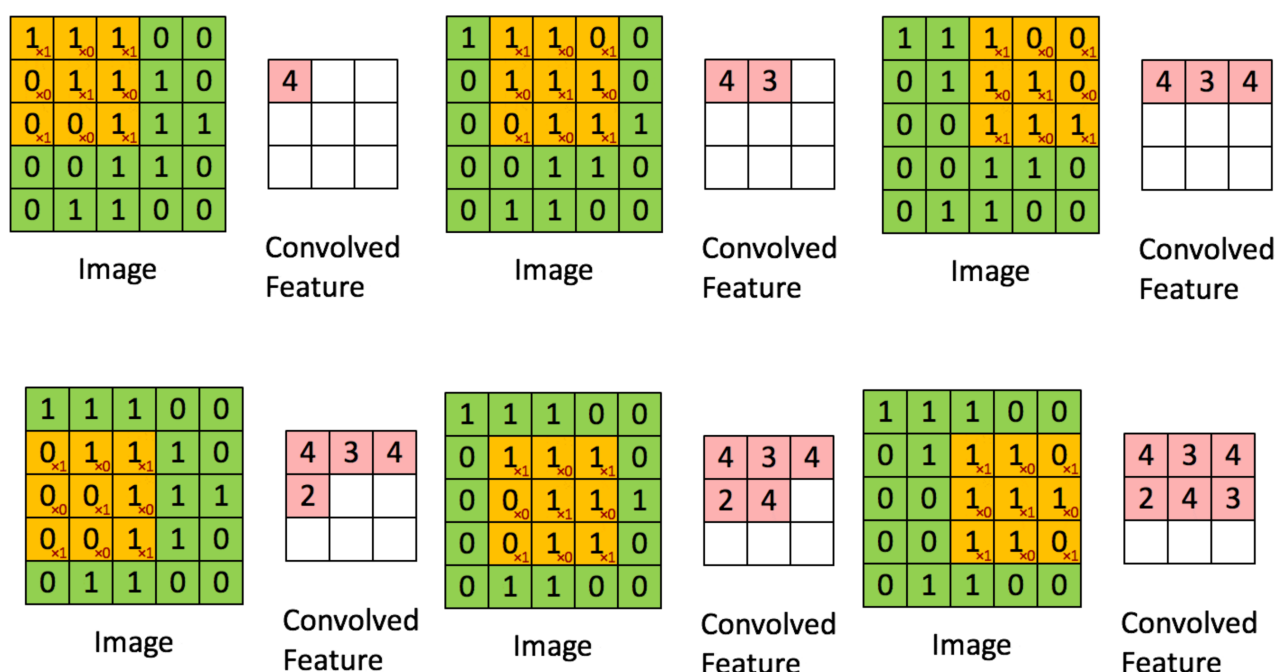
为便于阅读，本节（示例）承接上节（全连接层（Fully Connected Layer, FC）：从特征到分类—输出结果），先交代差异与共同点，再聚焦本节关键问题。

卷积层、池化层、全连接层各司其职。卷积层负责提取图像中的局部特征，池化层对提取的特征进行降维以避免过拟合，全连接层则类似于传统神经网络，用于输出分类结果。

卷积层的运算 卷积层的运算过程如下图，用一个 3×3 卷积核 扫描一张 5×5 像素的图片：

表 6.1: 卷积核

1	0	1
0	1	1
1	0	1



池化层的运算 例子：原始图片是 20×20 的，我们对其进行下采样，采样窗口为 10×10 ，最终将其下采样成为一个 2×2 大小的特征图。

6.4.1.4 CNN 让计算机“看”世界更高效

相较于早期的全连接神经网络（如 MLP），CNN 在计算效率和视觉感知能力上取得了突破性提升。这种优势主要体现在以下几个方面：

- **稀疏连接 (Sparse Connectivity)：降低计算复杂度** 传统神经网络使用全连接结构，每个神经元都与所有输入像素相连，导致参数数量庞大，计算复杂度极高。而 CNN 通过局

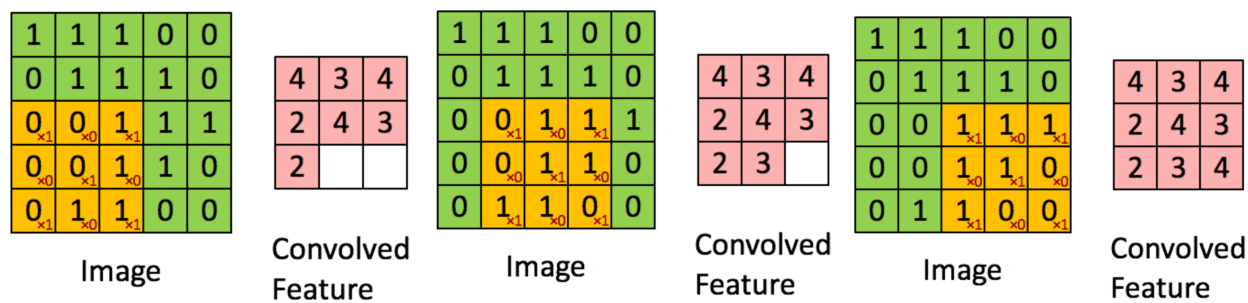


图 6.12: 卷积操作

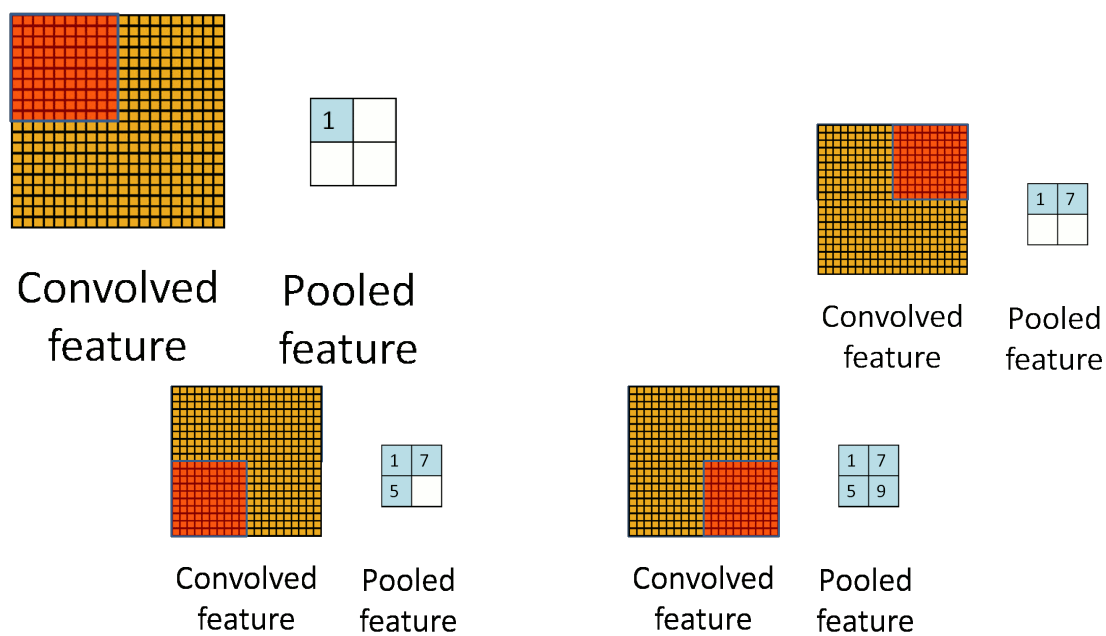


图 6.13: 池化操作

部感受野（Local Receptive Field）让神经元仅与局部区域连接，大幅减少了计算量，同时保留了关键特征信息，使得训练更加高效。

- **权重共享（Weight Sharing）：减少参数，提高泛化能力** 在全连接神经网络中，每个神经元都有独立的权重，而 CNN 采用卷积操作共享权重，即同一个卷积核在整个图像上滑动，使得不同位置的相同特征可以被相同的参数识别。这不仅减少了模型参数，也提高了泛化能力，使 CNN 在较少训练数据的情况下仍能保持良好的性能。
- **平移不变性（Translation Invariance）：提高识别的稳定性** 传统神经网络将图像摊平为一维向量，对图像的位置变化极为敏感。

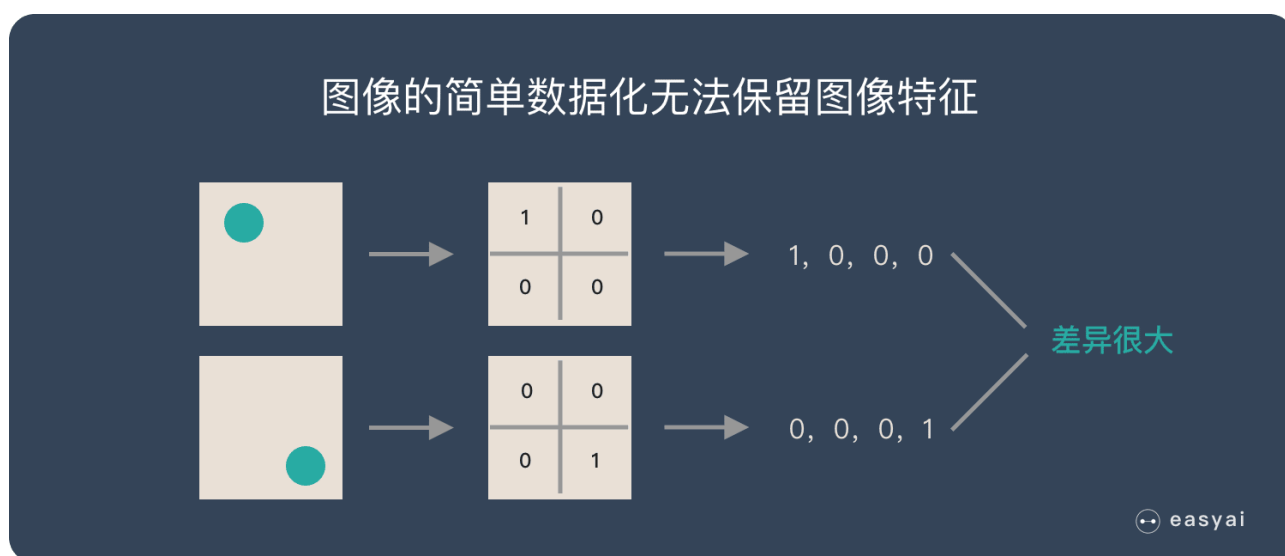


图 6.14: 平移不变性

例如，一张图片中的物体稍微移动，网络可能会输出完全不同的结果。如图 6.14 所示，假设某个位置有绿色圆点则标记为 1，否则标记为 0。在传统神经网络中，如果图像中的绿色圆点位置发生变化，即使图像的内容本质上没有改变，输入数据的数值表示却会发生巨大变化。这导致神经网络在处理相同物体但位置不同的情况下，产生截然不同的计算结果，显然不符合图像处理的要求。CNN 通过滑动卷积核，使得模型能够学习到与位置无关的特征，即无论物体出现在哪个位置，模型都能正确识别。这种特性让 CNN 更加接近人类的视觉系统，能够在平移、旋转、缩放等图像变换下稳定地识别出相同的物体。

- **自动提取特征，避免手工设计** 早期的神经网络或传统机器学习方法需要手工设计特征，如边缘检测、颜色直方图等，而 CNN 采用端到端学习（End-to-End Learning），能够自动从数据中学习多层次的特征，减少了人为干预，提高了特征提取的适应性。
- **降维能力强，有效减少冗余信息** 传统的全连接神经网络在处理大规模图片时，数据量过大，导致训练困难。而 CNN 通过池化（Pooling）操作进行降维，保留主要特征的同时，去除冗余信息，使得模型可以处理更大的数据集，而不会因计算量过大而受限。

卷积神经网络在保证计算效率的同时，能够捕捉高维数据中的深层特征。这种能力让 CNN 成为处理图像和视频数据的首选工具，并推动了深度学习在计算机视觉多个领域的突破性进

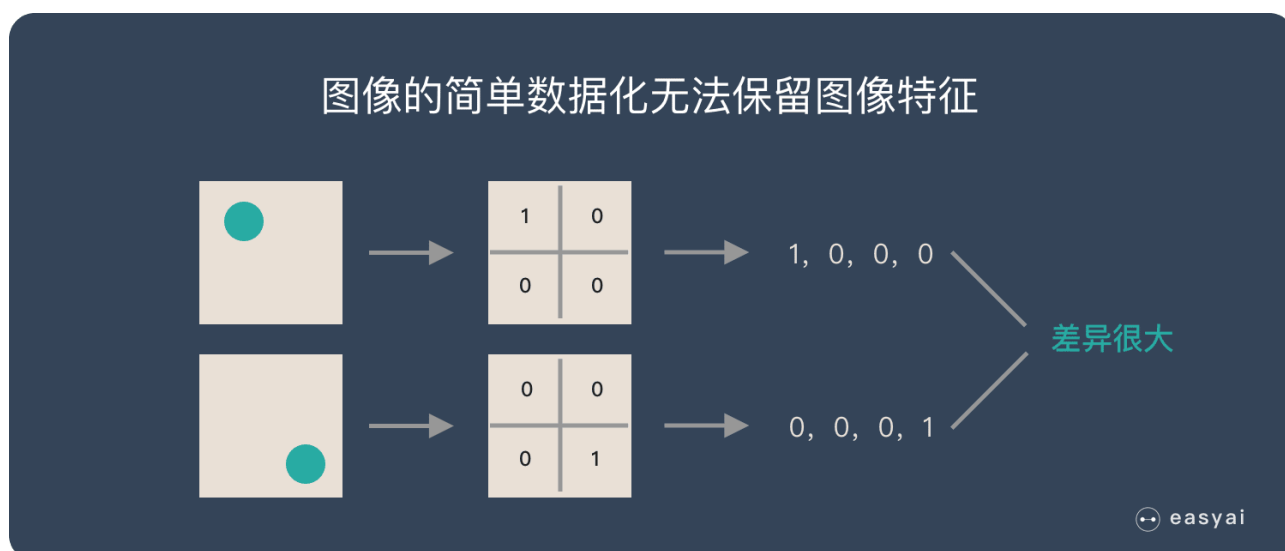


图 6.15: 平移不变性

展：

- **图像分类与检索**：CNN 在 ImageNet 图像分类挑战赛中屡次刷新纪录，奠定了其在计算机视觉中的核心地位。
- **目标检测**：Faster R-CNN、YOLO 等模型结合 CNN，实现了高效的目标识别与定位。
- **医学影像分析**：CNN 广泛应用于医学诊断，如肺部 CT 识别、病理切片分析等，提升了自动化医疗检测的精准度。
- **自动驾驶**：特斯拉等自动驾驶系统利用 CNN 进行视觉感知，识别行人、车辆、交通标志等关键信息。
- **图像生成**：GAN（生成对抗网络）基于 CNN 结构，能够生成高质量的合成图像，实现风格迁移、人脸生成等任务。

CNN 以其强大的特征提取能力，使计算机视觉迈向更高效、更智能的方向，为自动化、医疗、娱乐等多个领域带来了革命性的变革。

6.4.2 循环神经网络（RNN）：如何建模序列数据？

在深度学习的发展历程中，卷积神经网络（CNN）彻底改变了计算机视觉领域，使机器能够高效地识别静态图像。然而，现实世界中的许多数据并非是孤立静态的，而是**具有时间依赖性的序列数据**，如语音、文本、时间序列信号，甚至视频帧的连续变化。人类在理解语言时，也不是逐字逐句地分析，而是结合上下文来理解整体语义。例如，当你听到一句话的前半部分，大脑会自动预测可能的后续内容。相比之下，传统（前馈）神经网络（如 MLP 和 CNN）默认假设所有输入样本是相互独立的，无法捕捉数据点之间的时间关联，因此在面对涉及“记忆”能力的序列任务时表现受限。为了解决这一问题，循环神经网络（Recurrent Neural Network, RNN）应运而生。

6.4.2.1 Why：为什么需要 RNN？

如果说 CNN 关注的是空间结构，那么 RNN 关注的则是时间结构。传统的 CNN 处理图像时，能够提取边缘、纹理等静态特征，但当数据具有动态变化时，仅仅关注单个时刻的信息是不够的。例如：

- 在语音识别中，一个单词的发音会受到前后音节的影响，单独分析某个时刻的音频信号是远远不够的。
- 在机器翻译中，当前单词的含义依赖于前后语境，例如“Le Chat”应翻译为“The Cat”，但如果仅看“Le”本身而不结合上下文，无法正确翻译。
- 在时间序列预测（如股票市场分析）中，未来的市场趋势往往受到过去多个时间点的数据影响，而不能仅看当前股价。

循环神经网络通过引入**循环连接（Recurrent Connection）**，让当前的输出不仅依赖于当前输入，还与之前的隐藏状态相关联，形成了时间上的信息流。RNN 通过**隐藏状态（Hidden State）**来存储先前时间步的信息，并在每个时间步更新这个状态，从而让模型能够“记住”之前的信息。

6.4.2.2 How—RNN 的数学直觉：如何让网络“记住”信息？

RNN 的结构如图 6.16 所示：在 RNN 中，当前时间步 t 的隐藏状态 h_t 由前一时间步的隐

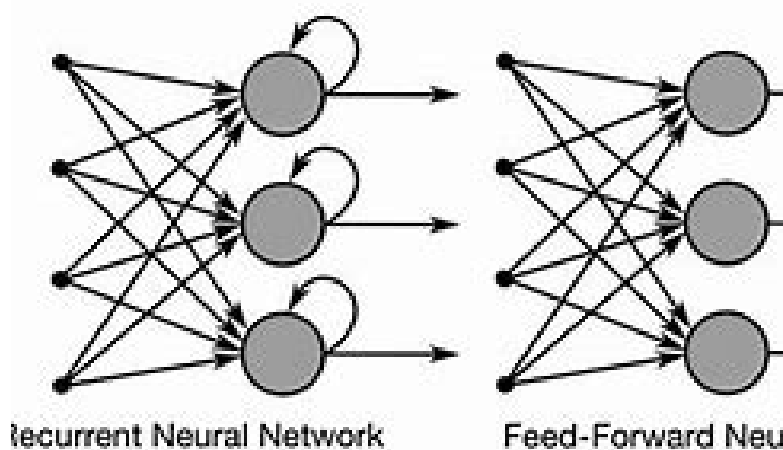


图 6.16: RNN

藏状态 h_{t-1} 和当前输入 x_t 共同决定，递归计算公式如下：

$$h_t = f(W_h h_{t-1} + W_x x_t + b)$$

其中， x_t 是当前时间步的输入， h_t 是当前隐藏状态—用于存储过去的信息， h_{t-1} 是前一时间步的隐藏状态， W_x 和 W_h 是模型学习到的权重矩阵， f 是非线性激活函数（如 $\tanh$ 或 ReLU ）。RNN 通过隐藏状态的循环更新，让当前计算能够结合过去的信息。即使某个输入已经过去，

它的信息仍然能够通过隐藏状态影响后续的计算，从而使 RNN 具备了“记忆”能力。其关键特性包括：

1. **时间步共享参数**：RNN 在不同的时间步使用相同的权重 W_x 和 W_h ，避免了随着时间增加参数数量的爆炸。
2. **状态递归更新**：当前状态 h_t 依赖于前一个时间步的状态 h_{t-1} ，从而在序列中形成信息的时间依赖性。
3. **适用于变长输入**：与传统神经网络不同，RNN 可以接受任意长度的输入序列，使其能够处理可变长的文本、语音等任务。

RNN 强大的序列建模能力使其能够在时间维度上积累信息，从而发现时间序列数据中的模式和结构，进行预测、生成文本，甚至实现智能对话。相比之下，MLP、CNN 等前馈神经网络更像是“只依赖当前感知”的人，每次只能处理当前的数据，而 RNN 更像是“有长期记忆”的人，它会参考过去的信息，并用它们来影响当前的决策。这种能力使得 RNN 成为处理文本、语音、时间序列数据、金融预测等动态信息的重要模型之一。

6.4.2.3 RNN 的局限性：记忆并不总是可靠

“天下没有免费的午餐”。尽管循环连接赋予了 RNN 一定的记忆能力，但也带来了一些局限性，其中最核心的问题就是长程依赖。假设我们正在阅读一篇文章，某个关键名词出现在第一段，我们希望在第十段的结论部分仍能准确引用这个名词。但由于信息在网络中传播的时间过长，前面学到的内容可能早已被“遗忘”。这一问题的根源在于 RNN 训练中的反向传播算法，长时间步传播中会引起了梯度消失或梯度爆炸。在训练 RNN 时，通常使用反向传播算法来计算梯度并更新参数：

$$\frac{\partial L}{\partial W_h} = \prod_{t=1}^T \frac{\partial h_t}{\partial h_{t-1}} \cdot \frac{\partial L}{\partial h_T}$$

在 RNN 中，隐藏状态 h_t 依赖于前一时间步的隐藏状态 h_{t-1} 进行更新，这种依赖关系在多个时间步中不断传播。如果梯度的绝对值小于 1，随着时间步数 T 的增加，梯度会指数级衰减趋近于 0，梯度更新几乎停滞，前面时间步的影响越来越小，无法有效影响后续计算。这使得 RNN 难以学习长程依赖关系，无法记住较远的历史信息。例如在文本任务中，网络可能无法有效地连接前后远距离的词语关系。相反，当梯度的绝对值大于 1 时，梯度会在反向传播过程中指数级放大，导致梯度爆炸，进而使得参数更新幅度过大，造成训练不稳定，甚至无法收敛。虽然梯度裁剪（Gradient Clipping）等方法可以缓解梯度爆炸问题，但并不能完全解决 RNN 在长序列任务中的表现瓶颈。

6.4.2.4 LSTM 与 GRU：赋予 RNN 更强的记忆能力

为了解决 RNN 在长期依赖问题上的局限性，研究者提出了长短时记忆网络（Long Short-Term Memory, LSTM）和门控循环单元（Gated Recurrent Unit, GRU）。它们在 RNN 的基础上增加了“门控机制”，使网络能够选择性地保留重要信息，同时抑制——“忘记”——不必要的信息，从

而更有效地捕捉长期依赖关系。选择性记住或遗忘信息，使得模型能够长时间

LSTM：引入记忆单元，保留长期信息 LSTM 通过输入门（Input Gate）、遗忘门（Forget Gate）和输出门（Output Gate）控制信息流，使得重要的信息能够在更长时间内保留，而不重要的信息则被遗忘。LSTM 的核心计算如下：

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) = \sigma(W_f h_{t-1} + U_f x_t + b_f) \quad (\text{遗忘门}) \quad i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) = \sigma(W_i h_{t-1} + U_i x_t + b_i)$$

LSTM 使得较远时间步的信息仍然能够有效影响当前决策，因此在自然语言处理、语音识别等任务中表现优异。

GRU：简化 LSTM 结构，提高计算效率 GRU 使用重置门（Reset Gate）和更新门（Update Gate）来控制信息的更新，相比 LSTM 具有更少的参数，因而计算效率更高：

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad (\text{更新门}) \quad r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

其中， z_t 负责决定保留多少过去信息， r_t 控制遗忘多少历史信息。GRU 通过减少门控结构，使计算更高效，同时避免梯度消失。GRU 在许多应用中表现与 LSTM 相当，但计算成本更低，因此在部分场景下更受青睐。

6.4.2.5 RNN 在现实中的应用

RNN 及其变种擅长处理时间序列任务，在多个领域取得了广泛应用：

- **自然语言处理（NLP）**：如机器翻译（如 Google 翻译）、文本摘要、聊天机器人、语音转文字（ASR）。
- **时间序列预测**，如股票价格预测、天气预报、金融数据建模。
- **语音识别**：如苹果 Siri、谷歌语音助手等语音交互系统。
- **视频分析**：用于动作识别、视频字幕生成等任务。

6.4.2.6 RNN 与 CNN：互补还是竞争？

CNN 适用于提取局部空间特征，而 RNN 则擅长建模序列数据，尤其适合捕捉时间依赖性。因此，许多任务会结合 CNN 和 RNN 的优势。例如，在视频分析中，CNN 负责提取图像帧中的空间信息，而 RNN 则用于捕捉时间上的动态变化。此外，Transformer 模型（如 BERT、GPT）的崛起也为序列建模提供了新思路，进一步推动了 NLP 领域的发展。

6.4.2.7 总结

RNN 通过引入隐藏状态和循环连接，实现了对序列数据的建模，使得深度学习能够处理时间相关性强的任务。然而，它的长期依赖问题限制了其应用。在此基础上，LSTM 和 GRU

通过门控机制改进了记忆能力，使其能够在更长的时间范围内保留重要信息。这些技术共同推动了自然语言处理、语音识别、时间序列预测等任务的发展，使得机器可以更好地理解和处理动态数据。

6.4.3 Transformer：为什么能取代 RNN？

在处理序列数据时，RNN 和其变种（如 LSTM 和 GRU）凭借着递归的结构有效地捕捉了时间上的依赖关系，曾是最主流的方法。然而，随着任务复杂性的增加，RNN 的局限性逐渐显现出，主要体现在**长程依赖的捕捉和计算效率的瓶颈**两个方面。从数学上看，RNN 本质上是一种递归迭代计算，每个时刻的状态都依赖于前一时刻的状态。这种设计导致两个问题：

- **梯度消失 (Vanishing Gradient)**：远距离信息随着链式求导而指数级地衰减，导致难以捕捉长序列中的远程依赖。
- **难以并行计算**：由于依赖前一个状态，计算只能逐步推进，无法高效利用现代硬件的并行能力。

通俗一点讲，RNN 像一位“健忘”的故事讲述者，讲着讲着就容易忘记更早的内容。同时，更糟糕的是，因为需要逐步计算隐藏状态从而难以并行加速，RNN 还面临着计算效率的瓶颈。Transformer 模型正是为了解决这些数学和计算瓶颈而诞生。它凭借创新的**自注意力 (Self-Attention) 机制**，彻底颠覆了 RNN 逐步推进的计算范式，使模型在任意位置都能同时关注所有其他位置的信息，避免了链式计算的梯度衰减问题。此外，通过引入**并行计算**，Transformer 能高效地进行批量处理，大幅提升训练速度。

Transformer 的出现，彻底重塑了自然语言处理 (NLP) 和其他序列建模任务的格局，成为当前序列数据处理的主流方法。Transformer 是一种基于 Attention 机制的网络架构，其全连接特点使其有别于 CNN、RNN 等网络，可以直接捕获序列数据的长距离依赖。在 Google 论文 [22] “Attention Is All You Need”中，其结构如图 6.17 所示。

自注意力机制 (Self-Attention)：如何打破时序束缚？

想象一下，当我们在阅读一段文字时，大脑不会逐字逐句地分析，而是快速地捕捉上下文中的关键信息，借此来推断某个词的具体含义，而自动忽略掉其它无关的信息。这种“快速捕捉关键关系”的直觉，正是 Transformer 自注意力机制背后的灵感来源。从数学的角度来看，Transformer 的自注意力机制本质上是对序列中元素之间的关系进行“注意力权重”的分配，以实现信息的高效流动，然后根据这些权重，动态调整每个元素对整体语义的贡献。具体地，自注意力机制定义为：

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

这里的三个关键矩阵分别是查询 (Query, Q)、键 (Key, K) 和值 (Value, V)， $\sqrt{d_k}$ 为缩放因子。通过计算查询 (Q) 与键 (K) 的内积，我们得到一个注意力矩阵，刻画了每个单词与其他单词的相关性强弱。而除以 $\sqrt{d_k}$ 则是从数值稳定性出发的巧妙设计，避免了点积过大造成

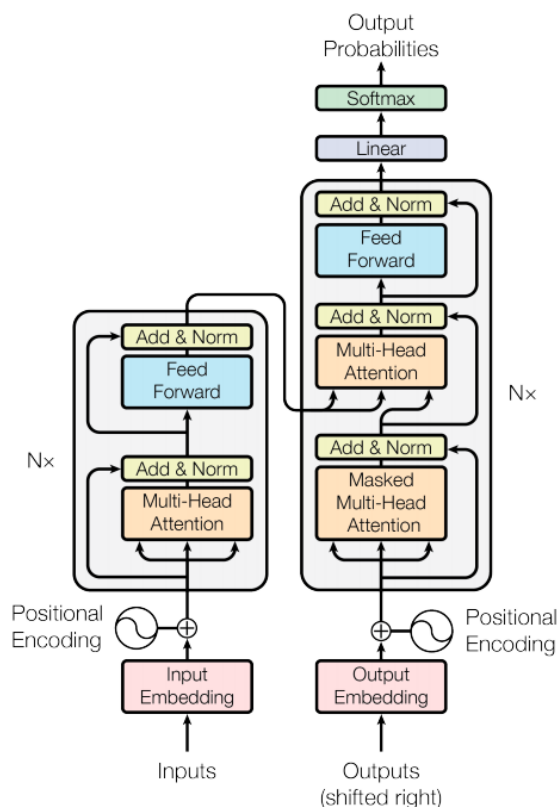


图 6.17: Transformer 架构

的梯度爆炸问题。这种简洁而巧妙的数学设计，使得 Transformer 能并行地、直接地计算序列中所有单词之间的相关性，而无需像 RNN 一样逐步推进。自注意力机制允许 Transformer 直接对序列中任意位置的元素进行交互，无论这些元素在序列中的距离多远，都能迅速建立起关联。这种机制突破了 RNN 难以建模长程依赖的瓶颈。举个例子，在理解句子“The money in the bank was taken away（银行里的钱被取走了）”，Transformer 不会孤立地看待“bank”，而是同时关注整个句子中其他所有单词之间的联系。通过“money”和“taken away”等关键词，它能迅速推断这里的“bank”指的是“银行”，而非“河岸”。这一点正体现了自注意力机制在语义建模上的优势——能够瞬间捕捉远距离的语义依赖关系。这种高效且并行的语义关联捕捉，正是自注意力机制带来的显著优势。

长程依赖的建模：不再受限于时间的局限

传统的 RNN 的数学结构本质上是递归式的，当前时刻的隐藏状态依赖于前一时刻状态的计算：

$$h_t = f(h_{t-1}, x_t) = f(W h_{t-1} + U x_t + b)$$

当进行梯度反向传播时，每一步梯度都会涉及连乘一个雅可比矩阵 W 的过程：

$$\frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}} = \prod_{i=k+1}^t W^T \text{diag}(f'(h_{i-1}))$$

这种链式结构导致远距离的信息贡献随着序列长度增加呈指数衰减，从而出现“梯度消失”的现象——远距离信息难以被有效记忆。相比之下，Transformer 通过自注意力机制，能够直接建立序列任意两个位置之间的关系，而非逐步递归，避免了链式求导导致的梯度衰减问题。比如，在机器翻译任务中，当面对存在歧义的句子如“I saw the man with a telescope”时，Transformer 无需逐词递推，而是同时评估句子中所有词语的相互关联性，从整体上精确判断出句子的意思究竟是“我用望远镜看到那个男人”还是“我看到那个拿着望远镜的男人”。也正因为如此，Transformer 在长程依赖关系的建模中表现出强大的优势。

并行计算：从“乡间小路”到高速公路

RNN 由于其顺序性质，每次计算都依赖于上一个时间步的结果。这种递归形式难以并行化，每个时刻的计算必须等待前一个时刻完成，这在大规模数据训练中效率极低。而 Transformer 通过矩阵运算一次性处理输入序列中所有位置之间的关系，使得整个序列能够并行计算。数学上看，这种并行的相关性计算得益于注意力矩阵 $\frac{QK^T}{\sqrt{d_k}}$ 。特别是除以 $\sqrt{d_k}$ 的操作，防止内积过大导致梯度爆炸，在数值稳定性和梯度传播上都有明显的优势。这种数学巧思不仅确保了 Transformer 在长序列上的高效建模，也充分利用了现代硬件（如 GPU）的并行计算优势。在处理大规模数据时，Transformer 的表现远超 RNN。可以形象地说，RNN 的顺序计算如同乡间小道上步行的旅人，必须一步一步慢慢走；而 Transformer 则好比“多车道高速公路”同时通行大量车辆，能够同时高速前进，效率和速度有质的飞跃。

位置编码（Positional Encoding）：如何让“非顺序”架构理解顺序？

尽管自注意力机制让 Transformer 能够同时关注序列中所有元素的关联性，但这种机制本身并不自然具备对序列顺序的感知能力。换句话说，Transformer 并不自然知道数据的时间或空间顺序。但在处理文本时，句子的顺序对于含义至关重要。为了赋予模型对序列顺序的感知能力，Transformer 引入了位置编码（Positional Encoding），为序列中每个元素引入一个额外的位置特征，让模型明确感知到元素在序列中的位置信息。Transformer 使用一组特殊设计的正弦和余弦函数，将位置信息编码成与输入数据维度相同的向量，并直接加到输入向量中。具体而言，第 pos 个位置的编码可以表示为：

$$\text{PE}_{(\text{pos}, 2i)} = \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right), \quad \text{PE}_{(\text{pos}, 2i+1)} = \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right),$$

其中， pos 表示序列中元素的位置（从 0 或 1 开始）， i 是向量维度的索引（从 0 到 $d_{\text{model}}/2 - 1$ ）， d_{model} 是输入特征向量的维度。为什么选择正弦和余弦函数来编码位置？这背后的数学思维非常巧妙：

- **周期性和平滑性：**正弦和余弦函数都是周期函数，位置编码随着位置的增加呈现稳定的周期性变化，这使得模型可以泛化到比训练时更长的序列，而不会失去位置信息的区分能力。

- **相对位置的线性组合**：位置编码的周期性设计使得不同位置之间的相对关系可以通过向量内积、加减等运算体现出来。也就是说，Transformer 可以通过简单的向量计算，捕捉元素间的相对位置关系。
- **可扩展性与稳定性**：通过使用不同的频率（由 $10000 \frac{2i}{d_{\text{model}}}$ 控制），位置编码向量在不同维度上有着不同的变化速率，这种分层的频率设置帮助模型捕捉不同尺度下的位置关系，使 Transformer 更灵活地理解和建模序列结构。

这种设计不仅能够在数值上区分不同位置，同时也能很好地泛化到任意长度的序列，因为正弦余弦函数在不同周期上的变化模式是稳定且可预测的。通过位置编码，Transformer 在没有显式顺序的自注意力机制中，巧妙地引入了顺序信息。这种数学设计使模型在计算过程中不仅考虑元素之间的相对关系，还能够感知和保留序列顺序的重要信息，从而准确地对文本、语音等序列数据建模。Transformer 通过自注意力机制、并行计算、长程依赖建模和位置编码等一系列数学和计算上的创新克服了 RNN 在处理长序列任务时的局限性，不仅有效解决了长程依赖导致的梯度消失问题，而且极大提升了模型的并行计算能力和训练效率。这种新范式彻底重塑了自然语言处理（NLP）领域，并推动了 GPT、BERT 等大规模语言模型的出现与快速发展，使 Transformer 成为机器翻译、文本生成和情感分析等任务的核心架构，逐步取代 RNN，成为当前序列建模的主流选择。

6.4.3.1 小结：神经网络的不同架构与数学思维

深度学习架构的演进，本质上体现了不同数学建模思维的碰撞与融合。无论是 CNN、RNN 还是 Transformer，本质上都是在探索更有效、更高效的函数逼近方式，使得神经网络能够以适应不同的数据类型和任务需求。图 6.18 是 RNN、CNN、自注意力机制架构的对比图。从数

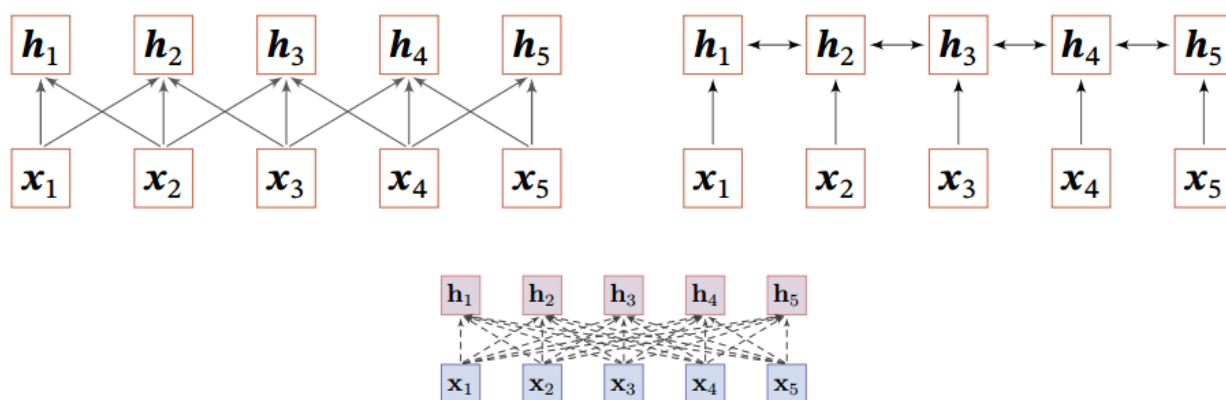


图 6.18: RNN、CNN、自注意力机制架构对比

学角度看，这些架构的区别在于它们如何处理输入数据的依赖关系：

- CNN 是一种局部依赖结构：

$$h_t = f(x_{t-1}, x_t, x_{t+1})$$

CNN 通过局部计算和共享权重，减少了计算复杂度，并利用层次化特征学习来逼近高

维空间中的映射关系。

- RNN 本是递归依赖结构：

$$h_t = f(h_{t-1}, x_t)$$

RNN 通过递归结构建模序列信息，使得时间序列数据可以在一个高维状态空间中展开，形成动态映射，从而使神经网络能够捕获数据的时间相关性，但其链式计算也导致了梯度消失与计算效率瓶颈。

- Transformer 的 Self-Attention 机制是一种动态的全局依赖模型：

$$h_t = \sum_i \lambda_{i,t} x_i$$

这里的动态体现在 $\lambda_{i,t}$ 的计算上

$$[\lambda_{1,t}, \dots, \lambda_{n,t}] = \text{softmax} \left(\frac{q_t K^T}{\sqrt{d_k}} \right)$$

这里，每个输出位置 $h_t, t = 1, \dots$ ，是输入序列中所有元素的动态加权组合，其权重 $\lambda_{i,t}$ 取决于输入之间的关系。如果 $\lambda_{i,t}$ 与 t 无关，就退化成普通的全连接网络。Transformer 通过矩阵运算并行建模全局依赖关系，利用注意力机制在信息流动上引入可变权重，从而增强数据的表达能力。这种全局、动态的关系捕捉方式，让 Transformer 在处理长程依赖问题时显著优于 RNN，同时通过矩阵并行计算克服了 RNN 的效率瓶颈。

从更广泛的视角来看，深度学习架构的发展，是基于数据特性、计算效率和数学优化的一系列选择。不同架构的演变，正是人类理解世界方式的映射——不仅要学会观察细节，还要能够记住过去，更要能在海量信息中快速抓住核心。与传统的还原论（Reductionism）方法论“1+1=2”不同，深度学习背后的方法论本质上属于复杂性科学的范畴。在深度学习中，我们并非逐个分解问题再逐个求解，而是通过网络整体来学习数据中的深层次映射关系，从而更加高效地逼近复杂世界的真实规律。神经网络架构的多样性，让深度学习不仅仅是一个简单的计算模型，而是一种能够适应不同认知需求的数学工具。在所有人工智能技术中，深度学习无疑是最具变革力的一种。它已经悄悄走出实验室，走进了每一个人的日常生活。今天，很多你以为“顺理成章”的体验背后，其实都有深度学习在默默运作。

6.4.3.1.1 视觉与语言：两大主战场 深度学习最早在两个领域展现出压倒性的优势：一是**计算机视觉**，二是**自然语言处理**。在计算机视觉方面，深度学习让机器学会了“看”。从手机刷脸解锁到自动驾驶识别道路和行人；从 B 超、CT 影像中识别病灶，到让医生辅助诊断更高效准确；从安防监控到艺术风格迁移……CNN 等视觉模型已经深入到医疗、交通、安防、娱乐等多个行业。**自然语言处理**是近年深度学习最为耀眼的应用领域之一，尤其随着大语言模型的兴起，人机交互的方式正在被彻底改写。从翻译软件精准地将“Bonjour”译为“你好”，到智能客服能理解你话语中的意图，背后都是深度学习模型对语言的深度建模能力。而像 ChatGPT、DeepSeek 这样的大语言模型，更是能更是能写稿、答题、陪聊，展现出前所未有的语言理解与生成能力。

6.4.3.1.2 “无感嵌入”：深度学习已成为生活背景的运转引擎 深度学习已经悄无声息地嵌入了我们生活的方方面面。那些看似“顺理成章”的日常体验，其实背后都有深度学习在默默运作：当你刷脸解锁手机，一张自拍照会被送入训练好的图像识别模型进行比对；你打开抖音、B 站，平台推荐的视频之所以“懂你”，是因为神经网络正在学习你的每一次点击偏好；你用 Midjourney 生成图像，或与 ChatGPT 写稿、提问，其实都在调用庞大的语言或图像生成模型，在“想象”你输入背后的世界；你在刷信用卡、申请贷款、配置理财产品的背后时，银行系统正通过深度模型评估你的风险等级、识别潜在欺诈——做着不声不响却关键的一票。当你坐进一辆自动驾驶汽车时，深度学习模型就在悄悄接管驾驶任务。它实时处理来自摄像头、雷达和传感器的大量数据，识别红绿灯、行人、障碍物，预测周围车辆的意图——每一次转弯、每一次刹车、每一次变道背后，都有一个深度神经网络在“驾驶座”上做出判断。但今天的智能驾驶早已不止“识别与避障”。比如华为的智能驾驶系统，已经能够在复杂路况中灵活“加塞”：在拥堵路段准确判断时机、协调刹车与加速动作，从容插入车流，就像一位经验老道的老司机。过去我们以为自动驾驶只是程序执行，而现在，它的表现开始令人怀疑：它是否真的在“思考”？深度学习让汽车不只是看清世界，而是学会了如何在变化中做决策、在规则中找弹性，这标志着从“感知”迈向“认知”的进化。

6.4.3.1.3 医疗健康：AI 成为“第二双眼” 在医疗领域，深度学习正悄然成为医生的得力助手。当你生病检查时，一张 CT 或 MRI 影像会被送入训练良好的模型，AI 能在数秒内标记出可疑区域——这些病灶有时连经验丰富的医生也难以察觉。相比之下，AI 不会疲劳、不受主观偏差影响，它通过海量图像学习而来的“直觉”，正在显著提升诊断的效率与准确率。不仅如此，AI 还能在电子病历中识别潜在规律，辅助判断疾病风险与个性化治疗方案。在新药研发中，深度学习也加快了分子结构的筛选与模拟，大幅缩短研发周期。

6.4.3.1.4 从生活到科学：深度学习成为科研的“加速器” 深度学习不仅在日常生活中“润物细无声”，也正以“AI for Science”的姿态，进入科学探索的核心场域。从处理数据到参与建模，从模式识别到提出假设，AI 正在成为科研过程中的“第二大脑”。在材料科学和量子物理中，深度神经网络已被用于模拟复杂系统的行为，如多体波函数、电荷分布等，绕开传统数值解法的计算瓶颈；在高能物理实验中，它能从亿万次粒子碰撞中高效筛选出极其稀有的信号事件；在脑科学中，深度网络正被用于重建神经元连接图谱，帮助科学家洞察大脑的运行机制；在新药发现中，生成模型甚至可以“想象”从未存在过的分子结构，极大地提升了分子筛选和合成路径设计的效率。令人瞩目的例子，莫过于 AlphaFold 的成功。它利用深度神经网络预测了成千上万种蛋白质的三维折叠结构，攻克了生物学界几十年来未解的“折叠难题”。2024 年诺贝尔生理学或医学奖也将这一成就纳入褒奖，正式肯定了 AI 在结构生物学中的革命性贡献。这意味着，深度学习正从科学家的“数据助手”升级为“提出假设、参与建模”的科研合作者，正在悄然重塑科学研究的范式。它使人工智能不再只是理解世界的工具，更开始解释世界、预测世界，乃至参与知识的创造。深度学习，正在引领一场新的科研方法论革命——一场由数据驱动、由模型引领、由智能加速的科学革命。

6.4.3.1.5 未来：理解世界，更是创造世界 从视觉到语言，从医疗到科研，深度学习正悄然成为我们生活中的“通才助手”。它不仅分担大量认知与决策任务，更让机器从“听话的执行者”进化为真正的“协作者”，具备理解意图、举一反三的能力。人工智能的未来，已不是科幻，而是正在发生的日常。在你未曾注意的每一秒，深度学习正以看得见和看不见的方式，悄悄提升着我们的生活效率、信息体验与决策品质。更重要的是，深度学习已经不再只是技术工具，而正成为这个时代的“数字基础设施”——就像水、电、网络一样，支撑着我们的工作与生活，重塑了内容分发的方式，重新定义了人机交互的边界，也正在重新构建许多传统行业的运作逻辑。所以说，深度学习不仅改变了 AI 技术的演化轨迹，更开启了一场理解世界、解释世界，乃至创造世界的智能革命。在这场变革中，我们不只是旁观者，而是真正的参与者与受益者。深度学习让机器变得越来越聪明，甚至在某些方面超过了人类。但别担心，它们不会篡位，只会让我们的生活更便利。未来，这样的智能体将如影随形，成为我们工作和生活的得力助手，很可能会帮助你成为“升级版”的自己。

深度学习让人工智能进入了“会看、会听、会说、能生成”的新时代，它像一台“经验驱动的智能引擎”，正在推动世界发生深刻变化。但这台引擎再强大，也不是没有极限。正如牛顿力学解释不了光速之上，深度学习也不是无所不能。它强在“经验”，但弱在“理解”；善于“拟合”，却不擅“推理”。当我们沉醉于它带来的各种惊艳时，也必须冷静地看到它面临的几大核心挑战。

1. 数据饥渴：没有数据，它寸步难行 深度学习的本质是一种“统计学习”，需要依靠大量训练样本来“看例子、学规律”。模型的性能好坏，很大程度上取决于数据是否充足、覆盖是否广泛。这就导致深度学习对数据的依赖性极强。训练一个模型往往需要数万张图像、数百万条文本或语音数据作为“食粮”。一旦进入“数据稀缺”场景，比如罕见病、特殊气候、低资源语言等小样本实验场景，模型的性能便会显著下降，甚至完全失效。可以说，深度学习就像一个“见多识广但不太会举一反三”的学生，只能在自己见过的样本范围内展现聪明才智。它擅长归纳，但不擅类比；善于模仿，却难以通解。这种“高性能但低泛化”的特性，决定了深度学习的边界，也启发了诸如小样本学习、迁移学习等新兴方法的研究。

2. 黑盒本质：模型的决策为何“不可解释”？ 深度学习的强大之处，恰恰也是它难以理解的原因。在传统机器学习中，研究者往往手工设计特征，有明确的输入-处理-输出逻辑链条。但深度学习则完全不同——它通过多层非线性网络自行“学习”特征，在隐藏层中构造了一个高度抽象、难以人类解读的“特征表示空间”，中间特征存在于一个我们无法直接感知的“机器空间”。这就像是让模型在我们看不到的“维度”中进行决策。你可以看到它的输出非常准确，却很难追溯它的思考路径。正如有人调侃：“深度学习是一门黑盒艺术——只要你愿意砸算力，它就能给你一个看起来很棒的答案；但你永远搞不清，它到底是怎么想出来的。”

从知识结构上看，深度学习的知识象限属于“可统计、不可推理”的范畴。它善于从海量数据中提取统计相关性，却难以进行具有逻辑结构的因果推理。深度学习的“知识”是模式间的相关性，而不是逻辑因果。也就是说，它可以判断“A 常伴随 B”，但很难理解“为什么 A 会导致 B”。更哲学性地说，深度学习面对的是一种“剪不断、理还乱”的非线性状态，它的模型参数

与中间表示早已超出人类直接理解的范畴。人类不再是设计者，而变成了调参者，这正是深度学习时代的新现实。

3. 算力成本：背后是“能耗与资源”的账本 要训练一个大模型，你可能需要数万张 GPU 卡，数周时间，数百万美元的成本。这已经不再是普通研究者“玩得起”的领域。不仅如此，高算力也意味着高能耗。一项研究表明，训练一个 GPT-3 规模的大模型，其碳排放量相当于一架飞机飞越大西洋数十次。这让我们不得不思考：为了“智能”，我们是否付出了过高的能源代价？当然，新的算法（如模型压缩、蒸馏、剪枝）正试图缓解这一问题，但“效率与能力”的张力，依然是深度学习未来发展的核心议题之一。

4. 鲁棒性脆弱：看似聪明，实则“胆小” 你只需要在图像中加几个像素级的干扰，人眼完全察觉不到，AI 却可能立刻认错物体；一句话中的词序轻轻调整，大模型就可能产生歧义，甚至胡言乱语。这就是深度学习的“鲁棒性问题”——模型对输入的微小扰动极其敏感。这种脆弱性让我们质疑：一个需要在真实世界中部署的“智能系统”，是否真的可靠？鲁棒性差的本质原因是：它学到的并不是“真正的规则”，而是训练数据里的“表面模式”。换句话说，它不是真的理解了猫，只是学会了“猫的照片在训练集中长什么样”。

5. 理解力有限：它“知道很多”，但“懂得很少” 一个大模型可以流利对话、创作文章、答数学题，但它真的“理解”语言吗？我们常说，AI 是在“理解”用户输入、“理解”图片内容，但这其实是一种比喻式表达。模型的本质，是在大量样本中“拟合模式”，而非真正地建构“知识图景”。它知道“水能灭火”，却无法像人类一样理解“为什么”；它知道“猫有耳朵”，却难以推理“如果耳朵受伤会怎样”。所以说，它拥有庞大的经验，却缺乏深刻的理解；能做出判断，却无法真正解释。这也决定了它在推理、常识、因果建模等方面的局限。

麻烦守恒定律：从“调特征”到“调网络” 在经典机器学习时代，模型的表现高度依赖于人类专家手动提取的特征。研究者需要根据先验知识，从原始数据中精心设计出各种特征 (Feature)，再交由算法进行分类或回归，这一过程被称为“特征工程” (Feature Engineering)。这种方法虽然可控、透明，每一步都“看得见、想得通”，但高度依赖经验与直觉。特征选得好，模型效果可能立竿见影；选得差，再高明的算法也无济于事。因此，早期的机器学习专家常常将大量时间花在“调特征”上，既辛苦又低效。可以说，传统机器学习更像是一门“手工艺术”——靠的是人的经验、技巧、灵感，而不是一个自动化的智能系统。后来，研究者逐渐发现，可以让神经网络从数据中自动学习特征。特征表示学习 (Feature Representation Learning) 随之兴起。模型不再依赖人工“筛选”特征，而是通过多层网络自动提取抽象表示。研究者们终于从自寻特征的苦役中解脱了出来。随着网络不断加深和复杂化，学习能力得到了极大提升，这也催生了一个新的名字——“深度学习” (Deep Learning)。然而，解脱的代价也很快到来。深度学习学到的特征，深藏于高维空间与层层神经元之间，早已超越了人类直觉和逻辑的可解释范围。这种“自动学习”的过程虽然强大，却也彻底变成了一个“黑盒”。为了让神经网络的学习性能表现

得更好，我们只能依据经验，不断尝试性地进行网络结构和网络超参数的调整。我们不再“调特征”，却掉入了“调网络”的新领域——隐藏层的层数、每层的神经元个数、激活函数、学习率、优化器，依旧是一项高度依赖经验的“玄学劳动”。网络越深，模型越强，但也越难调试、越难解释，也越难信任。于是人工智能领域就有了这样的调侃：“有多少人工，就有多少智能”。你看，在这个世界上，存在着一个“麻烦守恒定律”：“麻烦不会消失，只会转移。”就机器学习而言，从“人工选特征”转移到“人工调模型”。

未来可期，但仍需警醒 面对深度学习的诸多挑战，科学界并未停止努力。小样本学习（Few-shot Learning）正在尝试缓解对大数据的强依赖，可解释人工智能（Explainable AI, XAI）力图揭开“黑盒”的神秘面纱，模型剪枝与压缩技术正致力于降低能耗、提升效率，而多模态学习则在努力增强模型的理解力与泛化能力，让模型更接近真正的“认知”。然而不可否认的是，当前的深度学习，仍是一套“强拟合、弱推理”的技术体系。它擅长从数据中发现相关性，却尚未掌握像人类那样基于因果与逻辑的推理能力。它的成功更多是统计学意义上模式捕捉的“聪明”，而不是认知科学意义上的“理解”。我们尚未找到真正“让机器理解世界”的路径。但也正因如此，深度学习才如此引人入胜：它既是答案的一部分，也是问题的一部分。深度学习虽强大，但不万能。理解深度学习的局限，是理解它的真正开始。深度学习并不是魔法，而是一种“经验式认知系统”的数学构造。它的成功让我们惊叹，也提醒我们保持反思的清醒。深度学习的局限，正是我们真正理解它的起点。理解它的强大，才能放心使用；理解它的盲点，才能真正推动它向“更强智能”迈进。科学的本质之一是揭示“我们是怎么知道的”，深度学习的发展为这一追问提供了技术上的回答。从模拟生物神经元开始，到构建多层网络结构，深度学习的每一次跃进，不仅是人工智能技术的突破，而且是一种科学方法论的进步，也是一场关于“如何理解世界”的科学思维革新。深度学习的发展历程在某种程度上也加深了我们对智能本质和自身认知机制的新思考。本章从基本原理出发，系统介绍了深度学习的核心结构（如全连接神经网络、CNN、RNN、Transformer）、优化方法及其在视觉、语言、医疗、科学研究等领域的广泛应用。深度学习已经走出了实验室，成为推动社会智能化变革的底层力量。从数学角度看，深度学习是一种通过多层神经网络进行“表示学习”的方法，它将原始数据逐层映射为更抽象的特征表达，构建起一个能够捕捉非线性结构与复杂模式的函数系统。它的核心能力，不仅在于强大的模式识别，更在于能够从原始数据中自动生成中间表征——即便这些表征对人类并不直观，却对机器决策至关重要。在方法论上，深度学习体现了一种“复杂性科学”的哲学转向。不同于传统机器学习中“分而治之”的还原主义思路，深度学习强调系统的整体性、非线性与多层级嵌套，力图在数据中直接学习全貌而非规则。这种系统性构造，使得深度学习不仅是一种技术框架，更是一种新的“智能范式”。但深度学习的成功，也并不意味着它已经穷尽了智能的全部路径。它依然是“经验驱动”的，仍以“归纳模式识别”为主，距离“因果推理”与“可解释认知”尚有一段距离。它解决了传统 AI 在感知层面的瓶颈，却也暴露出在推理、理解、常识、样本效率等方面的不足。我们正在进入一个从“数据喂养”走向“通用智能”的转型阶段。深度学习正处于这个转折点的中心。它像一面镜子，让我们在构建机器智能的过程中，重新反思人类自身的认知机制：我们如何理解、如何抽象、如何决策、如何创造？深度学习不只是让

机器更聪明，更在促使我们重新认识什么是“理解”。未来，它会走向何方？是强化现有的统计学习框架，还是迈向更具推理能力的“混合智能”？是继续加深网络结构，还是构建更轻量、更普适的模型？是走向自监督、少样本与通用人工智能，还是与人类智能形成更紧密的协同系统？这一切，仍在展开之中。但可以肯定的是，深度学习已经让我们重新相信：理解世界、甚至创造世界，是可以被建模、被优化、被实践的。

参考文献

- [1] “A Logical Calculus of Ideas Immanent in Nervous Activity”. In: *Bulletin of Mathematical Biophysics* 5 (1943), pp. 127–147.
- [2] Yoshua Bengio. “Practical recommendations for gradient-based training of deep architectures”. In: *Neural networks: Tricks of the trade: Second edition*. Springer, 2012, pp. 437–478.
- [3] George Cybenko. “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of control, signals and systems* 2.4 (1989), pp. 303–314.
- [4] Kunihiro Fukushima and Sei Miyake. “Neocognitron: A new algorithm for pattern recognition tolerant of deformations and shifts in position”. In: *Pattern recognition* 15.6 (1982), pp. 455–469.
- [5] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Multilayer feedforward networks are universal approximators”. In: *Neural Networks* 2.5 (1989), pp. 359–366.
- [6] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Universal approximation of an unknown mapping and its derivatives using multilayer feedforward networks”. In: *Neural networks* 3.5 (1990), pp. 551–560.
- [7] David H Hubel and Torsten N Wiesel. “Receptive fields and functional architecture of monkey striate cortex”. In: *The Journal of physiology* 195.1 (1968), pp. 215–243.
- [8] Han Jiawei, Kamber Micheline, and Pei Jian. *Data Mining: Concepts and Techniques*. -3rd. Morgan Kaufmann, 2011.
- [9] Fukushima K and Miyake S. “Neocognitron: A self-organizing neural network model for a mechanism of visual pattern recognition”. In: *Competition and cooperation in neural nets*. Springer, Berlin, Heidelberg, 1982, pp. 267–285.
- [10] Diederik P Kingma and Jimmy Ba. “Adam: A method for stochastic optimization”. In: *arXiv preprint arXiv:1412.6980* (2014).
- [11] Yann Le Cun, Boser Brian, and et al. Denker John S. “Handwritten digit recognition with a back-propagation network”. In: *Advances in neural information processing systems*. 1990, pp. 396–404.
- [12] Yann Le Cun et al. “Handwritten digit recognition: Applications of neural net chips and automatic learning”. In: *Neurocomputing: Algorithms, Architectures and Applications*. Springer. 1990, pp. 303–318.
- [13] Yann LeCun et al. “Backpropagation applied to handwritten zip code recognition”. In: *Neural computation* 1.4 (1989), pp. 541–551.

- [14] Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.
- [15] Minsky Marvin and A Papert Seymour. “Perceptrons”. In: *Cambridge, MA: MIT Press* 6.318–362 (1969), p. 7.
- [16] Warren S McCulloch and Walter Pitts. “A logical calculus of the ideas immanent in nervous activity”. In: *The bulletin of mathematical biophysics* 5 (1943), pp. 115–133.
- [17] Tom Mitchell. *Machine Learning*. McGraw-Hill Education, 1997.
- [18] Michael Nielsen. *Neural Networks and Deep Learning*. Accessed: 2025-03-03. 2015. URL: <http://neuralnetworksanddeeplearning.com/>.
- [19] Hastie T, Tibshirani R, and Friedman J. 统计学习基础 (*The Elements of Statistical Learning*). 北京: 世界图书出版公司, 2015.
- [20] Alan Turing. *Computing Machinery and Intelligence*. Oxford, 1950.
- [21] Vladimir Naumovich Vapnik. 统计学习理论的本质. 清华大学华夏出版社, 2000.
- [22] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).
- [23] 于剑. 机器学习—从公理到算法. 北京: 清华大学出版社, 2017.
- [24] [意] 翁贝托·博塔兹尼. 余婷婷译. 尖叫的数学—令人惊叹的数学之美. 湖南科学技术出版社, 2021.
- [25] 周志华. 机器学习. 清华大学出版社, 2016.
- [26] Vladimir N. Vapnik. 张学工译. 统计学习理论的本质. 北京: 清华大学出版社, 2000.
- [27] 张玉宏. 深度学习之美—AI 时代的数据处理与最佳实践. 电子工业出版社, 2018.
- [28] 张玉宏. 深度学习之美: AI 时代的数据处理与最佳实践. 电子工业出版社, 2018.
- [29] Tom Mitchell. 曾华军等译. 机器学习. 北京: 机械工业出版社, 2002.
- [30] 王珏, 周志华, 周傲英. 机器学习及其应用. Vol. 4. 清华大学出版社有限公司, 2006.

参考文献

- [1] “A Logical Calculus of Ideas Immanent in Nervous Activity”. In: *Bulletin of Mathematical Biophysics* 5 (1943), pp. 127–147.
- [2] Yoshua Bengio. “Practical recommendations for gradient-based training of deep architectures”. In: *Neural networks: Tricks of the trade: Second edition*. Springer, 2012, pp. 437–478.
- [3] George Cybenko. “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of control, signals and systems* 2.4 (1989), pp. 303–314.
- [4] Kunihiro Fukushima and Sei Miyake. “Neocognitron: A new algorithm for pattern recognition tolerant of deformations and shifts in position”. In: *Pattern recognition* 15.6 (1982), pp. 455–469.
- [5] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Multilayer feedforward networks are universal approximators”. In: *Neural Networks* 2.5 (1989), pp. 359–366.
- [6] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Universal approximation of an unknown mapping and its derivatives using multilayer feedforward networks”. In: *Neural networks* 3.5 (1990), pp. 551–560.
- [7] David H Hubel and Torsten N Wiesel. “Receptive fields and functional architecture of monkey striate cortex”. In: *The Journal of physiology* 195.1 (1968), pp. 215–243.
- [8] Han Jiawei, Kamber Micheline, and Pei Jian. *Data Mining: Concepts and Techniques*. -3rd. Morgan Kaufmann, 2011.
- [9] Fukushima K and Miyake S. “Neocognitron: A self-organizing neural network model for a mechanism of visual pattern recognition”. In: *Competition and cooperation in neural nets*. Springer, Berlin, Heidelberg, 1982, pp. 267–285.
- [10] Diederik P Kingma and Jimmy Ba. “Adam: A method for stochastic optimization”. In: *arXiv preprint arXiv:1412.6980* (2014).
- [11] Yann Le Cun, Boser Brian, and et al. Denker John S. “Handwritten digit recognition with a back-propagation network”. In: *Advances in neural information processing systems*. 1990, pp. 396–404.
- [12] Yann Le Cun et al. “Handwritten digit recognition: Applications of neural net chips and automatic learning”. In: *Neurocomputing: Algorithms, Architectures and Applications*. Springer. 1990, pp. 303–318.
- [13] Yann LeCun et al. “Backpropagation applied to handwritten zip code recognition”. In: *Neural computation* 1.4 (1989), pp. 541–551.

- [14] Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.
- [15] Minsky Marvin and A Papert Seymour. “Perceptrons”. In: *Cambridge, MA: MIT Press* 6.318–362 (1969), p. 7.
- [16] Warren S McCulloch and Walter Pitts. “A logical calculus of the ideas immanent in nervous activity”. In: *The bulletin of mathematical biophysics* 5 (1943), pp. 115–133.
- [17] Tom Mitchell. *Machine Learning*. McGraw-Hill Education, 1997.
- [18] Michael Nielsen. *Neural Networks and Deep Learning*. Accessed: 2025-03-03. 2015. URL: <http://neuralnetworksanddeeplearning.com/>.
- [19] Hastie T, Tibshirani R, and Friedman J. 统计学习基础 (*The Elements of Statistical Learning*). 北京: 世界图书出版公司, 2015.
- [20] Alan Turing. *Computing Machinery and Intelligence*. Oxford, 1950.
- [21] Vladimir Naumovich Vapnik. 统计学习理论的本质. 清华大学华夏出版社, 2000.
- [22] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).
- [23] 于剑. 机器学习—从公理到算法. 北京: 清华大学出版社, 2017.
- [24] [意] 翁贝托·博塔兹尼. 余婷婷译. 尖叫的数学—令人惊叹的数学之美. 湖南科学技术出版社, 2021.
- [25] 周志华. 机器学习. 清华大学出版社, 2016.
- [26] Vladimir N. Vapnik. 张学工译. 统计学习理论的本质. 北京: 清华大学出版社, 2000.
- [27] 张玉宏. 深度学习之美—AI 时代的数据处理与最佳实践. 电子工业出版社, 2018.
- [28] 张玉宏. 深度学习之美: AI 时代的数据处理与最佳实践. 电子工业出版社, 2018.
- [29] Tom Mitchell. 曾华军等译. 机器学习. 北京: 机械工业出版社, 2002.
- [30] 王珏, 周志华, 周傲英. 机器学习及其应用. Vol. 4. 清华大学出版社有限公司, 2006.